

关于每个程序员都应该了解的内存知识

乌尔里希·德雷
珀 Red Hat, Inc.
drepper@redhat.com

2007年11月21日

摘要

随着CPU核心变得越来越快且数量越来越多，对于大多数程序而言，内存访问现在已成为性能瓶颈，并将在一段时间内继续保持这一状态。硬件设计师提出了越来越多的复杂内存处理和加速技术——例如CPU缓存——但这些技术无法在没有程序员协助的情况下实现最佳效果。遗憾的是，大多数程序员对计算机内存子系统的结构及其使用成本以及CPU缓存的成本知之甚少。本文解释了现代商品硬件中使用的内存子系统的结构，并说明了为何开发出了CPU缓存、它们的工作原理以及程序应如何利用这些缓存以达到最优性能。

1 引言

在早期，计算机要简单得多。系统中的各个组件，如CPU、内存、大规模存储和网络接口等，是同步开发的，并且它们的性能相当均衡。例如，内存和网络接口提供的数据速度并不（显著）快于CPU。

这种情况在计算机的基本结构稳定下来，硬件开发人员专注于优化各个子系统之后发生了变化。突然间，计算机中某些组件的性能显著落后于其他部分，并且出现了瓶颈问题。特别是由于成本原因，大规模存储和内存子系统的改进速度相对于其他组件更为缓慢。

大规模存储设备的缓慢问题主要通过软件技术来解决：操作系统将最常使用（且最有可能被使用的）数据保留在主内存中，其访问速度比硬盘快几个数量级。还在存储设备本身增加了缓存存储，这无需更改操作系统即可提升性能。¹ 在本文中，我们不会进一步详细讨论针对大规模存储访问的软件优化技术。

与存储子系统不同，消除主内存作为瓶颈的难度要大得多，并且几乎所有的解决方案都需要对硬件进行改动。为了实现这一目标——

¹Changes are needed, however, to guarantee data integrity when using storage device caches.

目前这些变化主要表现为以下几种形式：

- RAM硬件设计（速度与并行性）。
- 内存控制器设计。
- CPU缓存。
- 设备的直接内存访问（DMA）。

本文主要讨论CPU缓存以及内存控制器设计的一些影响。在探讨这些主题的过程中，我们将研究直接内存访问（DMA）并将其纳入更广泛的图景中。然而，我们首先会概述当今通用硬件的设计。这是理解高效利用内存子系统的问题和局限性的前提条件。我们还将详细学习不同类型的RAM，并说明为什么这些差异仍然存在。

本文件并非全面且最终的文档。它仅限于通用硬件，并进一步局限于其中的一部分硬件。此外，许多主题将仅以达成本文目标所需的细节程度进行讨论。对于此类主题，建议读者查找更详细的文档。

当涉及到特定操作系统的细节和解决方案时，本文仅描述Linux。文中不会包含任何其他操作系统的信息。作者无意讨论这些内容对其他操作系统的影响。如果读者认为需要使用不同的操作系统，则必须向其供应商要求提供类似此文档的资料。

在开始之前，有一点需要说明。文中包含多个“通常”一词及其类似的限定词的使用。这里讨论的技术存在以下情况：

在实际应用中存在许多变体，本文仅讨论最常见的主流版本。对于这项技术很难做出绝对性的陈述，因此文中会使用一些限定词。

文档结构

本文主要面向软件开发人员。文中并未深入介绍硬件技术细节，因此对关注硬件的读者帮助较小。但在进入针对开发人员的实际内容之前，我们必须先奠定大量基础。

为此，第二部分详细介绍了随机存取存储器（RAM）。本部分内容虽好了解，但并非理解后续章节所必需。在需要引用该部分内容的地方会添加适当的回指链接，以便焦虑的读者可以先跳过大部分内容。

第三部分详细介绍了CPU缓存的行为。图表的使用使得文本不至于过于枯燥乏味。这部分内容对于理解文档其余部分至关重要。第四部分简要描述了虚拟内存的实现方式，这也是理解后续内容的基础。

第五节详细介绍了非一致内存访问（NUMA）系统。

第六节是本文的核心部分。它汇集了前几节的所有信息，并为程序员提供了在各种情况下编写高效代码的建议。不耐烦的读者可以直接从这一节开始阅读，如有必要，可以回溯到前面的部分以重温相关技术细节。

第7节介绍了可以帮助程序员更好地完成工作的工具。即使对技术有全面的理解，也很难明确非平凡软件项目中的问题所在。有些工具是必不可少的。

在第8节中，我们最终展望了未来短期内可能出现的技术或一些值得拥有的技术。

报告问题

作者计划长期更新此文档。这不仅包括因技术进步而必要的更新，也包括修正错误。愿意报告问题的读者可将邮件发送给作者。请在报告中包含确切的版本信息。该版本信息可在文档的最后一页找到。

感谢

我想感谢John Ray Fuller以及LWN团队（特别是Jonathan Corbet，他承担了将作者的英语风格转化为更为传统的表达方式这一艰巨任务）。Markus Armbruster对文本中的问题和遗漏提供了许多宝贵的反馈。

关于本文档

本文的标题是对大卫·戈德伯格的经典论文《每个计算机科学家都应了解的浮点运算知识》[12]的一种致敬。尽管如此，这篇论文仍未被广泛知晓，然而它应当是任何敢于从事严肃编程工作的人的必修课。

关于PDF文件：xpdf绘制的部分图表质量较差。建议使用evince或必要时使用Adobe的程序来查看。如果使用evince，请注意尽管阅读器没有像其他程序那样指示，文档中广泛使用了超链接。

2 当今的通用硬件

理解通用硬件的重要性是因为专用硬件正在逐渐退步。如今，扩展更多情况下是通过水平扩展而非垂直扩展来实现的，这意味着使用许多较小且相互连接的通用计算机比使用少数几台非常大且异常快速（以及昂贵）的系统更具成本效益。这是因为高速且廉价的网络硬件广泛可用。尽管在某些情况下大型专用系统仍然有其存在的必要性，并且这些系统仍提供商业机会，但总体市场规模远不及通用硬件市场的规模。红帽公司截至2007年预计，未来产品的“标准构建模块”将是每台最多配备四个插槽的计算机，每个插槽装有一个四核CPU，在英特尔CPU的情况下将启用超线程技术。这意味着数据中心的标准系统将拥有多达64个虚拟处理器。虽然会支持更大规模的机器，但目前认为四插槽、四核心的情况是最佳选择，并且大多数优化措施都是针对此类系统的。

大型商品部件构建的计算机在结构上存在显著差异。尽管如此，通过集中讨论最重要的区别，我们可以涵盖超过90%的此类硬件。请注意，这些技术细节变化迅速，因此读者应考虑本文撰写的时间。

多年来，个人计算机和小型服务器标准化采用了一种芯片组结构，该结构分为两部分：北桥和南桥。图2.1展示了这种结构。

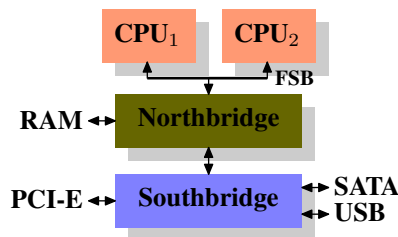


图2.1：北桥与南桥结构示意图

所有CPU（在之前的例子中是两个，但可以更多）通过一条公共总线（前端总线，FSB）连接到北桥。北桥包含内存控制器等组件，其实现决定了计算机使用的RAM芯片类型。不同类型的RAM，如DRAM、Rambus和SDRAM，需要不同的内存控制器。

为了与其他系统设备通信，北桥必须与南桥进行交互。南桥，通常被称为I/O桥，负责处理输入输出相关事务。

²Hyper-threading enables a single processor core to be used for two or more concurrent executions with just a little extra hardware.

通过多种不同的总线与设备进行通信。目前，PCI、PCIe、SATA和USB总线最为重要，但南桥也支持PATA、IEEE 1394、串行端口和并行端口。旧系统中北桥连接有AGP插槽，这是出于性能原因，因为北桥与南桥之间的连接速度不够快。然而，如今所有的PCI-E插槽都直接连接到南桥。

这样的系统结构具有若干值得注意的后果：

- 所有从一个CPU到另一个CPU的数据通信都必须通过与北桥通信所使用的同一总线进行。
- 与内存（RAM）的所有通信都必须通过北桥芯片进行。
- 内存只有一个端口。³
- CPU与南桥相连的设备之间的通信需通过北桥进行。

在该设计中，几个瓶颈问题立即显现。其中一个瓶颈涉及设备访问RAM的问题。在PC早期阶段，所有与桥接器上的设备通信都必须通过CPU进行，这严重影响了整个系统的性能。为了解决这个问题，一些设备开始具备直接内存访问（DMA）的能力。借助北桥的帮助，DMA允许设备无需CPU的干预（及其固有的性能成本），直接将数据存储和接收在RAM中。如今，所有连接到任何总线上的高性能设备都可以利用DMA功能。虽然这大大减轻了CPU的工作负担，但也导致了北桥带宽的竞争问题，因为DMA请求与来自CPU的RAM访问相互竞争。因此，必须考虑这个问题。

第二个瓶颈在于北桥到RAM的总线。具体细节取决于部署的内存类型。在较旧的系统中，只有一个总线连接所有RAM芯片，因此无法实现并行访问。最近的RAM类型需要两条独立的总线（对于DDR2而言称为通道，详见第8页），这将可用带宽翻倍。北桥会在各个通道之间交错存取内存。更近期的内存技术（如FB-DRAM）增加了更多的通道。

在可用带宽有限的情况下，为了性能考虑，需要安排内存访问以最小化延迟。正如我们将看到的，处理器的速度要快得多。

³We will not discuss multi-port RAM in this document as this type of RAM is not found in commodity hardware, at least not in places where the programmer has access to it. It can be found in specialized hardware such as network routers which depend on utmost speed.

并且在使用CPU缓存的情况下仍需等待访问内存。如果多个超线程、核心或处理器同时访问内存，内存访问的等待时间会更长。这同样适用于DMA操作。

访问内存不仅仅是并发的問題。访问模式本身也会极大地影响内存子系统的性能，尤其是在存在多个内存通道的情况下。在第2.2节中，我们将详细介绍RAM的访问模式。

在一些更高级的系统中，北桥芯片实际上并不包含内存控制器。相反，北桥可以通过连接多个外部内存控制器（例如，在以下示例中为四个）来实现与内存的通信。

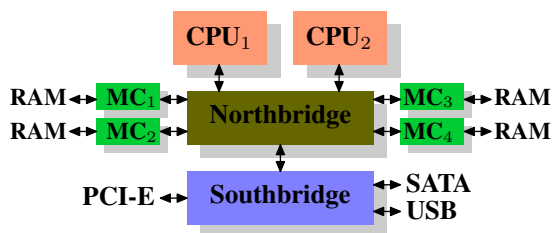


图2.2: 带有外部控制器的北桥

这种架构的优势在于存在多条内存总线，因此总的可用带宽增加。该设计还支持更多的内存。并发的内存访问模式通过同时访问不同的内存银行来减少延迟。特别是在如图2.2所示多个处理器直接连接到北桥的情况下，这种情况尤为明显。对于这样的设计，主要限制是北桥的内部带宽，而这在该架构中（来自Intel）是非常出色的。

使用多个外部内存控制器并不是增加内存带宽的唯一方式。另一种越来越流行的方法是将内存控制器集成到CPU中，并为每个CPU配置相应的内存。这种架构在基于AMD Opteron处理器的对称多处理器（SMP）系统中得到了广泛应用。图2.3展示了这样的系统。英特尔从Nehalem处理器开始支持通用系统接口（CSI），这基本上是相同的方法：集成内存控制器，每个处理器可以拥有本地内存的可能性。

在这种架构下，内存银行的数量与处理器数量相同。在四路CPU机器上，无需复杂的北桥芯片即可将内存带宽提升四倍。将内存控制器集成到CPU中还有一些额外的优势；我们不会深入探讨这一点。

⁴For completeness it should be mentioned that such a memory controller arrangement can be used for other purposes such as “memory RAID” which is useful in combination with hotplug memory.

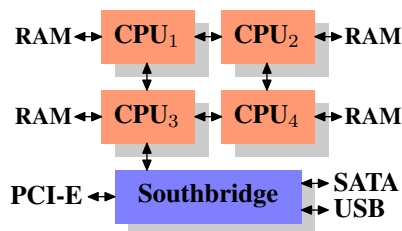


图2.3: 集成内存控制器

更深入地探讨这项技术。

这种架构也存在一些缺点。首先，由于系统中的所有内存仍然必须对所有处理器开放访问权限，因此内存不再具有统一性（这就是为什么称其为NUMA架构——非一致内存访问架构）。本地内存（连接到某个处理器的内存）可以以通常的速度进行访问。然而，当访问连接到其他处理器的内存时情况就不同了。在这种情况下，需要使用处理器之间的互连。从CPU₁访问连接到CPU₂的内存需要跨过一个互连。而当同一CPU访问连接到CPU₄的内存时，则需要跨越两个互连。

每种通信都伴随着一定的成本。当我们描述访问远程内存所需额外的时间时，我们会提到“NUMA因素”。如图2.3所示的架构中，每个CPU有两个层次：直接相邻的CPU和距离为两个互连设备远的一个CPU。在更复杂的机器上，层级的数量可能会显著增加。还有一些机器架构（例如IBM的x445系列和SGI的Altix系列）具有多种类型的连接。CPU被组织成节点；在一个节点内部，访问内存的时间可能是均匀的或只有很小的NUMA因素。然而，在节点之间的连接可能非常昂贵，并且NUMA因素可能会很高。

现有的商品NUMA机器已经存在，并且在未来可能会扮演更重要的角色。预计从2008年底开始，所有的SMP机器都将采用NUMA架构。由于NUMA相关的成本问题，识别程序是否在NUMA机器上运行变得非常重要。在第5节中，我们将讨论更多类型的机器架构以及Linux内核为这些程序提供的某些技术。

除了本节其余部分所描述的技术细节之外，还有若干因素会影响内存（RAM）的性能。这些因素无法通过软件进行控制，因此未在本节中涵盖。感兴趣的读者可以在第2.1节了解其中一些因素。掌握这些因素有助于更全面地理解内存技术，并可能在购买计算机时做出更好的决策。

以下两节将从门级详细讨论硬件细节以及内存控制器与DRAM芯片之间的访问协议。程序员可能会发现这些信息很有启发性，因为它们解释了为什么RAM访问方式是这样的。不过，对于急于了解更贴近日常生活主题的读者来说，可以直接跳到2.2.5节。

2.1 内存类型

多年来，RAM的类型多种多样，每种类型的特性各不相同，有时差异甚大。如今，较早的那些类型仅对历史学家具有研究价值。我们不会深入探讨这些早期类型的细节。相反，我们将集中讨论现代RAM类型，并且只会浅尝辄止，探索一些可以通过性能特征为内核或应用程序开发人员所观察到的细节。

首先，有趣的问题集中在为什么在同一台机器中会有不同类型的RAM。更具体地说，为什么既有静态RAM（SRAM）又有动态RAM（DRAM）。前者速度更快，并且提供相同的功能。既然SRAM如此优越，为何不将机器中的所有RAM都用SRAM呢？答案正如人们所预料的那样，是成本问题。生产SRAM和使用SRAM的成本都要比DRAM高得多。这两个成本因素都很重要，尤其是后者的重要性日益增加。为了理解这些差异，我们需要考察SRAM和DRAM中存储一位数据的具体实现方式。

在本节的其余部分，我们将讨论RAM实现的一些底层细节。为了尽量减少细节层次，我们将从“逻辑层面”来探讨信号，而不是硬件设计师实际需要的层面。这种程度的细节对于我们目前的目的来说是不必要的。

2.1.1 静态随机存取存储器（SRAM）

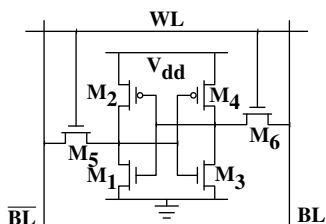


图2.4：6T静态随机存取存储器（SRAM）

图2.4展示了由6个晶体管组成的SRAM单元结构。该单元的核心是由M1到M4这四个晶体管构成的两个交叉耦合反相器。它们具有两种稳定状态，分别表示0和1。只要电源V_{dd}有效，这种状态就是稳定的。

在其他上下文中，SRAM可能表示“同步RAM”。

若需访问存储单元的状态，则提升字线WL。这使得存储单元的状态可立即在BL和BL_{bar}上读取。如果必须覆盖存储单元的状态，首先将BL和BL_{bar}设置为所需的值，然后提升WL。由于外部驱动器比四个晶体管（M1至M4）更强，因此可以覆盖旧状态。

参见[20]以获取关于细胞工作原理的更详细描述。对于以下讨论，重要的是要注意到

- 一个单元需要六个晶体管。也有使用四个晶体管的变体，但它们存在一些缺点。
- 维持单元的状态需要持续供电。
- 单元状态在字访问线WL升高的几乎瞬间即可读取。该信号与其他由晶体管控制的信号一样，矩形化（在两种二进制状态之间快速切换）。
- 单元状态稳定，无需进行刷新周期。

还有其他速度较慢但功耗更低的SRAM形式可供选择，但由于我们关注的是快速RAM，因此这些变体在此不作考虑。然而，这些慢速变体由于接口更简单，在系统中使用时比动态RAM更为便捷。

2.1.2 动态随机存取存储器（DRAM）

动态随机存取存储器（DRAM）在结构上比静态随机存取存储器（SRAM）简单得多。图2.5展示了典型的DRAM单元设计结构。它仅由一个晶体管和一个电容组成。这种复杂性的巨大差异当然意味着它的工作方式与静态随机存取存储器截然不同。

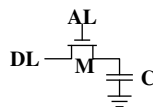


图2.5：1T型动态随机存取存储器（DRAM）

动态RAM单元的状态存储在电容C中。晶体管M用于保护对该状态的访问。要读取单元的状态，需要将访问线AL置高；这会根据电容中的电荷情况，在数据线DL上产生或不产生电流。要向单元写入数据，则需先适当设置数据线DL，然后将访问线AL保持在高位足够长的时间以对电容充电或放电。

动态随机存取内存（DRAM）的设计存在若干复杂问题。使用电容意味着读取操作会导致数据丢失，因此在每次读取后必须进行刷新以恢复数据。

单元中的电容放电。这一过程不能无限重复，电容必须在某个时刻重新充电。更糟糕的是，为了容纳大量的单元（目前常见的芯片包含10万个或更多的单元），电容的容量必须非常小（处于飞法范围或更低）。一个完全充满电的电容仅能储存几万到十万电子。尽管电容的电阻很高（大约几个太欧姆），但其电量耗散所需的时间却很短。这一问题被称为“漏电”。

这种泄漏是为什么DRAM单元必须不断刷新的原因。如今，大多数DRAM芯片每隔64毫秒就必须进行一次刷新。在刷新周期内，由于刷新本质上是一个被丢弃结果的内存读操作，因此无法访问内存。对于某些工作负载来说，这种开销可能会导致多达50%的内存访问停滞（参见[3]）。

由于单元中存储的电荷很小，从单元读取的信息无法直接使用。数据线必须连接到一个能够区分整个有效电荷范围内存储的0或1的检测放大器。

读取一个单元会导致电容上的电荷被耗尽。这意味着每次读操作之后都必须进行一次给电容重新充电的操作。这是通过将感测放大器的输出反馈到电容来自动完成的。不过，这确实意味着读取内存内容需要额外的能量和时间。

第四個问题是电容的充放电过程并非瞬间完成。感测放大器接收到的信号不是矩形波，因此必须使用保守估计来确定单元输出何时可用。电容充放电公式的表达式为

$$Q_{\text{Charge}}(t) = Q_0(1 - e^{-\frac{t}{RC}})$$

$$Q_{\text{Discharge}}(t) = Q_0 e^{-\frac{t}{RC}}$$

这意味着电容的充电和放电需要一定的时间（由容量C和电阻R决定）。这也意味着感测放大器能够检测到的电流不会立即可用。图2.6展示了充电和放电曲线。X轴以RC（电阻乘以电容）为单位，这是一个时间单位。

与静态随机存取存储器不同，当访问字线被置高时输出会立即可用，在动态随机存取存储器中，电容器放电到足够程度总是需要一些时间。这种延迟严重限制了DRAM的速度。

简单的方法也有其优势。主要的优势在于尺寸。实现一个DRAM单元所需的芯片面积远远小于SRAM单元。

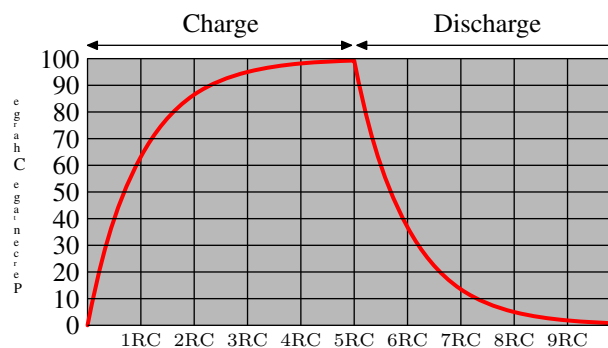


图2.6：电容充放电时序

SRAM 单元还需要为维持状态的晶体管提供单独的电源。DRAM

总体而言，（成本上的显著）差异决定了最终选择。除了专门的硬件——例如网络路由器——之外，我们不得不使用基于DRAM的主存。这给程序员带来了巨大的影响，这是我们将在本文剩余部分讨论的内容。但在那之前，我们需要进一步探讨一下实际使用的DRAM单元的一些细节。

2.1.3 动态随机存取存储器访问DRAM访问

一个程序使用虚拟地址选择内存位置。处理器将其转换为物理地址，最终由内存控制器根据该地址选择对应的RAM芯片。为了在RAM芯片上选择单个内存单元，物理地址的部分内容以若干地址线的形式传递。

单独从内存控制器地址映射每个内存位置是完全不切实际的：4GB 的 RAM 需要 2^{32} 条地址线。相反，地址被编码为一个二进制数，并通过一组较少的地址线传递。以这种方式传送到 DRAM 芯片的地址必须先经过解复用处理。具有 N 条地址线的解复用器将有 2^N 条输出线。这些输出线可以用来选择内存单元。对于容量较小的芯片，采用这种直接方法并没有太大问题。

但是，如果存储单元的数量增加，这种方法就不再适用了。一个容量为1Gbit的芯片需要30条地址线和2条选择线。当速度不能被牺牲时，解复用器的大小会随着输入线路数量呈指数级增长。对于30条地址线的解复用器来说，除了其复杂性（尺寸和时间）之外，还需要大量的芯片面积。更重要的是，传输

我讨厌那些SI前缀。对我来说，吉比特始终是 2^{30} 位而不是 10^9 位。

30个地址线上的同步脉冲传输要比仅仅传输“只有”15个脉冲困难得多。较少的线路也需要被精确布置成相同的长度或进行适当的定时。

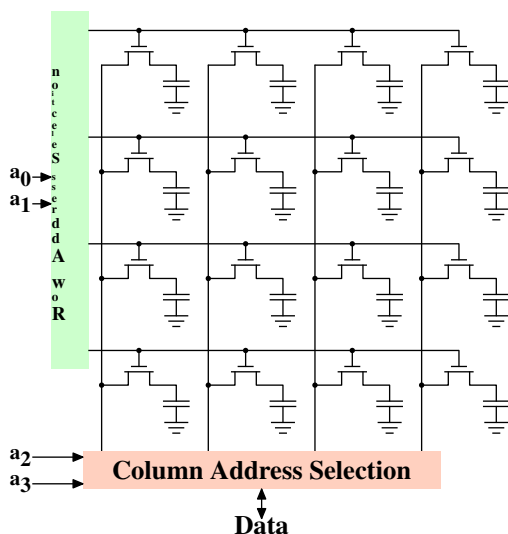


图2.7：动态随机存取存储器原理图

图2.7展示了DRAM芯片的高层次视图。DRAM单元组织成行和列。如果所有单元都排列在一行中，则DRAM芯片需要一个巨大的解复用器。采用阵列方法，设计只需要一个解复用器和半个大小的复用器。这在各个方面都是巨大的节省。例如，在地址线a0和a1通过RAS解复用器选择整个行单元的地址线时，读取时所有单元的内容都会提供给CAS复用器。根据地址线a2和a3，一个列的内容会被提供到DRAM芯片的数据引脚上。在多个DRAM芯片中并行进行多次操作，以生成与数据总线宽度相对应的位数总量。

对于写入操作，新的单元值会被放置在数据总线上，并且当使用RAS和CAS选中该单元时，它将被存储到单元中。设计相当直接明了。但实际上——显然地——存在许多更多复杂性。需要明确规定信号之后的数据总线上的数据可用延迟时间。正如前一节所述，电容器不会瞬间放电。从单元发出的信号非常微弱，因此必须进行放大处理。对于写入操作，则需明确在RAS和CAS完成之后，数据必须保留在总线上多长时间才能成功将新值存储到单元中（再次强调，电容）。

⁷Modern DRAM types like DDR3 can automatically adjust the timing but there is a limit as to what can be tolerated.

⁸Multiplexers and demultiplexers are equivalent and the multiplexer here needs to work as a demultiplexer when writing. So we will drop the differentiation from now on.

⁹The line over the name indicates that the signal is negated.

这些时序常数对于DRAM芯片的性能至关重要（因为监视器不会瞬间充满或排空）。我们将在下一节中讨论这一话题。

一个次要的可扩展性问题是，每块RAM芯片连接30条地址线是不可行的。芯片引脚是宝贵的资源。数据必须尽可能并行传输（例如，每次64位）已经是一个“问题”了。内存控制器必须能够寻址每个RAM模块（由若干RAM芯片组成）。如果出于性能原因需要同时访问多个RAM模块，并且每个RAM模块都需要自己的30条或更多地址线，则对于8个RAM模块而言，内存控制器仅用于地址处理就需要惊人的240根引脚。

为了应对DRAM芯片长期以来存在的二级可扩展性问题，这些芯片会对其地址本身进行复用。这意味着地址会被分成两部分传输。第一部分由地址位（如图2.7中的a0和a1）组成，用于选择行。这一选择状态保持有效直到被撤销。然后第二部分，即地址位a2和a3，用于选择列。关键的区别在于只需要两条外部地址线即可完成操作。需要额外几条线路来指示RAS和CAS信号何时可用，但这相对于将地址线数量减半而言是一个很小的代价。然而，这种地址复用也带来了一套新的问题，我们将在第2.2节中讨论这些问题。

2.1.4 结论

不必担心本节的细节有些令人感到难以承受。本节的重点内容是：

- 并非所有内存都是SRAM的原因有很多。
- 内存单元需要被单独选中以供使用。
- 地址线的数量直接决定了内存控制器、主板、DRAM模块和DRAM芯片的成本。
- 读取或写入操作的结果需要一段时间才能得到。

以下部分将详细介绍访问DRAM内存的实际过程。我们不会深入讨论访问SRAM的过程，因为SRAM通常是直接寻址的，这主要是出于速度考虑以及SRAM容量有限的原因。目前，SRAM主要用于CPU缓存和片上存储器中，在这些地方连接线路短且完全受CPU设计者控制。关于CPU缓存的话题稍后会进行讨论，但我们需要了解的是，SRAM单元具有一定的最大速度，该速度取决于对SRAM所投入的努力程度，其速度可能仅略低于其他选项。

比CPU核心慢一个或两个数量级。

2.2 DRAM 访问技术细节

在介绍DRAM的部分中，我们看到DRAM芯片通过复用地址来节省地址引脚资源。我们也了解到访问DRAM单元需要时间，因为这些单元中的电容不会瞬间放电以产生稳定的信号；此外，DRAM单元必须定期刷新。现在是时候将所有这些因素结合起来，看看它们是如何决定DRAM访问方式的。

我们将专注于当前技术；异步DRAM及其变体已不再相关，因此不予讨论。有兴趣的读者可参考文献[3]和[19]。我们也不会谈及Rambus DRAM（RDRAM），尽管该技术尚未过时，但它已经很少用于系统内存。我们将专门讨论同步DRAM（SDRAM）及其继任者双倍数据率DRAM（DDR）。

同步动态随机存取存储器（SDRAM）顾名思义，是相对于时间源工作的。内存控制器提供时钟信号，该时钟的频率决定了前端总线（FSB）的速度——这是DRAM芯片使用的接口。截至撰写本文时，800MHz、1,066MHz甚至1,333MHz的频率已经可用，并且更高频率（如1,600MHz）已宣布将在下一代产品中推出。但这并不意味着总线实际使用的就是这么高的频率。相反，今天的总线是双倍或四倍泵送的，这意味着每周期传输数据两次或四次。为了销售目的，制造商倾向于将一个200MHz的四倍泵送总线宣传为“有效”的800MHz总线。

对于今天的SDRAM，每次数据传输包含64位——8字节。因此，前端总线的传输速率是8字节乘以有效总线频率（对于四倍泵送的200MHz总线为6.4GB/s）。这听起来很多，但实际上这只是突发速度，即永远不会超过的最大速度。正如我们现在将要看到的，与RAM模块通信的协议在数据无法传输时会有大量空闲时间。我们必须理解并尽量减少这一空闲时间，以实现最佳性能。

2.2.1 读取访问协议

图2.8展示了DRAM模块上某些连接器在三个不同颜色阶段的活动。如通常所示，时间从左向右流动。省略了许多细节。这里我们只讨论总线时钟、RAS和CAS信号以及地址和数据总线。读取周期始于内存控制器将行地址通过地址总线提供出来。

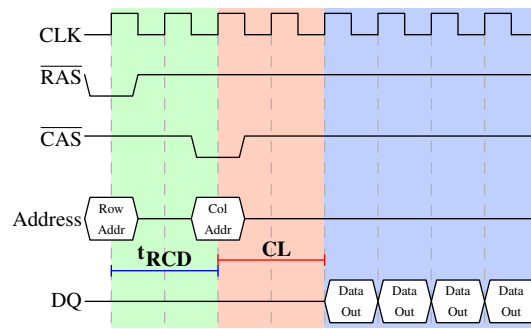


图2.8：SDRAM读取访问时序

总线信号下降并降低 $\overline{\text{RAS}}$ 信号。所有信号都在时钟（CLK）的上升沿被读取，因此只要在读取时刻信号是稳定的，即使不是完全方波也没有关系。设置行地址会使得RAM芯片开始锁存所选行的内容。

CAS信号可以在 t_{RCD} （RAS到CAS延迟）个时钟周期后发送。随后，通过在地址总线上提供列地址并降低CAS线来传输该地址。在这里我们可以看到地址的两部分（大致为一半，除此之外没有其他意义）是如何在同一地址总线上进行传输的。

现在地址转换已完成，数据可以开始传输。RAM芯片需要一些时间来为此做准备。这段延迟通常被称为CAS延迟（CL）。在图2.8中，CAS延迟为2。根据内存控制器、主板和DRAM模块的质量不同，这个延迟值可能会更高或更低。延迟也可能有半数值。例如，在CL=2.5的情况下，第一个数据将在蓝色区域的第一个falling边沿时可用。

经过如此多的准备来获取数据，只传输一个字显然会很浪费。这就是为什么DRAM模块允许内存控制器指定要传输多少数据的原因。通常的选择是2、4或8个字。这样可以在不重新执行RAS/CAS序列的情况下填充缓存中的整个行。内存控制器还可以在不重置行选择的情况下发送新的CAS信号。这样一来，连续的内存地址可以被更快地读取或写入，因为不需要发送RAS信号且行不必去激活（见下文）。保持行“打开”是由内存控制器决定的。始终投机性地将其保持打开状态，在实际应用中会有不利之处（参见[3]）。新的CAS信号仅受RAM模块的命令速率限制（通常指定为Tx，其中x是一个如1或2这样的值；对于每周期都接受新命令的高性能DRAM模块来说，它将是1）。

在这个示例中，SDRAM每个周期输出一个字。这是第一代产品的做法。DDR能够每周期传输两个字。这缩短了传输时间，但没有改变延迟。原则上

显然，DDR2 的工作原理相同，尽管在实际应用中看起来有所不同。这里无需深入细节。重要的是要指出，DDR2 可以实现更快的速度、更低的成本、更高的可靠性，并且更加节能（详见文献 [6] 获取更多信息）。

2.2.2 预充电与激活

图2.8并未涵盖整个周期。它仅展示了访问DRAM的完整周期的一部分。在发送新的RAS信号之前，当前锁定的行必须被去激活，并且新行必须预充电。这里我们集中讨论通过显式命令来完成这一操作的情况。有一些协议改进可以在某些情况下避免这额外步骤，尽管如此，预充电引入的延迟仍然会影响操作。

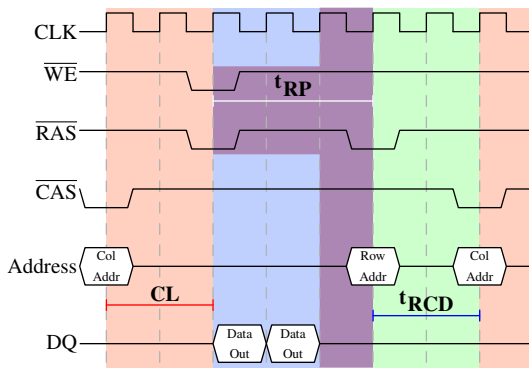


图2.9: SDRAM 预充电与激活

图2.9展示了从一个CAS信号到另一个行的CAS信号的活动。使用第一个CAS信号请求的数据如之前所述，在CL个周期后可用。在示例中，请求了两个字节，而在简单的SDRAM上，传输这两个字节需要两个周期。或者想象在一个DDR芯片上请求四个字。

即使在命令速率为一的 DRAM 模块上，预充电命令也不能立即发出。必须等待传输数据所需的时间，在这种情况下需要两个周期。这恰好与缓存行大小 CL 相同，但这只是巧合。预充电信号没有专用线路；相反，一些实现通过同时降低写使能（WE）和行地址选通（RAS）线来发出它。这种组合本身没有任何实际意义（详见文献 [18] 编码细节）。

预充电命令发出后，需要等待tRP（行预充电时间）个周期才能选中该行。在图2.9中，大部分时间（用紫色表示）与内存传输时间（浅蓝色部分）重叠。这很好！但是tRP大于传输时间，因此下一个RAS信号会被延迟一个周期。

如果我们继续参考图中的时间线，会发现下一个数据传输在上一个传输停止后的第5个周期开始。这意味着数据总线是

仅使用七个周期中的两个。将此乘以前端总线（FSB）的速度，对于800MHz的总线而言，理论上的6.4 GB/s会降至1.8GB/s。这种情况很糟糕，必须避免。第6节中描述的技术有助于提高这一数值。但程序员通常也需要做出相应的努力。

对于一个SDRAM模块，还有一种时序值我们尚未讨论。在图2.9中，预充电命令仅受数据传输时间的限制。另一个约束是，在接收到RAS信号后，SDRAM模块需要一段时间才能对另一行进行预充电（记为tRAS）。这个数值通常较高，大约是tRP值的两到三倍。如果在发出RAS信号之后只跟随一个CAS信号，并且数据传输在几个周期内完成，则会出现问题。假设在图2.9中，初始的CAS信号直接由RAS信号触发，而tRAS为8个周期。那么预充电命令就需要延迟一个额外的周期，因为tRCD、CL和tRP（由于其总和大于数据传输时间）仅为7个周期。

DDR模块通常使用一种特殊记号表示：w-x-y-z-T。例如：2-3-2-8-T1，这表示：

w	2	CAS Latency (CL)
x	3	RAS-to-CAS delay (tRCD)
y	2	RAS Precharge (tRP)
z	8	Active to Precharge delay (tRAS)
T	T1	Command Rate

还有许多其他定时常数会影响命令的发出和处理方式。这五个常数实际上足以决定模块的性能，尽管如此。

了解这些信息对于解读某些测量结果是有用的。在购买计算机时，知道这些细节尤为重要，因为它们与前端总线（FSB）和SDRAM模块速度一起，是决定计算机速度最重要的因素之一。

极具冒险精神的读者还可以尝试调整系统设置。有时BIOS允许更改这些值中的一个或全部。SDRAM模块具有可编程寄存器，可以在其中设置这些值。通常BIOS会选取最佳默认值。如果内存模块的质量较高，则可能可以减少某个延迟而不会影响计算机的稳定性。互联网上众多超频网站提供了大量关于如何进行此类操作的文档。不过，请自行承担风险，并且不要说你没有被警告过。

2.2.3 重新充电

关于DRAM访问的一个常被忽视的话题是刷新。如第2.1.2节所述，DRAM单元必须不断进行刷新。这一过程并不会自动发生

完全透明地对系统其余部分不可见。当一行¹⁰重新充电时，无法进行访问。[3]中的研究发现，“[s]urprisingly, DRAM re-fresh organization can 影响 performance dramatically”。

每个DRAM单元必须根据JEDEC（联合电子设备工程委员会）规范每隔64毫秒刷新一次。如果一个DRAM阵列有8,192行，这意味着内存控制器平均需要每7.8125毫秒发出一次刷新命令（刷新命令可以排队，因此实际上两次请求之间的最大间隔可能更长）。内存控制器负责调度刷新命令。DRAM模块会记录最近一次刷新的行地址，并在每次新的刷新请求时自动增加地址计数器。

程序员对此无能为力，既无法干预刷新操作也无法控制命令的发出时间。但在解读测量结果时，必须牢记DRAM生命周期中的这一部分。如果在当前正在进行刷新的操作行中需要检索关键数据，处理器可能会被阻塞很长时间。每次刷新所需的时间取决于DRAM模块。

2.2.4 内存类型

值得花些时间来讨论当前和即将成为主流的内存类型。我们将从单数据率（SDR）同步动态随机存取存储器（SDRAM）开始，因为它们是双数据率（DDR）SDRAM的基础。SDRs相对简单，其内存单元与数据传输速率相同。

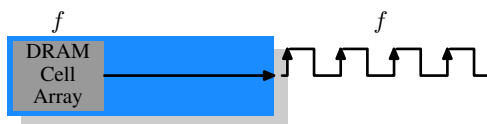


图2.10：SDR SDRAM操作

在图2.10中，DRAM单元阵列可以以与内存总线传输速率相同的速度输出存储内容。如果DRAM单元阵列的工作频率为100MHz，则单个单元的总线数据传输速率为100Mb/s。所有组件的频率 f 相同。提高DRAM芯片的吞吐量代价高昂，因为能耗会随着频率增加而上升。当单元数量庞大时，这种成本变得难以承受。¹¹实际上问题更为严重，因为提高频率通常还需要提升电压以维持系统的稳定性。DDR SDRAM（即回溯称作DDR1）设法在不增加任何相关频率的情况下提高了吞吐量。

¹⁰Rows are the granularity this happens with despite what [3] and other literature says (see [18]).

¹¹Power = Dynamic Capacity $\times$ Voltage² $\times$ Frequency.

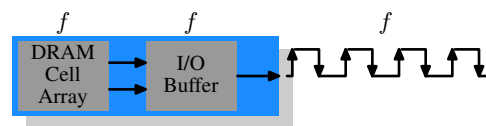


图2.11：DDR1 SDRAM操作

SDR与DDR1之间的区别，如图2.11所示并从名称可以猜测到的是，在每个时钟周期内传输的数据量增加了一倍。即，DDR1芯片在时钟上升沿和下降沿都进行数据传输。这种技术有时被称为“双泵”总线。为了实现这一点而不提高存储单元阵列的频率，需要引入一个缓冲器。该缓冲器每条数据线上保存两个比特。这进一步要求，在图2.7所示的存储单元阵列中，数据总线由两条线路组成。实现这一功能非常简单：只需使用相同的列地址并行访问两个DRAM单元即可。为此对存储单元阵列所做的修改也非常小。

SDR DRAM仅以其频率命名（例如，PC100表示100 MHz SDR）。为了使DDR1 DRAM听起来更出色，市场人员不得不提出一个新的命名方案，因为其频率并未改变。他们采用了一个包含DDR模块可维持的字节传输速率的新名称（这些模块具有64位总线）：

$$100\text{MHz} \times 64\text{bit} \times 2 = 1,600\text{MB/s}$$

因此，频率为100MHz的DDR模块被称为PC1600。通过提供1600 MB/s的速度，所有市场要求都得到了满足；虽然实际改进仅为两倍，但听起来要好得多。

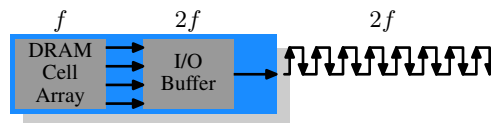


图2.12：DDR2 SDRAM操作

为了进一步提升内存技术，DDR2 包含了更多的创新。如图 2.12 所示，最明显的改变是总线频率的翻倍。频率加倍意味着带宽也加倍。由于这种频率的翻倍对存储单元阵列来说并不经济，现在要求 I/O 缓冲器在每个时钟周期内传输四个比特，并将其发送到总线上。这意味着 DDR2 模块的变化仅在于使 DIMM 的 I/O 缓冲器组件能够在更高的速度下运行。这当然是可行的，并且不会显著增加能耗，因为这只是模块中的一个微小部件而非整个模块。这些名称由制造商定义为

我接受这个两倍的因素，但我不必喜欢这些膨胀的数字。

eters为DDR2设计的命名方案与DDR1类似，仅在计算值时将因子2替换为4（我们现在有一个四倍泵浦总线）。表2.1展示了当今使用的模块名称。

Array Freq.	Bus Freq.	Data Rate	Name (Rate)	Name (FSB)
133MHz	266MHz	4,256MB/s	PC2-4200	DDR2-533
166MHz	333MHz	5,312MB/s	PC2-5300	DDR2-667
200MHz	400MHz	6,400MB/s	PC2-6400	DDR2-800
250MHz	500MHz	8,000MB/s	PC2-8000	DDR2-1000
266MHz	533MHz	8,512MB/s	PC2-8500	DDR2-1066

表2.1: DDR2模块名称

命名还有一个细节。CPU、主板和DRAM模块使用的前端总线（FSB）速度是通过*effective*频率来指定的。也就是说，它考虑了时钟周期两侧的数据传输，从而夸大了这个数字。因此，一个工作在133MHz的模块，在266MHz的总线上，其FSB“频率”为533MHz。

DDR3（真实版本，而非用于显卡的假GDDR3）规范要求沿用从DDR2过渡时的一些变化。电压将从DDR2的1.8V降低到DDR3的1.5V。由于功耗计算公式中电压的平方项，仅此一项就带来了约30%的改进。再加上芯片尺寸减小及其他电气性能提升，DDR3在同一频率下可以实现功耗减半。或者，在更高频率下，可以在相同的功率范围内运行。又或者，在容量翻倍的情况下，仍能保持同样的发热水平。

DDR3模块的单元阵列将以外总线速度的四分之一运行，并且需要一个8位I/O缓冲器，而DDR2则只需要4位。详见图2.13所示的电路图。

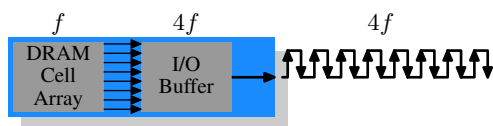


图2.13: DDR3 SDRAM操作

初始阶段，DDR3模块的CAS延迟可能会略高一些，仅仅是因为DDR2技术更为成熟。这会导致DDR3仅在频率高于DDR2所能达到的频率时才变得有用，并且即便如此，在带宽比延迟更重要的情况下才会更加适用。目前已经有关于1.3V模块的讨论，这些模块可以实现与DDR2相同的CAS延迟。无论如何，由于更快总线带来的高速可能性将超过增加的延迟影响。

DDR3的一个可能问题是，对于1,600 Mb/s及以上的传输速率，每个通道上的模块数量会受到限制。

可以将该要求简化为仅适用于一种频率。在早期版本中，这一要求适用于所有频率，因此人们可以期望将来某个时刻这一限制会被完全解除。否则，系统的容量将会受到严重限制。

表2.2展示了我们可能见到的DDR3模块名称。JEDEC迄今已确定前四种类型。鉴于英特尔45nm处理器的前端总线速率为1,600Mb/s，1,866Mb/s的速度则主要用于超频市场。在DDR3生命周期末期，我们将更有可能看到这种情况。

Array Freq.	Bus Freq.	Data Rate	Name (Rate)	Name (FSB)
100MHz	400MHz	6,400MB/s	PC3-6400	DDR3-800
133MHz	533MHz	8,512MB/s	PC3-8500	DDR3-1066
166MHz	667MHz	10,667MB/s	PC3-10667	DDR3-1333
200MHz	800MHz	12,800MB/s	PC3-12800	DDR3-1600
233MHz	933MHz	14,933MB/s	PC3-14900	DDR3-1866

表2.2: DDR3模块名称

所有DDR内存都存在一个问题：增加的总线频率使得创建并行数据总线变得困难。一个DDR2模块有240个引脚。所有连接到数据和地址引脚的线路必须布线，使其长度大致相同。更大的问题是，如果在同一总线上级联多个DDR模块，则每个额外模块都会使信号越来越失真。DDR2规范仅允许每条总线（即通道）最多两个模块，并且在高频下，DDR3规范只允许一个模块。由于每通道有240个引脚，单个北桥芯片无法合理地驱动超过两条通道。替代方案是使用外部内存控制器（如图2.2所示），但这会增加成本。

这意味着，普通主板最多只能安装四根DDR2或DDR3内存模块。这种限制严重制约了系统可拥有的内存容量。即使是旧的32位IA-32处理器也能处理高达64GB的RAM，并且家庭用户的内存需求也在增长，因此必须采取措施解决这一问题。

一种解决方案是如第2节所述，在每个处理器中加入内存控制器。AMD在其Opteron系列产品中采用了这种方法，而英特尔也将通过其CSI技术实现这一方案。只要一个处理器能够使用的合理数量的内存可以连接到单个处理器上，这种做法就能起到作用。但在某些情况下，这并非可行，并且这样的架构设置会引入NUMA架构及其负面影响。在一些特定情形下，需要另一种解决方案。

英特尔针对大型服务器机器的这一问题，至少目前而言，提出的解决方案称为全缓冲双列直插内存模组（Fully Buffered DRAM, FB-DIMM）。FB-DIMM模块使用与当今DDR2模块相同的内存芯片，因此相对便宜。区别在于与内存控制器的连接方式。FB-DIMM采用串行总线（而非并行数据总线），Rambus DRAM也采用了类似的方案。

当年也是如此，SATA 是 PATA 的继任者，PCI Express 同样是 PCI/AGP 的继任者。串行总线可以在更高的频率下运行，从而逆转了序列化带来的负面影响，并且甚至可以增加带宽。使用串行总线的主要效果包括

- 每个通道可以使用更多的模块。
- 2. 可以使用北桥/内存控制器上的更多通道。
- 3. 串行总线设计为全双工（两条线路）。
- 4. 实现差分总线（每个方向两条线路）的成本足够低，因此可以提高速度。

FB-DRAM模块仅有69个引脚，而DDR2则有240个。由于总线的电气效应可以更好地处理，因此级联FB-DRAM模块要容易得多。根据FB-DRAM规范，每个通道最多可支持8个DRAM模块。

与双通道北桥的连接要求相比，现在可以用更少的引脚驱动6通道的FB-DRAM：2条通道使用240个引脚，而6条通道仅需每通道69个引脚。每个通道的布线也更加简单，这有助于降低主板的成本。

全双工并行总线对于传统的 DRAM 模块来说成本过高，复制所有这些线路的成本太高。而使用串行线路（即使它们是差分信号，如 FB-DRAM 所需），情况则不同，因此串行总线被设计为全双工模式，在某些情况下仅通过这种方式理论上的带宽就可以翻倍。但这并不是唯一利用并行性来增加带宽的地方。由于一个 FB-DRAM 控制器可以同时运行多达六个通道，即使对于 RAM 容量较小的系统，也可以通过使用 FB-DRAM 来提高带宽。例如，在一个 DDR2 系统中，如果四个模块仅使用两个通道，则相同容量可以通过使用普通 FB-DRAM 控制器来处理四个通道。串行总线的实际带宽取决于所使用的 DDR2（或 DDR3）芯片类型。

我们可以总结其优点如下：

	DDR2	FB-DRAM
Pins	240	69
Channels	2	6
DIMMs/Channel	2	8
Max Memory ¹³	16GB ¹⁴	192GB
Throughput ¹⁵	~10GB/s	~40GB/s

¹³ Assuming 4GB modules.

¹⁴ An Intel presentation, for some reason I do not see, says 8GB...

¹⁵ Assuming DDR2-800 modules.

使用多条通道上的多个DIMM时，FB-DRAM存在一些缺点。信号在链中的每个DIMM处都会延迟——尽管这种延迟很小——从而增加了总延迟。另一个问题是，驱动串行总线的芯片由于极高频率和需要驱动总线的需求而消耗大量能量。但是，在相同容量和频率下，FB-DRAM始终可以比DDR2和DDR3更快，因为最多四个DIMM各自拥有独立通道；对于大型内存系统而言，使用通用组件时，DDR无法提供解决方案。

2.2.5 结论

本节应已阐明，访问DRAM并非一个任意快速的过程。至少与处理器运行速度以及访问寄存器和缓存的速度相比，并不算快。重要的是要记住CPU与内存频率之间的差异。例如，对于一款运行在2.933GHz的Intel Core 2处理器和1.066GHz前端总线（FSB）而言，其时钟比为11:1（注意：该1.066GHz总线是四倍泵送）。内存总线上每停顿一个周期，则处理器会停顿11个周期。对于大多数机器来说，实际使用的DRAM速度更慢，从而增加了延迟。在接下来的章节中讨论停顿时，请记住这些数字。

时序图显示，DRAM模块能够实现较高的持续数据传输率。整个DRAM行可以在不产生任何停顿的情况下被传输。数据总线可以始终保持满载状态。对于DDR模块而言，这意味着每周可传输两个64位字。而对于DDR2-800模块和双通道配置，则意味着数据传输速率为12.8GB/s。

但是，除非特别设计为顺序访问，DRAM访问并不总是连续的。使用非连续内存区域意味着需要预充电和新的RAS信号。这时性能会下降，DRAM模块也需要帮助。预充电越早进行，并且尽早发出RAS信号，在实际使用行时所受的惩罚就越小。

硬件和软件预取（见第6.3节）可以用来增加时间上的重叠并减少停顿。预取还有助于将内存操作在时间上移位，使得在数据实际需要时的争用较少。当一轮产生的数据必须存储而下一轮所需的数据又必须读取时，这种情况经常出现。通过将读取操作在时间上提前，写入和读取操作不必几乎同时发出。

2.3 其他主内存用户

除CPU外，还有其他系统组件可以访问主内存。高性能卡如网络控制器和大容量存储控制器无法完全避免与这些组件发生内存访问冲突。

为了直接访问主内存（直接内存访问，DMA），它们会读取或写入数据，而不是通过CPU管道传输所需的数据。如图2.1所示，这些卡可以直接通过南桥和北桥与内存通信。其他总线，例如USB，即使不使用DMA，也需要前端总线带宽，因为南桥也是通过前端总线连接到处理器的。

虽然直接内存访问（DMA）确实有益，但它也意味着对前端总线（FSB）带宽的竞争加剧。在高DMA流量期间，CPU可能会比平时更频繁地因等待主存数据而停滞不前。如果硬件配置得当，则有办法解决这一问题。如图2.3所示的架构可以确保计算使用未受DMA影响节点上的内存。还可以将南桥（South bridge）连接到每个节点上，从而在所有节点的FSB上均匀分配负载。可能性多种多样。在第6节中，我们将介绍有助于实现软件改进的技术和编程接口。

最后值得一提的是，一些低成本系统没有独立的专用视频内存。这些系统会使用部分主内存作为视频内存。由于访问视频内存的频率很高（对于一个分辨率为1024x768、每像素位深为16bpp、刷新率为60 Hz的显示器来说，数据传输速率约为94MB/s），而系统内存不像显卡上的RAM那样具有双端口设计，因此这会显著影响系统的性能和延迟。当性能是首要考虑因素时，最好忽略这类系统，因为它们带来的麻烦远大于其价值。购买这些机器的人也知道他们无法获得最佳的性能表现。

3 CPU缓存

如今的CPU比25年前复杂得多。在那个时代，CPU核心的频率与内存总线的频率相当，内存访问速度仅略慢于寄存器访问速度。然而，在90年代初期情况发生了巨大变化：CPU设计者提高了CPU核心的频率，但内存总线的频率和RAM芯片的性能并未同比例提升。这并非因为无法制造更快的RAM，正如前文所述，虽然技术上可行，但从经济角度来看并不划算。与当前的CPU核心相比，同等速度的RAM要贵出许多倍。

如果在一台配备少量但非常快速的内存（RAM）的机器和另一台配备大量相对较快内存的机器之间进行选择，在工作集大小超过前者的小容量内存且访问诸如硬盘等二级存储介质的成本较高的情况下，后者总是会胜出。问题在于二级存储的速度，通常是硬盘，必须用于存放被换出的工作集部分。访问这些磁盘的速度比甚至DRAM访问要慢几个数量级。

幸运的是，这并不一定是一个非此即彼的决定。计算机可以在大量DRAM之外还配备少量高速SRAM。一种可能的实现方式是将处理器地址空间的一部分专门用于存放SRAM，其余部分则用于DRAM。此时操作系统的任务就是优化数据分布，充分利用SRAM。在这种情况下，SRAM基本上充当了处理器寄存器集的扩展。

虽然这是一种实现方式，但不可行。忽略将此类SRAM支持的内存物理资源映射到进程虚拟地址空间的问题（这个问题本身已经非常困难），这种方法要求每个进程在软件中管理该内存区域的分配。内存区域的大小因处理器而异（即，不同处理器具有不同的昂贵SRAM支持的内存量）。构成程序的一部分的每个模块都会声称其快速内存份额，这会通过同步需求引入额外成本。简而言之，拥有快速内存带来的好处将完全被资源管理开销所抵消。

因此，SRAM不再受操作系统或用户的控制，而是成为一种由处理器透明使用的资源。在这种模式下，SRAM用于临时存储（即缓存）主内存中即将被处理器使用的数据。这是可能的，因为程序代码和数据具有时间和空间局部性。这意味着，在短时间内，相同的代码或数据很有可能会被再次访问。

代码重用意味着代码中很可能存在循环，使得同一段代码会被反复执行（这是*spatial locality*的理想情况）。数据访问也应尽量局限于小的区域。即使在短时间内使用的内存地址不接近，但从长远来看，相同的数据很有可能被重复使用（*temporal locality*）。对于代码而言，这意味着在一个循环中调用一个位于地址空间其他位置的函数。尽管该函数在内存中的位置可能很远，但对它的调用却会在时间上非常集中。对于数据来说，则意味着在同一时刻使用的总内存量（工作集大小）应被理想地限制，但由于RAM访问的性质（*random*），实际使用的内存地址并不接近。认识到局部性存在的事实是当今CPU缓存概念的关键所在。

一个简单的计算可以展示缓存理论上能有多大的效果。假设访问主内存需要200个周期，而访问缓存则只需要15个周期。那么，如果代码使用了100个数据元素各100次，在没有缓存的情况下将花费2,000,000个周期进行内存操作；但如果所有数据都能被缓存，则只需花费168,500个周期。这相当于提高了91.5%的效率。

SRAM用于缓存的大小远远小于主内存。根据作者在工作站上使用CPU缓存的经验，缓存的大小通常为主内存大小的千分之一左右（如今：4MB缓存和4GB主内存）。这本身并不构成问题。如果工作集（当前正在处理的数据集合）的大小小于缓存的大小，则不会产生影响。但是计算机并非无缘无故地配备大容量主内存。在多进程系统中，工作集的大小往往是所有单独进程以及内核所需空间之和，因此工作集通常会大于缓存。

为了应对缓存容量有限的问题，需要制定一系列有效的策略来确定何时何地应该将哪些数据存储在缓存中。由于工作集中并非所有数据在同一时刻都会被使用，因此可以采用技术手段暂时用其他数据替换掉部分缓存中的数据。甚至可以在实际需要之前就进行这种预取操作。这样就可以异步地执行程序，从而减少访问主内存的成本。通过这些技术和更多方法的运用，可以使缓存看起来比其实际容量更大。我们将在第3.3节中讨论这些问题。一旦所有技术手段都被充分利用之后，就需要程序员来协助处理器了。这将在第6节中进行详细讨论。

3.1 大背景下的CPU缓存

在深入探讨CPU缓存的实现技术细节之前，一些读者可能会发现先详细了解缓存在现代计算机系统整体架构中的作用是有益的。

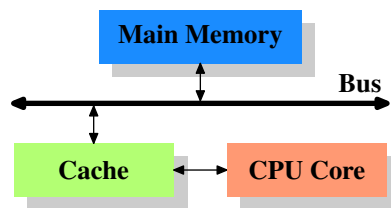


图3.1：最小缓存配置

图3.1展示了最小的缓存配置。这对应于早期部署CPU缓存系统的架构。此时，CPU核心不再直接连接到主内存。所有加载和存储操作都必须通过缓存进行。CPU核心与缓存之间的连接是一个特殊的快速连接。在简化的表示中，主内存和缓存都连接到系统总线（FSB），该总线也可用于与其他系统组件通信。我们在第2.2节中介绍了“FSB”这一术语；在此部分我们忽略北桥的存在，假定它是为了方便CPU与主内存之间的通信而存在。

尽管过去几十年来的大多数计算机都采用了冯·诺伊曼架构，但经验表明将用于代码和数据的缓存分开是有利的。自1993年以来，英特尔一直使用独立的代码缓存和数据缓存，并且从未回头。代码和数据所需的内存区域基本上是相互独立的，这就是为什么独立缓存能够更好地发挥作用的原因。近年来又出现了一个优势：大多数处理器的指令解码步骤较慢；缓存已解码的指令可以加快执行速度，尤其是在由于预测错误或无法预测的分支导致流水线为空的情况下。

不久之后，在引入缓存系统后，系统的复杂性增加。缓存与主内存之间的速度差异再次扩大，以至于添加了另一级的缓存，这一级比一级缓存更大但更慢。由于经济原因，单纯增大一级缓存的大小并不是一个可行的选择。如今，甚至有配备了三级缓存的机器在常规使用中。配备这种处理器的系统如图3.2所示。随着单个CPU内核心数量的增加，未来缓存层级的数量可能会进一步增多。

图3.2展示了三级缓存，并介绍了我们在文档其余部分将使用的术语。L1d是第一级数据缓存，L1i是第一级指令缓存等。请注意，这是一张示意图；实际上从核心到主内存的数据流无需经过任何较高级别的缓存。

¹⁶In even earlier systems the cache was attached to the system bus just like the CPU and the main memory. This was more a hack than a real solution.

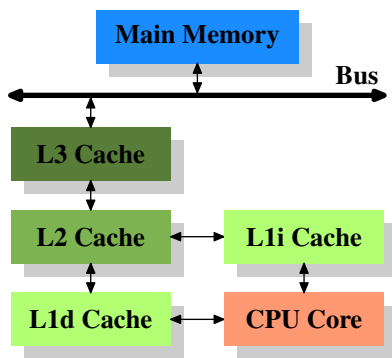


图3.2: 带有三级缓存的处理器

内存。CPU设计者在设计缓存接口时拥有很大的自由度。对于程序员来说，这些设计选择是不可见的。

此外，我们还有具有多个核心的处理器，每个核心可以拥有多个“线程”。核心与线程之间的区别在于，不同的核心各自独立拥有一套（几乎全部）硬件资源的副本。除非它们同时使用相同的资源——例如对外部连接的同时访问，否则这些核心可以完全独立运行。而线程则共享处理器几乎所有的资源。英特尔实现的线程仅拥有各自的寄存器，并且即使这样也并非所有寄存器都是独立的，有些寄存器是共用的。因此，现代CPU的整体架构如图3.3所示。

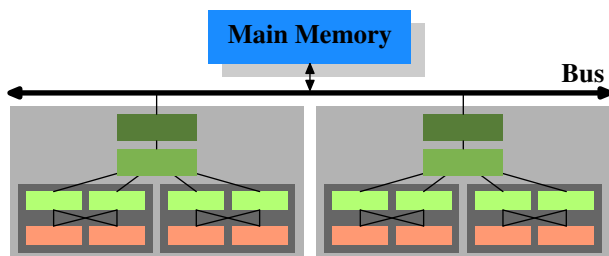


图3.3: 多处理器、多核、多线程

在图中我们有两个处理器，每个处理器包含两个核心，每个核心又包含两条线程。这些线程共享一级缓存（L1缓存）。核心（用深灰色阴影表示）各自拥有独立的一级缓存。CPU的所有核心则共用更高层级的缓存。这两个处理器（用浅灰色阴影表示的大方框）当然不会共享任何缓存。所有这一切在讨论多进程和多线程应用程序的缓存影响时将非常重要。

¹⁷Early multi-core processors even had separate 2nd level caches and no 3rd level cache.

3.2 高层次缓存操作

为了理解使用缓存的成本和收益，我们必须将第二部分中关于机器架构和RAM技术的知识与前一部分所描述的缓存结构结合起来。

默认情况下，CPU核心读取或写入的所有数据都会存储在缓存中。有些内存区域无法被缓存，但这只是操作系统实现者需要关心的问题；对于应用程序开发者来说是不可见的。还有一些指令允许程序员故意绕过某些缓存。这将在第6节中讨论。

如果CPU需要一个数据字，则首先会搜索缓存。显然，缓存不可能包含整个主存储器的内容（否则就不需要缓存了），但由于所有内存地址都可以被缓存，因此每个缓存条目都会使用该数据字在主存储器中的地址。这样一来，对某个地址的读取或写入请求就可以在缓存中查找匹配的标签。在此上下文中，地址可以是虚拟地址也可以是物理地址，具体取决于缓存实现方式的不同。

由于标签除了实际内存之外还需要额外的空间，因此选择一个字作为缓存的粒度是低效的。对于x86机器上的32位字来说，标签本身可能需要32位或更多。此外，鉴于空间局部性是缓存设计的基本原则之一，忽略这一点是不好的。由于相邻内存很可能一起被使用，因此也应该将它们一起加载到缓存中。记住我们在第2.2.1节中学到的内容：RAM模块在能够连续传输多个数据字而无需新的CAS甚至RAS信号的情况下更为有效。所以存储在缓存中的条目不是单个字而是由几个连续字组成的“行”。早期的缓存这些行长度为32字节；如今的标准是64字节。如果内存总线宽度为64位，这意味着每条缓存行需要8次传输。DDR支持这种传输模式高效地实现这一点。

当处理器需要内存内容时，整个缓存行会被加载到L1d中。每个缓存行的内存地址通过根据缓存行大小屏蔽地址值来计算得出。对于64字节的缓存行来说，这意味着低6位被清零。这些被丢弃的位用作缓存行内的偏移量。剩余的部分在某些情况下用于定位缓存中的行，并作为标签使用。实际上，一个地址值会被分成三个部分。对于32位地址，它可能如下所示：



缓存行大小为20时，低0位被使用。

作为缓存行内的偏移量。接下来的S位选择“缓存组”。我们很快会详细讨论为什么使用缓存组而不是单个槽来组织缓存行的原因。目前，理解有 2^S 个缓存组是足够的。这留下了顶部 $32-S-O=T$ 位，这些位构成了标签。T位与每个缓存行相关联，用于区分存储在同一缓存组中的所有不同项。由于同一组内的所有缓存行的S位地址都是相同的，因此不需要存储这些位。

当一条指令修改内存时，处理器仍然需要先加载一个缓存行，因为没有任何一条指令一次就能修改整个缓存行（特殊情况：写组合操作如第6.1节所述）。因此，在执行写操作之前，缓存行的内容必须被加载。缓存不能保存部分缓存行。对于已被写入但尚未写回到主内存的缓存行来说，它被称为“脏”缓存行。一旦数据被写回主内存，“脏”标志就会清除。

为了在缓存中加载新数据，几乎总是首先需要腾出空间。L1d的驱逐将缓存行推入L2（使用相同的缓存行大小）。这当然意味着还需要为L2腾出空间。这可能会进一步将内容推入L3，最终进入主内存。每次驱逐的成本逐渐增加。这里描述的是AMD和VIA处理器偏好的*exclusive cache*模型。Intel实现的是*inclusive caches*¹⁹，其中L1d中的每个缓存行也在L2中存在。因此从L1d驱逐数据要快得多。有足够的L2缓存时，为在两个地方存储内容而浪费内存的缺点可以忽略不计，并且在驱逐时会带来好处。独占式缓存的一个可能优势是加载新缓存行只需触及L1d而不必访问L2，这可能会更快。

CPU可以在不更改处理器架构所定义的内存模型的情况下，自行管理缓存。例如，处理器可以利用几乎没有或很少的内存总线活动，主动将脏缓存行写回主内存，这是完全可以接受的。x86和x86-64处理器之间、不同制造商之间甚至同一制造商的不同型号之间的各种缓存架构差异，充分证明了内存模型抽象的强大之处。

在对称多处理器（SMP）系统中，CPU的缓存不能独立工作。所有处理器应始终看到相同的内存内容。保持这种统一的内存视图称为“缓存一致性”。如果一个处理器仅仅查看自己的缓存和主内存，那么：

¹⁸ All cache lines with the same S part of the address are known by the same alias.

¹⁹ This generalization is not completely correct. A few caches are exclusive and some inclusive caches have exclusive cache properties.

在这种情况下，它无法看到其他处理器中脏缓存行的内容。从一个处理器直接访问另一个处理器的缓存将会极其昂贵，并且会成为一个巨大的瓶颈。相反，处理器会检测到当另一处理器想要读取或写入某个缓存行时的情况。

如果检测到写访问并且处理器的缓存中有一份干净的缓存行副本，则该缓存行会被标记为无效。未来的引用将需要重新加载该缓存行。需要注意的是，另一个CPU上的读访问不需要进行无效化操作，多个干净的副本可以同时存在。

更复杂的缓存实现允许另一种情况的发生。假设一个处理器的缓存中某条缓存行被标记为脏状态，而另一个处理器想要读取或写入该缓存行。在这种情况下，主内存中的数据已经过时，请求的处理器必须从第一个处理器获取该缓存行的内容。通过监听机制，第一个处理器会注意到这种情况，并自动将数据发送给请求的处理器。这一操作绕过了主内存，在某些实现中，内存控制器应该能够察觉到这种直接传输并更新主内存中的缓存行内容。如果访问是为了写入，则第一个处理器会将其本地缓存行副本标记为无效。

随着时间的推移，开发出了若干缓存一致性协议。其中最重要的是MESI协议，我们将在第3.3.4节中介绍该协议。所有这些可以归纳为几条简单的规则：

- 一条脏缓存行不在其他任何处理器的缓存中存在。
- 同一缓存行的干净副本可以存在于任意数量的缓存中。

如果能够维持这些规则，处理器即使在多处理器系统中也能高效地使用缓存。所有处理器需要做的就是监控彼此的写访问，并将地址与本地缓存中的地址进行比较。在下一节中，我们将进一步探讨实现细节及其成本。

最后，我们应该至少给出缓存命中和未命中的相关成本的印象。这些是英特尔为Pentium M列出的数值：

To Where	Cycles
Register	≤ 1
L1d	~ 3
L2	~ 14
Main Memory	~ 240

这些是实际测量的访问时间，以CPU周期为单位。值得注意的是，对于片上L2缓存

访问时间的很大一部分（甚至可能是大多数）是由走线延迟引起的。这是物理限制，随着缓存大小的增加，这一问题只会变得更加严重。只有通过工艺缩减（例如英特尔产品线中从墨罗姆的60nm缩小到奔仁的45nm）才能改善这些数值。

表中的数字看起来很高，但幸运的是，并非每次缓存加载和缺失都需要支付全部成本。部分成本可以被隐藏起来。当今的处理器都使用不同长度的内部流水线来解码指令并为执行做准备。准备工作的一部分包括从内存（或缓存）中加载值到寄存器。如果内存加载操作可以在流水线中的早期阶段启动，它可能会与其他操作并行进行，并且整个加载成本可能被隐藏起来。这种情况在L1d缓存中经常发生；对于一些具有较长流水线的处理器，在L2缓存中也有可能实现这一点。

启动内存读取操作存在许多障碍。这可能是由于缺乏足够的资源来执行内存访问，或者是因为另一条指令导致加载指令的最终地址较晚才确定。在这种情况下，无法完全隐藏加载操作的成本。

对于写操作，CPU 不必等到值安全存储在内存中才继续。只要后续指令的执行效果与值已存入内存时相同，则 CPU 可以采取捷径，提前开始执行下一条指令。借助可以保存常规寄存器不再包含的值的影子寄存器，甚至可以在不完整的写操作中更改要存储的值。

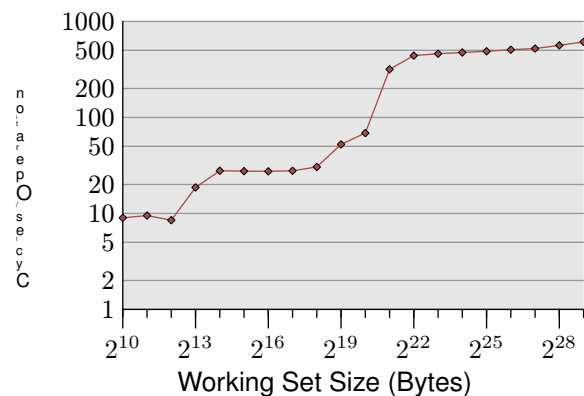


图3.4：随机写入访问时间

图3.4展示了缓存行为的影响。稍后我们将讨论生成这些数据的程序，它是一个简单的模拟程序，该程序以随机方式反复访问可配置大小的内存区域。每个数据项具有固定大小。元素的数量取决于所选的工作负载。

图中的X轴表示工作集大小。Y轴显示处理一个元素所需的平均CPU周期数；请注意，Y轴的刻度是 logarithmic（对数）的。在所有此类图表中，X轴的刻度同样是对数的。工作集的大小始终以2的幂次方形式表示。

图中显示了三个明显的平台期。这并不令人惊讶：特定的处理器具有L1d和L2缓存，但没有L3缓存。根据经验我们可以推断出L1d大小为2¹³字节，而L2大小为2²⁰字节。如果整个工作集可以完全容纳在L1d中，则每个元素的操作周期低于10个。一旦超出L1d的大小，处理器需要从L2加载数据，平均时间上升到大约28个周期。当L2不再足够时，时间会跳升至480个周期以上。此时许多或大多数操作都需要从主内存中加载数据。更糟糕的是：由于数据被修改，脏缓存行还需要写回。

此图应能充分说明优化编码以改善缓存使用的重要性。我们讨论的不是几个微不足道的百分比差异，而是可能存在的数量级上的改进。在第6节中，我们将讨论一些有助于编写更高效代码的技术。下一节将详细介绍CPU缓存设计。这些知识虽有帮助但并非阅读本文其余部分所必需，因此可以跳过该节。

3.3 CPU缓存实现细节

缓存实现者面临的问题是，主内存中的每个单元都有可能被缓存。如果程序的工作集足够大，则意味着有许多主内存位置在争夺缓存中的每一个位置。此前曾提到，缓存与主内存大小的比例为1比1000并不罕见。

3.3.1 关联性

可以实现一种缓存，其中每个缓存行都可以保存任何内存位置的副本（参见图3.5）。这种缓存被称为全相联映射。为了访问一个缓存行，处理器核心必须将每个缓存行的标签与请求地址的标签进行比较。该标签由地址中除偏移量以外的所有部分组成（也就是说，在第15页的图中，S为零）。

存在这样的缓存实现方式，但通过查看当前使用的L2缓存的参数可以发现这种方式是不切实际的。假设有一个4MB大小、缓存行大小为64B的L2缓存，那么该缓存将包含65,536个条目。为了达到足够的性能，缓存逻辑必须能够在几个时钟周期内从所有这些条目中找到与给定标签匹配的一个。实现这一目标所需的

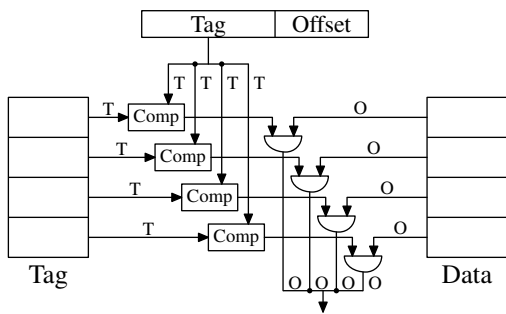


图3.5: 全相联缓存示意图

这将带来巨大的实现效益。

对于每个缓存行都需要一个比较器来比较大标签（注意，S为零）。每条连接旁边的字母表示位宽。如果没有给出，则默认为单比特线。每个比较器需要比较两个T位宽的值。然后根据比较结果选择并提供适当的缓存行内容。这要求将O数据线路的集合合并到与缓存桶数量相等的数量中。实现一个单一比较器所需的晶体管数量非常大，尤其是它必须工作得非常快。迭代比较器不可用。唯一减少比较器数量的方法是通过迭代比较标签来减少它们的数量。但由于同样的原因，迭代比较器不可行：耗时太长。

全相联缓存适用于小容量缓存（例如，某些英特尔处理器上的快表就是全相联的），但这些缓存的实际大小非常有限，最多也就几十个条目。

对于L1i、L1d以及更高层次的缓存，需要采用不同的方法。可以限制搜索范围。在最极端的情况下，每个标签只映射到一个缓存条目。计算很简单：对于4MB/64B大小且包含65,536个条目的缓存，可以通过使用地址中的第6至21位（共16位）直接寻址每一个条目。低6位是缓存行内的索引。

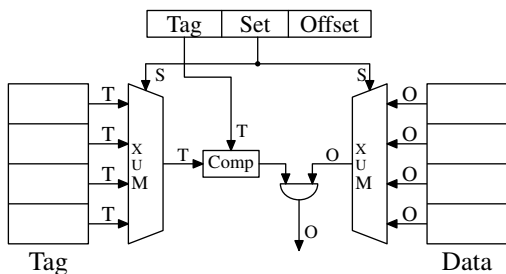


图3.6: 直接映射缓存示意图

如图3.6所示，这样的*direct-mapped cache*实现速度快且相对简单。它需要

仅有一个比较器、一个多路复用器（在本图中由于标签和数据分离而有两个，但这并不是设计的硬性要求），以及一些逻辑用于选择有效的缓存行内容。尽管速度需求使得比较器变得复杂，但目前只有一个这样的组件；因此可以投入更多精力使其更快。这种方法的实际复杂性在于多路复用器。一个简单的多路复用器中的晶体管数量随着 $O(\log N)$ 增长，其中N是缓存行的数量。这在可接受范围内，但在某些情况下可能会变慢，此时可以通过增加多路复用器中用于并行化部分工作和提高速度的晶体管面积来提升速度。随着缓存大小的增长，总晶体管数可以缓慢增长，使得此解决方案非常有吸引力。但其缺点是：只有在程序使用的地址相对于直接映射所用位均匀分布时才能很好地工作。如果不满足这一条件（通常情况下确实如此），某些缓存条目会被频繁使用并因此被反复驱逐，而其他条目则几乎不被使用或保持空闲状态。

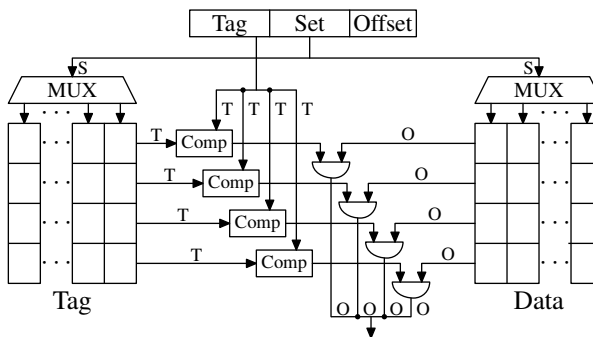


图3.7: 组相联缓存示意图

此问题可通过使缓存采用*set associative*的方式来解决。集合关联型缓存结合了全相联和直接映射缓存的优点，大大避免了这些设计的缺点。图3.7展示了集合关联型缓存的设计。标签和数据存储被划分为若干组，其中一组由缓存行地址选定。这与直接映射缓存类似。但是，在缓存中，对于每个集值并非只有一个元素，而是为相同的集值缓存少量的值。所有集合成员的标签会并行比较，类似于全相联缓存的工作方式。

结果是一个不易被不幸或故意选择具有相同组号的地址所击败的缓存。同时，缓存的大小不受经济上可实现的比较器数量的限制。如果缓存增大（如图所示），只是列的数量增加，而行的数量不变。只有在提高缓存的关联度时，行数（因此也是比较器的数量）才会增加。如今，处理器使用的L2缓存的关联级别可达24或更高。而L1缓存通常使用8个组即可满足需求。

L2 Cache Size	Associativity							
	Direct		2		4		8	
	CL=32	CL=64	CL=32	CL=64	CL=32	CL=64	CL=32	CL=64
512k	27,794,595	20,422,527	25,222,611	18,303,581	24,096,510	17,356,121	23,666,929	17,029,334
1M	19,007,315	13,903,854	16,566,738	12,127,174	15,537,500	11,436,705	15,162,895	11,233,896
2M	12,230,962	8,801,403	9,081,881	6,491,011	7,878,601	5,675,181	7,391,389	5,382,064
4M	7,749,986	5,427,836	4,736,187	3,159,507	3,788,122	2,418,898	3,430,713	2,125,103
8M	4,731,904	3,209,693	2,690,498	1,602,957	2,207,655	1,228,190	2,111,075	1,155,847
16M	2,620,587	1,528,592	1,958,293	1,089,580	1,704,878	883,530	1,671,541	862,324

表3.1: 缓存大小、关联度和缓存行大小的影响

鉴于我们的缓存大小为4MB，每行64B，并且采用8路组相联结构，我们得到的缓存共有8,192个组，地址定位时仅使用了标签中的13位。为了确定缓存组中是否存在对应的缓存行，需要比较该组内的8个标签。这一操作可以在极短的时间内完成。通过实验可以验证这一点是有意义的。

表3.1展示了通过更改L2缓存大小、缓存行大小和关联集大小，程序（在此情况下为gcc，据Linux内核人员所述是最重要的一项基准测试）的L2缓存未命中的数量。在第7.2节中，我们将介绍用于此测试所需的模拟缓存工具。

为了保持原文的简洁性和专业性，以下是精准翻译后的最终文本：这关系到所有这些值的是缓存大小是考虑到这句话在中文中表达不够流畅和完整，建议补充使其更符合中文习惯：这说明所有这些值之间的关系在于缓存大小是
缓存行大小 × 关联度 × 集数

地址通过以下方式映射到缓存中：使用

$$\begin{aligned} O &= \log_2 \text{cache line size} \\ S &= \log_2 \text{number of sets} \end{aligned}$$

如第15页的图所示。

图3.8使表格中的数据更加易于理解。它展示了固定缓存行为32字节的数据。观察给定缓存大小下的数字，我们可以看到相联度确实有助于显著减少缓存未命中次数。对于8MB的缓存，从直接映射变为两路组相联缓存可以几乎减少44%的缓存未命中。处理器可以在集相联缓存中比在直接映射缓存中保留更多的工作集。

在文献中偶尔可以看到这样的说法：引入相联性等同于将缓存容量翻倍。这种说法在某些极端情况下确实成立，例如从4MB缓存跳到8MB缓存时的情况。但进一步提高相联度却并非如此简单。如数据所示，后续的增益表现为

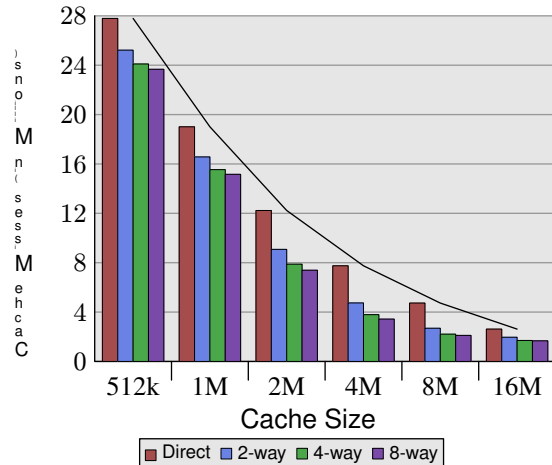


图3.8: 缓存大小与关联性 (CL=32)

虽然效果较小，但我们也不能完全忽视这些影响。在示例程序中，峰值内存使用量为5.6MB。因此，在8MB缓存的情况下，同一缓存组不太可能被多次（超过两次）使用。随着工作集的增大，节省的效果会更高，这一点从较小缓存尺寸下关联性的更大收益中看起来。

一般来说，单线程工作负载下将缓存的关联性增加到8以上似乎几乎没有效果。随着超线程处理器和使用共享二级缓存的多核处理器的出现，情况发生了变化。现在基本上有两个程序同时访问同一个缓存，这实际上使缓存的关联性减半（对于四核处理器则为四分之一）。因此可以预期，在核心数量增加的情况下，共享缓存的关联性应该随之增大。一旦无法再继续提高（16路组相联已经很困难），处理器设计者就需要开始使用三级及以上的共享缓存，而二级缓存可能由部分核心共用。

另一项我们可以从图3.8中研究的效果是缓存容量的增加如何有助于性能提升。没有了解工作集的相关知识，无法解读这些数据。

缓存的大小。显然，一个与主内存一样大的缓存会比较小的缓存带来更好的结果，因此一般来说，并没有一个最大的缓存尺寸，在可测量的好处范围内存在限制。注：此处“并没有一个最大的缓存尺寸”，在可测量的好处范围内存在限制”意译为更符合中文表达习惯的说法。“在可测量的好处范围内存在限制”可能需要根据上下文进一步调整表述以准确传达原意。原意是缓存大小因数量级来说，没有理论上的最高缓存问题，并非其使用的性能提升是可量化的。问题在于，并非所有使用的内存都是连续的，因此即使配备了16M缓存和5.6M的工作集，仍然会发生冲突（参见2路组相联16MB缓存相较于直接映射版本的优势）。但是可以肯定的是，在相同工作负载下，32MB缓存带来的好处将微乎其微。但谁说工作集必须保持不变呢？随着时间的推移，工作负载在增长，缓存大小也应随之增加。在购买机器时，如果需要选择愿意支付的缓存大小，则测量工作集大小是有意义的。为什么这很重要可以从第21页的图表中看出。

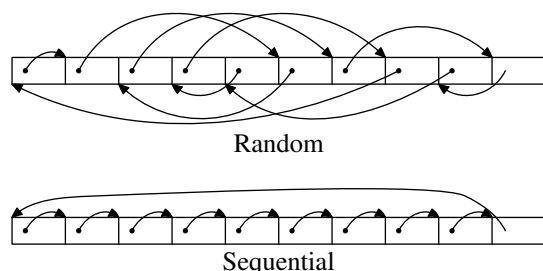


图3.9：测试内存布局

有两种类型的测试。在第一次测试中，元素是顺序处理的。测试程序跟随指针 n ，但数组元素被链接成链表形式，使得它们按照内存中的实际位置顺序遍历。如图3.9下半部分所示，最后一个元素有一个回指。在第二次测试（见图3.9上半部分），数组元素以随机顺序进行遍历。在这两种情况下，数组元素都形成一个循环单向链表。

3.3.2 缓存效应的测量

所有图表均通过测量一个能够模拟任意大小工作集、读写访问以及顺序或随机访问的程序生成。我们在图3.4中已经看到了一些结果。该程序创建了一个与工作集大小相对应的数组，其元素类型如下：

```
````struct l { struct l *n; long int
pad[NPAD]; };```` 注：该代码
片段为结构体定义，其中`pa
d`数组保持原样，无需翻译
。
```

所有条目都通过指针以循环链表的形式连接起来，可以是顺序排列也可以是随机排列。从一个条目移动到下一个条目时始终使用指针，即使元素是顺序布局也不例外。填充元素承载有效负载，并且其大小可以任意增大。在某些测试中会修改数据，在其他测试中程序仅执行读操作。

在性能测量中，我们讨论的工作集大小是由多个 `struct l` 类型的元素组成的数组。一个包含  $2^N$  字节的工作集包含  $2^N / \text{sizeof}(\text{struct l})$  个元素。注：由于给定文本中的“ $2^N$ ”可能是一个排版错误或占位符，在没有更多上下文的情况下保留原样。如果这里的“ $2^N$ ”是数学表达式的一部分，正确的翻译应为“ $2^N$ ”。

显然，`struct l` 的 `sizeof` 大小取决于 `NPAD` 的值。对于 32 位系统，`NPAD=7` 表示每个数组元素的大小为 32 字节；对于 64 位系统，则为 64 字节。

**单线程顺序访问** 最简单的情况是对列表中的所有条目进行一次简单的遍历。列表元素是连续且紧密排列的。处理顺序无论是从前向后还是从后向前都无关紧要，处理器可以同样高效地处理这两种方向。在这里以及后续的所有测试中，我们测量的是处理单个列表元素所需的时间。时间单位为处理器周期。图3.10展示了结果。除非另有说明，所有测量均在64位模式下的Pentium 4机器上进行，这意味着结构体 `l` 与 `NPAD=0` 的大小为8字节。

首次测量受到噪声干扰。所测工作负载太小，无法过滤掉系统其他部分的影响。我们可以安全地假设所有值都在4个周期的水平。考虑到这一点，我们可以看到三个明显的层次：

- 直到工作集大小为  $2^{14}$  字节。
- 从2字节到2字节。此处的“2字节”应理解为表达形式，实
- 从2字节及以上。

这些步骤很容易解释：处理器具有16kB的L1d缓存和1MB的L2缓存。由于缓存在系统其他部分也被使用，因此在从一个级别过渡到另一个级别的过程中没有明显的界限，因为缓存并非仅用于程序数据。具体来说，L2缓存是统一缓存，并且也用于指令（注：Intel采用包含式缓存）。

或许不太符合预期的是不同工作集大小的实际时间。L1d缓存命中的时间如预期：P4处理器上L1d命中后的加载时间为大约4个周期。但L2访问情况如何？一旦L1d缓存不足以容纳数据，人们可能会认为每个元素的访问时间会达到14个周期或更多，因为这是L2的访问时间。然而，结果显示只需要约9个周期即可完成。这种差异

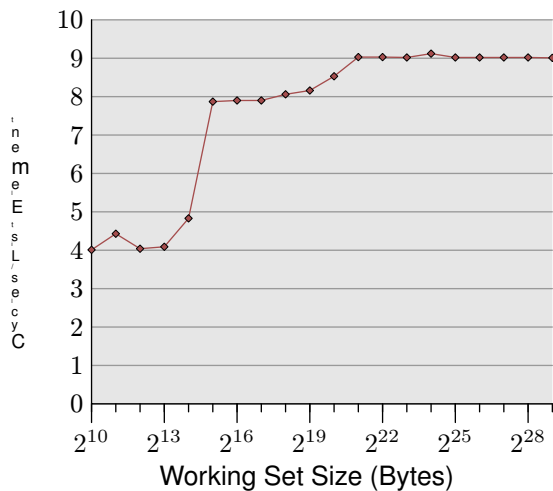


图3.10: 顺序读访问, NPAD=0

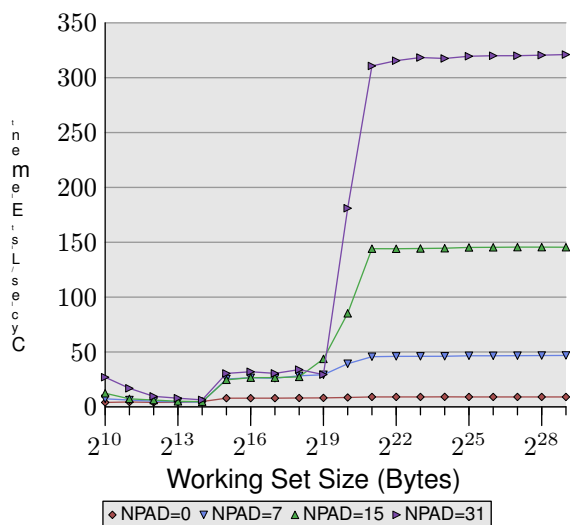


图3.11: 不同大小的顺序读取

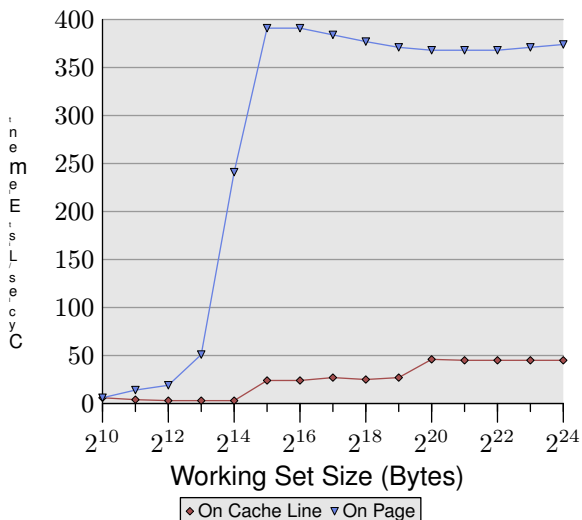


图3.12: 连续读取的快表影响

可以由处理器中的高级逻辑来解释。为了提前使用连续的内存区域, 处理器预取下一个缓存行。这意味着当实际使用下一个缓存行时, 它已经加载了一半。因此, 等待下一个缓存行完全加载所需的延迟远小于L2访问时间。

当工作集大小超过L2缓存的容量时, 预取的效果更加明显。之前我们提到, 主内存访问需要花费200个周期。只有通过有效的预取, 处理器才能将访问时间保持在低至9个周期的水平。从200和9之间的差异可以看出, 这种效果非常理想。

我们可以间接地观察到在预取时处理器的行为。图3.11展示了相同工作集大小下的时间, 但这次我们看到的是结构1的不同大小的图表。这意味着列表中的元素数量减少了, 但每个元素的大小增加了。不同的大小导致了列表中连续的 $n$ 个元素之间的距离增大。在图中的四种情况中, 这些距离分别为0、56、120和248字节。

在底部我们可以看到图3.10中的那条线, 但这次它几乎是一条直线。其他情况的时间表现则糟糕得多。在这张图表中, 我们同样可以看到三个不同的层次, 并且可以看出小工作集大小的测试中有较大的误差(忽略这些误差)。只要只涉及L1d缓存时, 各条曲线基本上相互匹配。由于不需要预取操作, 所有元素大小在每次访问时都直接命中L1d缓存。

对于L2缓存命中的情况, 我们看到新增的三条指令几乎一致, 但它们处于更高的层级(大约在28层)。这是L2访问时间所在的层级。这意味着从L2预取到L1d基本上是被禁用的。即使使用NPAD=7, 在每次循环迭代时都需要一个新的缓存行; 而对于NPAD=0, 循环需要迭代八次之后才会需要下一个缓存行。由于预取逻辑无法在每个周期加载新的缓存行, 因此我们会在每次迭代中看到从L2加载的停顿。

当工作集大小超过L2容量时, 情况变得更加有趣。现在所有四条曲线差异明显。不同的元素大小显然在性能差异中扮演了重要角色。处理器应当识别步长的大小, 并且对于NPAD=15和31, 由于元素大小小于预取窗口, 不应预取不必要的缓存行(参见6.3.1节)。当硬件预取器无法跨页边界进行预取时, 就会阻碍预取工作的效果。每次增加元素大小都会使硬件调度器的效率降低50%。如果允许硬件预取器跨页边界工作, 并且下一个页面不在内存中或无效, 则操作系统必须介入以定位该页面。

页表缺失。这意味着程序会遇到一个它自身并未发起的缺页异常。这是完全不可接受的，因为处理器无法判断页面不存在是由于未被加载还是根本就不存在。在后一种情况下，操作系统必须终止该进程。无论如何，在 NPAD=7 及更高版本中，每个列表元素需要一条缓存行，硬件预取器几乎无能为力。由于处理器只是读取一个字然后加载下一个元素，根本没有时间从内存中加载数据。

另一个导致性能下降的重要原因是快表缓存的缺失。如第4节所述，快表缓存存储了虚拟地址转换为物理地址的结果。由于需要极高的访问速度，快表缓存非常小。当访问的页面数量超过快表中记录的虚拟到物理地址映射条目时，系统必须不断重复进行地址转换操作，这是一项非常昂贵的操作。随着数据元素大小的增加，每次快表查找的成本分摊到更少的元素上，这意味着每个列表元素所需的总TLB条目计算量会更高。

为了观察快表（TLB）的影响，我们可以运行一个不同的测试。在一次测量中，我们像往常一样顺序排列元素。对于占用整个缓存行的元素，我们使用 NPAD=7。对于第二次测量，我们将每个列表元素放置在一个单独的页面上。每页其余部分保持不变，并且不计入工作集大小的总和。结果是，在第一次测量中，每次列表迭代都需要一个新的缓存行，并且每隔64个元素就需要加载一个新页面。而在第二次测量中，每次迭代都要求加载位于新页面上的新的缓存行。

如图3.12所示。测量是在与图3.11相同的机器上进行的。由于可用RAM的限制，工作集大小不得被限制为 $2^{24}$ 字节，这需要1GB的空间来将对象放置在单独的页面上。较低的红色曲线正好对应于图3.11中的NPAD=7曲线。我们可以看到显示L1d和L2缓存大小的明显阶梯状变化。第二条曲线看起来完全不同。重要特征是当工作集大小达到 $2^{13}$ 字节时开始出现的巨大峰值，这是TLB缓存溢出的时候。以64字节为元素大小，我们计算得出TLB缓存有64个条目。由于程序锁定内存以防止其被换出，因此没有缺页异常影响成本。

如图3.12所示，计算物理地址并将其存储在快表中所需的周期数非常高。虽然该图表展示了极端情况，但现在应该清楚的是，这是一个导致性能下降的重要因素。

<sup>20</sup>Yes, this is a bit inconsistent because in the other tests we count the unused part of the struct in the element size and we could define NPAD so that each element fills a page. In that case the working set sizes would be very different. This is not the point of this test, though, and since prefetching is ineffective anyway this makes little difference.

对于较大的NPAD值，TLB缓存的效率降低。由于在读取L2缓存或主内存中的缓存行之前必须计算物理地址，因此地址转换的开销是累加到内存访问时间上的。这在一定程度上解释了为什么NPAD=31每个列表元素的总成本高于RAM理论访问时间。

我们可以通过观察列表元素被修改时的测试数据来进一步了解预取实现的细节。图3.13展示了三条线。在所有情况下，元素宽度均为16字节。第一条线是现在熟悉的列表遍历操作，作为基准参考。第二条线标记为“Inc”，仅将当前元素的pad[0]成员递增后继续访问下一个元素。第三条线标记为“Addnext0”，它将next元素的pad[0]列表元素加到当前列表元素的pad[0]成员上。

朴素的假设是，“Addnext0”测试运行得更慢，因为它需要执行更多的工作。在访问下一个列表元素之前，必须从该元素加载一个值。因此，令人惊讶的是，在某些工作集大小下，这个测试实际上比“Inc”测试更快。解释这一现象的原因在于，从下一个列表元素的加载基本上是一个强制性的预取操作。每当程序前进到下一个列表元素时，可以确定该元素已经存在于L1d缓存中。结果表明，“Addnext0”的性能与简单的“Follow”测试相当，只要工作集大小能够容纳在L2缓存内。

“Addnext0”测试比“Inc”测试更快地耗尽L2缓存，因为它需要从主内存加载更多的数据。这就是为什么当工作集大小为 $2^{21}$ 字节时，“Addnext0”测试达到28个周期的水平。“Follow”测试则只达到了14个周期的水平，而前者是后者的两倍。这也很好解释：由于其他两个测试会修改内存，在L2缓存驱逐以腾出空间给新的缓存行时，不能简单地丢弃数据，而是必须将其写回到内存中。这意味着FSB上的可用带宽减半，因此从主内存传输数据到L2所需的时间翻倍。

最后一个影响顺序且高效的缓存处理的因素是缓存的大小。这一点显而易见，但仍需指出。图3.14展示了Increment基准测试中元素大小为128字节（NPAD=15，在64位机器上）时的时间表现。这次我们从三台不同的机器上获取了测量结果。前两台是P4处理器，最后一台是Core2处理器。前两台机器通过缓存大小的不同来区分：第一台处理器的L1d缓存为32k，L2缓存为1M；第二台处理器的L1d缓存为16k，L2缓存为512k，并且还有一个2M的L3缓存。Core2处理器则拥有32k的L1d缓存和4M的L2缓存。

图表中有趣的部分并不一定是Core2处理器相对于其他两个处理器的表现（尽管这也很令人印象深刻）。这里的主要关注点是工作集大小为多少的区域。

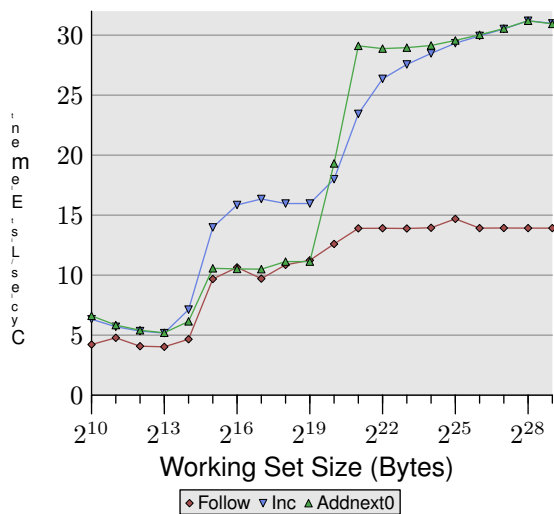


图3.13: 顺序读取和写入, NPAD=1

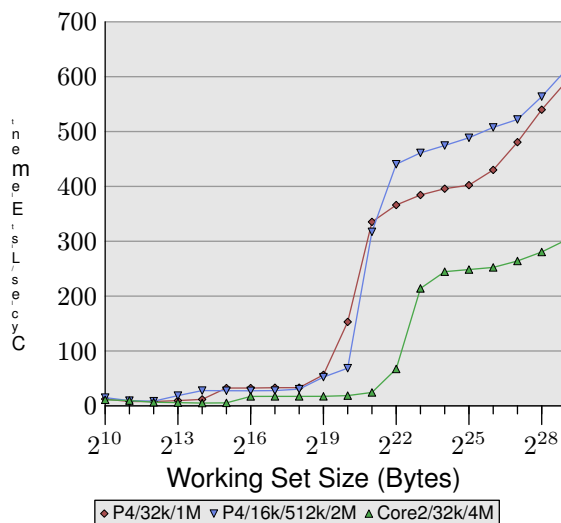


图3.14: 更大L2/L3缓存的优势

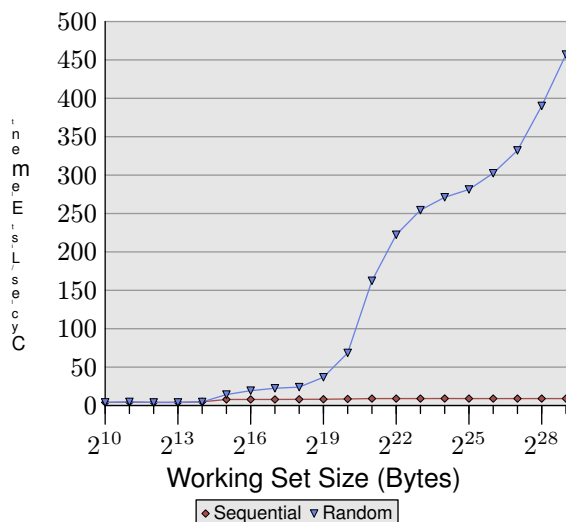


图3.15: 顺序读取与随机读取, NPAD=0

过大而无法完全容纳于相应的最后一级缓存, 从而导致主内存的大量参与。

如预期的那样, 最后一级缓存越大, 在对应L2访问成本的低水平区域曲线保持的时间就越长。值得注意的是这种性能优势。第二颗处理器(稍旧一些)在处理 $2^{32}$ 字节的工作集时, 速度是第一颗处理器的两倍。这一切都要归功于更大的最后一级缓存大小。配备4MB L2缓存的Core2处理器表现得更好。

对于随机工作负载而言, 这可能意义不大。但如果可以将工作负载调整到与最后一级缓存大小相匹配的程度, 则程序性能可以大幅提升。这就是为什么有时花更多的钱购买具有更大缓存的处理器是值得的。

**单线程随机访问** 我们已经看到, 处理器能够通过预取将缓存行加载到L2和L1d中来隐藏大部分主内存甚至L2访问延迟。然而, 这仅在内存访问是可预测的情况下才能有效工作。

如果访问模式不可预测或随机, 情况则完全不同。图3.15比较了顺序访问(与图3.10相同)时每个列表元素的时间与工作集中列表元素随机分布时的时间。顺序由随机化的链表决定。处理器无法可靠地预取数据。这只能偶然奏效, 如果短时间内使用的元素在内存中也被彼此接近的话。

图3.15中有两个重要的观察点。首先是工作集大小增加时所需的大量周期数。机器能够以200到300个周期访问主存, 而在这里我们达到了450个周期甚至更多。之前我们也见过这种现象(参见图3.11)。自动预取在此反而起到了不利的作用。

第二个值得注意的点是, 该曲线并未像顺序访问情况下那样在多个平台期趋于平坦, 而是持续上升。为了说明这一点, 我们可以测量不同工作集大小下的L2访问情况。结果见图3.16和表3.2。

该图显示, 当工作集大小超过L2缓存容量时, 缓存缺失率(L2访问次数 / L2缺失次数)开始上升。曲线与图3.15中的曲线形式相似: 快速上升、略微下降后再次上升。它与每列表元素周期数的图表有很强的相关性。L2缺失率会持续增长, 直到最终接近100%。在工作集(以及RAM)足够大的情况下, 任意随机选取的缓存行位于L2缓存或正在被加载的概率可以被任意减小。

Set Size	Sequential					Random				
	L2 Hit	L2 Miss	#Iter	Ratio Miss/Hit	L2 Accesses per Iteration	L2 Hit	L2 Miss	#Iter	Ratio Miss/Hit	L2 Accesses per Iteration
2 <sup>20</sup>	88,636	843	16,384	0.94%	5.5	30,462	4721	1,024	13.42%	34.4
2 <sup>21</sup>	88,105	1,584	8,192	1.77%	10.9	21,817	15,151	512	40.98%	72.2
2 <sup>22</sup>	88,106	1,600	4,096	1.78%	21.9	22,258	22,285	256	50.03%	174.0
2 <sup>23</sup>	88,104	1,614	2,048	1.80%	43.8	27,521	26,274	128	48.84%	420.3
2 <sup>24</sup>	88,114	1,655	1,024	1.84%	87.7	33,166	29,115	64	46.75%	973.1
2 <sup>25</sup>	88,112	1,730	512	1.93%	175.5	39,858	32,360	32	44.81%	2,256.8
2 <sup>26</sup>	88,112	1,906	256	2.12%	351.6	48,539	38,151	16	44.01%	5,418.1
2 <sup>27</sup>	88,114	2,244	128	2.48%	705.9	62,423	52,049	8	45.47%	14,309.0
2 <sup>28</sup>	88,120	2,939	64	3.23%	1,422.8	81,906	87,167	4	51.56%	42,268.3
2 <sup>29</sup>	88,137	4,318	32	4.67%	2,889.2	119,079	163,398	2	57.84%	141,238.5

表3.2：顺序访问和随机访问的L2缓存命中与未命中，NPAD=0

缓存缺失率的增加本身就能解释一部分成本。但还有一个因素需要考虑。从表3.2中的L2/#Iter列可以看出，程序每次迭代中使用的L2总次数在增长。每个工作集都比前一个大一倍。因此，在没有缓存的情况下，我们预计主内存访问量会翻倍。但在有缓存且（几乎）完全可预测的条件下，数据表明顺序访问时L2使用量仅略有增加。这种增加完全是由于工作集大小的增长所致，并无其他原因。

对于随机访问，每当工作集大小翻倍时，每个元素的访问时间会增加一倍以上。这意味着由于工作集大小仅翻倍，列表中每个元素的平均访问时间增加了。其背后的原因是快表未命中率上升。在图3.17中，我们看到了NPAD=7随机访问的成本。这次随机化进行了修改：正常情况下整个列表作为一个块进行随机化（用标签∞表示），而其他11条曲线则显示了以更小的块执行随机化的结果。对于标记为“60”的曲线，每组60页（245.760字节）单独进行随机化处理。这意味着在进入下一个块中的元素之前，必须遍历当前块中所有列表元素。这导致任何时刻使用的快表条目数量受到限制。

NPAD=7 的元素大小为 64 字节，这对应于缓存行的大小。由于列表元素的随机顺序，硬件预取器的影响微乎其微，尤其是在涉及少数几个元素时更是如此。这意味着 L2 缓存未命中率与整个列表在一个块内的随机化差异不大。随着块大小的增加，测试性能逐渐逼近单个块随机化的曲线。这表明此测试案例的性能显著受到快表（TLB）未命中的影响。如果能够降低 TLB 未命中次数，则性能将大幅提升。

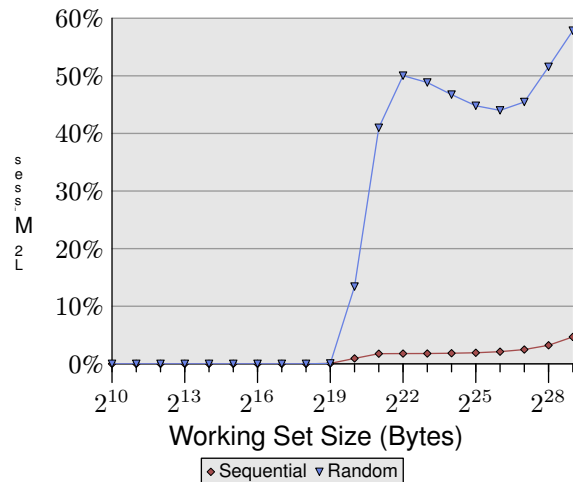


图3.16：L2d缺失率

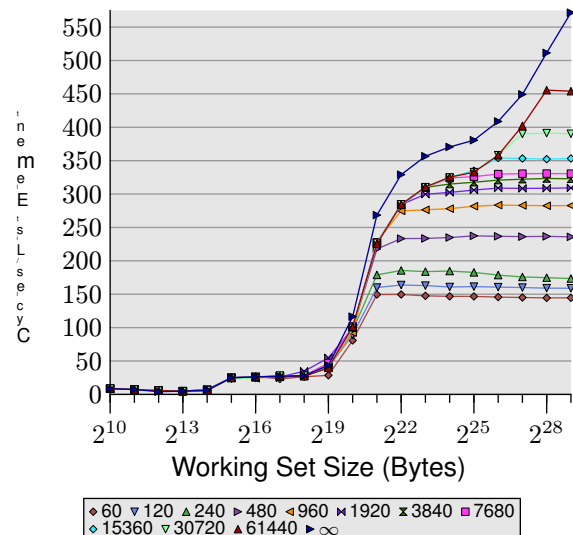


图3.17：页级随机化，NPAD=7

(在稍后将看到的一项测试中, 这一数值可高达38%。下一条缓存行是必需的。在下一节中, 我们将看到当前这一功能是如何实现的。

### 3.3.3 写操作行为

在我们开始分析多个执行上下文(线程或进程)使用同一段内存时的缓存行为之前, 我们必须先探讨一下缓存实现的一个细节。缓存应该是连贯一致的, 并且这种一致性对于用户级代码来说应该是完全透明的。然而, 内核代码则有所不同; 它偶尔需要进行缓存刷新操作。

这具体意味着, 如果缓存行被修改, 则在此之后系统的结果与完全没有缓存且主内存位置本身已被修改的情况相同。这一功能可以通过两种方式或策略来实现:

- 写通缓存实现;
- 写回缓存实现。

写直达缓存是实现缓存一致性的最简单方式。当缓存行被写入时, 处理器会立即将该缓存行也写入主内存中。这样可以确保在任何时候, 主内存和缓存始终保持同步。每当替换缓存行时, 可以直接丢弃缓存内容。这种缓存策略虽然简单, 但速度并不快。例如, 一个不断修改局部变量的程序即使数据可能仅被短暂使用且不会在其他地方用到, 也会在前端总线上产生大量流量。

写回策略更为复杂。在这种情况下, 处理器不会立即将修改过的缓存行写回到主内存中, 而是仅将该缓存行标记为脏状态。当将来某个时刻该缓存行从缓存中被移除时, 脏位会指示处理器在丢弃内容之前将其数据写回主内存。

写回缓存有机会表现出显著更好的性能, 这也是为什么在一个配备良好处理器的系统中, 大多数内存都采用这种方式进行缓存。处理器甚至可以利用前端总线上多余的容量来存储一个缓存行的内容, 在该缓存行必须被清空之前。这使得脏位可以被清除, 并且当缓存空间需要腾出时, 处理器只需丢弃该缓存行即可。

但是写回实现存在一个显著的问题。当多个处理器(或核心或超线程)同时访问同一内存时, 必须确保所有处理器在任何时候都能看到相同的内存内容。如果一条缓存行在一个处理器上是脏的(即尚未写回到主内存), 而另一个处理器尝试读取相同的位置, 则该读操作不能直接访问主内存。相反, 需要使用第一个处理器缓存中的内容。

在继续之前, 还有两种缓存策略需要提及:

- 写组合; 以及
- 非缓存ABLE (注: 原文中的"uncacheable"似乎有误或为特定上下

这两种策略用于地址空间中的特殊区域, 这些区域没有实际的RAM作为后盾。内核为这些地址范围设置这些策略(在x86处理器上使用内存类型范围寄存器MTRRs), 其余过程会自动完成。MTRRs还可用于选择写通和写回策略之间。

写结合是一种有限的缓存优化, 通常用于诸如显卡等设备上的RAM。由于向这些设备传输数据的成本远高于本地RAM访问成本, 因此避免过多的数据传输尤为重要。仅仅因为缓存行中的一个字被写入就将整个缓存行进行传输是低效的, 尤其是在下一个操作会修改该缓存行中下一个字的情况下。可以想象这种情况很常见, 在屏幕上水平相邻的像素所对应的内存通常也是相邻的。正如其名称所示, 写结合会在缓存行写回设备之前合并多个写访问。在理想情况下, 整个缓存行会被逐字节地修改, 并且只有在最后一个字被写入之后, 该缓存行才会被写回到设备中。这可以显著提高设备上RAM的访问速度。

最终存在不可缓存的内存。这通常意味着该内存位置根本没有由RAM支持。它可能是某个特殊地址, 被硬编码为在CPU外部实现某些功能。对于商用硬件而言, 这种情况最常见于映射到总线(如PCIe等)上连接的卡和设备的内存地址范围。在嵌入式电路板中, 有时会发现可以用来开启或关闭LED的这种内存地址。缓存这样的地址显然是个坏主意。在此上下文中, LED用于调试或状态报告, 因此希望尽快看到其变化。由于PCIe卡上的内存可以在不与CPU交互的情况下发生变化, 所以不应缓存此类内存。

### 3.3.4 多处理器支持

在前一节中, 我们已经指出了多处理器环境下出现的问题。即使是多核处理器, 对于那些不被共享的缓存层级(至少是L1d缓存)也存在此类问题。

提供一个处理器直接访问另一个处理器缓存的途径完全不切实际。首先, 这种连接的速度根本不够快。实际可行的替代方案是将缓存内容传输过去。

如果需要的话，也要考虑另一个处理器。注意，这同样适用于在同一处理器上未共享的缓存。

现在的问题是，缓存行传输何时必须发生？这个问题很容易回答：当一个处理器需要读取或写入另一个处理器缓存中脏的缓存行时。但是，一个处理器如何确定某个缓存行在另一个处理器的缓存中是否为脏状态呢？假设仅仅因为某缓存行被另一个处理器加载就认为它是脏的状态是次优的选择（最多只能说是这样）。通常情况下，大多数内存访问都是读取操作，并且由此产生的缓存行不会是脏状态。处理器对缓存行的操作非常频繁（当然，这也是我们撰写这篇论文的原因），这意味着在每次写入操作后广播有关更改的缓存行信息将是不切实际的。

经过多年的发展，形成了MESI缓存一致性协议（修改、独占、共享、无效）。该协议的名字来源于使用MESI协议时缓存行可以处于的四种状态：修改、独占、共享和无效。

修改：本地处理器已修改缓存行。这也意味着它是任何缓存中唯一的副本。

独占：缓存行未被修改，但已知不会加载到任何其他处理器的缓存中。

共享状态：缓存行未被修改，并可能存在于另一个处理器的缓存中。

无效：缓存行处于无效状态，即未被使用。

经过多年的发展，该协议从早期较为简单但效率较低的版本演变而来。借助这四种状态，可以高效地实现写回缓存，并同时支持不同处理器对只读数据的并发访问。

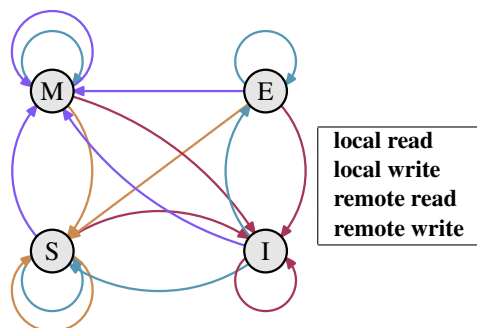


图3.18：MESI协议状态转换

处理器通过监听其他处理器的工作，即所谓的窥探机制，可以轻松完成状态变化。某些操作会在外部引脚上宣布，从而导致

处理器缓存的外部可见状态。问题缓存行的地址在地址总线上可见。以下描述了各状态及其转换（如图3.18所示），并将指出总线参与的情况。

初始时所有缓存行均为空，因此状态也为无效。若数据以写入方式加载到缓存中，则缓存状态变为修改。若数据是以读取方式加载的，则新的状态取决于是否有其他处理器也加载了该缓存行。如果有其他处理器也加载了该缓存行，则新状态为共享，否则为独占。

如果本地处理器读取或写入一个修改状态的缓存行，指令可以使用当前的缓存内容，并且状态不会改变。如果有第二个处理器想要从该缓存行读取数据，则第一个处理器必须将其缓存内容发送给第二个处理器，然后将状态更改为共享。发送的数据也会被内存控制器接收并处理，存储在主存中。如果没有这一步骤，缓存行无法标记为共享状态。如果第二个处理器想要写入该缓存行，第一个处理器会发送缓存行的内容，并将其本地状态标记为无效。这就是著名的“请求所有权”（RFO）操作。在最后一级缓存中执行此操作，就像从I→M转换一样，相对昂贵。对于写通缓存，我们还需要加上将新缓存内容写入更高一级缓存或主内存所需的时间，进一步增加了成本。

如果缓存行处于共享状态，且本地处理器从中读取数据，则无需改变状态，可以直接从缓存中完成读请求。若本地处理器对该缓存行进行写操作，则可以使用该缓存行，但其状态会变为修改状态。此时还需将其他所有可能存在于其他处理器中的该缓存行副本标记为无效。因此，需要通过RFO消息向其他处理器宣布写操作。如果第二个处理器请求读取该缓存行，则无需采取任何行动。主内存中包含当前数据，并且本地状态已经是共享状态。若第二个处理器想要对该缓存行进行写入（RFO），则只需将该缓存行标记为无效，不需要总线操作。

独占状态与共享状态几乎完全相同，唯一的区别在于本地写操作无需在总线上进行通告。本地缓存被认为是唯一持有该特定缓存行的缓存。这可以带来巨大的优势，因此处理器会尽量将尽可能多的缓存行保持在独占状态而不是共享状态。后者仅在信息当时不可用时作为备选方案。完全省略掉独占状态也不会导致功能问题，但性能将会受到影响，因为从E到M的状态转换比从S到M的状态转换要快得多。

从上述状态转换的描述中，多处理器操作特有的开销应该已经很明显了。是的，填充缓存仍然很昂贵，但现在我们还需要注意RFO消息。每当需要发送此类消息时，性能都会变差。

RFO消息在以下两种情况下是必要的：

- 一个线程从一个处理器迁移到另一个处理器，并且所有的缓存行都必须移动到新的处理器一次。
- 两个不同的处理器真正需要一条缓存行。<sup>21</sup>

在多线程或多进程程序中，总是存在一些同步需求；这种同步是通过内存实现的。因此，有一些有效的RFO消息。尽管如此，仍需尽量减少这些消息的发生频率。还有其他产生RFO消息的原因，在第6节中我们将解释这些场景。缓存一致性协议的消息必须在系统中的所有处理器之间进行分发。MESI转换只有在确定系统中的所有处理器都有机会回复该消息后才能发生。这意味着，回复所需的最大可能时间决定了缓存一致性的速度。在NUMA系统中总线冲突是可能的，延迟可能会很高，并且当然，巨大的流量也会导致性能下降。这些都是避免不必要的通信量的重要原因。

存在一个与多处理器系统相关的问题。这个问题的具体表现因机器而异，但其本质总是存在的：前端总线（FSB）是一个共享资源。在大多数机器中，所有处理器通过一条单一的总线连接到内存控制器（见图2.1）。如果单个处理器就能使总线达到饱和状态（这种情况很常见），那么两个或四个处理器共用同一总线将会进一步限制每个处理器可用的带宽。

即使每个处理器都有自己的总线连接到内存控制器（如图2.2所示），仍然存在一条通向内存模块的总线。通常这是一条总线，但在图2.2中的扩展模型中，对同一内存模块的同时访问仍会限制带宽。

在AMD模型中，每个处理器可以拥有本地内存。所有处理器确实能够快速并发访问各自的本地内存，特别是当集成内存控制器发挥作用时。但是多线程和多进程程序——至少从时间上来看，这种情况会导致问题：

<sup>21</sup> At a smaller level the same is true for two cores on the same processor. The costs are just a bit smaller. The RFO message is likely to be sent many times.

<sup>22</sup> Which is why we see nowadays, for instance, AMD Opteron systems with three sockets. Each processor is exactly one hop away given that the processors only have three hyperlinks and one is needed for the Northbridge connection.

时间——必须访问相同的内存区域以进行同步。

并行性严重受限于实现必要同步所需的有限带宽。程序需要精心设计，以最小化不同处理器和核心对同一内存位置的访问。以下测量将展示这一点以及与多线程代码相关的其他缓存效应。

为了确保能够理解在同一处理器上并发使用相同缓存行所带来的问题的严重性，我们将在此展示之前使用的同一程序的一些更多性能图表。这次，同时运行的线程数量超过一个。这里测量的是任意一条线程最快运行时间。这意味着当所有线程完成时，完整运行所需的时间会更高。所用机器具有四个处理器；测试中使用最多四个线程。所有处理器共享一条连接到内存控制器的总线，并且只有一条连接到内存模块的总线。

图3.19展示了顺序只读访问对于128字节条目的性能（NPAD=15在64位机器上）。对于单线程的曲线，我们可以预期其类似于图3.11所示。测量数据来自不同的机器，因此实际数值会有所不同。

该图的重要部分当然是在运行多个线程时的行为。请注意，在遍历链表的过程中，没有对内存进行任何修改，并且也没有尝试同步这些线程。尽管不需要发送任何读取并获取（RFO）消息，所有缓存行都可以共享，但在使用两个线程时最快的那个线程性能会下降多达18%，而在使用四个线程时则最多可下降34%。由于无需在处理器之间传输任何缓存行，这种性能下降完全是由以下一个或两个瓶颈引起的：从处理器到内存控制器的共享总线和从内存控制器到内存模块的总线。一旦工作集大小超过该机器中的L3缓存，所有三个线程都将预取新的列表元素。即使在使用两个线程的情况下，可用带宽也不足以实现线性扩展（即没有运行多个线程带来的惩罚）。

当我们修改内存时，情况变得更加糟糕。图3.20展示了顺序Increment测试的结果。该图表的Y轴采用对数刻度，因此请不要被表面上看似微小的区别所迷惑。即使运行两个线程，我们仍然大约有18%的性能损失，而当运行四个线程时，则出现了惊人的93%的性能损失。这意味着预取流量与写回流量在使用四个线程时几乎饱和了总线。

我们使用对数尺度来展示L1d缓存范围内的结果。可以看出，一旦有更多的

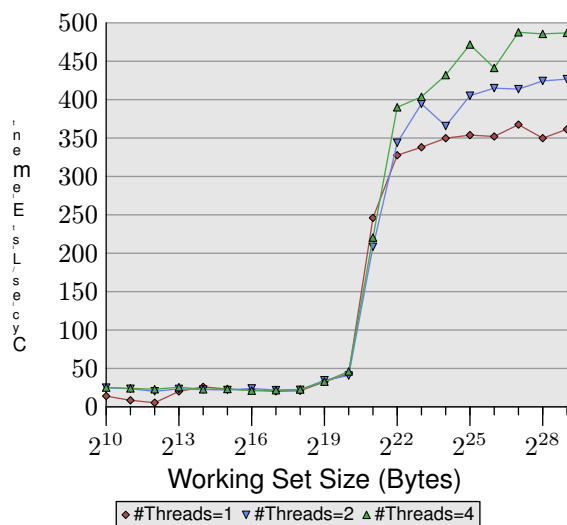


图3.19: 顺序读访问, 多线程

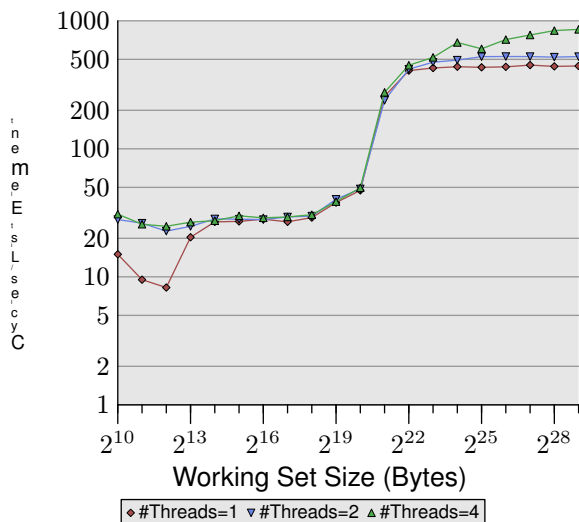


图3.20: 顺序增量, 多线程

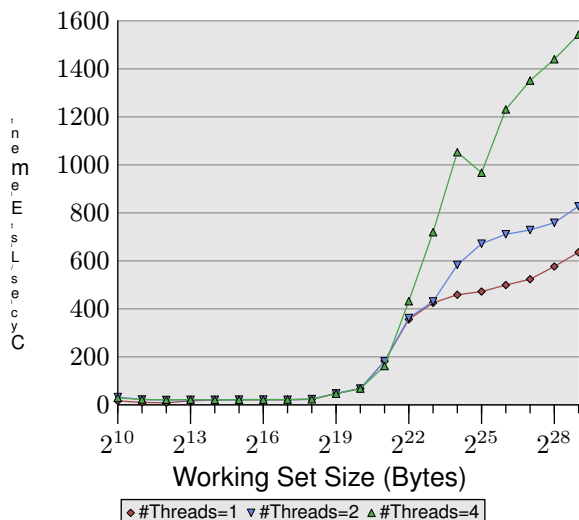


图3.21: 随机Addnextlast, 多线程

当多个线程运行时, L1d缓存基本上是无效的。单线程访问时间超过20个周期仅在L1d缓存不足以容纳工作集时发生。当多线程运行时, 即使是最小的工作集大小下, 这些访问时间也会立即出现。

一个问题的方面在这里没有展示出来。使用这个特定的测试程序很难测量到这一点。尽管该测试修改了内存, 因此我们预期会看到更多的RFO消息, 但在多线程情况下, 并未观察到L2范围内的更高成本。程序需要大量使用内存, 并且所有线程必须同时访问相同的内存区域。在没有大量的同步机制的情况下, 要实现这一点非常困难, 而这又会导致执行时间主要受同步操作的支配。

最终在图3.21中给出了Add-nextlast测试中随机访问内存的统计数据。此图主要目的是展示令人震惊的高数值。在这种极端情况下, 处理单个列表元素现在需要大约1500个时钟周期。使用更多线程的做法更加值得怀疑。我们可以将多线程使用的效率总结在一张表格中。

#Threads	Seq Read	Seq Inc	Rand Add
2	1.69	1.69	1.54
4	2.98	2.07	1.65

表3.3: 多线程效率

表格显示了在第28页的三幅图中, 工作集大小最大的多线程运行效率。数字表示测试程序使用两个或四个线程时, 在最大工作集大小下所能达到的最佳加速比。对于两个线程, 理论上的加速比上限为2; 而对于四个线程, 则为4。两个线程的情况还算可以, 但四个线程的最后一个测试结果表明, 扩展到超过两个线程几乎没有什么意义, 额外的好处微乎其微。如果我们以另一种方式表示图3.21中的数据, 这一点会更加明显。

图3.22中的曲线显示了加速比, 即相对于单线程执行代码的相对性能。我们必须忽略最小规模的数据, 因为这些测量结果不够准确。在L2和L3缓存范围内, 我们可以看到确实实现了几乎线性的加速效果。我们分别接近了2倍和4倍的速度提升。但是, 一旦L3缓存不足以容纳工作集时, 性能数据急剧下降。它们下降到两线程和四线程的加速比相同(见表3.3第四列)。这也是为什么很少能找到主板上带有超过四个CPU插槽且共用同一内存控制器的原因之一。具有更多处理器的机器必须采用不同的设计(参见第5节)。

这些数字并不具有普遍性。在某些情况下, 即使工作集能够完全容纳于最后一级缓存中, 也不会

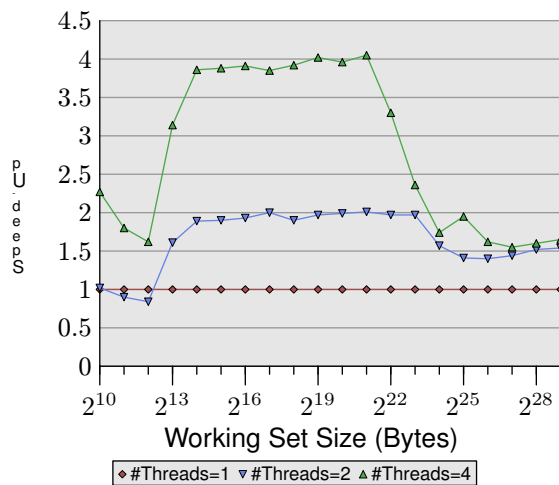


图3.22：通过并行性实现的加速效应

允许线性加速。事实上，这种情况很常见，因为线程通常不像此测试程序中那样解耦。另一方面，在使用大工作集时仍然可以利用多于两个线程的优势。要做到这一点需要仔细考虑。我们将在第6节讨论一些方法。

**特殊情形：超线程** 超线程（有时称为对称多线程，MT）由CPU实现，并且是一种特殊情况，因为单个线程实际上不能真正并发运行。它们共享几乎所有的处理资源，除了寄存器集之外。各个核心和CPU仍然并行工作，但每个核心上实现的线程受到这种限制。理论上，每个核心可以有多个线程，但迄今为止，英特尔的CPU最多每个核心有两个线程。CPU负责时间复用这些线程。仅此而已意义不大。真正的优势在于，当当前运行的超线程因延迟而无法使用资源时（如由内存访问引起的延迟），CPU可以调度另一个超线程并利用可用资源，例如算术逻辑单元（ALU）。在大多数情况下，这种延迟是由内存访问造成的。

如果两个线程在一个超线程核心上运行，该程序只有在两个线程的运行时间都低于单线程代码的运行时间时才更高效。这是通过重叠不同内存访问的等待时间来实现的，而这些等待时间通常会依次发生。一个简单的计算显示了要达到一定加速比所需的最小缓存命中率要求。

程序的执行时间可以用一个简单的模型进行近似，该模型仅包含一级缓存（参见[16]）。

$$T_{\text{exe}} = N [(1 - F_{\text{mem}})T_{\text{proc}} + F_{\text{mem}}(G_{\text{hit}}T_{\text{cache}} + (1 - G_{\text{hit}})T_{\text{miss}})]$$

变量的含义如下：

$N$  = 指令数量。 $F_{\text{mem}}$  = 访问内存的占比。 $G_{\text{hit}}$  = 加载命中缓存的比例。 $T_{\text{proc}}$  = 每条指令所需时钟周期数。 $T_{\text{cache}}$  = 缓存命中的时钟周期数。 $T_{\text{miss}}$  = 缓存未命中的时钟周期数。 $T_{\text{exe}}$  = 程序执行时间。

为了使用两个线程有意义，每个线程的执行时间必须不超过单线程代码执行时间的一半。两边唯一的变量是缓存命中次数。如果我们解方程求得不使线程执行速度减慢50%或更多的最小缓存命中率，就可以得到图3.23所示的图表。

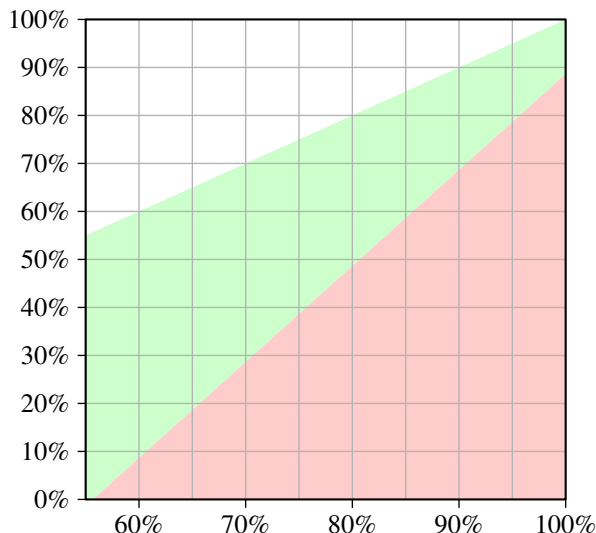


图3.23：加速比的最小缓存命中率

输入（在X轴上测量）是单线程代码的缓存命中率 $G_{\text{hit}}$ 。Y轴显示多线程代码的缓存命中率。这个值永远不会高于单线程命中的比率，因为否则单线程代码也会使用改进后的代码。对于单线程命中率——在这种特定情况下低于55%时——程序在所有情况下都可以从使用线程中获益。由于缓存未命中导致CPU大部分时间处于空闲状态，因此可以运行第二个超线程。

绿色区域为目标。如果线程的性能下降低于50%，且每个线程的工作负载减半，那么合并后的运行时间可能小于单线程的运行时间。对于所建模的处理器（使用了带有超线程的P4处理器的数据），一个程序

单线程代码的命中率为60%时，双线程程序需要至少10%的命中率。这通常是可行的。但如果单线程代码的命中率达到95%，那么多线程代码则需要至少80%的命中率。这会更困难。特别是，这是超线程的问题所在，因为现在每个超线程可用的有效缓存大小（这里指L1d，在实际应用中还包括L2等）被减半了。两个超线程使用同一个缓存来加载数据。如果这两个线程的工作集是非重叠的，则原来的95%命中率可能会减半，因此远低于所需的80%。

超线程仅在有限的情况下才有用。单线程代码的缓存命中率必须足够低，使得根据上述方程式和减少后的缓存大小，新的命中率仍能满足目标要求。只有在这种情况下，使用超线程才具有任何意义。实际效果是否更快取决于处理器能否充分重叠一个线程中的等待时间与其他线程的执行时间。并行化代码所带来的开销必须加入到新的总运行时间内，而这种额外成本往往不可忽视。

在第6.3.4节中，我们将看到一种技术，其中线程紧密协作，通过共同的缓存进行的紧密耦合实际上是一种优势。如果程序员愿意投入时间和精力来扩展代码，这种技术可以应用于许多情况。

显然，如果两个超线程执行完全不同的代码（即操作系统将这两个线程视为独立的处理器来运行不同的进程），那么缓存大小确实会被减半，这意味着缓存未命中会显著增加。除非缓存足够大，否则这种操作系统调度的做法值得怀疑。除非该机器的 workload 由设计上能够从超线程中受益的进程组成，否则最好在计算机的BIOS中关闭超线程功能。

### 3.3.5 其他细节

到目前为止，我们讨论了地址由三部分组成：标签、组索引和缓存行偏移。但实际上使用的地址是什么？当今所有相关的处理器都为进程提供虚拟地址空间，这意味着有两种不同的地址类型：虚拟地址和物理地址。

虚拟地址的问题在于它们并不唯一。随着时间的推移，同一个虚拟地址可能会指向不同的物理内存地址。不同进程中的相同地址也很可能指向不同的物理地址。因此，总是最好使用物理内存地址吗？

在此问题中，执行过程中使用的虚拟地址必须借助内存管理单元（MMU）转换为物理地址。这是一个非简单的操作。在执行指令的流水线中，物理地址可能仅在后续阶段才可用。这意味着缓存逻辑必须非常迅速地确定该内存位置是否已被缓存。如果可以使用虚拟地址，则可以在流水线的早期阶段进行缓存查找，并且在缓存命中时可提供内存内容。结果是，更多的内存访问成本可以通过流水线隐藏起来。

处理器设计人员目前使用虚拟地址标记一级缓存。这些缓存相对较小，可以轻松清除。如果进程的页表树发生变化，则至少需要部分清空缓存。如果处理器具有允许指定已更改的虚拟地址范围的指令，则可能可以避免完全刷新。考虑到一级指令缓存（L1i）和一级数据缓存（L1d）的低延迟（~3个周期），使用虚拟地址几乎是必须的。

对于较大的缓存，包括L2、L3等缓存，需要使用物理地址标记。这些缓存的延迟较高，虚拟到物理地址的转换可以在时间内完成。由于这些缓存容量较大（即在清空时会丢失大量信息），并且由于主内存访问延迟的原因，重新填充它们需要很长时间，因此频繁清空这些缓存的成本很高。

通常不需要了解这些缓存中地址处理的细节。它们是不可更改的，所有影响性能的因素通常是应该避免或与高成本相关的事项。超出缓存容量会导致问题，并且如果大多数使用的缓存行都落入同一个组内，则所有缓存都会很快遇到问题。后者可以通过虚拟寻址的缓存来避免，但对于使用物理地址寻址的缓存来说，用户级进程无法避免这种情况。唯一需要注意的是，在可能的情况下，不要将同一物理内存位置映射到同一进程中两个或多个虚拟地址上。

缓存的另一个细节是替换策略，这对程序员来说通常不太感兴趣。大多数缓存首先淘汰最近最少使用（LRU）的元素。这是一种始终适用的良好默认策略。随着关联度增大（由于未来可能增加更多核心，关联度确实可能会进一步提高），维护LRU列表的成本越来越高，因此我们可能会看到采用不同的替换策略。

关于缓存替换策略，程序员能做的并不多。如果缓存使用物理地址标签，则无法得知虚拟地址与缓存组之间的映射关系。可能所有逻辑页的缓存行都被映射到了相同的缓存组中。

<sup>23</sup> Another reason to keep hyper-threads enabled is debugging. SMT is astonishingly good at finding some sets of problems in parallel code.

留下大量的缓存未被使用。实际上，操作系统的职责是确保这种情况不会频繁发生。

随着虚拟化的出现，情况变得更加复杂。现在甚至操作系统也无法控制物理内存的分配。虚拟机监控器（VMM，又称Hypervisor）负责物理内存的分配。

程序员所能做的最佳实践是：a) 充分利用逻辑内存页面；b) 尽可能使用较大的页面大小，以最大程度地多样化物理地址。更大的页面尺寸还具有其他好处，但这又是另一个话题（参见第4节）。

### 3.4 指令缓存

不仅处理器使用的数据会被缓存，处理器执行的指令也会被缓存。然而，与数据缓存相比，指令缓存的问题要少得多。原因有几点：

- 代码的执行量取决于所需代码的大小。代码的大小通常取决于问题的复杂性。问题的复杂性是固定的。
- 程序的数据处理由程序员设计，而程序的指令通常是由编译器生成的。编译器作者了解良好代码生成的规则。
- 程序流程比数据访问模式更易于预测。当今的CPU非常擅长检测模式，这有助于预取操作。
- 代码始终具有很好的空间和时间局部性。

程序员应遵循一些规则，这些主要涉及如何使用工具。我们将在第6节中讨论这些问题。在这里，我们仅谈论指令缓存的技术细节。

自从CPU的核心时钟频率大幅提升，缓存（甚至一级缓存）与核心之间的速度差异也随之增大以来，CPU的设计中便引入了流水线。这意味着指令的执行被划分为多个阶段：首先解码一条指令，然后准备参数，最后执行该指令。这样的流水线可以很长（例如Intel的Netburst架构有20个阶段）。长流水线意味着如果流水线停滞（即指令流中断），则需要一段时间才能恢复到正常速度。流水线停滞的情况包括无法正确预测下一条指令的位置或加载下一条指令所需时间过长（例如，当它必须从内存中读取时）。

因此，CPU设计者会花费大量时间和芯片面积在分支预测上，以尽量减少流水线停滞的发生。

在复杂指令集计算（CISC）处理器上，解码阶段也会消耗一定时间。x86 和 x86-64 处理器尤其受影响。近年来，这些处理器不再将原始字节序列缓存到一级指令缓存（L1i），而是缓存经过解码的指令。在这种情况下，L1i 被称为“追踪缓存”（trace cache）。使用追踪缓存使得在命中缓存时可以跳过流水线的初始步骤，这对解决流水线停滞问题尤其有利。

如前所述，L2及以上的缓存是统一缓存，同时包含代码和数据。显然这里的代码是以字节序列列表形式缓存的，并未进行解码。

为了获得最佳性能，只需遵循与指令缓存相关的几条规则：

生成尽可能小的代码。但在为了利用流水线而进行软件流水线处理的情况下，有时需要生成更多的代码，或者使用小代码带来的开销过高时除外。

帮助处理器做出良好的预取决策。这可以通过代码布局或显式预取来实现。

这些规则通常由编译器的代码生成阶段来强制执行。程序员可以采取一些措施，我们将在第6节中进行讨论。

#### 3.4.1 自修改代码

在早期的计算机时代，内存资源十分宝贵。人们想方设法减小程序大小以腾出更多空间来存储程序数据。其中一个常用的技巧是随着时间改变程序本身的内容。这种自修改代码（SMC）至今偶尔仍会出现，如今大多出于性能考虑或用于安全漏洞利用。

SMC 通常应避免使用。虽然它一般能够正确执行，但在某些边界情况下可能会出现错误，如果不正确处理还会导致性能问题。显然，被修改的代码无法保留在包含解码指令的追踪缓存中。即使代码从未被执行（或长时间未被执行），如果追踪缓存没有被使用，处理器也可能遇到问题。当一个即将执行的指令在已经进入流水线后被更改时，处理器必须丢弃大量已完成的工作并重新开始。甚至有些情况下，处理器的大部分状态都需要被抛弃。

最后，由于处理器假设——出于简化原因，并且在99.999999%的所有情况下确实如此——

由于代码页不可变，L1i 实现不使用 MESI 协议，而是简化后的 SI 协议。这意味着如果检测到修改，则必须做出许多悲观假设。

强烈建议尽可能避免使用 SMC。内存资源不再那么稀缺了。最好编写单独的函数，而不是根据特定需求修改一个函数。也许有一天可以将 SMC 支持设为可选，并通过这种方式检测试图以此方式修改代码的攻击代码。如果绝对必须使用 SMC，则写入操作应绕过缓存，以免在 L1i 中需要的数据与 L1d 中的数据产生问题。更多信息请参见第 6.1 节。

在 Linux 系统中，通常很容易识别包含自修改代码（SMC）的程序。使用常规工具链编译的所有程序代码都是写保护的。程序员必须在链接时进行复杂的操作才能生成一个可执行文件，在该文件中代码页是可写的。当这种情况发生时，现代 Intel x86 和 x86-64 处理器具有专用的性能计数器，用于统计自修改代码的使用次数。借助这些计数器，即使由于权限放宽导致程序成功运行，也能够很容易地识别出包含 SMC 的程序。

### 3.5 缓存未命中因素

当内存访问未命中缓存时，成本会急剧上升。有时这种情况无法避免，因此理解实际的成本以及可以采取哪些措施来缓解问题是非常重要的。

#### 3.5.1 缓存与内存带宽

为了更好地了解处理器的能力，我们会在最优条件下测量可用带宽。这一测量尤其有趣，因为不同版本的处理器差异很大。这就是为什么本节包含多台不同机器的数据。用于衡量性能的程序使用了 x86 和 x86-64 处理器的 SSE 指令一次性加载或存储 16 字节数据。工作集从 1kB 增加到 512MB，就像我们在其他测试中所做的那样，并测量每周期可以加载或存储多少字节。

图 3.24 展示了在 64 位 Intel Netburst 处理器上的性能。对于能够完全放入 L1d 的工作集大小，处理器能够在每个周期读取完整的 16 字节，即每周期执行一次加载指令（movaps 指令一次性移动 16 字节）。测试中并未对读取的数据进行任何操作，仅测试了读取指令本身。一旦 L1d 不再足够使用时，性能会急剧下降到每周期不到 6 字节的水平。在  $2^{18}$  字节处的台阶是由于 DTLB 缓存耗尽导致的，这意味着每次访问新页面都需要额外的工作。由于读取

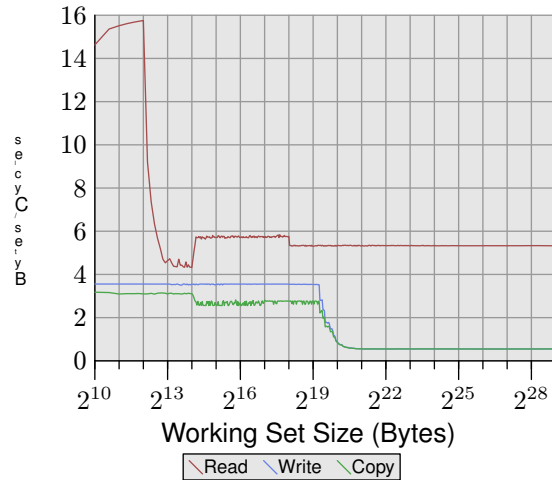


图3.24: Pentium 4 带宽

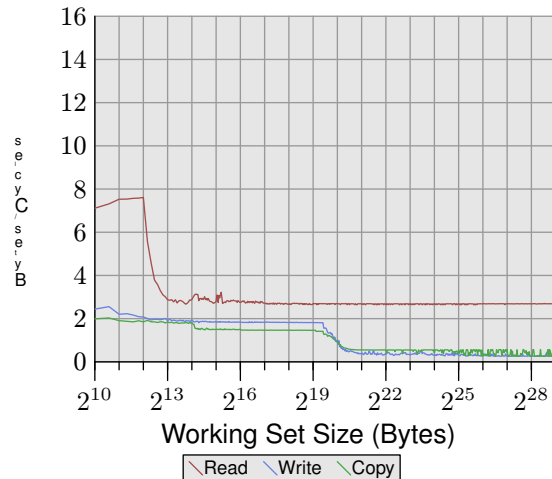


图3.25: P4带宽（启用2个超线程）

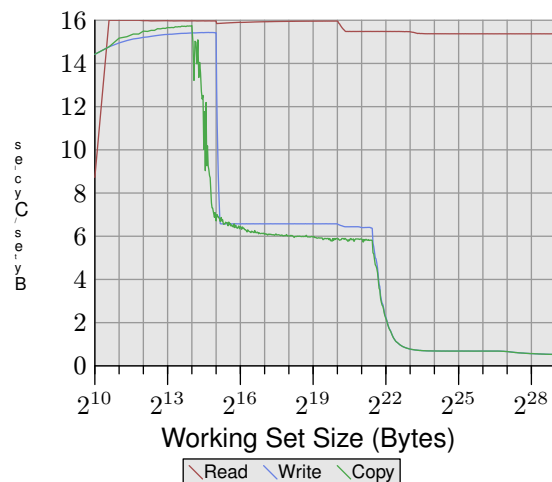


图3.26: Core 2 带宽

顺序预取如果能够完美预测访问次数，并且前端总线（FSB）可以以大约每周期5.3字节的速度流式传输工作集大小的内存内容。然而，预取的数据并不会传播到L1d缓存中。这些当然是一些在实际程序中永远无法达到的数值，可将其视为实践中的极限值。

更令人惊讶的不仅是读取性能，还有写入和复制性能。即使对于较小的工作集大小，写入性能也从未超过每周期4字节。这表明，在这些Netburst处理器中，英特尔选择了在L1d缓存使用写穿模式（Write-Through mode），在这种模式下，性能显然受到L2缓存速度的限制。这也意味着从一个内存区域复制到另一个不重叠的内存区域时，复制测试的性能并不会显著下降。必要的读取操作非常快，并且可以部分与写入操作并行执行。写入和复制测量中最值得注意的细节是当L2缓存不再足够时性能急剧下降。此时性能降至每周期0.5字节！这意味着写入操作比读取操作慢了十倍。这表明优化这些操作对于程序性能至关重要。

在图3.25中，我们展示了在同一处理器上运行两个线程的结果，每个线程绑定到处理器的其中一个超线程。图表与前一张图保持相同的比例尺，以突出差异。由于测量两个并发线程的问题，曲线有些抖动。结果如预期所示。由于超线程共享所有资源（除了寄存器），每个线程只有半个缓存和带宽可用。这意味着即使一个线程需要等待很长时间，并且可以将执行时间让给另一个线程，但这并不会产生任何影响，因为另一个线程也需要等待内存访问。这真正体现了使用超线程的最糟糕情况。

与图3.24和图3.25相比，图3.26和图3.27中英特尔酷睿2处理器的结果看起来大不相同。这是一款具有共享L2缓存的双核处理器，其L2缓存大小是P4机器上L2缓存的四倍。但这只能解释写入和复制性能下降延迟的原因。

其他更大的差异也显而易见。在整个工作集范围内，读取性能徘徊在每周期最优的16字节左右。超过 $2^{20}$ 字节后的读取性能下降再次是因为工作集过大导致DTLB缓存不足所致。达到这些高数值意味着处理器不仅能够预取数据并在时间内传输数据，还意味着数据被预取到了L1d缓存中。

写入和复制性能也存在显著差异。处理器没有采用写通策略；写入的数据存储在L1d缓存中，仅在必要时才会被逐出。这使得写入速度接近于理论极限。

每个周期最优16字节。一旦L1d不再足够，性能会显著下降。与Netburst处理器类似，写入性能明显较低。由于读取性能很高，这里的差异更加突出。实际上，当连L2都不再足够时，速度差距甚至扩大到20倍！但这并不意味着Core 2处理器表现不佳。相反，它们的性能始终优于Netburst核心。

在图3.27中，测试运行了两个线程，每个线程分别在一个Core 2处理器的两个核心上执行。这两个线程访问相同的内存区域，但不一定完全同步。读取性能的结果与单线程情况没有显著差异，但在多线程测试案例中出现一些额外的抖动是意料之中的。

有趣的是，当工作集大小可以完全放入L1d缓存时，写入和复制性能表现与数据必须从主内存读取的情况相同。如图所示，两个线程竞争同一内存位置，并且需要发送针对缓存行的RFO消息。问题在于这些请求并未以L2缓存的速度处理，尽管两颗核心共享该缓存。L1d缓存不再足够容纳修改后的条目时，每个核心的L1d缓存中的修改项将被刷新到共享的L2缓存中。此时性能显著提升，因为L1d缓存未命中的数据可以由L2缓存提供服务，并且只有在数据尚未被刷新的情况下才需要RFO消息。这就是为什么我们看到工作集大小为这些值时速度减少了50%的原因。渐近行为符合预期：由于两颗核心共享同一前端总线（FSB），每个核心只能获得一半的FSB带宽，因此对于大型工作集而言，每条线程的性能大约是单线程情况的一半。

由于即使是同一供应商的不同处理器版本之间也存在显著差异，因此研究其他供应商的处理器性能也是十分值得的。图3.28展示了AMD家族10h Opteron处理器的性能。该处理器配备64kB L1d缓存、512kB L2缓存和2MB L3缓存。L3缓存被所有核心共享。性能测试的结果如图3.28所示。

观察这些数据的第一个细节是，如果L1d缓存足够的话，处理器每周期可以处理两条指令。读取性能超过每周期32字节，即使写入性能也高达每周期18.7字节，表现相当高。然而，读取曲线很快趋于平坦，并且在达到每周期2.3字节时变得非常低。在此测试中，处理器并未进行有效的数据预取。

另一方面，写入曲线根据各级缓存的大小表现不同。当L1d缓存达到全容量时性能峰值出现，然后降至每周期6字节（对于L2），再降至每周期2.8字节（对于L3），最终如果连L3都无法容纳所有数据，则为每周期0.5字节。



图3.27: 双线程下的Core 2带宽

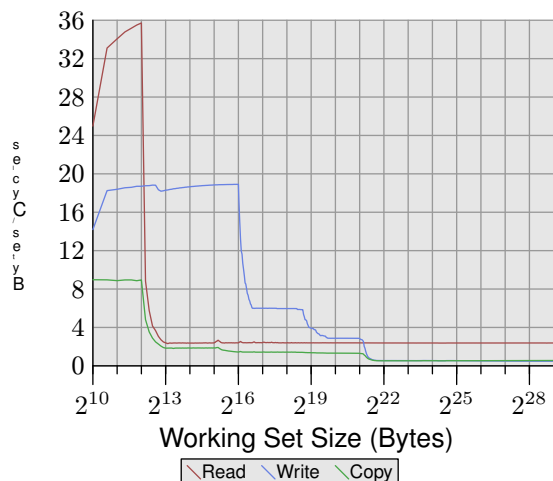


图3.28: AMD Family 10h Opteron 带宽



图3.29: AMD Fam 10h 双线程带宽

数据。L1d缓存的性能优于（较旧的）Core 2处理器，L2访问速度相同（尽管Core 2的缓存更大），而L3和主内存的访问速度较慢。

复制性能不会优于读取或写入的性能。这就是为什么我们最初看到曲线主要受读取性能影响，随后则由写入性能主导。

Opteron处理器的多线程性能如图3.29所示。读取性能基本不受影响，每个线程的L1d和L2缓存照常工作，而在此情况下L3缓存也没有很好地进行预取操作。两个线程不会对L3缓存造成过大的压力以满足其需求。然而，在此测试中写入性能存在较大问题。所有共享的数据都必须通过L3缓存传输。这种共享似乎效率较低，即使L3缓存的大小足以容纳整个工作集，其成本也显著高于一次L3访问的成本。将该图与图3.27进行比较可以看出，Core 2处理器的两个线程在适当的工作集大小范围内以共享L2缓存的速度运行。而Opteron处理器仅在一个非常小的工作集大小范围内才能达到这种性能水平，并且即使在此范围接近于L3缓存的速度，但L3缓存速度慢于Core 2的L2缓存。

### 3.5.2 关键字载入

内存从主存传输到缓存时，是以小于缓存行大小的块进行传输。今天一次传输64字节，而缓存行大小为64或128字节。这意味着每个缓存行需要8次或16次传输。

DRAM芯片可以以突发模式传输这些64字节的块。这样可以在无需内存控制器进一步命令及其可能关联的延迟的情况下填满缓存行。如果处理器预取缓存行，这可能是最佳的操作方式。

当程序访问数据缓存或指令缓存发生未命中（这意味着，这是由于数据首次使用而导致的强制性缓存未命中，或者由于有限的缓存大小需要驱逐缓存行而引起的容量型缓存未命中）时，情况会有所不同。缓存行中所需的用于继续执行程序的那个字可能并不是该缓存行中的第一个字。即使在突发模式和双倍数据速率传输下，每个独立的64位块到达的时间也会有明显的不同。每个块比前一个块晚4个或更多的CPU周期到达。如果程序需要继续执行的那个字是缓存行中的第八个字，则该程序必须在第一个字到达后等待额外的30个或更多周期。

事情并不一定非要如此。内存控制器可以按不同的顺序请求缓存行中的字。处理器可以告知内存控制器程序正在等待的字，即 *critical word*。

内存控制器可以首先请求该字。一旦该字到达，程序就可以继续执行，而缓存行的其余部分仍在到来，并且缓存尚未处于一致状态。这种技术称为“关键字优先及早重启”。

如今处理器实现了这种技术，但有些情况下无法做到。如果处理器预取数据而关键字未知，则在预取操作进行期间，处理器若请求缓存行则必须等待关键字到达，且无法影响其顺序。

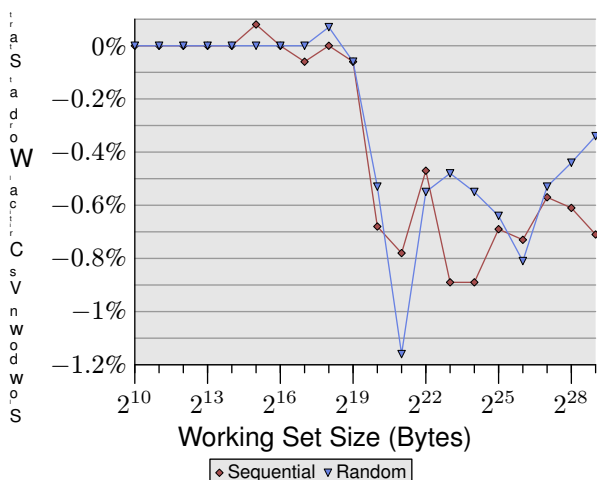


图3.30: 缓存行末的关键字

即使进行了这些优化，缓存行中关键字的位置仍然很重要。图3.30展示了顺序访问和随机访问的Follow测试结果。该图显示了使用指针在第一个单词与指针在最后一个单词时运行测试的速度下降情况。元素大小为64字节，对应于缓存行大小。数据波动较大，但可以看出，一旦L2缓存不足以容纳工作集大小，关键字位于末尾的情况性能大约慢0.7%。顺序访问似乎受到的影响更大一些。这与前述预取下一个缓存行时的的问题一致。

### 3.5.3 缓存放置

缓存相对于超线程、核心和处理器的位置不由程序员控制。但是，程序员可以确定线程的执行位置，在这种情况下，缓存与所用CPU的关系就变得重要了。

在这里，我们不会详细讨论何时选择哪些核心来运行线程。我们将仅描述程序员在设置线程亲和性时必须考虑的架构细节。

超线程在定义上共享一切，但不包括寄存器集。这其中包括L1缓存。这里没有更多可说的了。真正的有趣之处在于处理器的各个核心。每个核心至少拥有自己的L1缓存。除此之外，目前并没有太多共通的细节：

- 早期的多核处理器根本没有共享缓存。
- 后来的英特尔型号为双核处理器配备了共享的L2缓存。对于四核处理器，我们需要为每对两个核心处理各自的独立L2缓存。没有更高层次的缓存存在。
- AMD 的家族 10h 处理器具有独立的 L2 缓存和统一的 L3 缓存。

处理器供应商的宣传材料中已经有很多关于各自型号优势的内容。如果各个核心处理的工作集不重叠，那么没有共享缓存会带来一定的优势。这种方法对于单线程程序来说效果很好。由于这种情况在今天仍然很常见，因此这种做法表现得并不差。但是总会有一定程度上的重叠存在。所有缓存中都包含了最常使用的公共运行时库的部分内容，这意味着一些缓存空间被浪费了。

完全共享除L1缓存以外的所有缓存（如Intel双核处理器所做）可以带来显著优势。如果运行在两个核心上的线程的工作集有大量重叠，可用的总缓存内存将增加，从而使工作集可以在不降低性能的情况下变得更大。如果没有重叠，Intel的高级智能缓存管理机制应防止任何一个核心独占整个缓存。

当两个核心各自使用大约一半的缓存来存储其工作集时，就会出现一些摩擦。缓存需要不断权衡两个核心的缓存使用情况，而为了重新平衡所执行的驱逐操作可能会选择不当。为了解决这些问题，我们来看看另一个测试程序的结果。

测试程序有一个进程持续使用SSE指令读取或写入一块2MB的内存区域。选择2MB是因为这是Core 2处理器L2缓存大小的一半。该进程被固定在一个核心上，而第二个进程则被固定在另一个核心上。这个第二进程读取和写入一个可变大小的内存区域。图表显示了每周期内读取或写入的字节数。共展示了四种不同的图表，对应于每个进程组合的读写情况。其中读/写图是针对背景进程的，该进程始终使用2MB的工作集进行写操作，而被测进程则使用可变大小的工作集进行读操作。

图中有趣的部分是在2<sup>20</sup>字节和2<sup>23</sup>字节之间。如果两个核心的L2缓存是完全独立的，我们可以预期性能会表现为

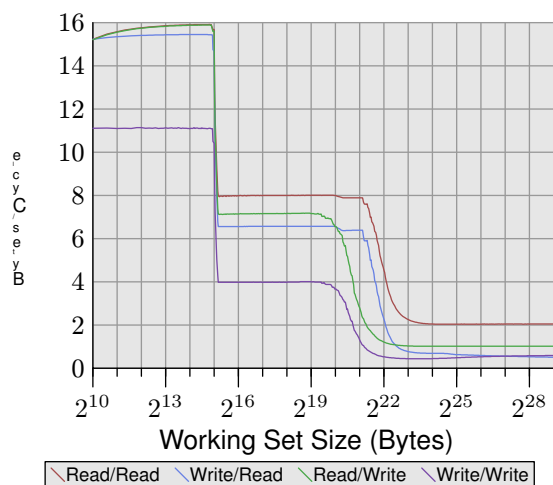


图3.31：两个进程的带宽

所有四个测试在L2缓存耗尽时都会减少2<sup>21</sup>到2<sup>22</sup>字节。如图3.31所示，实际情况并非如此。当后台进程进行写操作时，这种情况最为明显。性能开始下降发生在工作集大小达到1MB之前。两个进程不共享内存，因此不会生成RFO消息。这些都是纯粹的缓存淘汰问题。智能缓存处理存在一些问题，导致每个核心实际体验到的缓存大小更接近于1MB而不是可用的2MB。只能希望，如果跨核心共享缓存仍然是未来处理器的一个特性，用于智能缓存处理的算法将会得到修正。

拥有四个核心但配备两个L2缓存的处理器只是在更高层次缓存引入之前的权宜之计。这种设计相对于单独的插槽和双核处理器并无显著性能优势。两个核心通过同一总线进行通信，从外部来看就是前端总线（FSB），没有特别快速的数据交换通道。

多核处理器缓存设计的未来将在于更多层级的设计。AMD的10h处理器家族已经开始了这一趋势。是否会有更低级别的缓存在一部分核心之间共享仍有待观察（在2008年的处理器中，L2缓存并未被共享）。额外的缓存层次是必要的，因为高速且频繁使用的缓存无法在多个核心间共享，否则性能会受到影响，并需要非常大的缓存和高相联度。无论是缓存大小还是相联度，都必须随着共享缓存的核心数量而扩展。使用一个大容量的L3缓存以及合理尺寸的L2缓存是一个合理的权衡方案。虽然L3缓存速度较慢，但理论上其被频繁访问的程度不如L2缓存。

对于程序员而言，所有这些不同的设计在进行调度决策时意味着复杂性。在做出调度决策时，开发者需要考虑多种因素。注意，为了保持原有段落结构的简洁性和符合中文表达习惯，这里将原句进行了适当的拆分和重组。原文中的“meaning complexity when making scheduling decisions”被翻译为“在进行调度决策时意味着复杂性”，并根据中文语境添

了解工作负载和机器架构的细节以实现最佳性能。幸运的是，我们有支持来确定机器架构。接口将在后续章节中介绍。

### 3.5.4 总线影响

前端总线在机器的性能中扮演着核心角色。缓存内容只能以内存连接允许的速度进行存储和加载。我们可以通过在一个程序上运行两台仅在内存模块速度上有差异的机器来展示这一点。图3.32展示了NP AD=7在64位机上的Addnext0测试（将下一个元素pad[0]的内容加到自身的pad[0]元素）的结果。这两台机器都配备了Intel Core 2处理器，第一台使用了667 MHz DDR2内存模块，第二台则使用了800MHz的内存模块（提高了20%）。

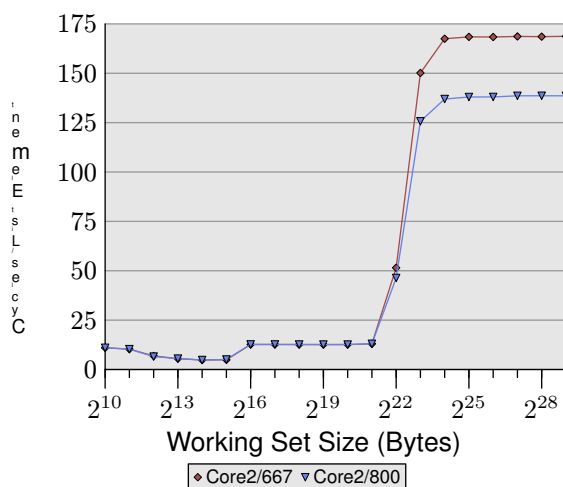


图3.32：前端总线速度的影响

这些数据表明，在大工作集大小下，FSB确实承受了很大压力，并且我们确实看到了显著的好处。在此测试中测得的最大性能提升为18.2%，接近理论最大值。这说明更快的FSB确实可以带来巨大的收益。当工作集能够完全放入缓存（这些处理器具有4MB L2缓存）时，这一点并不那么关键。需要注意的是，我们这里测量的是一个程序的表现。系统的工作集包括所有并发运行进程所需的内存。因此，即使是较小的程序也有可能轻易超过4MB或更多的内存需求。

如今，英特尔的一些处理器支持高达1333MHz的前端总线（FSB）速度，这意味着又增加了60%的速度。未来将会出现更高的速度。如果速度很重要且工作集大小较大，则快速RAM和高FSB速度绝对物有所值。不过需要注意的是，即使处理器可能支持更高的FSB速度，主板/北桥芯片组也可能不支持。因此，必须仔细检查规格说明。

## 4 虚拟内存

处理器的虚拟内存（VM）子系统实现了提供给每个进程的虚拟地址空间。这使得每个进程都认为自己是系统中唯一的。虚拟内存的优势已在其他地方详细描述，这里不再重复。相反，本节集中讨论虚拟内存子系统的实际实现细节及其相关成本。

虚拟地址空间由CPU的内存管理单元（MMU）实现。操作系统需要填充页表数据结构，但大多数CPU会自行完成其余的工作。实际上这是一个相当复杂的机制；理解它的最好方法是介绍用于描述虚拟地址空间的数据结构。

内存管理单元（MMU）执行地址翻译的输入是一个虚拟地址。通常对其值几乎没有——甚至没有——限制。在32位系统上，虚拟地址是32位值，在64位系统上则是64位值。例如，在某些系统（如x86和x86-64架构）中，实际使用的地址涉及另一层间接寻址：这些架构使用段来简单地将一个偏移量添加到每个逻辑地址上。我们可以忽略这部分地址生成过程，因为它很简单，并且对于内存处理的性能而言，程序员无需关心这一点。

### 4.1 最简单的地址转换

虚拟地址转换为物理地址的过程非常有趣。MMU可以在页级别上重新映射地址。与访问缓存行时类似，虚拟地址会被划分为不同的部分。这些部分用于索引各种表格，而这些表格则用于构建最终的物理地址。在最简单的模型中，我们只有一级表格。

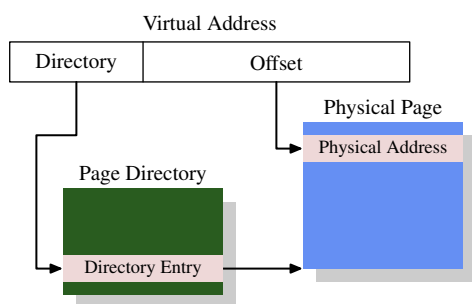


图4.1：一级地址转换

图4.1展示了虚拟地址的不同部分是如何被使用的。顶部的部分用于选择页目录中的一个条目；该目录中的每个条目可以

操作系统单独设置。页目录项确定了物理内存页面的地址；页目录中的多个条目可以指向同一个物理地址。内存单元的完整物理地址由页目录中的页地址与虚拟地址的低位部分结合确定。页目录项还包含有关该页的一些附加信息，例如访问权限。

页目录的数据结构存储在主内存中。操作系统需要分配一段连续的物理内存，并将该内存区域的基础地址存入一个特殊寄存器。然后，虚拟地址中的相应位用作页目录的索引，而页目录实际上是一个由目录项组成的数组。

对于一个具体的例子，x86机器上用于4MB页面的布局如下。虚拟地址中的偏移量部分大小为22位，足以寻址4MB页面内的每个字节。虚拟地址剩余的10位选择页目录中的一个条目（共1024个）。每个条目包含一个指向4MB页面的10位基地址，该基地址与偏移量组合形成完整的32位地址。

### 4.2 多级页表

4MB 页面并不是常态，因为许多操作系统必须执行的操作都需要与内存页面对齐，使用 4MB 页面会浪费大量内存。在 32 位机器上（以及很多情况下在 64 位机器上），标准页面大小为 4KB。在这种情况下，虚拟地址中的偏移部分只有 12 位。这使得页目录的选择器有 20 位。一个包含  $2^{20}$  个条目的表是不切实际的。即使每个条目仅为 4 字节，该表也会占用 4MB 的空间。由于每个进程可能都有自己的独立页面目录，系统中的大量物理内存将被这些页面目录占据。

使用多级页表是解决方案。这些级别形成一个巨大的、稀疏的页目录；未实际使用的地址空间区域不需要分配内存。因此，这种表示方法更加紧凑，使得可以在不影响性能的情况下将许多进程的页表保留在内存中。

今天的最复杂页表结构包含四个层级。图4.2展示了这种实现的示意图。在这个例子中，虚拟地址至少被划分为五个部分。其中四部分是各级目录的索引。第4级目录通过CPU中的特殊寄存器进行引用。从第4级到第2级目录的内容是指向下一级目录的引用。如果目录项标记为空，则显然不需要指向任何更低层级的目录。这样，页表树可以稀疏且紧凑。第1级目录条目，就像图4.1所示一样，包含部分物理地址和辅助数据（如访问权限）。

<sup>24</sup>Segment limits on x86 are performance-relevant but that is another story.

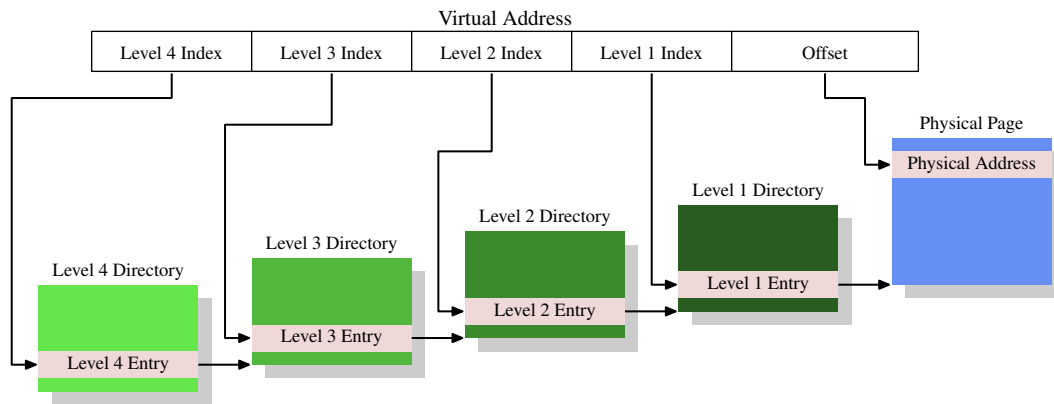


图4.2：四级地址转换

为了确定与虚拟地址对应的物理地址，处理器首先确定最高级目录的地址。该地址通常存储在一个寄存器中。然后，CPU 使用对应于该目录的虚拟地址中的索引部分，并利用此索引选择合适的条目。这个条目是下一个目录的地址，使用虚拟地址的下一部分进行索引。这一过程持续进行，直到到达一级目录，在这一点上，目录条目的值就是物理地址的高位部分。通过将虚拟地址中的页偏移位添加到该值中来完成物理地址的构建。此过程称为页树遍历。一些处理器（如 x86 和 x86-64）在硬件中执行此操作，而其他处理器则需要操作系统提供支持。

每个在系统上运行的进程可能需要自己的页表树。部分共享这些树是有可能的，但这较为罕见。因此，为了性能和可扩展性，页表树所需的内存应尽可能小。理想情况下，使用的虚拟地址空间中的内存应该紧密相邻；实际使用的物理地址并不重要。一个小型程序可能只需要在第2、3、4级各使用一个目录，并且一级目录中只需几个即可运行。对于采用x86-64架构、每页大小为4KB且每个目录包含512个条目的系统，仅通过4个目录（每一级一个）就可以寻址到2MB的内存空间；而连续的1GB内存则可以通过第2至第4级各使用一个目录以及一级使用512个目录来实现寻址。

假设内存可以连续分配是过于简单化的。出于灵活性的考虑，进程的栈和堆区域通常被分配在地址空间的几乎相反两端。这使得任一区域都可以根据需要尽可能地扩展。这意味着很可能需要两个二级目录，并且相应地需要更多的低级目录。

但即使这样也并不总是与当前实践相符。出于安全原因，可执行文件的各个部分（代码、数据、堆、栈、动态共享对象（DSO）等）的情况有所不同。

共享库以随机地址进行映射[9]。这种随机化扩展到了各个部分之间的相对位置；这意味着进程使用的各种内存区域在整个虚拟地址空间中分布广泛。通过限制随机化的地址位数，可以将范围缩小，但在大多数情况下，这并不会允许一个进程仅使用一两个目录来运行第二级和第三级缓存。注：原文中的“directories”在上下文中可能指的是页表或段表的层级结构，但为了保持术语精准并符合中文习惯，在这里翻译为“目录”可能会引起歧义，因此建议根据具体技术背景进行调整或进一步明确其含义。

如果性能确实比安全更重要，可以关闭随机化功能。此时，操作系统通常会将所有DSO在虚拟内存中连续加载。

### 4.3 优化页表访问

所有页表的数据结构都保存在主内存中；操作系统在此构建和更新这些表格。当创建一个进程或更改一页表时，CPU会收到通知。使用上述描述的页表行走过程，页表用于将每个虚拟地址解析为物理地址。更具体地说：在解析虚拟地址的过程中，至少需要每一级目录中的一个条目。这最多需要四次内存访问（对于运行中的进程的一次访问而言），速度较慢。可以将这些目录表项视为普通数据并将其缓存到L1d、L2等缓存中，但这仍然会太慢。

从虚拟内存早期开始，CPU设计者就采用了一种不同的优化方法。简单的计算表明，仅将目录表项保留在L1d和更高层次的缓存中会导致极差的性能。每个绝对地址计算都需要进行与页表深度相对应数量的L1d访问。由于这些访问依赖于前一次查找的结果，因此无法并行化处理。在具有四级页表结构的机器上，仅此一项就需要至少12个周期。再加上L1d未命中概率的影响，结果是流水线根本无法掩盖这种延迟。额外的L1d访问还会抢占宝贵的缓存带宽。

因此，不仅仅是缓存目录表项，物理页地址的完整计算也被缓存起来。由于代码和数据缓存有效的原因，这种缓存地址计算同样有效。由于虚拟地址中的页面偏移部分在计算物理页地址时不起任何作用，只有虚拟地址的其余部分被用作缓存标记。根据页面大小的不同，这意味着成百或上千条指令或数据对象会共享相同的标记，并因此具有相同的物理地址前缀。

存储计算值的缓存称为转换后备缓冲区（TLB）。由于需要极高的速度，它通常是一个小缓存。现代CPU提供了多级TLB缓存，就像其他缓存一样；较高层次的缓存更大但速度较慢。为了弥补L1TLB的小容量，常常采用全相联结构，并使用最近最少使用（LRU）淘汰策略。近年来，该缓存的大小在增加，在此过程中被改为组相联结构。因此，在需要添加新条目时，被淘汰替换的可能不再是最老的条目。

如上所述，用于访问快表的标签是虚拟地址的一部分。如果标签在缓存中匹配成功，则通过将虚拟地址中的页偏移量与缓存值相加来计算最终的物理地址。这是一个非常快速的过程；必须如此，因为对于使用绝对地址的每条指令以及某些情况下使用物理地址作为键进行L2查找的情况，都需要有可用的物理地址。如果TLB查找未命中，则处理器需要执行页表遍历操作；这可能会相当耗时。

预取代码或数据可以通过软件或硬件隐式地为快表（TLB）预取条目，前提是地址位于另一个页面上。由于硬件可能会发起无效的页表遍历操作，因此不允许通过硬件进行此类预取。程序员不能依赖硬件预取来为 TLB 条目进行预取，必须显式使用预取指令完成这一任务。与数据和指令缓存一样，TLB 也可以出现在多个级别中。同样地，TLB 通常有两种形式：指令 TLB（ITLB）和数据 TLB（DTLB）。较高级别的 TLB 如 L2TLB 通常是统一的，就像其他缓存一样。

#### 4.3.1 使用快表时的注意事项

快表（TLB）是处理器核心的全局资源。运行在该处理器核心上的所有线程和进程共用同一个TLB。由于虚拟地址到物理地址的转换依赖于所安装的页表树，因此当页表发生变化时，CPU不能盲目地重用缓存条目。每个进程都有不同的页表树（但同一进程内的各个线程除外），内核以及如果存在的话虚拟机管理程序（hypervisor）也是如此。

进程的地址空间布局也可能发生变化。解决这个问题有以下两种方法：

- 每当页表树发生变化时，快表会被冲洗。
- 快表条目的标签被扩展，以唯一标识它们所指的页表树。

在第一种情况下，每当执行上下文切换时都会清空快表（TLB）。由于大多数操作系统中，从一个线程或进程切换到另一个通常需要执行一些内核代码，因此 TLB 清空仅限于离开（有时也包括进入）内核地址空间。在虚拟化系统中，当内核必须调用 VMM 时也会发生这种情况，在返回路径上同样如此。如果内核和/或 VMM 不必使用虚拟地址，或者可以重用与发起系统/VMM 调用的进程或内核相同的虚拟地址（即，地址空间重叠），则只有在处理器从离开内核或 VMM 后恢复执行不同的进程或内核时才需要清空 TLB。

清空快表（TLB）虽然有效但代价高昂。例如，在执行系统调用时，内核代码可能仅包含几千条指令，并且这些指令可能会触及少数几页新内存（或者在某些架构下，如Linux中的大页面）。这种情况下，只会替换与触及的页面数量相等的快表项。对于Intel Core2架构而言，其ITLB和DTLB分别有128个和256个条目，在进行完全清空时，可能会无谓地清除超过100个和200个条目（分别为）。当系统调用返回到同一进程时，所有这些被清空的快表项可以再次使用，但它们已经不存在了。同样的情况也适用于内核或VMM中经常使用的代码。每次进入内核时，即使内核和VMM的页表通常不会改变，因此理论上可以长时间保留快表条目，但仍需从头开始填充快表。这也解释了为什么当今处理器中的TLB缓存没有更大：程序运行时间很可能不足以填满所有这些条目。

这一事实当然没有逃过 CPU 架构师的眼睛。优化缓存刷新的一种可能方法是个别失效快表条目。例如，如果内核代码和数据位于特定的地址范围内，则只需将该地址范围内的页面从快表中移除。这只需要比较标签信息，并且因此成本不高。这种方法在地址空间的一部分发生变化时也很有用，例如通过调用 `munmap` 引起的变化。

一个更好的解决方案是扩展用于TLB访问的标签。除了虚拟地址的一部分之外，如果为每个页表树（即，进程的地址空间）添加一个唯一的标识符，则TLB无需完全清空。内核、VMM和

各个进程都可以拥有唯一的标识符。该方案的唯一问题是可用作快表标签的位数非常有限，而地址空间的数量却不受限制。这意味着必须重复使用某些标识符。当这种情况发生时，如果可能的话，TLB需要部分清空。所有具有重复使用的标识符的条目都必须被清空，但希望这只是一个较小的集合。

扩展的快表标签在外围虚拟化领域也具有优势，当系统上运行多个进程时尤为明显。如果每个可运行进程的内存使用量（以及相应的快表条目使用量）受到限制，则该进程最近使用的快表条目在再次调度时很可能仍然存在于快表中。此外还有两个额外的优势：

特殊地址空间，如内核和虚拟内存管理器使用的地址空间，通常只在短时间内被访问；之后控制权往往返回到最初发起进入该地址空间的地址空间。如果没有标签，通常只会执行一两次的快表刷新操作。有了标签后，调用地址空间中的缓存翻译可以得到保留，并且由于内核和虚拟内存管理器的地址空间很少更改快表条目，因此之前的系统调用等所使用的翻译仍然可以继续使用。

2. 在同一进程的两个线程之间切换时，完全不需要进行快表刷新。如果没有扩展的快表标签，在进入内核模式时会破坏第一个线程的快表条目。

一些处理器已经实现了一段时间的扩展标签。AMD在Pacifica虚拟化扩展中引入了1位标签扩展，即地址空间标识符（ASID）。在这种虚拟化的上下文中，该1位ASID用于区分VMM的地址空间与客户域的地址空间。这使得操作系统能够在进入VMM（例如处理缺页异常时）或控制返回到客户机时避免清空客户的快表条目和VMM的快表条目。未来架构将允许使用更多位数，其他主流处理器也可能效仿并支持此功能。

#### 4.3.2 影响快表性能的因素

影响快表（TLB）性能的因素有几个。首先是页面大小。显然，页面越大，可以容纳的指令或数据对象就越多。因此，较大的页面尺寸减少了整体地址转换的数量，意味着所需使用的快表缓存条目也更少。如今大多数架构都允许使用多种不同的页面大小；某些大小可以在同一时间并发使用。例如，x86/x86-64处理器的标准页面大小为4kB，但它们也可以分别使用4MB和2MB的页面。IA-64和PowerPC则支持以64kB作为基础页面大小。

使用大页大小会带来一些问题。用于大页的内存区域在物理内存中必须是连续的。如果将物理内存管理的单位大小提升到虚拟内存页面的大小，那么浪费的内存将会增加。所有类型的内存操作（如加载可执行文件）都需要对齐到页面边界。这意味着，在平均情况下，每个映射都会在物理内存中浪费半个页面大小的空间。这种浪费很容易累积起来，因此它为合理的物理内存分配单位大小设定了上限。

增加x86-64系统中大页的单元大小到2MB显然是不切实际的，这个尺寸过大。但这又意味着每个大页必须由许多小页组成，并且这些小页在虚拟内存中必须是连续的。以每页4KB的单位页面大小分配2MB的连续物理内存是非常具有挑战性的，需要找到一个包含512个连续页面的空闲区域。系统运行一段时间后，随着物理内存变得碎片化，这将极其困难（甚至不可能）。

在Linux系统中，必须使用特殊的hugetlbfs文件系统在系统启动时分配这些大页。固定数量的物理页面被预留用于专用于大虚拟页面。这会占用可能不会始终使用的资源。此外，这是一个有限的池；增加其大小通常意味着需要重启系统。然而，在性能至关重要的情况下、资源充足且复杂的设置不是主要障碍的情况下，使用大页是最佳选择。数据库服务器就是一个例子。

增加最小虚拟页大小（而不是可选的大页面）也会带来问题。内存映射操作（例如加载应用程序）必须符合这些页面大小的要求，无法进行更小的映射。对于大多数架构而言，可执行文件各部分的位置具有固定的关联关系。如果页面大小超过了构建可执行文件或动态共享对象时所考虑的范围，则加载操作将无法完成。这一点需要特别注意。4.3展示了如何确定ELF二进制文件的对齐要求。这些信息编码在ELF程序头中。在这个例子中，对于一个x86-64二进制文件，值为 $200000_{16} = 2,097,152 = 2\text{MB}$ ，这对应于处理器支持的最大页面大小。

使用更大的页面大小还会产生另一个效果：页表树的层次数量减少。由于虚拟地址中对应于页面偏移的部分增加，留给页目录处理的位数就减少了。这意味着，在发生快表未命中时，需要完成的工作量会减小。

除了使用大页大小之外，还可以通过以下方式减少

```
$ eu-readelf -l /bin/ls
Program Headers:
 Type Offset VirtAddr PhysAddr FileSiz MemSiz Flg Align
...
LOAD 0x000000 0x0000000000400000 0x0000000000400000 0x0132ac 0x0132ac R E 0x200000
LOAD 0x0132b0 0x00000000006132b0 0x00000000006132b0 0x001a71 0x001a71 RW 0x200000
...
```

图4.3: ELF程序头指示的对齐要求

通过将同时使用的数据移动到较少的页面上来减少所需的快表条目数量。这与我们上面讨论的一些缓存优化类似。只不过现在所需对齐的数量较大。鉴于快表条目的数量相当有限，这种优化可能非常重要。

#### 4.4 虚拟化的影响

操作系统镜像的虚拟化将越来越普遍；这意味着内存管理又增加了一层。进程的虚拟化（基本上是 jails）或操作系统容器不在此列，因为只涉及一个操作系统。技术如 Xen 或 KVM 可以在处理器的支持下或无需处理器支持的情况下执行独立的操作系统映像。在这种情况下，只有一个软件直接控制对物理内存的访问。

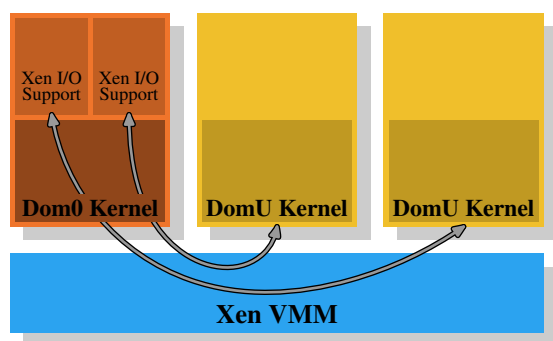


图4.4: Xen虚拟化模型

在 Xen（见图4.4）的情况下，Xen VMM就是那部分软件。然而，VMM 并没有实现许多其他硬件控制功能。与早期系统的 VMM（以及 Xen VMM 的第一个版本）不同，内存和处理器之外的硬件由特权 Dom0 域进行管理。目前，这基本上是与非特权 DomU 内核相同的内核，并且在内存处理方面它们并无二致。重要的是，VMM 将物理内存分配给 Dom0 和 DomU 内核，这些内核随后会像直接运行于处理器上一样实现常规的内存管理功能。

为了实现虚拟化所需的域隔离，内存管理需要：

在 Dom0 和 DomU 内核中，内存管理并不赋予其不受限制的物理内存访问权限。VMM 不会通过分配单独的物理页并让客操作系统处理地址映射的方式来分发内存；这种方式无法防止因故障或恶意行为导致的安全风险。相反，VMM 为每个客域创建自己的页表树，并使用这些数据结构来分配内存。好处在于可以控制对页表树管理信息的访问权限：如果没有适当的权限，代码将无法执行任何操作。

这种访问控制在 Xen 提供的虚拟化中被利用，无论是半虚拟化还是硬件（又称全）虚拟化。来宾域为其每个进程构建页面表树的方式，在半虚拟化和硬件虚拟化之间故意非常相似。每当来宾操作系统修改其页面表时，VMM 会被调用。然后 VMM 使用来宾域中的更新信息来更新自己的影子页表。这些是实际由硬件使用的页表。显然，这个过程相当昂贵：每次对页面表树的修改都需要调用一次 VMM。虽然在没有虚拟化的情况下更改内存映射并不便宜，但现在变得更加昂贵了。

额外的成本可能非常高，考虑到从客户操作系统到虚拟机管理程序（VMM）再到客户操作系统的转换本身已经非常昂贵。这就是为什么处理器开始具备避免创建影子页表的附加功能的原因。这不仅是因为速度问题，还因为减少了 VMM 的内存消耗。英特尔推出了扩展页表（EPTs），而 AMD 则称之为嵌套页表（NPTs）。这两种技术的基本原理是让客户操作系统的页表从“客户虚拟地址”生成“主机虚拟地址”。然后必须使用每个域特有的 EPT/NPT 树，将这些主机虚拟地址进一步转换为实际的物理地址。这使得内存处理速度接近无虚拟化情况下的速度，因为大多数用于内存处理的 VMM 条目都被移除了。这也减少了 VMM 的内存使用量，因为现在只需要为主机中的每个域（而不是每个进程）维护一个页表树。

额外的地址转换步骤的结果也存储在快表中。这意味着快表并不存储虚拟物理地址，而是存储查找操作的完整结果。此前已经解释过这一点。

AMD的Pacifica扩展引入了地址空间标识符（ASID），以避免每次进入时快表（TLB）的刷新。初始发布的处理器扩展中，ASID占用1位，这足以区分虚拟机管理程序和客户操作系统。Intel则使用虚拟处理器ID（VPIDs）来实现相同的目的，只是它们的数量更多。但是，每个来宾域中的VPID是固定的，因此无法用于标记不同的进程并在该级别避免TLB刷新。

虚拟化操作系统中每个地址空间修改所需的工作量是一个问题。然而，基于VMM的虚拟化还存在另一个固有问题：无法避免内存处理的两层架构。但内存管理本身就很复杂（尤其是在考虑如NUMA这样的复杂情况时，请参见第5节）。Xen采用单独的VMM方法使得最优（甚至良好的）内存处理变得困难，因为所有内存管理实现中的复杂性，包括“简单”的事情如内存区域的发现等，都必须在VMM中重复实现。操作系统拥有完整且优化的实现；我们真的希望避免重复这些工作。

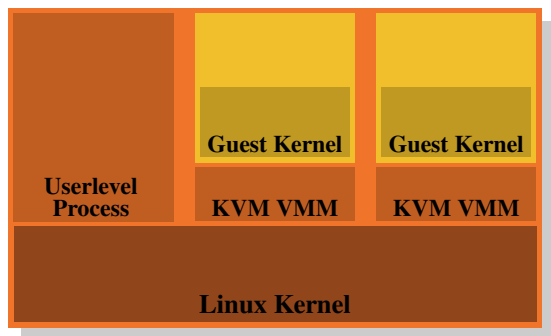


图4.5：KVM虚拟化模型

这就是为什么将VMM/Dom0模型推向极致成为如此有吸引力的替代方案。图4.5展示了KVM Linux内核扩展如何尝试解决这一问题。没有单独运行在硬件上的VMM来控制所有客户机；相反，一个普通的Linux内核接管了这项功能。这意味着Linux内核中完整且复杂的内存管理功能被用来管理系统内存。客户域与普通用户级进程一起运行，在创建者所说的“客户模式”下工作。虚拟化功能（半虚拟化或全虚拟化）由KVM VMM控制，这只是另一个使用内核实现的特殊KVM设备来控制客户机的用户级进程。

这种模型相比Xen模型的独立VMM而言，其优势在于：尽管在使用客操作系统时仍然存在两个内存管理器，但只需要一个实现即可，即Linux内核中的实现。无需再在类似Xen VMM这样的另一段代码中重复同样的功能。

这将导致更少的工作量、更少的 bug，以及可能减少内存管理器在两个层次上交互时的摩擦，因为 Linux 客户机中的内存管理器与运行在裸硬件上的外层 Linux 内核中的内存管理器做出了相同的假设。

总体而言，程序员必须意识到，在使用虚拟化的情况下，缓存未命中的成本（指令、数据或 TLB 缺失）甚至高于没有虚拟化时的成本。任何能够减少这种开销的优化措施在虚拟化环境中将更加有效。随着时间的推移，处理器设计者会通过 EPT 和 NPT 等技术逐渐减小这一差异，但这种差距永远不会完全消失。

## 5 NUMA支持

在第2节中，我们看到，在某些机器上，访问物理内存的特定区域的成本取决于访问的来源。这种硬件需要操作系统和应用程序特别小心处理。我们将首先介绍一些NUMA硬件的细节，然后讨论Linux内核为NUMA提供的部分支持。

### 5.1 NUMA硬件

非均匀内存架构越来越普遍。在最简单的NUMA形式中，处理器可以拥有本地内存（见图2.3），访问成本低于其他处理器的本地内存。这种类型的NUMA系统的成本差异不大，即NUMA因子较低。

非统一内存访问（NUMA）也特别用于大型机器。我们已经描述了多个处理器访问同一内存时的问题。对于通用硬件，所有处理器都会共享同一个北桥芯片（暂不考虑AMD Opteron NUMA节点，它们有自己的问题）。这使得北桥成为了一个严重的瓶颈，因为所有的内存流量都必须通过它进行路由。当然，大型机器可以使用自定义硬件来替代北桥芯片，但是除非所使用的内存芯片具有多个端口——即可以从多个总线上访问——否则仍然存在瓶颈。多端口RAM复杂且昂贵，因此几乎从未被采用。

在更高的复杂度层次上，AMD使用的模型中，通过一种互连机制（AMD的情况是Hyper Transport技术，该技术是从Digital许可而来的）为未直接连接到RAM的处理器提供访问。除非愿意任意增加直径（即任意两个节点之间的最大距离），否则以这种方式形成的结构大小会受到限制。

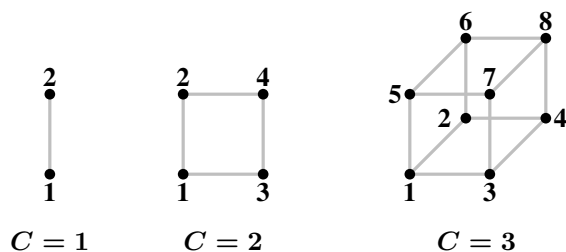


图5.1: 超立方体

一种高效的连接节点拓扑是超立方体，它将节点数量限制为 $2^C$ （其中每个节点具有 $C$ 个互连接口）。在拥有 $2^C$ 颗CPU和 $C$ 条互连线路的所有系统中，超立方体的直径最小。图5.1展示了前三个超立方体。每个超立方体的直径为

AMD的第一代Opteron处理器每个处理器有三条超传输链路。至少有一个处理器需要通过一条链路连接南桥芯片，这意味着当前可以直接且高效地实现顶点数为 $C=2$ 的超立方体结构。下一代产品将拥有四条链路，届时将能够支持顶点数为 $C=3$ 的超立方体结构。

这并不意味着更大的处理器集合无法被支持。有些公司开发了交换单元，允许使用更大规模的处理器集合（例如，Newisys的Horus）。但是，这些交换单元会增加NUMA因子，并且在达到一定数量的处理器后效果会逐渐减弱。

下一步是将多个CPU组连接起来，并为它们实现共享内存。所有此类系统都需要专门的硬件，绝非普通的商品化系统。这种设计在不同的复杂度层次上存在多种形式。一种仍然接近普通机器的设计是IBM x445及其类似机型。这些机器可以作为常规的4U、8路服务器购买，配备x86和x86-64处理器。然后，两台（最多四台）这样的机器可以通过连接工作为具有共享内存的一台大型机。所使用的互连引入了显著的NUMA因素，操作系统以及应用程序都必须予以考虑。

在另一端，像SGI的Altix这样的机器专门设计用于互连。SGI的NUMalink互连结构非常快，并且延迟很低；这两个特性都是高性能计算（HPC）的要求，特别是在使用消息传递接口（MPI）时。缺点当然是这种复杂性和专业性成本非常高昂。它们使得相对较低的NUMA因子成为可能，但由于这些机器可以拥有的CPU数量（数千个）以及互连容量有限，NUMA因子实际上是动态变化的，并且根据工作负载的不同可能会达到不可接受的水平。

更常见的是使用高速网络将许多商品机器连接起来形成集群的解决方案。这些并不是NUMA机器；它们没有实现共享地址空间，因此不属于本文讨论的任何一类。

### 5.2 操作系统对NUMA的支持

为了支持非统一内存访问（NUMA）机器，操作系统必须考虑内存的分布特性。例如，如果一个进程在某个处理器上运行，则该进程地址空间分配的物理RAM应理想情况下来自本地内存。否则，每条指令都需要访问远程内存以获取代码和数据。还有一些特殊情况仅存在于NUMA机器中需要特别注意。动态共享对象（DSO）的文本段通常在一个机器中只存在一次。

物理RAM。但如果DSO被所有CPU上的进程和线程使用（例如基本运行时库如libc），这意味着除了少数处理器外，其他所有处理器都需要进行远程访问。操作系统理想情况下会将此类DSO“镜像”到每个处理器的物理内存中，并使用本地副本。这是一种优化措施，而非必要要求，通常难以实现，并且可能不被支持或仅在有限程度上支持。

为了避免情况恶化，操作系统不应将进程或线程从一个节点迁移到另一个节点。在普通的多处理器机器上，操作系统已经应尽量避免迁移进程，因为从一个处理器迁移到另一个处理器意味着缓存内容会丢失。如果需要通过迁移进程或线程来平衡负载，则操作系统通常可以选择任意一个新的具有足够容量的处理器。但在NUMA环境中，新处理器的选择受到一定限制。新的目标处理器不应比旧处理器访问该进程使用的内存成本更高；这缩小了可选的目标列表。如果没有符合这一标准的空闲处理器可用，操作系统别无选择，只能迁移到一个内存访问成本更高的处理器上。

在这种情况下有两种可能的前进方式。首先，可以期望这种情况是暂时的，并希望该进程能够迁移到更适合的处理器上运行。或者，操作系统也可以将该进程的内存迁移至更接近新使用的处理器的物理页面中。这是一个相当昂贵的操作。可能需要复制大量的内存，尽管不一定是一次性完成。在此过程中，至少在短时间内，必须暂停该进程以确保对旧页的修改能够正确地迁移到新的位置。为了使页面迁移高效快速，还有一系列其他要求。简而言之，除非绝对必要，否则操作系统应尽量避免进行此类操作。

通常不能假设NUMA机器上的所有进程使用相同数量的内存，因此，在处理器之间分配进程时，内存使用量也不会均匀分布。实际上，除非运行在这些机器上的应用程序非常特殊（这在高性能计算领域较为常见，但在其他领域则不然），否则内存使用情况会非常不均衡。有些应用将占用大量内存，而另一些应用几乎不会使用内存。如果总是将内存分配到发起请求的处理器本地，则迟早会导致问题，因为运行大型进程的节点最终可能会耗尽其本地内存。

为了应对这些问题，内存默认情况下不会独占分配在本地节点上。为了充分利用整个系统的内存，默认策略是将内存进行条带化。这保证了系统中所有内存的均衡使用。作为副作用，由于访问所有使用的内存的平均成本不变，因此可以在处理器之间自由迁移进程。对于较小的NUMA因子，条带化是可以接受的，但仍然不是最优的（参见第5.4节的数据）。

这是一项优化措施，有助于系统避免严重问题，并在正常运行时使其更加可预测。但这也降低了整体系统的性能，在某些情况下会显著降低。这就是为什么Linux允许每个进程选择内存分配规则的原因。一个进程可以选择不同的策略用于自身及其子进程。我们将在第6节介绍可用于此目的的接口。

### 5.3 已发布信息

内核通过 sys 虚拟文件系统（sysfs）发布有关处理器缓存的信息。

`/sys/devices/system/cpu/cpu*/缓存`

在第6.2.1节中，我们将看到可以用来查询各种缓存大小的接口。这里重要的是缓存的拓扑结构。上述目录包含子目录（命名为index\*），这些子目录列出了CPU所拥有的各种缓存的信息。就拓扑结构而言，type、level和shared\_cpu\_map文件是重要的文件。对于Intel Core 2 QX6700处理器，信息如表5.1所示。

		type	level	shared_cpu_map
cpu0	index0	Data	1	00000001
	index1	Instruction	1	00000001
	index2	Unified	2	00000003
cpu1	index0	Data	1	00000002
	index1	Instruction	1	00000002
	index2	Unified	2	00000003
cpu2	index0	Data	1	00000004
	index1	Instruction	1	00000004
	index2	Unified	2	0000000c
cpu3	index0	Data	1	00000008
	index1	Instruction	1	00000008
	index2	Unified	2	0000000c

表5.1：Core 2 CPU缓存的sysfs信息

这段数据的含义如下：

- 每个核心都配备了三个缓存：L1i、L1d 和 L2。
- L1数据缓存和L1指令缓存不与其他核心共享，每个核心都有自己的缓存组。这可以通过共享的shared\_cpu\_map位图中只有一个位置被设置为1来表示。
- cpu0 和 cpu1 的 L2 缓存是共享的，同理，cpu2 和 cpu3 的 L2 缓存也是共享的。

如果CPU有更多级缓存，就会有更多的索引目录。

对于四插槽、双核Opteron机器，缓存信息如表5.2所示。可以看出这些

知识来源将在稍后说明，这里可以确定cpu0到cpu3是核心处理器。

		type	level	shared_cpu_map
cpu0	index0	Data	1	00000001
	index1	Instruction	1	00000001
	index2	Unified	2	00000001
cpu1	index0	Data	1	00000002
	index1	Instruction	1	00000002
	index2	Unified	2	00000002
cpu2	index0	Data	1	00000004
	index1	Instruction	1	00000004
	index2	Unified	2	00000004
cpu3	index0	Data	1	00000008
	index1	Instruction	1	00000008
	index2	Unified	2	00000008
cpu4	index0	Data	1	00000010
	index1	Instruction	1	00000010
	index2	Unified	2	00000010
cpu5	index0	Data	1	00000020
	index1	Instruction	1	00000020
	index2	Unified	2	00000020
cpu6	index0	Data	1	00000040
	index1	Instruction	1	00000040
	index2	Unified	2	00000040
cpu7	index0	Data	1	00000080
	index1	Instruction	1	00000080
	index2	Unified	2	00000080

表5.2: Opteron CPU缓存的sysfs信息

处理器还配备了三个缓存: L1i、L1d 和 L2。核心之间不共享任何级别的缓存。该系统有趣的部分在于处理器拓扑结构。没有这些额外信息,就无法理解缓存数据。sys 文件系统通过以下文件公开了这些信息。

/sys/devices/system/cpu/cpu\*/拓扑结构

表5.3展示了在此SMP Opteron机器的层级结构中有趣的文件。

	physical_package_id	core_id	core_siblings	thread_siblings
cpu0	0	0	00000003	00000001
cpu1		1	00000003	00000002
cpu2	1	0	0000000c	00000004
cpu3		1	0000000c	00000008
cpu4	2	0	00000030	00000010
cpu5		1	00000030	00000020
cpu6	3	0	000000c0	00000040
cpu7		1	000000c0	00000080

表5.3: Opteron CPU拓扑的sysfs信息

结合表5.2和表5.3可以看出

没有任何一个CPU启用了超线程(每个线程兄弟位图中只有一位被设置)。

系统实际上共有四个处理器。

(物理包标识 0 到 3),

每个处理器具有两个核心, 并且

d) 没有任何核心共享缓存。

这正好对应了早期的Opteron处理器。

截至目前提供的数据中, 完全缺失了关于该机器NUMA架构性质的信息。任何SMP Opteron机器都是一种NUMA机器。对于这些数据, 我们需要查看sys文件系统中的另一个部分, 这部分存在于NUMA机器的层次结构下。

/sys/devices/system/节点

此目录包含系统中每个NUMA节点的子目录。在特定节点的目录下有一系列文件。表5.4展示了之前两张表格中描述的Opteron机器的重要文件及其内容。

	cpumap	distance
node0	00000003	10 20 20 20
node1	0000000c	20 10 20 20
node2	00000030	20 20 10 20
node3	000000c0	20 20 20 10

表5.4: Opteron节点的sysfs信息

这一信息将所有内容串联起来; 现在我们对这台机器的架构有了完整的了解。我们知道该机器有四个处理器, 每个处理器构成自己的节点, 这一点可以从节点目录下的cpumap文件中设置的位值看出。这些目录中的距离文件包含一组数值, 每一组对应一个节点, 表示在相应节点上内存访问的成本。在这个例子中, 所有本地内存访问的成本为10, 而对任何其他节点的所有远程访问成本为20。这意味着, 尽管处理器被组织成二维超立方体(参见图5.1), 但那些不直接相连的处理器之间的访问并不更昂贵。这些成本值的相对大小可以作为实际访问时间差异的一个估计。所有这些信息的准确性则是另一个问题。

## 5.4 远程访问成本

距离是相关的。在文献[1]中, AMD记录了四插槽机器的NUMA成本。对于写操作, 数字如图5.3所示。Writes

<sup>26</sup>This is, by the way, incorrect. The ACPI information is apparently wrong since, although the processors used have three coherent HyperTransport links, at least one processor must be connected to a Southbridge. At least one pair of nodes must therefore have a larger distance.

```

00400000 默认文件=/bin/cat 映射=3 N3=3 00504000 默认文件=/bin/cat 匿名=1 脏页=1 映射=2 N3=2 00506000 默认堆
匿名=3 脏页=3 活跃=0 N3=3 38a9000000 默认文件=/lib64/ld-2.4.so 映射=22 mapmax=47 N1=22 38a9119000 默认文件=/
lib64/ld-2.4.so 匿名=1 脏页=1 N3=1 38a911a000 默认文件=/lib64/ld-2.4.so 匿名=1 脏页=1 N3=1 38a9200000 默认文件=/l
ib64/libc-2.4.so 映射=53 mapmax=52 N1=51 N2=2 38a933f000 默认文件=/lib64/libc-2.4.so 38a943f000 默认文件=/lib64/li
bc-2.4.so 匿名=1 脏页=1 映射=3 mapmax=32 N1=2 N3=1 38a9443000 默认文件=/lib64/libc-2.4.so 匿名=1 脏页=1 N3=1 3
8a9444000 默认 匿名=4 脏页=4 活跃=0 N3=4 2b2bbcdce000 默认 匿名=1 脏页=1 N3=1 2b2bbcdce4000 默认 匿名=2 脏页
=2 N3=2 2b2bbcdce6000 默认文件=/usr/lib/locale/locale-archive 映射=11 mapmax=8 N0=11 7ffedcc7000 默认栈 匿名=2
脏页=2 N3=2

```

图5.2: /proc/PID/numa\_maps的内容

相比读取操作，写入操作更慢，这并不令人惊讶。有趣的部分是1跳和2跳情况的成本。实际上，两种1跳的情况成本略有不同。详情请参见[1]。我们需要记住的是，与0跳读取相比，2跳读取和写入分别慢30%和49%。2跳写入比0跳写入慢32%，比1跳写入慢17%。处理器节点和内存节点之间的相对位置会有很大影响。AMD下一代处理器将每个处理器配备四个一致性的HyperTransport链路。在这种情况下，四插槽机器的直径为一。当扩展到八插槽时，同样的问题会更加严重，因为具有八个节点的超立方体的直径是三。

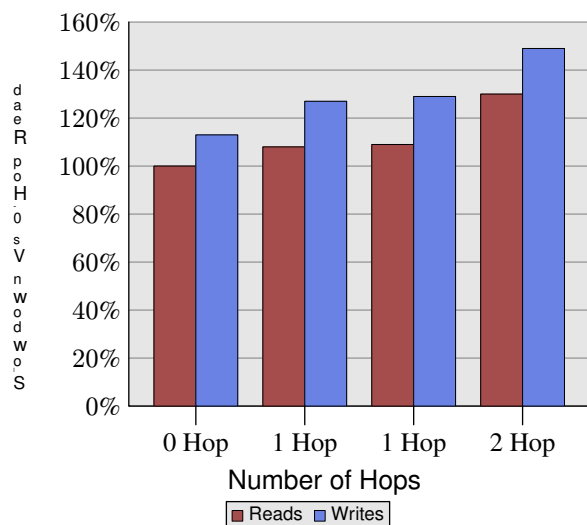


图5.3: 多节点读写性能

所有这些信息都是可用的，但使用起来较为繁琐。在第6.5节中，我们将介绍一个接口，以便更轻松地访问和利用这些信息。

系统提供的最后一项信息是：在

进程自身的状态。可以确定内存映射文件、Copy-On-Write (COW)<sup>27</sup> 页面和匿名内存在这系统的各个节点上的分布情况。对于每个进程，内核提供一个伪文件 /proc/PID/numa\_maps，其中 PID 是该进程的 ID，如图 5.2 所示。文件中的重要信息是 N0 到 N3 的值，这些值表示在节点 0 至 3 上为内存区域分配的页面数量。可以推测程序是在节点 3 的某个核心上执行的。程序本身及其被修改过的页面都分配在这个节点上。只读映射，例如 ld-2.4.so 和 libc-2.4.so 的第一个映射以及共享文件 locale-archive，则分配在其他节点上。

如图5.3所示，跨节点进行读取操作时，1跳和2跳读取的性能分别下降了9%和30%。在执行过程中，此类读取是必需的，并且如果未命中L2缓存，则每个缓存行都会产生这些额外开销。对于超出缓存大小的的大工作负载而言，如果内存远离处理器，则所有测量的成本都需要增加9%/30%。

为了在实际环境中观察其效果，我们可以按照第3.5.1节所述的方法测量带宽，但这次内存位于一个远程节点上，距离为一跳。将此次测试结果与使用本地内存的数据进行比较，可以参见图5.4。数据中存在一些明显的双向峰值，这是多线程代码测量中的问题，可以忽略不计。图表中重要的信息是读操作始终慢20%。这比图5.3中的9%要慢得多，后者很可能是未中断的读写操作的数据，并且可能指的是较早版本的处理器。

<sup>27</sup>Copy-On-Write is a method often used in OS implementations when a memory page has one user at first and then has to be copied to allow independent users. In many situations the copying is unnecessary, at all or at first, in which case it makes sense to only copy when either user modifies the memory. The operating system intercepts the write operation, duplicates the memory page, and then allows the write instruction to proceed.

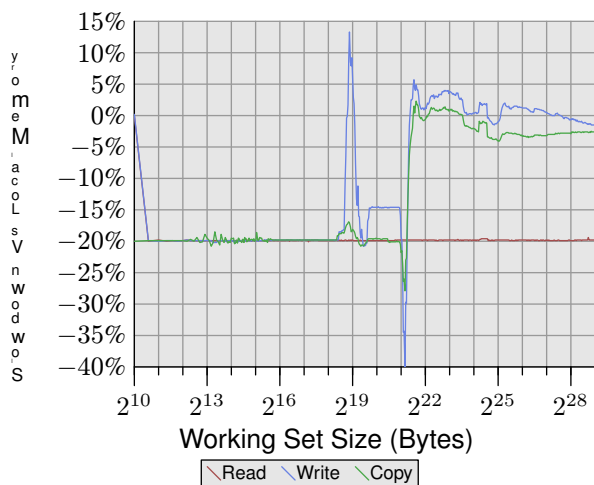


图5.4：操作远程内存

AMD知情。

对于工作集大小能够完全放入缓存的情况，写入和复制操作的性能也慢了20%。当工作集超过缓存大小时，写入操作的性能与本地节点上的操作相比没有明显差异。互连的速度足够快，可以跟上内存的速度。主要的影响因素是等待主内存的时间。

## 6 编程人员可以做什么

经过前几节的描述，很明显程序员可以通过各种方式影响程序的性能，无论是正面还是负面。而这里仅涉及与内存相关的操作。我们将从底层开始介绍这些机会，包括物理RAM访问和L1缓存的最低级别，一直到操作系统功能对内存处理的影响。

### 6.1 跳过缓存

当数据生成后不再（立即）被消费时，内存存储操作首先读取整个缓存行然后再修改缓存中的数据这一事实会对性能产生负面影响。此操作会将可能再次需要的数据驱逐出缓存，以腾出空间给即将不会使用不久的数据。对于大型数据结构，如矩阵，在其最后一个元素填充之前，由于规模庞大，会导致最先填入的元素被驱逐出去，使得写入缓存变得无效。

对于此类情况，处理器提供了对`non-temporal`写操作的支持。此处的“非临时性”意味着数据不久后不会被重新使用，因此没有理由将其缓存。这些非临时性写操作不会先读取缓存行再进行修改；相反，新内容会直接写入内存。

这听起来可能成本较高，但实际上并不一定如此。处理器会尝试使用写组合（参见3.3.3节）来填充整个缓存行。如果成功，则完全不需要任何内存读取操作。对于x86和x86-64架构，gcc提供了若干内联汇编指令：

```
```c
#include <emmintrin.h>
void _mm_stream_si32(int *p, int a);
void _mm_stream_si128(int *p, __m128i a);
void _mm_stream_pd(double *p, __m128d a);
#include <xmmmintrin.h>
void _mm_stream_pi(__m64 *p, __m64 a);
void _mm_stream_ps(float *p, __m128 a);
#include <ammintrin.h>
void _mm_stream_sd(double *p, __m128d a);
void _mm_stream_ss(float *p, __m128 a);
```
```

这些指令在一次性处理大量数据时最为高效。数据从内存中加载，经过一个或多个步骤的处理后，再写回内存。数据以“流”的形式通过处理器，因此得名这些`intrinsic`函数。

内存地址必须分别对齐到8字节或16字节。在使用多媒体扩展的代码中，可以使用正常的`_mm_store`内联函数进行替换。`/* 注：最后一句中的_mm_store保留原文格式，但中文表述建议调整为：“在使用多媒体扩展的代码中，可以将正常的_mm_store内联函数进行替换。”以更符合中文表达习惯。*/`

使用这些非临时版本时的情况。在第A.1节的矩阵乘法代码中，我们没有这样做，因为写入的值会在短时间内被重新使用。这是一个使用流式指令无益的例子。关于此代码的更多内容请参见6.2.1节。

处理器的写合并缓冲区只能暂时保存对缓存行进行部分写操作的请求。通常需要依次发出所有修改单个缓存行的指令，以便实际执行写合并。以下是一个示例说明如何做到这一点：

```
```c#include <emmintrin.h>```
```cvoid 设置字节(char *p, int c)```
{ __m128i i = _mm_set_epi8(c, c, c, c, c, c, c, c, c, c, c, c,
c, c, c); _mm_stream_si128((__m128i *)&p[0], i); _mm_stream_si128((__m128i *)&p[16], i); _mm_stream_si128((__m128i *)&p[32], i); _mm_stream_si128((__m128i *)&p[48], i);
}
```

假设指针 *p* 的对齐是适当的，调用此函数将把所指向的缓存行的所有字节设置为 *c*。写组合逻辑会看到生成的四个 `movntdq` 指令，并且只有在最后一个指令执行完毕后才发出一次写命令。总之，这段代码不仅避免了在写入之前读取缓存行，还避免了用可能很快不再需要的数据污染缓存。这在某些情况下可以带来巨大的好处。一个日常使用的示例是 C 运行时中的 `memset` 函数，在处理大块数据时应使用类似上述的代码序列。

某些架构提供了专门的解决方案。PowerPC 架构定义了 `dcbz` 指令，可用于清除整个缓存行。该指令并不会真正绕过缓存，因为会为结果分配一个缓存行，但不会从内存中读取数据。与非临时性存储指令相比，它的功能更为有限，只能将缓存行设置为全零，并且会污染缓存（如果数据是非临时性的），但无需写组合逻辑即可实现这一效果。

为了观察非临时性指令的实际效果，我们将使用一个新的测试程序来测量向矩阵写入数据的情况。该矩阵以二维数组的形式组织，并由编译器在内存中进行布局：最左边（第一个）索引指向行，所有元素在内存中顺序排列；右边（第二个）索引则指向某一行中的具体元素。测试程序以两种方式遍历矩阵：首先通过内层循环增加列号，然后通过内层循环增加行索引。这意味着我们能够获得

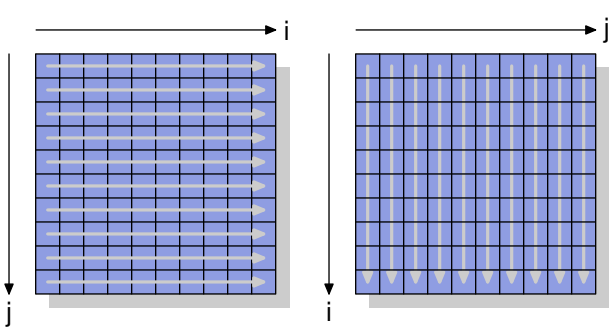


图6.1：矩阵访问模式

如图6.1所示的行为。

我们测量了初始化一个3000×3000矩阵所需的时间。为了观察内存的行为，我们使用不利用缓存的存储指令。在IA-32处理器上，为此会使用“非临时性提示”。作为对比，我们也测量了普通存储操作的结果。结果见表6.1。

|              | Inner Loop Increment |        |
|--------------|----------------------|--------|
|              | Row                  | Column |
| Normal       | 0.048s               | 0.127s |
| Non-Temporal | 0.048s               | 0.160s |

表6.1：时延矩阵初始化

对于使用缓存的常规写操作，我们得到了预期的结果：如果内存被顺序访问，则性能显著提升，整个操作仅耗时0.048秒，相当于约750 MB/秒，而随机访问则需要0.127秒（大约280 MB/秒）。矩阵规模足够大，使得缓存几乎无效。

我们主要关注的是绕过缓存的写操作。这里顺序访问的速度与使用缓存的情况一样快，这可能令人惊讶。造成这一结果的原因是处理器如上所述在执行写合并（write-combining）。此外，对于非临时性写操作，*memory ordering*规则有所放宽：程序需要显式插入内存屏障指令（x86和x86-64处理器中的sfence指令）。这意味着处理器有更多的自由来回写数据，并充分利用可用带宽。

在内层循环进行列式访问时，情况有所不同。未命中缓存的访问速度显著慢于命中缓存的情况（0.16秒，约225MB/s）。这里可以看出无法进行写合并，并且每个内存单元都必须单独寻址。这要求不断选择新的行以访问RAM芯片中的数据，伴随着所有相关的延迟。结果是未命中缓存的性能比命中缓存时差了25%。

在读取方面，直到最近，处理器除了使用非临时访问（NTA）预取指令提供弱提示外，并没有其他支持。对于读取操作而言，没有类似于写入合并的机制，这对无法缓存的内存（如内存映射I/O）尤其不利。英特尔通过SSE4.1扩展引入了NTA加载功能。它们使用少量流式加载缓冲区实现；每个缓冲区包含一个缓存行。对于给定的缓存行，首次执行movntdqa指令将该缓存行加载到缓冲区中，并可能替换另一个缓存行。后续对该缓存行进行16字节对齐访问时，可以从加载缓冲区以较低成本提供服务。除非有其他原因需要这样做，否则不会将缓存行加载到缓存中，从而允许大量内存的加载而不污染缓存。编译器为此指令提供了内联函数：

```
``c#include <smmintrin.h> __m128i _mm_stream_load_si128(
 __m128i *p);``
```

此内嵌函数应多次使用，每次传递16字节块的地址作为参数，直到每个缓存行都被读取。只有在所有缓存行都读取完毕后，才开始下一个缓存行的读取。由于存在几个流式读取缓冲区，因此可能可以同时从两个内存位置进行读取。

通过这次实验我们可以得出结论，现代CPU在处理未命中缓存的写操作以及最近甚至包括顺序读取访问时，能够进行很好的优化。这一知识在处理大型数据结构（这些数据结构仅使用一次）时会非常有用。其次，缓存可以在一定程度上掩盖随机内存访问的部分成本，但并非全部。在这个例子中，由于RAM访问实现的原因，随机访问比顺序访问慢70%。除非实现发生变化，在可能的情况下应尽量避免随机访问。

在关于预取的部分中，我们将再次讨论非临时标志。

## 6.2 缓存访问

希望提升程序性能的程序员会发现，专注于影响一级缓存的变化是最有效的，因为这可能会带来最佳的结果。我们将首先讨论这一层面，然后再扩展到其他层级。显然，所有针对一级缓存的优化也会对其他缓存产生影响。内存访问的所有优化主题都是相同的：提高局部性（空间和时间）并使代码和数据对齐。

### 6.2.1 优化一级数据缓存访问

在第3.3节中，我们已经看到了有效利用L1d缓存可以显著提升性能。

在本节中，我们将展示哪些代码更改可以帮助提升性能。延续上一节的内容，我们首先集中于优化顺序访问内存的策略。如第3.3节中的数据所示，当内存被顺序访问时，处理器会自动预取数据。

示例代码使用的是矩阵乘法。我们采用两个1000×1000的双精度元素方阵。对于那些忘记数学知识的人来说，给定两个矩阵A和B，其元素分别为a<sub>{ij}</sub>和b<sub>{ij}</sub>（其中i和j从0开始），它们的乘积为

$$(AB)_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{kj} = a_{i1} b_{1j} + a_{i2} b_{2j} + \cdots + a_{i(N-1)} b_{(N-1)j}$$

一个简单的C语言实现可以是这样的：

```
``for (i = 0; i < N; ++i) for (j = 0; j < N; ++j) for (k = 0; k
 < N; ++k) res[i][j] += mul1[i][k] * mul2[k][j];``
```

两个输入矩阵分别为mul1和mul2。结果矩阵res假设已初始化为全零。这是一个简洁明了的实现方法。但显而易见，我们遇到了图6.1中所解释的问题。虽然mul1是顺序访问的，但在内层循环中mul2的行号在递增。这意味着mul1被处理得像图6.1中的左矩阵，而mul2则像是右矩阵。这种情况显然不好。

有一种简单的补救方法可以尝试。由于矩阵中的每个元素会被多次访问，因此在使用之前重新排列（从数学角度讲即“转置”）第二个矩阵mul2可能是值得的。

$$(AB)_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{jk}^T = a_{i1} b_{j1}^T + a_{i2} b_{j2}^T + \cdots + a_{i(N-1)} b_{j(N-1)}^T$$

转置完成后（通常用上标“T”表示），我们现在依次遍历两个矩阵。就C代码而言，现在看起来像这样：

```
double tmp[N][N]; for (i = 0; i < N; ++i) for (j = 0; j < N; ++j)
 tmp[i][j] = mul2[j][i]; for (i = 0; i < N; ++i) for (j = 0; j
 < N; ++j) for (k = 0; k < N; ++k) res[i][j] += mul1[i][k] * tmp[j][k];
```

我们创建一个临时变量来存储转置后的矩阵。这需要访问额外的内存，但希望这笔开销能够通过减少每列1000次非顺序访问的成本得到补偿（至少在现代硬件上是这样的）。现在进行一些性能测试。在一个时钟速率为2666MHz的Intel Core 2处理器上的结果如下（以时钟周期为单位）：

|          | Original       | Transposed    |
|----------|----------------|---------------|
| Cycles   | 16,765,297,870 | 3,922,373,010 |
| Relative | 100%           | 23.4%         |

通过简单的矩阵变换，我们可以实现76.6%的加速！复制操作更是完全补偿了这一变化。那些1000次非顺序访问确实造成了很大影响。

接下来的问题是这是否已经是最佳方案。无论如何，我们需要一种不需要额外复制的方法作为替代。我们并不总是能够进行这种复制：矩阵可能过大，或者可用内存过小。

对替代实现的探索应从仔细审视所涉及的数学运算和原实现执行的操作开始。基本的数学知识告诉我们，只要每个加数恰好出现一次，结果矩阵中每个元素的加法操作顺序是无关紧要的。这种理解使我们能够寻找重新排列原代码内循环中加法操作顺序的解决方案。

现在让我们来分析原始代码执行过程中实际遇到的问题。mul2数组元素的访问顺序为：(0,0), (1,0), ……，(N-1,0), (0,1), (1,1), ……。其中，元素(0,0)和(0,1)位于同一缓存行中，但在内层循环完成一轮时，该缓存行早已被替换出去了。在这个例子中，每轮内层循环需要为每个矩阵访问1000个缓存行（对于Core 2处理器而言，每个缓存行为64字节），这远远超过了L1d缓存区32KB的容量。

但在执行内循环时，如果我们同时处理中间循环的两次迭代会怎样？在这种情况下，我们从保证在L1d缓存中的缓存行中使用两个双精度值。这样可以将L1d未命中率减半。这当然是一种改进，但根据缓存行大小的不同，可能仍然无法达到最佳效果。Core 2处理器的L1d缓存行为64字节。实际值可以通过查询获得。

```
sysconf (_SC_LEVEL1_DCACHE_LINESIZE)
```

在运行时或通过命令行使用 `getconf` 工具来获取配置，以便程序可以针对特定的缓存行大小进行编译。由于 `'sizeof(double)'` 为 8，

<sup>28</sup>We ignore arithmetic effects here which might change the occurrence of overflows, underflows, or rounding.

这意味着，为了充分利用缓存行，我们应该将中间循环展开8次。继续这一思路，为了有效地使用结果矩阵，即同时写入8个结果，我们也应该将外层循环展开8次。这里假设缓存行为64字节大小，但该代码在32字节缓存行的系统上也能很好地工作，因为这两种情况下的缓存行都能达到100%利用率。一般来说，最好通过使用 `getconf` 工具在编译时硬编码缓存行大小，例如：

```
gcc -DCLS=$(getconf 一级数据缓存行大小) ...
```

如果二进制代码需要通用性，应使用最大的缓存行大小。当一级数据缓存（L1d）非常小时，可能会导致并非所有数据都能放入缓存中，但这类处理器本身也不适合运行高性能程序。我们得到的代码大致如下：

```
#define SM (CLS / sizeof (double))

for (i = 0; i < N; i += SM)
 for (j = 0; j < N; j += SM)
 for (k = 0; k < N; k += SM)
 for (i2 = 0, rres = &res[i][j],
 rmul1 = &mul1[i][k]; i2 < SM;
 ++i2, rres += N, rmul1 += N)
 for (k2 = 0, rmul2 = &mul2[k][j];
 k2 < SM; ++k2, rmul2 += N)
 for (j2 = 0; j2 < SM; ++j2)
 rres[j2] += rmul1[k2] * rmul2[j2];
```

这看起来相当吓人。在某种程度上确实如此，但只是因为它包含了一些技巧。最明显的变化是我们现在有了六个嵌套循环。外层循环以SM（缓存行大小除以 `sizeof(double)`）为步长进行迭代。这样将乘法分解成几个较小的问题，这些问题可以更好地利用缓存局部性来处理。内层循环则遍历外层循环中缺失的索引。这里仍然有三个循环。唯一棘手的部分是 `k2` 和 `j2` 循环的顺序不同。这是因为在实际计算中，只有一个表达式依赖于 `k2`，而有两个表达式依赖于 `j2`。

这里的其余复杂性源于 `gcc` 在优化数组索引时不够智能。引入额外变量 `rres`、`rmul1` 和 `rmul2` 可以通过将公共表达式从内层循环中提取出来，从而优化代码。C 和 C++ 语言的默认别名规则并不能帮助编译器做出这些决策（除非使用 `restrict` 关键字，否则所有指针访问都可能是别名冲突的潜在来源）。这就是为什么 Fortran 仍然是数值编程首选语言的原因：它使得编写快速代码变得更加容易。

<sup>29</sup>In theory the `restrict` keyword introduced into the C language in the 1999 revision should solve the problem. Compilers have not caught up yet, though. The reason is mainly that too much incorrect code exists which would mislead the compiler and cause it to generate incorrect object code.

|          | Original       | Transposed    | Sub-Matrix    | Vectorized    |
|----------|----------------|---------------|---------------|---------------|
| Cycles   | 16,765,297,870 | 3,922,373,010 | 2,895,041,480 | 1,588,711,750 |
| Relative | 100%           | 23.4%         | 17.3%         | 9.47%         |

表6.2：矩阵乘法时序

如表6.2所示，通过避免复制操作，我们获得了额外6.1%的性能提升。此外，无需增加任何额外内存。输入矩阵可以任意大，只要结果矩阵也能容纳在内存中即可。这是通用解决方案的一个要求，我们现在已实现这一目标。

表6.2中还有一列未作解释。如今大多数现代处理器都包含对向量化操作的特殊支持。这些特殊指令通常被标榜为多媒体扩展，允许同时处理2、4、8或更多值。这些通常是单指令多数据（SIMD）操作，并辅以其他指令将数据调整到合适的形式。英特尔处理器提供的SSE2指令可以在一次操作中处理两个双精度数值。指令参考手册列出了提供访问这些SSE2指令的内联函数。如果使用了这些内联函数，程序运行速度会比原始代码快7.3%（相对于原代码）。结果是，该程序仅需原代码十分之一的时间即可完成执行。换算成人们熟悉的数值表示，我们从318 M FLOPS提升到了3.35 GFLOPS。由于此处只关注内存效应，将程序代码移至附录A.1中。

需要注意的是，在代码的最新版本中，mul2仍然存在一些缓存问题；预取操作依旧无法正常工作。但这些问题在不转置矩阵的情况下是无法解决的。也许缓存预取单元会变得更智能，能够识别出这些模式，那么就无需进行额外的修改了。单线程代码在一个2.66 GHz处理器上达到3.19 GFLOPS的表现还算不错。

我们在矩阵乘法的例子中优化了缓存行的使用。缓存行中的所有字节总是会被使用，我们只是确保在缓存行被替换之前将其完全利用。这无疑是一个特例。

数据结构往往占用一个或多个缓存行，但程序在任何时刻仅使用其中的少数几个成员的情况非常常见。在图3.11中，我们已经看到了当只使用少量成员时，大尺寸数据结构的影响。

图6.2展示了使用现在广为人知的程序进行的另一组基准测试结果。这次是对同一个列表元素中的两个值进行相加操作。在一种情况下，这两个元素位于同一缓存行中；而在另一种情况下，一个元素位于该列表元素的第一个缓存行中，另一个则位于最后一个缓存行中。图表显示了我们所经历的性能下降情况。

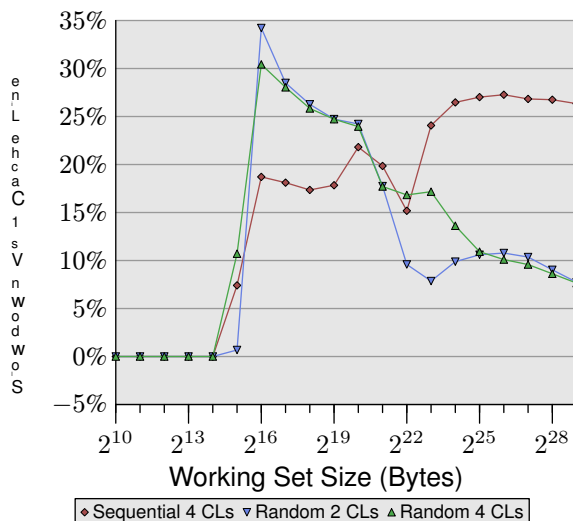


图6.2：跨越多个缓存行分布

不出所料，在所有情况下，如果工作集能够完全放入L1d缓存中，则不会产生负面影响。一旦L1d缓存不再足够使用时，通过在一个进程中使用两条缓存行而不是一条来支付相应的代价。红色线条显示了当列表在内存中顺序排列时的数据情况。我们可以看到典型的两步模式：当L2缓存足够使用时大约有17%的性能损失，而当主内存必须被使用时则约为27%的性能损失。

在随机内存访问的情况下，相关数据略有不同。工作集能够完全放入L2缓存中的情况下，性能下降幅度介于25%到35%之间。超出这一范围后，性能下降幅度降至大约10%左右。这不是因为惩罚（延迟）变小了，而是实际的内存访问成本变得不成比例地更高。数据还显示，在某些情况下，元素之间的距离确实有影响。随机4缓存行曲线显示出更高的惩罚，这是因为第一和第四缓存行被使用。

一种查看数据结构与缓存行布局差异的简便方法是使用pahole程序（参见[4]）。该程序会检查二进制文件中定义的数据结构。考虑包含以下定义的程序：

```
``cstruct foo { int a; long fill[7]; int b; };``
```

```

struct foo {
 int a; /* 0 4 */

 /* XXX 4 bytes hole, try to pack */

 long int fill[7]; /* 8 56 */
 /* --- cacheline 1 boundary (64 bytes) --- */
 int b; /* 64 4 */
}; /* size: 72, cachelines: 2 */
/* sum members: 64, holes: 1, sum holes: 4 */
/* padding: 4 */
/* last cacheline: 8 bytes */

```

图6.3: pahole运行结果

在64位机器上编译时，pa-hole的输出包含图6.3所示的内容。这些信息非常有价值。首先，它表明该数据结构使用了不止一个缓存行。此工具假设当前处理器的缓存行大小，但可以通过命令行参数覆盖这一值。特别是在数据结构大小仅略超缓存行限制且大量此类对象被分配的情况下，寻找压缩该结构的方法是有意义的。或许可以将某些元素的数据类型缩小，或者将一些实际上是标志位的字段用单个比特表示。

在示例中，压缩很简单，并且程序已经给出了提示。输出显示，在第一个元素之后存在一个四字节的空洞。这个空洞是由结构体对齐要求和填充元素引起的。很容易看出，大小为四个字节的元素b（由行尾的4表示）正好填满这个间隙。在这种情况下，间隙不再存在，并且数据结构可以完全放入一条缓存行中。pahole工具能够自行执行此优化操作。如果使用--reorganize参数并在命令行末尾添加结构体名称，则工具输出将包括优化后的结构和缓存行的使用情况。除了移动元素以填充空隙外，该工具还可以优化位字段并合并填充和空洞。更多细节请参见[4]。

当然，理想的情况是空洞大小恰好适合尾部元素。为了使这一优化有效，对象本身必须与缓存行对齐。我们稍后会详细讨论这一点。

pahole 输出还允许轻松查看元素是否需要重新排序，以便那些一起使用的元素也存储在一起。使用 pahole 工具可以很容易地确定哪些元素位于同一缓存行上，并且在必要时确定哪些元素需要重新排列以实现这一目标。这并不是一个自动过程，但该工具可以提供很大的帮助。

各个结构元素的位置及其使用方式也很重要。如第3.5.2节所示，如果关键指令位于缓存行末尾，则代码性能会更差。这意味着一个程序的性能会受到……

语法应始终遵循以下两条规则：

1. 将最有可能成为关键字段的结构体元素移动到结构体的开头。
2. 在访问数据结构时，如果访问顺序不由情况决定，则按照结构体内定义的顺序进行访问。

对于小规模结构，这意味着元素应当按照它们可能被访问的顺序进行排列。这必须以一种灵活的方式处理，以便其他优化措施（如填充空洞）也能应用。对于较大的数据结构，每个缓存行大小的块都应遵循这些规则进行排列。

如果对象本身并未按预期对齐，那么重新排列元素就不值得花费时间了。对象的对齐方式由数据类型的对齐要求决定。每种基本类型都有其特定的对齐要求。对于结构体类型而言，其中任何一个成员的最大对齐要求决定了整个结构体的对齐方式。这通常小于缓存行大小。这意味着即使结构体中的成员被排列以适应同一缓存行，分配的对象也可能没有与缓存行大小相匹配的对齐方式。确保对象具有在设计结构布局时使用的对齐方式有以下两种方法：

- 对象可以分配给其显式的对齐要求。对于动态分配，调用 malloc 只会分配与最严格标准类型（通常是 long double）对齐的对象。不过，可以通过使用 posix\_memalign 来请求更高的对齐方式。

```

```c#include <stdlib.h>int posix_memalign(void
**memptr, size_t align, size_t size);```

```

该函数将指向新分配内存的指针存储在由 `memptr` 指向的指针变量中。这块内存块大小为 `size` 字节，并且以 `align` 字节边界对齐。

对于编译器分配的对象（位于`.data`、`.bss`等段以及栈上），可以使用变量属性：

```
`struct strtpe variable __attribute__((aligned(64)));`
```

在这种情况下，变量在64字节边界上进行对齐，而不考虑`strtpe`结构的对齐要求。这同样适用于全局变量和自动变量。

对于数组而言，这种方法并不会像预期的那样工作。只有在每个数组元素的大小是对其齐值的整数倍时，数组的第一个元素才会对齐。这也意味着每一个单独的变量都必须适当标注。使用 `posix\_memalign` 并非完全免费，因为对齐要求通常会导致碎片化和/或更高的内存消耗。

- 用户定义类型的对齐要求可以通过使用类型属性来更改。

```
```cstruct strtpe { ...成员... } __attribute__((aligned(64)));```
```

这将导致编译器为所有对象分配适当的对齐方式，包括数组。然而，程序员必须确保为动态分配的对象请求适当的对齐方式。在此情况下，必须使用 `posix\_memalign` 函数。使用 gcc 提供的 `alignof` 操作符获取值，并将其作为 `posix\_memalign` 的第二个参数传递是非常简单的。

本节之前提到的多媒体扩展几乎总是要求内存访问地址对齐。即，对于16字节的内存访问，地址应为16字节对齐。x86和x86-64处理器具有处理非对齐访问的特殊内存操作变体，但这些操作速度较慢。这种严格的对齐要求对于大多数RISC架构来说并不新鲜，因为它们要求所有内存访问都完全对齐。即使某个架构支持非对齐访问，在某些情况下这也会比使用适当的对齐方式更慢，尤其是在不对齐导致一次加载或存储操作需要使用两个缓存行而不是一个时。

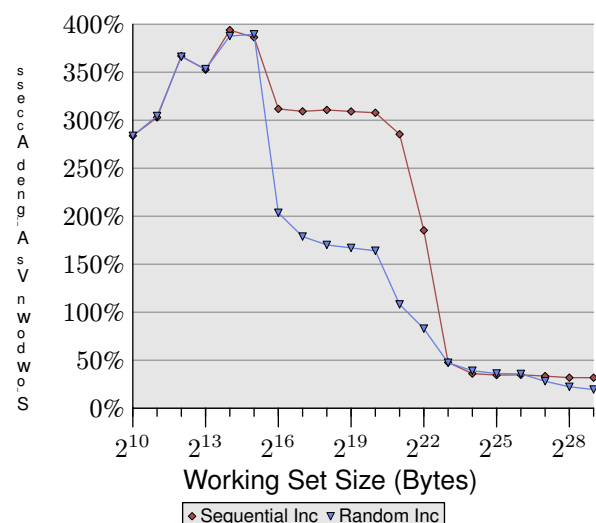


图6.4: 未对齐访问的开销

图6.4展示了未对齐内存访问的影响。现在广为人知的测试是在访问内存（顺序或随机）时递增一个数据元素，分别在列表元素对齐和故意错开的情况下进行测量。图表显示了由于未对齐访问导致程序性能下降的情况。对于顺序访问情况，影响比随机访问更为显著，因为在后者情况下，未对齐访问的成本部分被较高的内存访问成本掩盖。当工作集大小恰好适合L2缓存时，在顺序访问情况下，性能下降约为300%。这可以解释为L1缓存的有效性降低：一些递增操作现在会触及两个缓存行，并且开始处理列表元素通常需要读取两个缓存行。L1和L2之间的连接过于拥挤。

对于非常大的工作集大小，未对齐访问的影响仍然达到20%到30%，考虑到这些规模下的对齐访问时间已经很长，这样的影响是相当显著的。这张图应该表明必须认真对待对齐问题。即使架构支持未对齐访问，也不能将其视为“与对齐访问一样好”。

尽管这些对齐要求带来了一些问题。如果一个自动变量具有对齐要求，编译器必须确保在所有情况下都能满足这一要求。这并非易事，因为编译器无法控制调用站点及其处理栈的方式。这个问题可以通过两种方式来解决：

生成的代码会主动对栈进行对齐，必要时插入空隙。这要求代码检查对齐情况、创建对齐并后续取消对齐。

要求所有调用者都对栈进行对齐。

所有常用的可执行文件二进制接口（ABI）都遵循第二种方式。如果调用者违反规则且被调用者需要对齐，则程序可能会失败。然而，保持对齐并不免费。

函数使用的栈帧大小不一定是对齐倍数。这意味着如果从该栈帧调用其他函数，则需要填充字节。主要的区别在于，大多数情况下编译器知道栈帧的大小，并且因此它知道如何调整栈指针以确保从该栈帧调用的任何函数都满足对齐要求。实际上，大多数编译器会简单地将栈帧大小向上取整即可。

处理对齐问题的这种简单方法在使用可变长度数组（VLAs）或 `alloca` 时无法实现。在这种情况下，栈帧的总大小仅在运行时才能确定。此时可能需要主动控制对齐，从而使生成的代码（略微）变慢。

在某些架构上，只有多媒体扩展需要严格的对齐；这些架构上的栈对于普通数据类型总是最小限度地进行对齐，通常分别为32位和64位架构提供4字节或8字节的对齐。在这种系统中，强制执行对齐会带来不必要的开销。这意味着，在这种情况下，如果我们知道它从未被依赖，则可能希望消除严格的对齐要求。不执行多媒体操作且不调用其他函数的尾部函数（即仅执行单一功能的函数）不需要对齐。同样地，只调用那些不需要对齐的函数的功能也不需要对齐。如果可以识别出足够数量的此类函数集，程序可能希望放宽对齐要求。对于x86二进制文件，gcc支持放松栈对齐的要求。

```
-mpreferred-stack-boundary=2
```

如果将此选项设置为  $N$ ，则栈对齐要求将会被设定为  $2^N$  字节。因此，若使用值 2，则栈的对齐要求从默认值（即 16 字节）减少到仅 4 字节。在大多数情况下，这意味着不需要额外的对齐操作，因为正常的栈压入和弹出操作本来就在四字节边界上进行。此特定于机器的选项有助于减小程序大小，并且可以提高执行速度。但是，它不能应用于许多其他架构。即使对于 x86-64 架构来说，由于 x86-64 ABI 要求浮点参数通过 SSE 寄存器传递，而 SSE 指令需要完整的 16 字节对齐，因此通常也不适用。然而，在该选项可使用的情况下，它确实可以产生显著的影响。

高效地放置结构体元素和对齐并不是影响缓存效率的数据结构方面的唯一因素。如果使用了结构体数组，整个结构体的定义都会影响性能。记住这一点。

图3.11所示的结果：在这种情况下，数组元素中未使用的数据量不断增加。结果是预取效果逐渐降低，对于大规模数据集而言，程序效率也随之下降。

对于大型工作集而言，充分利用可用缓存至关重要。为了达到这一目的，可能需要重新组织数据结构。虽然程序员倾向于将概念上相关联的所有数据放在同一个数据结构中，但这未必是实现最高性能的最佳方法。假设我们有如下所示的数据结构：

```
``cstruct 订单 { double 价格;
bool 是否支付; const char *买家[5]; long 买家ID;};``
```

进一步假设这些记录存储在一个大的数组中，并且一个频繁运行的任务会累加所有未结账单的预期支付金额。在这种情况下，用于买家和 `buyer_id` 字段的内存会被不必要的加载到缓存中。根据图3.11中的数据，该程序的表现可能会比最优情况差上5倍。

将订单数据结构拆分为两部分要好得多，可以将前两个字段存储在一个结构中，而其他字段则存放在别处。这种改动无疑会增加程序的复杂性，但性能提升可能足以证明这一成本是合理的。

最后，让我们考虑另一种缓存使用优化方法，虽然这种方法也适用于其他级别的缓存，但主要影响的是 L1d 访问。如图 3.8 所示，增加缓存的关联性有助于正常操作。缓存越大，其关联性通常越高。L1d 缓存太大以至于无法完全实现全相联，但又不够大到可以拥有与 L2 缓存相同的关联性。如果工作集中许多对象落入同一个缓存组中，则可能会出现驱逐现象，即使大部分缓存未被使用，程序也可能经历延迟。这些缓存缺失有时被称为“冲突缺失”。由于 L1d 地址使用虚拟地址，因此程序员可以控制这种情况。如果一起使用的变量也存储在一起，则它们落入同一个组中的可能性会最小化。图 6.5 显示了这个问题出现的速度有多快。

在图中，使用特殊设置测量了熟悉的 Follow<sup>30</sup>与 NPAD=15 测试。X 轴表示两个列表元素之间的距离，以空闲单元为单位测量。

<sup>30</sup>The test was performed on a 32-bit machine, hence NPAD=15 means one 64-byte cache line per list element.

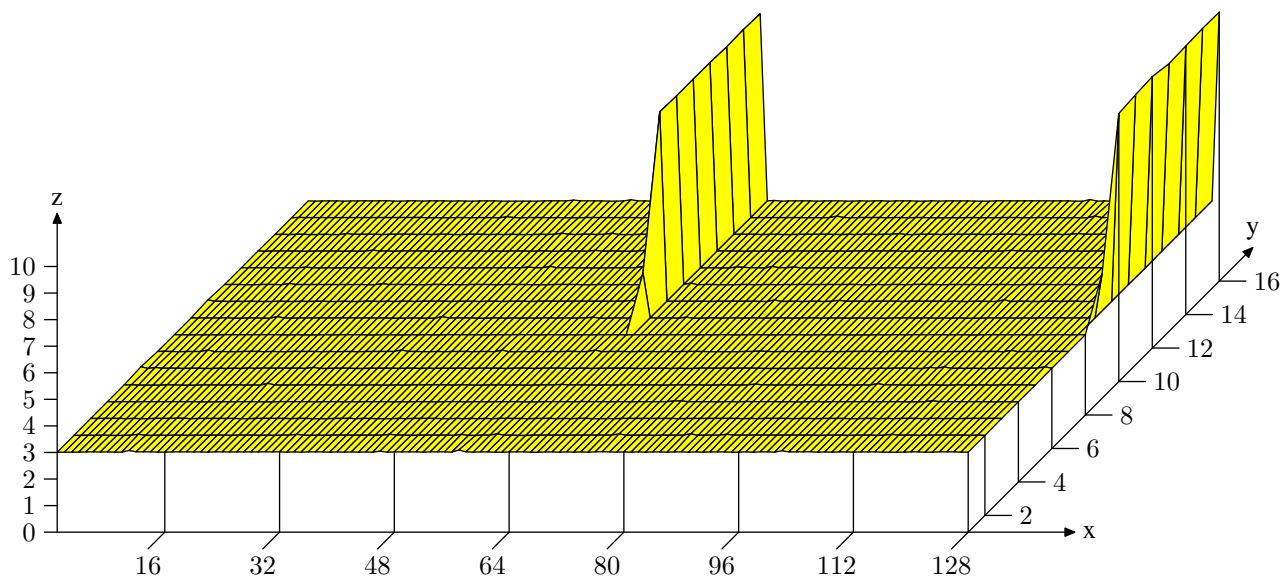


图6.5: 缓存关联性影响

列表元素之间的距离。换句话说，距离为2意味着下一个元素的地址比前一个元素的地址多出128字节。所有元素在虚拟地址空间中以相同的间隔布局。Y轴表示列表的总长度。使用的元素数量从1到16不等，这意味着工作集大小总共为64至1024字节。Z轴显示遍历每个列表元素所需的平均周期数。

图中所示的结果并不令人惊讶。如果使用的元素很少，所有数据都能放入L1d缓存中，访问时间仅为每个列表元素3个周期。对于几乎所有的列表元素排列情况也是如此：虚拟地址很好地映射到了L1d缓存行上，几乎没有冲突。但在图表中有两个特殊的距离值例外。当距离是4096字节的倍数（即64个元素的距离），且列表长度大于8时，每个列表元素的平均周期数会显著增加。在这种情况下，所有条目都在同一个集合中，一旦列表长度超过关联性，L1d中的条目就会被冲洗出来，并在下一轮从L2重新读取。这导致了大约每个列表元素需要额外花费约10个周期的成本。

通过该图我们可以确定所使用的处理器具有一个8路关联的32KB大小的L1d缓存。这意味着如果需要，可以使用此测试来确定这些值。对于L2缓存也可以测量相同的效果，但这里更为复杂，因为L2缓存使用物理地址进行索引，并且其容量要大得多。

程序员们应将此数据视为设置关联性值得重视的迹象。在实际世界中，以2的幂次边界布局数据的情况并不少见，但这正是可能导致上述问题的情形。

未对齐的访问会增加冲突不命中概率，因为每次访问可能都需要额外的一条缓存行。因此会导致效果变差和性能下降。

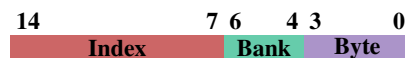


图6.6: AMD L1d缓存的银行地址

如果进行了上述优化，另一个相关的优化也成为可能。至少在AMD的处理器中，L1d缓存被实现为多个独立的银行。L1d每周期可以接收两个数据字，但前提是这两个字必须存储在不同的银行中或同一个具有相同索引号的银行内。银行地址编码在虚拟地址的低位部分，如图6.6所示。如果一起使用的变量也被集中存放在一起，则它们位于不同银行或同一具有相同索引号的银行中的概率会很高。

## 6.2.2 优化一级指令缓存访问

准备代码以充分利用L1i缓存需要类似的好用L1d缓存的技术。然而，除非程序员直接使用汇编语言编写代码，否则他们通常无法直接影响L1i的使用方式。如果使用编译器，则程序员可以通过指导编译器生成更好的代码布局来间接决定L1i的使用情况。

代码具有线性的优点，尤其是在跳转之间。在这期间，处理器可以高效地预取内存。然而，跳转会扰乱这种理想的状态，因为

- 跳转目标可能无法静态确定；

- 即使内存访问是静态的，如果所有缓存都未命中，也可能需要很长时间。

这些问题会导致执行停滞，可能对性能产生严重的影响。这就是为什么当今的处理器在分支预测（BP）上投入大量资源的原因。高度专业化的分支预测单元试图尽可能提前确定跳转的目标，以便处理器可以开始将新位置的指令加载到缓存中。它们使用静态和动态规则，并且越来越擅长识别执行中的模式。

尽快将数据加载到指令缓存中尤为重要。如第3.1节所述，指令在执行前必须先进行解码，并且为了加快这一过程（特别是在x86和x86-64架构中），实际缓存的是已经解码后的形式，而不是从内存读取的字节/单词形式。

为了充分利用L1指令缓存，程序员应关注代码生成的以下方面：

1. 尽可能减少代码大小。这需要与循环展开和内联等优化措施保持平衡。
2. 代码执行应为线性，避免出现停顿。
3. 在有意义的情况下对齐代码。

我们将探讨一些可用于根据这些方面优化程序的编译器技术。

编译器提供了启用不同优化级别的选项；也可以单独开启特定的优化。许多在高优化级别（如 gcc 的 -O2 和 -O3）下启用的优化主要涉及循环优化和函数内联。总体而言，这些是良好的优化措施。如果以这种方式进行优化的代码占程序总执行时间的重要部分，则可以提高整体性能。特别是函数内联允许编译器一次优化更大的代码块，从而生成更好地利用处理器流水线架构的机器码。无论是处理代码还是数据（通过消除死代码或传播值范围等方式），当能够将更大一部分程序视为一个单一单元时，效果会更好。

更大的代码尺寸意味着对L1i（以及L2和更高级别）缓存的压力增大。这会导致性能下降。较小的代码可以运行得更快。幸运的是，gcc提供了一个优化选项来指定这一点。如果使用-Os参数，编译器将针对代码大小进行优化。优化措施包括

<sup>31</sup>Bubbles describe graphically the holes in the execution in the pipeline of a processor which appear when the execution has to wait for resources. For more details the reader is referred to literature on processor design.

已知会增加代码大小的选项被禁用了。使用此选项通常会产生出人意料的结果。特别是如果编译器实际上无法充分利用循环展开和内联优化时，启用此选项会有很大收益。

内联也可以单独控制。编译器具有启发式算法和限制，用以指导内联操作；这些限制可以由程序员进行调整。`-finline-limit`选项指定了一个函数必须达到的大小阈值，超过该阈值则认为不宜进行内联。如果一个函数在多个地方被调用，在所有位置都进行内联会导致代码体积急剧膨胀。但情况不仅如此。假设函数`inlcand`分别在函数`f1`和`f2`中被调用，而这两个函数又在一个序列中依次被调用。

<pre>start f1   code f1   inlined inlcand   more code f1 end f1  start f2   code f2   inlined inlcand   more code f2 end f2</pre>	<pre>start inlcand   code inlcand end inlcand  start f1   code f1 end f1  start f2   code f2 end f2</pre>
-----------------------------------------------------------------------------------------------------------------------------------	-----------------------------------------------------------------------------------------------------------

表6.3：内联与不内联对比

表6.3展示了在不内联和两个函数均进行内联的情况下生成代码的可能样子。如果inlcand函数在f1和f2中都被内联，则生成代码的总大小为size f1 + + size f2 + + 2 × + size inlcand。如果不进行任何内联操作，那么总的代码大小会减少size inlcand。这表示如果f1和f2相继被调用时，L1i和L2缓存需要额外的存储空间来容纳这些代码。此外：若inlcand没有被内联，则其代码可能仍保留在L1i中，并且无需再次解码。另外：分支预测单元由于已经见过该代码，可能会更准确地预测跳转指令。如果编译器默认设置的可内联函数大小上限对程序来说不是最优选择，应将其调低。

然而，在某些情况下，内联总是有意义的。如果一个函数仅被调用一次，那么最好将其内联。这为编译器提供了更多的优化机会（如值范围传播，可能会显著改善代码质量）。尽管如此，内联可能受到选择限制的影响。对于这种情况，gcc 提供了一个选项来指定某个函数始终进行内联。添加 `always_inline` 函数属性会指示编译器执行其名称所暗示的操作。

在同一上下文中，如果某个函数尽管足够小但也应从内联，则可以使用 `noinline` 函数属性。即使对于经常从不同地方被调用的小函数来说，使用此属性也是有意义的。

在许多情况下，如果 L1i 缓存的内容可以被重用并且整体占用空间减小，这通常能够弥补额外函数调用的成本。现在的分支预测单元已经相当不错了。如果内联会导致更激进的优化，则情况会有所不同。这种情况必须根据具体情况进行决定。

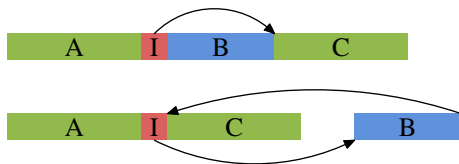
如果内联代码总是被使用，那么 `always_inline` 属性工作得很好。但如果情况并非如此呢？如果这个内联函数只是偶尔被调用：

```
``void fct(void) { ... 代码块A ...
if (条件) inlfct() ... 代码块C ... }
``
```

生成的此类代码序列通常与源代码结构相匹配。首先是代码块A，然后是一个条件跳转指令，如果条件为假，则向前跳转。接着是内联 `inlfct` 生成的代码，最后是代码块C。这看起来都很合理，但其实存在一个问题。

如果条件经常为假，则执行过程不再呈线性。中间有一段未使用的代码，不仅会因预取而污染 L1i 缓存，还可能导致分支预测出现问题。若分支预测错误，则条件表达式的效率将非常低下。

这是一个通用问题，并不特定于内联函数。每当使用条件执行且该执行偏向一边（即，表达式更常导致一个结果而非另一个）时，就有可能出现错误的静态分支预测，从而导致流水线中的空泡。可以通过告知编译器将较少执行的代码移出主代码路径来防止这种情况。在这种情况下，为 `if` 语句生成的条件跳转会跳到顺序之外的位置，如以下图所示。



上部分表示了简单的代码布局。如果区域B（例如，由上述内联函数 `inlfct` 生成）由于条件跳转I而很少被执行，则处理器的预取操作会将包含块B但很少使用的缓存行拉入缓存中。通过使用块重排，可以改变这种情况，如图下部分所示。

经常执行的代码在内存中是线性排列的，而很少执行的代码则被移动到不会影响预取和 L1i 效率的位置。

GCC 提供了两种实现这一目标的方法。首先，编译器在重新编译代码时可以考虑性能分析输出，并根据分析结果布局代码块。我们将在第7节中看到这种方法的具体工作原理。第二种方法是通过显式分支预测。GCC 识别 `__builtin_expect`：

```
long __builtin_expect(long EXP, long C);
```

此构造告知编译器表达式 `EXP` 最有可能取值为 `C`。返回值是 `EXP`。`__builtin_expect` 通常用于条件表达式中。在大多数情况下，它会用在布尔表达式的上下文中，在这种情况下，定义两个辅助宏更为方便：

```
#define 不大可能(expr) __builtin_expect(!(expr), 0) #define 可能(expr) __b
uiltin_expect(!(expr), 1)
```

这些宏可以如以下使用：

```
如果 (likely(a > 1))
```

如果程序员使用了这些宏，并且启用了 `-freorder-blocks` 优化选项，gcc 将会按照上述图示重新排列代码块。此选项在 `-O2` 级别启用，在 `-Os` 级别禁用。还有一个用于重新排列代码块的 gcc 选项（`-freorder-blocks-and-partition`），但由于其不支持异常处理，因此实用性有限。

小循环还有一种在某些处理器上的显著优势。Intel Core 2 的前端有一个名为循环流检测器（Loop Stream Detector, LSD）的特殊功能。如果一个循环包含不超过 18 条指令（其中没有调用子程序的指令），需要最多 4 次解码取指操作且每次取指 16 字节，分支指令数量不超过 4 条，并且该循环被执行超过 64 次，则此循环有时会被锁定在指令队列中，从而在再次使用时更快地获取。例如，通过外部循环多次进入的小内层循环就适用这种情况。即使没有这种专用硬件，紧凑的循环也具有一定的优势。

内联优化并非 L1i 缓存优化的唯一方面。另一个方面是代码对齐，这与数据对齐类似。然而，两者之间存在明显的差异：代码是一个基本上线性的块，不能随意放置在地址空间中，并且程序员无法直接通过编写程序来影响它，因为代码是由编译器生成的。尽管如此，还是有一些方面是可以由程序员控制的。

每条指令单独对齐是没有意义的。目标是使指令流保持顺序性。因此，只有在战略位置进行对齐才有意义。为了决定在哪里添加对齐，必须理解其可能带来的优势。一条指令位于缓存行的开头意味着该缓存行的预取被最大化利用。对于指令而言，这也意味着解码器更加高效。很明显，如果执行的是位于缓存行末尾的指令，则处理器需要准备好读取新的缓存行并进行解码。可能会出现一些问题（如缓存行未命中），这意味着相对于位于缓存行开头的指令，位于结尾处的指令在平均情况下执行效率较低。

结合后续的推断，如果控制刚刚转移到了问题指令（因此预取无效），问题最为严重。由此我们得出最终结论：代码对齐最有益的情况是：

- 在函数开始时；
- 仅通过跳转到达的基本块的开头；
- 在一定程度上，在循环的开始。

在前两种情况下，对齐几乎不会带来额外成本。执行过程从新的位置开始，并且如果我们选择将其置于缓存行的起始处，则可以优化预取和解码。编译器通过插入一系列无操作指令来填充由于代码对齐而产生的空隙，从而实现这种对齐方式。“死代码”会占用一些空间，但通常不会影响性能。

第三种情况略有不同：将每个循环的开始对齐可能会导致性能问题。问题是，一个循环的开始通常会紧随其他代码之后。如果不凑巧的话，在前一条指令和对齐后的循环开始之间会出现空隙。与之前的两种情况不同，这个空隙不能完全被忽略。执行完前一条指令后，必须立即执行循环中的第一条指令。这意味着，在前一条指令之后，要么需要填充一些无操作指令来填补空隙，要么需要一个无条件跳转到循环的开始处。这两种选择都不是免费的。特别是如果该循环本身不经常被执行的话，这些无操作指令或跳转可能会比对齐循环所节省的成本更高。

程序员可以通过三种方式影响代码的对齐。显然，如果代码是用

<sup>32</sup>For some processors cache lines are not the atomic blocks for instructions. The Intel Core 2 front end issues 16 byte blocks to the decoder. They are appropriately aligned and so no issued block can span a cache line boundary. Aligning at the beginning of a cache line still has advantages since it optimizes the positive effects of prefetching.

<sup>33</sup>For instruction decoding processors often use a smaller unit than cache lines, 16 bytes in case of x86 and x86-64.

汇编函数及其内部的所有指令都可以显式对齐。汇编器为所有架构提供了 ``.align`` 假指令来实现这一点。对于高级语言，必须告知编译器对齐要求。与数据类型和变量不同，这无法在源代码中指定，而是使用编译器选项：

`-falign-functions=N`

此选项指示编译器将所有函数对齐到大于N的下一个2的幂边界。这意味着最多会创建N字节空隙。对于使用大数值N的小函数来说，这是一种浪费。同样地，对于很少执行的代码也是如此。这种情况在包含流行和不太流行的接口的库中很常见。明智地选择选项值可以加快速度或通过避免对齐来节省内存。将N的值设为1或使用 `-fno-align-functions` 选项可以关闭所有对齐操作。

对于上述第二种情况——非顺序到达的基本块的开始位置，可以使用不同的选项进行控制：

`-falign-jumps=v7`

所有其他细节都相同，同样的关于内存浪费的警告依然适用。

第三种情况也有其特定选项：

`-falign-loops=N`

再次强调，相同的细节和警告同样适用。不过在这里，如前所述，地址对齐会在运行时带来开销，因为如果顺序访问到对齐的地址，则必须执行空操作或跳转指令。

gcc 还提供了一个用于控制对齐的选项（此处仅作完整说明）：``.falign-labels`` 会将代码中的每个标签（基本上是每个基本块的开始处）进行对齐。然而，在几乎所有情况下，这都会减慢代码执行速度，因此不建议使用此选项。

### 6.2.3 优化二级及更高级缓存访问

关于一级缓存优化的所有说法也适用于二级及更高级别缓存访问。关于最后一级缓存，还有两个额外的方面：

- 缓存未命中总是非常昂贵。虽然L1缓存缺失（希望）通常会命中L2及更高层次的缓存，从而限制了惩罚，但显然对于最后一级缓存来说没有后备措施。

- L2 缓存及更高层级的缓存通常由多个核心和/或超线程共享。因此，分配给每个执行单元的有效缓存大小通常小于总的缓存容量。

为了规避缓存未命中的高昂成本，工作集大小应与缓存容量相匹配。如果数据仅需访问一次，则显然没有必要进行这种匹配，因为缓存本身将无法发挥效用。这里讨论的是那些需要多次访问数据集的工作负载。在这种情况下，使用一个过大而无法完全容纳于缓存中的工作集将会产生大量的缓存未命中，即使预取操作成功执行，也会导致程序运行速度变慢。

一个程序即使面对过大的数据集也必须完成其任务。程序员的职责是通过最小化缓存未命中来优化工作方式。对于末级缓存，可以通过将任务分解为更小的部分来实现这一点，这与第50页所述的优化矩阵乘法非常相似。不过，一个不同之处在于，针对末级缓存处理的数据块可以更大一些。如果还需要进行L1缓存优化，则代码会变得更加复杂。想象一下，在矩阵乘法中，输入矩阵和输出矩阵无法同时放入末级缓存的情况。在这种情况下，可能需要同时优化L1缓存和末级缓存的访问。

一级缓存行的大小在许多处理器世代中通常是恒定的；即使有所变化，差异也会很小。假设使用较大的缓存行大小也不会造成太大问题。对于缓存较小的处理器，可以使用两行或多行缓存来代替一行。无论如何，将缓存行大小硬编码并针对其进行优化是合理的。

对于更高层次的缓存而言，如果程序需要具有通用性，则情况并非如此。这些缓存的大小差异很大，相差八倍或更多的情况并不罕见。由于较大的缓存尺寸会导致在除最大缓存机器外的所有机器上代码性能较差，因此无法将其作为默认值假设。相反的选择也不好：假设最小的缓存意味着浪费掉87%以上的缓存容量。这很糟糕；如图3.14所示，使用大缓存可以对程序的速度产生巨大的影响。

这意味着代码必须根据缓存行大小动态调整自身。这是针对特定程序的优化措施。在这里我们只能说，程序员应该正确计算程序的需求。不仅数据集本身是必需的，更高层次的缓存也被用于其他目的；例如，所有执行的指令都从缓存中加载。如果使用了库函数，这种缓存使用量可能会累积到相当大的程度。这些库函数可能还需要它们自己的数据，这进一步减少了可用内存。

一旦我们得到了内存需求的公式，就可以将其与缓存大小进行比较。如前所述，该缓存可能与其他多个核心共享。目前唯一获取正确信息而不硬编码知识的方法是通过/sys文件系统。在表5.2中，我们已经看到了内核发布的硬件信息。程序需要找到相应的目录：

```
/sys/devices/system/cpu/cpu*/缓存
```

对于最后一级缓存，可以通过该目录下的级别文件中最高的数值来识别。确定目录后，程序应读取该目录中的大小文件，并将其中的数值除以共享CPU映射文件中位掩码设置的位数。

这样计算出的值是一个安全的下限。有时程序对其他线程或进程的行为有更多的了解。如果这些线程被调度到一个共享缓存的核心或超线程上，并且已知其缓存使用量不会耗尽总缓存量中的份额，则所计算的限制可能过低，无法达到最优状态。是否应该使用超过公平份额的资源取决于具体情况。程序员需要做出选择，或者允许用户进行决策。

## 6.2.4 优化快表使用

有两种TLB使用的优化方法。第一种是减少程序所需使用的页面数量，这会自动降低TLB缺失次数。第二种是通过减少必须分配的较高层次目录表的数量来降低TLB查找成本。较少的表格意味着占用的内存更少，从而可能导致目录查找时更高的缓存命中率。

首先进行的优化与减少缺页异常密切相关。我们将在此详细讨论这一主题（见第7.5节）。虽然缺页异常通常是一次性的成本，但快表未命中则是一种持续的惩罚，因为快表缓存一般较小且经常被刷新。尽管缺页异常比快表未命中要昂贵得多，但如果程序运行时间足够长，并且某些部分频繁执行，则快表未命中的代价甚至可能超过缺页异常的成本。因此，从减少页面开销的角度来看，不仅要考虑缺页异常的视角，还要关注快表未命中的影响。区别在于，虽然减少缺页异常只需要对代码和数据进行页面级别的分组，但优化快表则需要任何时刻尽可能地减少使用的快表条目数量。

第二项快表优化更难以控制。所需使用的页目录数量取决于进程虚拟地址空间中所用地址范围的分布情况。地址空间中的位置分布广泛意味着需要更多的目录。

<sup>34</sup>There definitely will sometime soon be a better way!

地址空间布局随机化（ASLR）会导致上述情况的发生。在运行时，栈、DSO（动态共享对象）、堆以及可能的可执行文件的加载地址会被随机化，以防止攻击者猜测函数或变量的地址。

只有在最高性能至关重要的情况下才应关闭地址空间布局随机化（ASLR）。额外目录带来的开销很低，因此除了极少数极端情况外，这一步骤是不必要的。内核可以在任何时候进行的一种优化是确保单个映射不会跨越两个目录之间的地址空间边界。这样可以以最小限度限制 ASLR，但不足以显著削弱其效果。

当程序员直接受到影响时，是因为明确请求了一个地址空间区域。这种情况通常发生在使用 `mmap` 并带有 `MAP_FIXED` 标志时。以这种方式分配新的地址空间区域非常危险，并且很少见。不过这是可能的，如果确实使用了这种方法并且可以自由选择地址，则程序员应该了解最末级页目录的边界，并适当选择所需的地址。

### 6.3 预取技术

预取的目的在于隐藏内存访问的延迟。当今处理器的指令流水线和乱序（OOO）执行能力可以隐藏一些缓存命中的访问延迟，但要掩盖主内存访问的延迟，则需要极长的命令队列。有些不具备乱序执行功能的处理器试图通过增加核心数量来弥补这一不足，但这只有在所有使用的代码都已并行化的情况下才是一个糟糕的选择。

预取可以进一步帮助隐藏延迟。处理器会自行执行预取操作，这些操作由某些事件（硬件预取）触发，或者由程序明确请求（软件预取）。

#### 6.3.1 硬件预取

CPU启动硬件预取的触发条件通常是某种模式下的两个或多个缓存未命中序列。这些缓存未命中的目标可以是后续的或前续的缓存行。在早期实现中，只有相邻缓存行的缓存未命中会被识别。而在现代硬件中，步长也被识别，这意味着跳过固定数量的缓存行被视为一种模式并得到适当处理。

性能会因每次缓存未命中都触发硬件预取而受损。例如，对全局变量的随机内存访问非常常见，由此产生的预取操作大多会浪费前端总线带宽。因此，为了启动预取机制，至少需要两次缓存未命中。现代处理器

所有人都期望存在多条内存访问流。处理器会自动将每个缓存未命中分配到相应的流中，并在达到阈值时启动硬件预取。当今的CPU可以为较高层次的缓存跟踪八到十六个独立的流。

负责模式识别的单元与相应的缓存关联。L1d和L1i缓存可能配备预取单元。L2及更高层级的缓存大概率也配备了预取单元，且该预取单元被使用相同缓存的所有核心和超线程共享。因此，八个到十六个独立流的数量很快就会减少。

预取有一个主要弱点：它无法跨越页面边界。当意识到CPU支持按需分页时，原因就显而易见了。如果允许预取器跨过页面边界访问数据，则可能会触发操作系统事件以使该页面可用。这本身可能对性能产生负面影响。更糟糕的是，预取器不了解程序或操作系统的语义。因此，它可能会预取那些在实际运行中永远不会被请求的页面。这意味着预取器会在处理器可识别的模式下超出内存区域的末尾进行访问。这种情况不仅可能发生，而且非常有可能发生。如果由于预取而触发了操作系统从未预料到的页面请求，则操作系统甚至可能完全失去其原有的跟踪能力。

因此，重要的是要认识到，无论预取器在预测模式方面多么出色，程序在页面边界处仍会经历缓存未命中，除非它显式地进行预取或从新页面读取数据。这也是优化数据布局（如第6.2节所述）以最小化缓存污染、保持无关数据分离的另一个原因。

由于当前页大小的限制，现代处理器并没有非常复杂的逻辑来识别预取模式。在仍然占主导地位的4KB页面大小下，能够实现的功能是有限的。多年来，被识别的地址范围有所增加，但可能超出目前常用的512字节窗口并无太大意义。当前的预取单元无法识别非线性访问模式。这些模式很可能是真正的随机访问，或者至少是非重复性的，以至于没有必要尝试去识别它们。

如果硬件预取意外触发，所能采取的措施是有限的。一种可能的方法是尝试检测此问题并调整数据和/或代码布局。这可能会非常困难。可能存在一些特殊的局部解决方案，例如在x86和x86-64处理器上使用ud2指令。该指令本身无法执行，在间接跳转指令之后使用；它用作向指令发出信号的标志。

<sup>35</sup>Or non-instruction. It is the recommended undefined opcode.

处理器不应浪费资源解码后续的内存访问，因为执行将从不同的位置继续。但这是一种非常特殊的情况，在大多数情况下，人们不得不忍受这个问题。

完全或部分禁用整个处理器的硬件预取是可能的。在英特尔处理器上，使用模型特定寄存器（MSR）来实现这一功能（对于许多处理器而言为IA32 MIS C ENABLE寄存器的第9位；第19位仅禁用相邻缓存行的预取）。由于这是一个特权操作，在大多数情况下需要在内核中进行。如果性能分析显示，系统上运行的重要应用程序因硬件预取导致带宽耗尽和过早的缓存淘汰，则可以考虑使用此MSR。

### 6.3.2 软件预取

硬件预取的优势在于程序无需进行调整。正如前所述，其缺点是访问模式必须非常简单，并且预取不能跨越页面边界进行。因此，我们现在有了更多的可能性，其中最重要的是软件预取。软件预取确实需要通过插入特殊指令来修改源代码。一些编译器支持pragma以或多或少自动插入预取指令。在x86和x86-64架构上，Intel通常使用特定的编译器内联汇编约定来插入这些特殊指令：

```
#include <xmmintrin.h>
enum _mm_hint
{
 _MM_HINT_T0 = 3,
 _MM_HINT_T1 = 2,
 _MM_HINT_T2 = 1,
 _MM_HINT_NTA = 0
};
void _mm_prefetch(void *p,
 enum _mm_hint h);
```

程序可以使用`\_mm\_prefetch`内联函数来预取任何指针指向的内存。大多数处理器（特别是所有x86和x86-64处理器）会忽略无效指针导致的错误，这大大简化了程序员的工作。如果传递的指针引用的是有效内存，则预取单元将被指示将数据加载到缓存中，并在必要时替换其他数据。不必要的预取操作应尽量避免，因为这可能会降低缓存的有效性并消耗内存带宽（如果被淘汰的缓存行是脏数据，则可能会影响两条缓存行）。

使用`\_mm\_prefetch`内在函数的不同提示是实现定义的。这意味着每个处理器版本可以（略微）有所不同地实现它们。

一般来说，`\_MM\_HINT\_T0`提示会将数据加载到包含缓存的所有层级中（对于包含缓存而言），或者仅加载到最低级别的缓存中（对于独占缓存而言）。如果数据项已经在较高级别的缓存中，则会被加载到L1d缓存。而`\_MM\_HINT\_T1`提示则会将数据拉入L2缓存，而不将其放入L1d缓存。如果有L3缓存的话，`\_MM\_HINT\_T2`提示可以对其执行类似的操作。不过这些细节的规范性较弱，需要针对实际使用的处理器进行验证。通常而言，如果数据即将立即使用，则使用`\_MM\_HINT\_T0`是正确的做法。当然，这要求L1d缓存大小足够大以容纳所有预取的数据。如果即将使用的活跃工作集过大，将所有内容预取到L1d中则不是一个好主意，此时应考虑使用其他两种提示。

第四条提示符`\_MM\_HINT\_NTA`允许程序告知处理器以特殊方式处理预取的缓存行。NTA代表非临时对齐，我们在第6.1节中已经解释过。程序告诉处理器应尽可能避免用这些数据污染缓存，因为这些数据仅用于短时间内使用。因此，在加载时，对于包含性缓存实现而言，处理器可以避免将数据读入较低级别的缓存。当L1d中的数据被逐出时，无需将其推送到L2或更高级别的缓存中，而是可以直接写入内存。如果给出这条提示符，处理器设计者可能还会采用其他技巧。程序员在使用此提示符时必须小心：如果当前工作集过大并导致带有NTA提示符加载的缓存行被逐出，则会从内存重新加载数据。

。

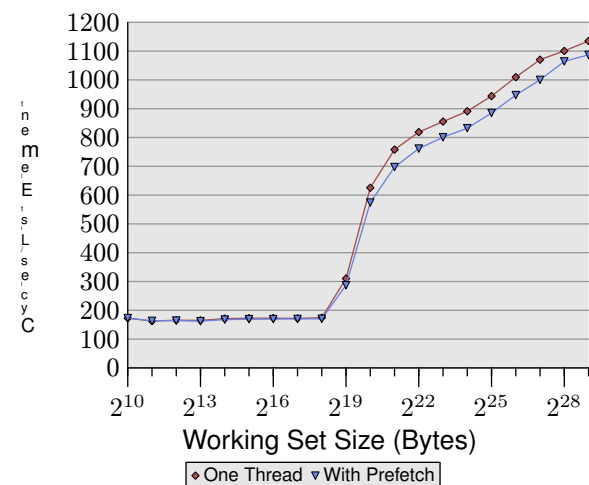


图6.7：预取时的平均值，NPAD=31

图6.7展示了使用熟悉的指针追逐框架进行测试的结果。列表在内存中随机布局。与之前的测试不同的是，程序实际上会在每个列表节点上花费一些时间（大约160个周期）。正如我们在图3.15的数据中所了解到的，一旦工作集大小超过最后一级缓存的容量，程序性能会显著下降。

我们现在可以通过提前发出预取请求来尝试改善这种情况。即，在每一轮循环中，我们预取一个新的元素。需要谨慎选择要预取的列表节点与当前正在处理的节点之间的距离。鉴于每个节点的处理时间为160个时钟周期，并且我们需要预取两个缓存行（NPAD=31），因此相距五个列表元素的距离就足够了。

图6.7的结果表明，预取确实有帮助。只要工作集大小不超过最后一级缓存的容量（该机器具有512kB = 2<sup>19</sup>B的L2缓存），各项数值完全相同。预取指令不会增加可测量的额外负担。一旦L2缓存容量被超过，预取可以节省50到60个周期，最多可达8%。虽然使用预取无法隐藏所有惩罚，但它确实能提供一些帮助。

AMD在其Opteron系列的10h家族中实现了一条新的指令：prefetchw。目前Intel平台尚无等效指令，且无法通过内联汇编接口使用。该prefetchw指令告诉CPU像其他预取指令一样将缓存行加载到L1缓存中。不同之处在于，这条缓存行会被立即置为“M”状态。如果后续没有对该缓存行进行写操作，则这会成为一个劣势。如果有一次或多次写入操作，这些写入操作将会被加速，因为无需改变缓存状态——这是在预取时已经完成的。这对于争用激烈的缓存行尤为重要，在其他处理器缓存中简单读取一条缓存行首先会使两个缓存中的状态变为“S”。

预取技术可以带来比这里实现的微薄8%更大的优势。但要正确实施预取却非常困难，尤其是当同一二进制代码需要在多种机器上都表现出良好性能时。CPU提供的性能计数器可以帮助程序员分析预取操作。可计数和采样的事件包括硬件预取、软件预取、有用的/使用的软件预取、各级缓存缺失等。在第7.1节中，我们将介绍其中一些事件。所有这些计数器都是特定于机器的。

在分析程序时，首先应关注缓存未命中。当找到大量缓存未命中的来源后，尝试为有问题的内存访问添加预取指令。这一操作应当一次在一个地方进行。通过观察测量有效预取指令的性能计数器的结果来检查每次修改的效果。如果这些计数器没有增加，则说明预取可能不正确，或者预取指令没有足够的时间从内存中加载数据，又或者是预取导致缓存中的有用数据被驱逐。

今天的gcc已经能够在一种情况下自动发出预取指令。如果一个循环正在遍历数组，可以使用以下选项：

-预取循环数组

编译器会判断预取是否合理，如果合理，则确定应该提前多远进行预取。对于小数组而言，这可能是一个缺点，并且如果在编译时无法得知数组的大小，结果可能会更差。GCC手册警告说，这种做法的效果高度依赖于代码的形式，在某些情况下甚至可能导致程序运行得更慢。程序员需要谨慎使用此选项。

### 6.3.3 特殊的预取类型：推测预取

现代处理器的乱序执行（OOO）能力允许在指令之间没有冲突的情况下重新排列这些指令。例如（这次使用IA-64作为示例）：

```
st8 [r4] = 12
add r5 = r6, r7;;
st8 [r18] = r5
```

这段代码序列将数值12存储在寄存器r4所指定的地址中，然后将寄存器r6和r7的内容相加并将结果存储在寄存器r5中。最后，它将该和值存储在由寄存器r18指定的地址中。关键点在于，由于没有数据依赖性，add指令可以在第一个st8指令之前执行或与之同时执行。但如果其中一个加数需要从内存加载会发生什么情况？

```
st8 [r4] = 12
ld8 r6 = [r8];;
add r5 = r6, r7;;
st8 [r18] = r5
```

额外的`ld8`指令从寄存器`r8`指定的地址加载值。这条加载指令与随后的加法指令之间存在明显的依赖关系（这就是为什么在这条指令后面有`;`，感谢提问）。关键在于新的`ld8`指令——不同于加法指令——不能被移动到第一个`st8`之前。处理器在解码阶段无法快速确定存储和加载是否冲突，即寄存器`r4`和`r8`是否可能具有相同的值。如果它们确实有相同值，则`st8`指令将决定载入到`r6`中的值。更糟糕的是，如果该加载指令未命中缓存，`ld8`可能会带来较大的延迟。IA-64架构为此情况支持推测性加载：

```
`ld8.a r6 = [r8];; [... 其他指令 ...] st8 [r4]
= 12`
```

清空寄存器 r6 的 v0 字段，将 r8 中的值加载到 r6；将 r7 加载到 r5 的 v1 字段中，并将 r6 的内容添加到 r5；将 r5 的 v2 字段中的值存储到由 r18 指向的位置。

新的 `\ld8.a` 和 `\ld8.c.clr` 指令属于一组，它们取代了之前代码序列中的 `\ld8` 指令。`\ld8.a` 指令是一个推测性加载指令，其值不能直接使用，但处理器可以开始执行相关工作。当到达 `\ld8.c.clr` 指令时，内容可能已经被加载（假设中间有足够的指令）。此指令的参数必须与 `\ld8.a` 指令相同。如果前面的 `\st8` 指令没有覆盖该值（即 r4 和 r8 相同），则无需执行任何操作，推测性加载完成其任务，并隐藏了加载延迟。若存储和加载确实冲突，则 `\ld8.c.clr` 会从内存重新加载该值，最终效果等同于正常的 `\ld8` 指令的语义。

推测性加载目前尚未广泛使用。但如示例所示，这是一种简单而有效的隐藏延迟的方法。预取基本上与此等效，但对于寄存器较少的处理器而言，推测性加载可能意义不大。推测性加载的一个（有时是很大的）优势在于可以直接将值加载到寄存器中，而不是加载到缓存行中，在该位置可能会再次被替换出去（例如，当线程被取消调度时）。如果具备推测功能，则应充分利用它。

### 6.3.4 辅助线程

当尝试使用软件预取时，经常会遇到代码复杂性的问题。如果代码需要遍历一个数据结构（在这种情况下是一个列表），则必须在同一循环中实现两个独立的迭代：一个是执行实际工作的正常迭代，另一个是向前查看以进行预取的迭代。这很容易变得足够复杂，以至于容易出错。

此外，还需要确定预测的范围。如果预测范围过小，则内存可能无法及时加载；而若预测范围过大，则刚刚加载的数据可能会再次被驱逐。另一个问题是，预取指令虽然不会阻塞并等待内存加载完成，但它们本身也需要时间来执行。这些指令需要解码，如果解码器过于繁忙（例如由于编写或生成的代码质量较高），则这种延迟可能变得明显。最后，循环中的代码大小会增加，这降低了 L1 缓存效率。若试图通过连续发出多个预取请求来避免部分成本（在第二个加载不依赖于第一个结果的情况下），将会遇到未完成的预取请求数量的问题。

另一种方法是将正常操作与预取完全分开进行。这可以通过使用两个正常的线程来实现。这两个线程必须确保：

显然需要调度预取线程以填充由两个线程访问的缓存。这里有两种特别值得提及的解决方案：

- 在同一核心上使用超线程（参见第29页）。在这种情况下，预取数据可以进入 L2 缓存（甚至可以进入 L1d 缓存）。
- 使用仅能执行预取和其他简单操作的“更简单”的线程，而不是 SMT 线程。这是处理器制造商可能探索的一个选项。

超线程的使用尤其引人关注。正如我们在第29页所见，如果超线程执行独立代码，则共享缓存会成为一个问题。然而，如果其中一个线程被用作预取辅助线程，则不会出现这个问题。相反，这是期望的效果，因为最低级别的缓存会被预先加载。此外，由于预取线程大部分时间处于空闲状态或等待内存访问，因此另一个超线程的正常操作不会受到太大干扰，前提是它本身不需要访问主内存。后者正是预取辅助线程所要防止的情况。

确保辅助线程不要过度领先是唯一需要小心的地方。它不能完全污染缓存，导致最早预取的值再次被驱逐。在 Linux 系统中，可以轻松使用 `futex` 系统调用 [7] 或以稍高的成本使用 POSIX 线程同步原语来完成同步。

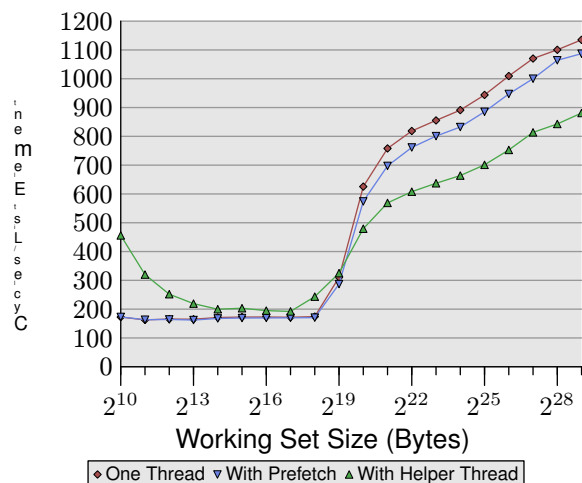


图6.8：辅助线程平均值，NPAD=31

图6.8展示了该方法的优点。这与图6.7中的测试相同，只是增加了额外的结果。新的测试创建了一个辅助线程，该线程大约提前处理了100个列表项，并读取（而不仅仅是预取）每个列表元素的所有缓存行。在这种情况下，每个列表元素有两条缓存行（在 32 位机器上，缓存行为 64 字节时，NPAD=31）。

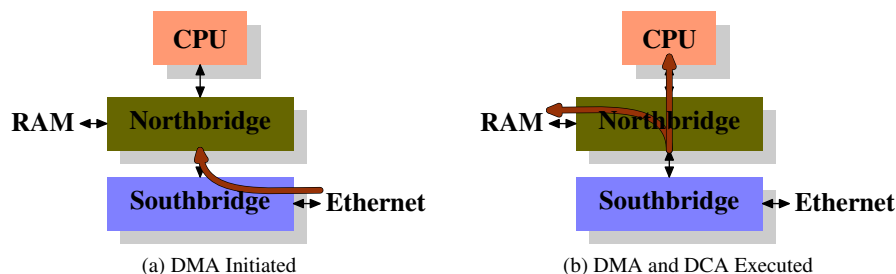


图6.9: 直接缓存访问

两个线程被调度到同一核心的两个超线程上。测试机器只有一个核心，但如果有多于一个核心，结果应该大致相同。我们将在第6.4.3节介绍的亲和性函数用于将线程绑定到适当的超线程上。

为了确定操作系统知道的哪些两个（或多个）处理器是超线程，可以使用libNUMA中的NUMA\_cpu\_level\_mask接口（参见附录D）。

```
```c#include <libNUMA.h>ssize_t NUMA_cpu_level_mask(size_t dest
size, cpu_set_t *dest, size_t srcsize, const cpu_set_t *src, unsigned int l
evel);```
```

此接口可用于确定通过缓存和内存连接的CPU层级结构。这里感兴趣的是第一级，它对应于超线程。为了在两个超线程上调度两条线程，可以使用libNUMA函数（省略错误处理代码）：

```
```cppcpu_set_t self; NUMA_cpu_self_current_mask(sizeof(self),
&self);cpu_set_t hts; NUMA_cpu_level_mask(sizeof(hts), &hts,
sizeof(self), &self, 1);CPU_XOR(&hts, &hts, &self);```
```

执行这段代码后，我们得到了两个CPU位集。self 可用于设置当前线程的亲和性，而 hts 中的掩码可用于设置辅助线程的亲和性。理想情况下，应在创建线程之前进行这些操作。在第6.4.3节中，我们将介绍设置亲和性的接口。如果没有超线程可用，NUMA\_cpu\_level\_mask 函数将返回1。这可以作为避免此优化的一个标志。

该基准测试的结果可能会令人惊讶（或者也可能不会）。如果工作集可以完全放入L2缓存中，那么其开销

辅助线程的性能损耗在10%到60%之间（大多数情况下偏向较低端，忽略最小的工作集大小，噪声太高）。这在意料之中，因为如果所有数据都已经在L2缓存中，那么预取辅助线程只是占用系统资源而没有对执行做出贡献。

一旦L2缓存容量不足时，情况就会发生变化。预取辅助线程有助于将运行时间减少约25%。尽管如此，我们仍然可以看到曲线呈上升趋势，这是因为预取操作无法被快速处理。不过，主线程执行的算术运算与辅助线程进行的内存加载操作可以相互补充。由于资源冲突最小化，这种协同效应得以产生。

该测试的结果应可应用于许多其他情况。超线程技术在这些情况下由于缓存污染问题通常不太有用，但在某些特定场景中却能发挥重要作用，应当加以利用。附录D介绍的NUMA库使得查找线程兄弟节点变得非常容易（参见该附录中的示例）。如果未提供此库，则可以通过sys文件系统使程序找到线程兄弟节点（参见表5.3中的thread\_siblings列）。一旦获取了这些信息，程序只需定义线程的亲和性，然后以两种模式运行循环：正常操作模式和预取模式。内存预取的数量应根据共享缓存的大小来决定，在此示例中L2缓存大小相关，并且程序可以通过查询获得其大小。

```
sysconf(_SC_LEVEL2_CACHE_SIZE)
```

是否需要限制辅助线程的进度取决于程序本身。通常最好确保有一定的同步机制，因为否则调度细节可能会导致显著的性能下降。

### 6.3.5 直接缓存访问

现代操作系统缓存未命中的一大来源是处理传入的数据流量。现代硬件，如网络接口卡（NIC）和磁盘控制器，具有将接收到的或读取的数据直接写入的能力。

内存操作无需涉及CPU。这对于当今设备的性能至关重要，但也带来了问题。假设一个来自网络的入站数据包：操作系统必须通过查看数据包头部来决定如何处理它。网卡将数据包放入内存中，然后通知处理器有新数据到达。由于处理器不知道数据何时会到达，甚至可能不清楚具体存储位置，因此无法预取数据，结果在读取头部时会发生缓存未命中。

英特尔在其芯片组和CPU中添加了技术来缓解这一问题[14]。其核心思想是将即将接收数据包的CPU缓存填充上该数据包的数据。此处数据包的有效载荷并不关键，这些数据通常会由更高层次的功能处理，无论是内核层面还是用户层面。而数据包头部用于决定如何处理该数据包，因此这部分信息需要立即获取。

网络I/O硬件已经具备直接内存访问（DMA）功能，可以将数据包写入内存控制器。这意味着它可以直接与北桥中的内存控制器通信（假设内存控制器未集成到CPU中）。内存控制器的另一端则是通过前端总线（FSB）与处理器进行接口连接。

直接缓存访问（DCA）的思想是扩展网卡与内存控制器之间的协议。图6.9中的第一个图展示了在具有北桥和南桥的常规机器中DMA传输开始时的情况。网卡连接到（或作为一部分集成于）南桥。它发起DMA访问，但提供了应推入处理器缓存的新包头信息。

传统的做法是在第二步中，DMA传输会直接与内存连接完成传输。对于带有DCA标志的DMA传输，北桥除了在前端总线上发送数据外，还会使用一个新的特殊DCA标志。处理器始终监控前端总线，如果识别到DCA标志，则尝试将指向处理器的数据加载到最低级别的缓存中。实际上，DCA标志只是一个提示；处理器可以自由选择是否忽略它。DMA传输完成后，会向处理器发出信号。

在处理数据包时，操作系统首先需要确定数据包的类型。如果未忽略DCA提示，则操作系统为识别数据包而执行的加载操作很可能命中缓存。将每包节省的数百个周期乘以每秒可以处理的数万个数据包，这些节省下来的周期会累积成非常显著的数量，尤其是在延迟方面更是如此。

没有集成I/O硬件（在此指网络接口卡）、芯片组和CPU，这样的优化是不可能实现的。因此，如果需要这项技术，则必须明智地选择平台。

## 6.4 多线程优化

当谈到多线程时，有三个方面与缓存的使用密切相关：

- 并发性
- 原子性
- 带宽

这些方面也适用于多进程情况，但由于多个进程大多是独立的，因此很难对其进行优化。可能的多进程优化方法是多线程场景下可用优化方法的一个子集。所以在这里我们将专门讨论后者。

在此上下文中，并发指的是进程在同一时间运行多个线程时所经历的内存效应。线程的一个特性是它们共享同一个地址空间，因此可以访问相同的内存区域。理想情况下，线程大多数时候使用的内存区域各不相同，在这种情况下这些线程之间的耦合度较低（例如共同输入和/或输出）。如果一个以上的线程使用同一数据，则需要协调；此时原子性就发挥作用了。最后，根据机器架构的不同，处理器可用的内存以及处理器间总线带宽是有限制的。我们将在这几个部分分别处理这三个方面——尽管它们当然紧密相关。

### 6.4.1 并发优化

最初，在本节中，我们将讨论两个相互矛盾的优化问题。一个多线程应用程序在其某些线程中使用共享数据。常规缓存优化要求将数据集中存放，以减少程序的内存占用，从而在任何时刻都能使更多的内存内容适应缓存。

虽然这种方法存在问题：如果多个线程写入同一内存位置，那么每个相应核心的L1d缓存中该缓存行必须处于‘E’（独占）状态。这意味着在最坏情况下，每次写访问都会发送一个RFO消息，导致普通的写操作突然变得非常昂贵。如果使用相同的内存位置，则需要同步（可能通过原子操作来实现，这将在下一节中讨论）。即使所有线程都在使用不同的内存位置并且看似是独立的，这个问题也是显而易见的。

图6.10展示了这种“伪共享”的结果。测试程序（见附录A.3）创建了若干线程，这些线程唯一的工作就是对一个内存位置进行递增操作（共5亿次）。测量的时间是从

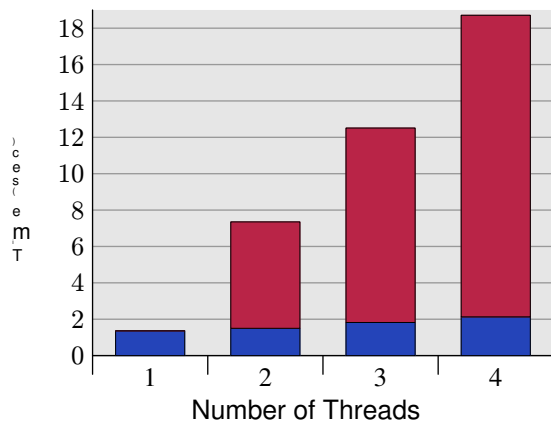


图6.10: 并发缓存行访问开销

程序从启动到等待最后一个线程结束为止的整个运行期间。线程被固定在单独的处理器上。该机器有四颗P4处理器。蓝色数值代表每个线程的内存分配位于不同的缓存行的情况下的运行结果。红色部分表示当将线程的位置移动到仅一个缓存行时所遭受的惩罚。

蓝色测量结果（使用单独缓存行所需的时间）符合预期。该程序在多线程情况下性能无损扩展。每个处理器将其缓存行保留在自己的L1d缓存中，且由于代码和数据读取量很小（实际上都被缓存了），因此没有带宽问题。所测量到的轻微增加实际上是系统噪声以及可能的一些预取效应（线程使用顺序缓存行）。

测量的开销是通过比较使用单一缓存行与为每个线程单独分配一个缓存行所需的时间来计算得出，分别为390%、734%和1,147%。这些较大的数字乍一看可能会令人惊讶，但考虑到所需的缓存交互情况，这应该是显而易见的。当某个处理器刚刚完成对缓存行的写操作后，该缓存行就会被拉取到另一个处理器的缓存中。除了当前持有缓存行的处理器外，所有其他处理器都会被延迟且无法执行任何操作。每个额外增加的处理器只会导致更多的延迟。

从这些测量结果可以看出，必须避免在程序中出现这种情况。鉴于巨大的性能损失，在许多情况下这个问题是显而易见的（分析工具会显示出代码位置），但现代硬件存在一个陷阱。图6.11展示了在同一处理器上运行代码时的等效测量结果（Intel Core 2 QX 6700，四核单处理器机器）。即使该处理器有两个独立的L2缓存，在测试案例中也没有表现出任何可扩展性问题。当重复使用同一个缓存行时会有一些额外开销，但这个开销并不会随着核心数量的增加而增大。<sup>36</sup> 如果

<sup>36</sup>I cannot explain the lower number when all four cores are used but it is reproducible.

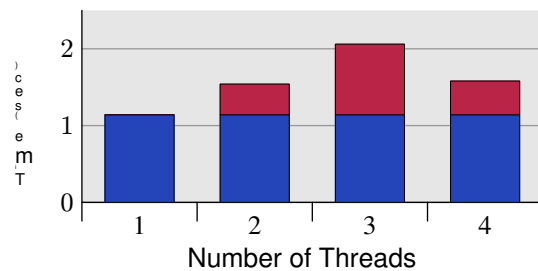


图6.11: 开销，四核

当然会看到与图6.10类似的结果。尽管多核处理器的使用越来越普遍，但仍有许多机器将继续使用多个处理器，因此正确处理这种场景非常重要，这可能意味着需要在真实的SMP机器上测试代码。

有一种非常简单的“解决办法”：将每个变量放在它自己的缓存行中。这里就出现了与之前提到的优化方法之间的冲突，具体来说，应用程序的内存占用会大幅增加。这显然是不可接受的；因此，有必要提出一个更为智能的解决方案。

需要识别哪些变量是由单个线程独占使用的，哪些变量在整个程序运行过程中仅被一个线程使用过，以及那些在某些时刻会被多个线程竞争的变量。针对这些不同场景可以采取不同的解决方案，并且这些方案都是有用的。区分这些变量的基本标准是：它们是否会被写入，以及这种写操作发生的频率如何。

从未被写入的变量以及仅初始化一次的变量基本上可以视为常量。由于RFO（读取并修改为独占状态）消息仅在写操作时需要，因此常量可以在缓存中共享（‘S’状态）。所以，这些变量无需特殊处理；将它们分组在一起是可以接受的。如果程序员正确地使用const关键字标记这些变量，则工具链会将这些变量从普通变量移动到.rodata（只读数据）或.data.rel.ro（重定位后为只读）段中。在这种情况下不需要采取其他特殊措施。如果由于某种原因无法正确使用const关键字标记变量，程序员可以通过将其分配给一个特殊的段来影响其位置。

当链接器构建最终的二进制文件时，它首先将所有输入文件中同名的部分追加在一起；然后根据链接脚本确定这些部分的顺序。这意味着，通过将那些实际上是常量但未被标记为常量的所有变量移至一个特殊部分，程序员可以将这些变量集中起来。这样就不会在它们之间频繁写入其他变量。通过适当对齐该部分中的第一个变量，

<sup>37</sup>Sections, identified by their names are the atomic units containing code and data in an ELF file.

可以保证不会发生伪共享。假设这个小例子：

```
``cint foo = 1; int bar __attribute__((section(".data.ro"))) = 2; int baz = 3; int xyzy __attribute__((section(".data.ro"))) = 4;``
```

如果编译器处理此输入文件，则会定义四个变量。有趣的是，变量 `foo` 和 `baz` 分别与 `bar` 和 `xyzy` 成对分组。如果没有属性定义，编译器将按照源代码中定义的顺序为这四个变量分配内存，并将其放置在一个名为 `.data`。<sup>38</sup> 的段落中。然而，根据当前代码，变量 `bar` 和 `xyzy` 被放在一个名为 `.data.ro` 的段落里。`.data.ro` 这个段名是任意选择的，但以 `.data.` 为前缀可以确保 GNU 链接器将该段与其他数据段一起放置。

同样的技术也可以应用于分离那些主要被读取但偶尔也会被写入的变量。只需选择一个不同的节名即可。这种分离在某些情况下似乎是有意义的，例如在Linux内核中。

如果一个变量仅由一个线程使用，可以采用另一种方式来指定该变量。在这种情况下，使用线程局部变量是有用且可行的（参见[8]）。在 `gcc` 中，C 和 C++ 语言允许使用 `__thread` 关键字定义线程特定的变量。

```
int foo = 1; __thread int bar = 2; int baz = 3; __thread int xyzy = 4;
```

变量 `bar` 和 `xyzy` 不在正常的数据段中分配；相反，每个线程都有自己的独立区域来存储此类变量。这些变量可以具有静态初始化器。所有线程局部变量都可以被其他所有线程访问到，但是除非一个线程将指向线程局部变量的指针传递给其他线程，否则其他线程无法找到该变量。由于变量是线程局部的，因此不会出现伪共享问题——除非程序人为地制造了这个问题。这种解决方案易于设置（编译器和链接器会完成所有工作），但它也有其代价。当创建一个线程时，它需要花费一些时间来初始化线程局部变量，这需要时间和内存。此外，访问线程局部变量通常比使用全局或自动变量更昂贵（参见[8]以了解如何在可能的情况下最小化成本的解释）。

<sup>38</sup>This is not guaranteed by the ISO C standard but it is how gcc works.

使用线程本地存储（TLS）的另一个缺点是，如果变量的使用转移到了另一个线程，则旧线程中变量的当前值对新线程不可用。每个线程中的变量副本都是独立的。通常这根本不是问题，但如果确实存在问题，则在切换到新线程时需要进行协调，在此期间可以复制当前值。

更大的问题是资源可能被浪费。如果任何时候只有一个线程使用某个变量，所有线程都必须为此付出内存方面的代价。如果一个线程不使用任何TLS（线程局部存储）变量，则懒加载的TLS内存区域分配机制可以避免这个问题（除了应用程序本身的TLS）。但如果一个线程在一个动态共享对象中仅使用了一个TLS变量，该对象中的其他所有TLS变量的内存也会被分配。在大规模使用TLS变量的情况下，这种情况可能会累积造成问题。

总的来说，最好的建议是

至少将只读（初始化后）变量与读写变量分开。可以进一步将几乎只读的变量作为第三类进行分离。

将一起使用的读写变量分组成一个结构体。使用结构体是确保所有这些变量的内存位置彼此靠近且被所有版本的gcc一致翻译的唯一方法。

将经常被不同线程写入的读写变量移到它们自己的缓存行上。这可能意味着在末尾添加填充以填满缓存行的剩余部分。如果与步骤2结合，这样做通常并不会真正浪费资源。延续上面的例子，我们可能会得到如下代码（假设 `bar` 和 `xyzy` 是配合使用的）：

```
int foo = 1;
int baz = 3;
struct {
 struct all {
 int bar;
 int xyzy;
 };
 char pad[CLSIZE - sizeof(struct all)];
} rwstruct __attribute__((aligned(CLSIZE))) =
{ { .bar = 2, .xyzy = 4 } };
```

一些代码更改是必要的（对 `bar` 的引用需要替换为 `rwstruct.bar`，同样适用于 `xyzy`），但仅此而已。编译器和链接器会完成其余的工作。<sup>39</sup>

4. 如果一个变量被多个线程使用，但每次使用都是独立的，则将该变量移入线程本地存储（TLS）。

<sup>39</sup>So far this code has to be compiled with `-fms-extensions` on the command line.

<pre>for (i = 0; i &lt; N; ++i)     __sync_add_and_fetch(&amp;var, 1);</pre>	<pre>for (i = 0; i &lt; N; ++i)     __sync_fetch_and_add(&amp;var, 1);</pre>	<pre>for (i = 0; i &lt; N; ++i) {     long v, n;     do {         v = var;         n = v + 1;     } while (!__sync_bool_compare_and_swap(&amp;var,                                            v, n)); }</pre>
1. Add and Read Result	2. Add and Return Old Value	3. Atomic Replace with New Value

图6.12: 循环中的原子增量操作

## 6.4.2 原子性优化

如果多个线程同时修改同一内存位置，处理器不会保证任何特定的结果。这是一个有意作出的决定，旨在避免在99.999%的情况下不必要的开销。例如，当一个内存位置处于“S”状态时，如果有两个线程需要同时增加其值，执行流水线不必等待缓存行进入“E”状态后再从缓存中读取旧值进行加法操作。相反，它会直接读取当前缓存中的值，并在缓存行进入“E”状态后写回新值。如果两个线程的两次缓存读取同时发生，则结果将与预期不符；一次加法将会丢失。

在并发操作可能发生的情况下，处理器提供了原子操作。例如，在确保对内存位置的加法操作可以以原子方式完成之前，不会读取旧值。除了等待其他核心和处理器之外，一些处理器甚至会将特定地址的原子操作信号发送给主板上的其他设备。所有这些都使得原子操作变得更慢。

处理器供应商决定提供不同的原子操作集。早期的RISC处理器，遵循“简化”的理念，提供的原子操作非常有限，有时仅包括原子位设置和测试操作。在另一端，我们有x86和x86-64架构，它们提供了大量的原子操作。常见的原子操作可以分为四类：

位测试操作以原子方式设置或清除一个位，并返回一个状态指示该位在操作前是否已被设置。

负载锁定/存储条件（LL/SC）<sup>41</sup> LL/SC 操作成对工作，其中特殊加载指令用于启动事务，并且最终的存储操作仅在该位置在此期间未被修改的情况下才能成功。存储操作会指示成功或失败，因此程序可以在必要时重复尝试。

比较并交换（CAS）这是一种三元操作，仅当当前值与第三个参数提供的值相同的情况下，才将第二个参数指定地址处的值更改为第一个参数提供的值。

原子算术操作 这些操作仅在x86-64架构上可用，能够对内存位置执行算术和逻辑运算。这些处理器支持非原子版本的此类操作，而RISC架构则不提供。因此，它们的应用范围受到限制也就不足为奇了。

一个架构要么支持LL/SC指令，要么支持CAS指令，不会同时支持两者。这两种方法基本上是等价的；它们都能同样有效地实现原子算术操作，但目前CAS似乎是更优选的方法。所有其他操作都可以间接使用它来实现。例如，原子加法：

```
``cint curval;int newval;do { curval = var; newval = curval + addend;} while (!CAS(&var, curval, newval));``
```

CAS调用的结果表明操作是否成功。如果返回失败（非零值），则循环重新执行，进行加法运算，并再次尝试CAS调用。此过程重复直至成功为止。值得注意的是，内存位置的地址需要在两条单独的指令中计算出来。LL/SC，代码看起来大致相同：

```
``cint 当前值; int 新值; do { 当前值 = v8 LL(var); 新值 = v9 当前值 + addend;} while (v10 SC(var, 新值));``
```

<sup>40</sup>HP Parisc still does not provide more...

<sup>41</sup>Some people use “linked” instead of “lock”, it is all the same.

<sup>42</sup>The CAS opcode on x86 and x86-64 can avoid the load of the value in the second and later iterations but, on this platform, we can write the atomic addition in a simpler way, with a single addition opcode.

在这里，我们必须使用一个特殊的加载指令（LL），并且不需要将当前内存位置的值传递给SC，因为处理器知道在此期间内存位置是否已被修改。

The big differentiators are x86 and x86-64, where we have the atomic operations, and here it is important to select the proper atomic operation to achieve the best result. Figure 6.12 shows three different ways to implement an atomic increment operation. All three produce different code on x86 and x86-64 while the code might be identical on other architectures. There are huge performance differences. The following table shows the execution time for 1 million increments by four concurrent threads. The code uses the built-in primitives of gcc (`__sync_*`).

1. Exchange Add	2. Add Fetch	3. CAS
0.23s	0.21s	0.73s

前两个数字相似；我们可以看到返回旧值略快一些。重要的是高亮显示的字段，即使用CAS操作的成本。不出所料，这要昂贵得多。原因有多个：1. 需要执行两次内存操作，2. CAS操作本身更复杂，并且需要条件判断，3. 整个操作必须在一个循环中完成，以防两个并发访问导致CAS调用失败。

现在读者可能会提出一个问题：为什么有人会使用复杂且冗长的代码来利用CAS指令？答案是：这种复杂性通常被隐藏起来了。如前所述，目前CAS是所有有趣架构中的统一原子操作。因此，有些人认为仅通过CAS定义所有的原子操作就足够了，这使得程序变得更简单。但是，正如数字所显示的那样，结果往往远非最优。CAS解决方案的内存处理开销非常大。以下展示了两个各自在独立核心上运行的线程的执行情况。

Thread #1	Thread #2	var Cache State
<code>v = var</code>		'E' on Proc 1
<code>n = v + 1</code>	<code>v = var</code>	'S' on Proc 1+2
<code>CAS(var)</code>	<code>n = v + 1</code>	'E' on Proc 1
	<code>CAS(var)</code>	'E' on Proc 2

在这一短暂的执行期间，缓存行的状态至少发生了三次变化；其中两次变化是由RFO引起的。此外，第二次CAS操作将会失败，因此该线程必须重新执行整个操作。在此过程中，同样的情况可能会再次发生。

相比之下，当使用原子算术操作时，处理器可以将执行加法（或其他运算）所需的加载和存储操作保持在一起。它可以确保并发发出的缓存行请求在原子操作完成之前被阻塞。

每个循环迭代因此最多会生成一个回写缓存请求，不会再有其他操作。

这意味着必须在可以利用原子算术和逻辑运算的层次上定义机器抽象。CAS不应被普遍用作统一机制。

对于大多数处理器而言，原子操作本身总是具有原子性。只有在不需要保持原子性的情况下，通过为这些情况提供完全独立的代码路径才能避免使用原子操作。这意味着需要更多的代码、条件判断以及进一步的跳转以直接执行适当的指令。

对于x86和x86-64架构而言，情况有所不同：相同的指令既可以用于原子操作也可以用于非原子操作。为了使这些指令具有原子性，需要在指令前添加一个特殊的前缀——锁前缀（lock前缀）。这使得在特定情况下不需要原子性的要求时，可以避免高昂的开销。例如，在编写库代码时，如果需要确保线程安全，则可以从这种机制中受益。编写代码时无需提供额外信息，该决策可以在运行时做出。技巧在于跳过锁前缀。此技巧适用于所有x86和x86-64处理器允许添加lock前缀的指令。

```
cmp $0, multiple_threads je 1f lock
```

请提供需要翻译的英文学术文本内容。目前提供的信息中缺少具体的待翻译内容。

如果这段汇编代码看起来晦涩难懂，不要担心，它其实很简单。第一条指令用于检查一个变量是否为零。在这个上下文中，非零值表示有多个线程在运行。如果该值为零，则第二条指令跳转到标签1。否则，执行下一条指令。这是关键部分：如果je指令没有发生跳转，则带有lock前缀的add指令被执行；否则，不带lock前缀的add指令被执行。

添加一个可能代价高昂的操作，如条件跳转（如果分支预测错误则代价高昂），似乎是有害的。确实如此：如果多个线程大多数时间都在运行，性能会进一步下降，尤其是当分支预测不正确时。但如果有很多情况下只有一个线程在使用，则代码速度显著提高。使用if-then-else结构的替代方案会在两种情况中都引入一个无条件跳转，这可能会更慢。鉴于一次原子操作大约耗时200个周期，采用该技巧（或if-then-else块）的临界点非常低。这确实是一种值得牢记的技术。不幸的是，这意味着gcc的\_\_sync\_\*原语无法使用。

当使用多个线程时，即使这些线程不会因为在同一颗处理器的不同核心上使用相同的缓存行而产生缓存争用，仍然可能存在潜在问题。每个处理器到内存的最大带宽由该处理器上的所有核心和超线程共享。根据机器架构（例如图2.1所示的架构），多个处理器可能会通过同一个总线或北桥连接到同一块内存。

处理器核心本身运行在这样的频率下，即使在理想条件下全速运行时，与内存的连接也无法满足所有加载和存储请求而无需等待。进一步将可用带宽除以共享到北桥连接的核心数、超线程数以及处理器数量，突然间并行性就变成了一个大问题。高效的程序可能会受到可用内存带宽的限制而无法发挥最佳性能。

图3.32显示，提高处理器前端总线（FSB）的速度可以带来很大帮助。因此，在处理器核心数量不断增加的情况下，我们也会看到FSB速度的提升。然而，如果程序使用了较大的工作集并且已经足够优化的话，这仍然不够。程序员需要准备好识别由于带宽限制引起的问题。

现代处理器的性能测量计数器允许观察前端总线（FSB）的竞争情况。在Core 2处理器中，NUS\_BNR\_DRV事件统计由于总线未准备好而导致核心等待的周期数。这表明总线使用率很高，从主内存加载数据或存储数据所需的时间比平时更长。Core 2处理器支持更多可以计数特定总线操作（如RFO）或总体FSB利用率的事件。后者在开发期间调查应用程序可扩展性的可能性时可能会很有用。如果总线利用率接近1.0，则应用程序的可扩展性机会将非常有限。

如果识别到带宽问题，可以采取几种措施。这些方法有时会相互矛盾，因此可能需要进行一些实验来确定最佳方案。一种解决方案是购买更快的计算机，如果有此类设备可供选择的话。增加前端总线（FSB）速度、使用更快速的RAM模块，并且有可能使用靠近处理器的本地内存，都可以——并且很可能会有所帮助。不过这可能会花费大量资金。如果某个程序仅在一台（或少数几台）机器上运行，则硬件的一次性购置费用可能比重新编写该程序的成本更低。然而，通常来说，最好还是从改进程序入手。

在优化程序代码以避免缓存未命中之后，提高带宽利用率的唯一选项就是更好地将线程分配到可用的核心上。默认情况下，内核中的调度器会根据自身的策略将一个线程分配给处理器。当从一个核心移动线程到另一个核心时，通常会被避免。

调度器实际上并不了解工作负载的详细情况。它可以从缓存未命中等信息中收集一些数据，但在许多情况下这些信息帮助不大。

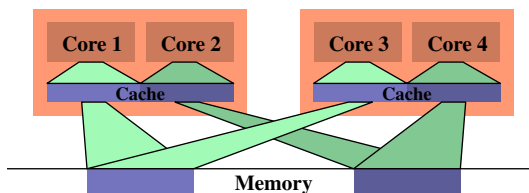


图6.13: 无效调度

当两个线程被调度到不同的处理器（或不同缓存域中的核心）上，并且它们使用相同的数据集时，会大量占用内存总线。图6.13展示了这种情况。核心1和核心3访问相同的数据（通过访问指示器和内存区域的颜色一致来表示）。同样地，核心2和核心4也访问相同的数据。但是这些线程被调度到了不同的处理器上。这意味着每个数据集都必须从内存中读取两次。这种情况下可以有更好的处理方式。

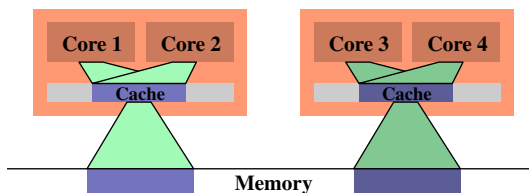


图6.14: 高效调度

如图6.14所示，理想情况下应呈现的状态。现在使用的总缓存大小减少了，因为核心1和2以及核心3和4共享同一数据集。数据集只需从内存读取一次。

这是一个简单的例子，但由此可以扩展到许多其他情况。如前所述，内核中的调度器无法了解数据的使用情况，因此程序员必须确保调度操作高效进行。实际上，可供传达这一需求的内核接口并不多，只有一个：定义线程亲和性。

线程亲和性是指将一个或多个线程分配给一个或多个核心。调度程序在决定运行该线程的位置时，只会在这几个核心中选择（仅限这些核心）。即使其他核心处于空闲状态也不会被考虑。这听起来可能像是一个缺点，但实际上这是必须付出的代价。如果过多的线程独占一组核心，则剩余的核心可能会大部分时间都处于闲置状态，唯一的解决办法是更改亲和性设置。默认情况下，线程可以在任何核心上运行。

有若干接口可用于查询和更改线程的亲和性：

```
``c#define _GNU_SOURCE
#include <sched.h>``
``cint sched_setaffinity(pid_t pid, size_t size, const cpu_set_t *cpuset);
int sched_getaffinity(pid_t pid, size_t size, cpu_set_t *cpuset);``
```

这两个接口用于单线程代码。pid 参数指定了要更改或确定哪个进程的亲和性。调用者显然需要相应的权限才能执行此操作。第二个和第三个参数指定核心的位掩码。第一个函数要求填充该位掩码，以便设置亲和性。第二个函数使用所选线程的调度信息来填充位掩码。这些接口在 <sched.h> 中声明。

在该头文件中还定义了 cpu\_set\_t 类型以及若干用于操作和使用此类对象的宏。

```
``c#define _GNU_SOURCE #include <sched.h> #define CPU_SETSIZE #define CPU_SET(cpu, cpusetp) #define CPU_CLR(cpu, cpusetp) #define CPU_ZERO(cpusetp) #define CPU_ISSET(cpu, cpusetp) #define CPU_COUNT(cpusetp) ``
```

CPU\_SETSIZE 指定了可以在数据结构中表示的 CPU 数量。其余三个宏用于操作 cpu\_set\_t 对象。初始化对象时应使用 CPU\_ZERO；另外两个宏则用于选择或取消选择单个核心。CPU\_ISSET 用于测试特定处理器是否属于该集合。CPU\_COUNT 返回集合中选定的核心数量。cpu\_set\_t 类型提供了一个合理的默认值，作为可表示的 CPU 数量上限。随着时间推移，这个数值无疑会显得太小；到那时类型将被调整。这意味着程序始终需要考虑大小限制。上述便利宏根据 cpu\_set\_t 的定义隐式处理了大小问题。如果需要更动态的大小管理，则应使用一组扩展宏：

```
``c#define _GNU_SOURCE
#include <sched.h>``
#define CPU_SET_S(cpu, setsize, cpusetp) #define CPU_CLR_S(cpu, setsize, cpusetp) #define CPU_ZERO_S(setsize, cpusetp) #define CPU_ISSET_S(cpu, setsize, cpusetp) #define CPU_COUNT_S(setsize, cpusetp)
```

这些接口多了一个大小参数。为了能够动态分配不同大小的CPU集，提供了三个宏定义：

```
``c#define _GNU_SOURCE #include <sched.h> #define CPU_ALLOC_SIZE(count) #define CPU_ALLOC(count) #define CPU_FREE(cpuset) ``
```

‘CPU\_ALLOC\_SIZE’ 宏的返回值是指定可以处理 ‘count’ 个 CPU 的 ‘cpu\_set\_t’ 结构所需的字节数。为了分配这样的内存块，可以使用 ‘CPU\_ALLOC’ 宏。通过这种方式分配的内存必须调用 ‘CPU\_FREE’ 进行释放。这些宏在幕后可能会使用 ‘malloc’ 和 ‘free’，但这不一定始终如此。

最后，定义了若干针对CPU集对象的操作：

```
#define _GNU_SOURCE
#include <sched.h>
#define CPU_EQUAL(cpuset1, cpuset2)
#define CPU_AND(destset, cpuset1, cpuset2)
#define CPU_OR(destset, cpuset1, cpuset2)
#define CPU_XOR(destset, cpuset1, cpuset2)
#define CPU_EQUAL_S(setsize, cpuset1, cpuset2)
#define CPU_AND_S(setsize, destset, cpuset1, cpuset2)
#define CPU_OR_S(setsize, destset, cpuset1, cpuset2)
#define CPU_XOR_S(setsize, destset, cpuset1, cpuset2)
```

这两组四个宏可以用于检查两组是否相等，并对集合执行逻辑与、或和异或操作。这些操作在使用某些libNUMA函数时非常有用（参见附录D）。

一个进程可以使用 sched\_getcpu 接口来确定它当前正在哪个处理器上运行。

```
``c#define _GNU_SOURCE #include <sched.h> int sched_getcpu(void);``
```

结果是CPU在CPU集中的索引。由于调度的特性，这个数字并不总是100%准确。线程可能在返回结果和回到用户级之间被移动到不同的CPU上。程序必须始终考虑这种不准确性。更重要的是，在任何情况下，线程允许运行的CPU集合。此集合可以通过 sched\_getaffinity 获取。该集合会被子线程和进程继承。线程不能依赖于其在整个生命周期中的稳定性。亲和性掩码可以从外部设置（参见原型中的pid参数）。

以上；Linux还支持CPU热插拔，这意味着CPU可以从系统中消失——因此也会从亲和力CPU集（affinity CPU set）中消失。

在多线程程序中，各个线程正式上没有由POSIX定义的进程ID，因此上述两个函数无法使用。相反，《pthread.h》声明了四个不同的接口：

```
#define _GNU_SOURCE #include <pthread.h> int pthread_setaffinity_np(pthread_t th, size_t size, const cpu_set_t *cpuset);int pthread_getaffinity_np(pthread_t th, size_t size, cpu_set_t *cpuset);int pthread_attr_setaffinity_np(pthread_attr_t *at, size_t size, const cpu_set_t *cpuset);int pthread_attr_getaffinity_np(pthread_attr_t *at, size_t size, cpu_set_t *cpuset);
```

第一个两个接口基本上与我们已经看到的两个接口相同，只是它们的第一个参数使用线程句柄而不是进程ID。这允许在进程中单独指定各个线程。这也意味着这些接口不能从另一个进程中使用，仅限于在同一进程中使用。第三个和第四个接口使用线程属性。这些属性用于创建新线程时设置。通过设置属性，可以从一开始就将线程调度到特定的一组CPU上运行。与在线程启动后选择目标处理器相比，在此阶段进行选择可以在多个层面上带来优势，包括（尤其是）内存分配（参见第6.5节中的NUMA）。

说到NUMA，亲和性接口在NUMA编程中也扮演着重要角色。我们稍后会回到这个话题。

到目前为止，我们讨论了两个线程的工作集重叠的情况，此时将这两个线程安排在同一核心上是有意义的。相反的情况也可能存在：如果两个线程处理的是独立的数据集，那么将它们调度在同一个核心上可能会成为一个问题。两个线程会争夺相同的缓存，从而降低彼此有效使用缓存的效果。其次，需要将两个数据集都加载到同一缓存中；实际上这增加了必须加载的数据量，因此可用带宽减半。

在这种情况下，需要设置线程的亲和性，以确保它们不能被调度到同一核心上。这与之前的情况相反，因此在进行任何更改之前，理解要优化的情形是非常重要的。

优化缓存共享以提升带宽性能是关键所在。

实际上，NUMA编程的一个方面将在下一节中介绍。只需将“内存”的概念扩展到缓存即可。随着缓存级别的增加，这一点将变得越来越重要。因此，在NUMA支持库中提供了解决多核调度问题的方案。请参见附录D中的代码示例，以了解如何在硬编码系统细节或深入研究/sys文件系统的深度的情况下确定亲和性掩码。

## 6.5 NUMA编程

对于NUMA编程而言，迄今为止所提到的所有关于缓存优化的内容同样适用。差异仅在更低层次开始显现。NUMA引入了访问地址空间不同部分时的不同成本。在具有统一内存访问的情况下，我们可以进行优化以最小化缺页异常（参见第7.5节），但这基本上就是全部了。所有页面都是平等的。

NUMA 改变了这一点。访问成本可能取决于所访问的页面。不同的访问成本也增加了优化内存页本地性的必要性。由于 Intel（针对 x86、x86-64 和 IA-64）和 AMD（针对 Opteron）都在使用 NUMA，大多数 SMP 机器不可避免地会采用 NUMA 架构。随着每个处理器核心数量的增加，我们可能会看到用于非数据中心环境以及对 CPU 使用要求极高的办公环境中 SMP 系统的数量显著减少。对于大多数家用计算机来说，仅需一个处理器即可满足需求，因此不会出现 NUMA 相关问题。但这 a) 并不意味着程序员可以忽视 NUMA 的存在，b) 也不意味着没有相关的其他问题。

如果考虑到非统一内存访问（NUMA）的扩展，很快就会发现这一概念也可以应用于处理器缓存。使用相同缓存的核心上的两个线程比不共享缓存的核心上的线程协作更快。这并不是一个虚构的例子：

- 早期的双核处理器没有共享L2缓存。
- 英特尔的Core 2 QX 6700和QX 6800四核芯片配备了两个独立的L2缓存。
- 如早先所料，随着芯片上核心数量的增加以及统一缓存的需求，我们将拥有更多层级的缓存。

缓存形成了自己的层次结构；线程在核心上的放置对于共享（或不共享）各种缓存变得重要。这与非一致性内存访问架构（NUMA）面临的问题并无太大差异，因此这两个概念可以统一起来。即使是只关注非对称多处理器（SMP）机器的人也应阅读本节内容。

在第5.3节中，我们看到Linux内核提供了许多对NUMA编程有用且必要的信息。然而，收集这些信息并不容易。目前可用的NUMA库

在Linux上的实现对此目的完全不够。作者目前正在构建一个更为合适的版本。

现有的NUMA库libnuma，作为numactl套件的一部分，不提供对系统架构信息的访问。它只是一个围绕可用系统调用的封装，并附带一些常用操作的便捷接口。当前Linux系统上可用的系统调用包括：

mbind 绑定指定的内存页。

设置内存绑定策略（set\_mempolicy）设置默认的内存绑定策略。注意：为了严格遵循给定规则，这里保留了英文原文中的函数名“set\_mempolicy”不作翻译，并直接将描述性名称作为符合中文习惯的表述方式。由于原句非常简短，直接翻译后保持原有结构即可。

migrate\_pages 将指定节点集上某个进程的所有页面迁移到不同的节点集。

移动页面 将选定的页面移至指定节点 或 请求关于页面的信息。

这些接口声明在与libnuma库一起提供的<numaif.h>头文件中。在深入探讨之前，我们必须理解内存策略的概念。

### 6.5.1 内存策略

定义内存策略的目的是允许现有代码在不进行重大修改的情况下，在NUMA环境中合理运行。该策略会被子进程继承，从而可以使用numactl工具。此工具可用于启动具有指定策略的应用程序等。

Linux内核支持以下策略：

使用 MPOL\_BIND 策略时，内存仅从指定的一组节点分配。如果无法满足此要求，则分配失败。

MPOL\_PREFERRED策略下，内存优先从指定节点集分配。若无法满足，则考虑从其他节点分配内存。

MPOL\_INTERLEAVE 内存从指定的节点中均匀分配。对于基于 VMA 的策略，节点的选择依据是虚拟内存区域中的偏移量；对于基于任务的策略，则通过一个连续递增的计数器来选择节点。

MPOL\_DEFAULT 根据区域的默认设置进行分配。

此列表似乎递归地定义了策略。这种说法只说对了一半。实际上，内存策略形成一个层次结构（见图6.15）。如果某个地址受VMA策略覆盖，则使用该策略。有一种特殊的策略用于

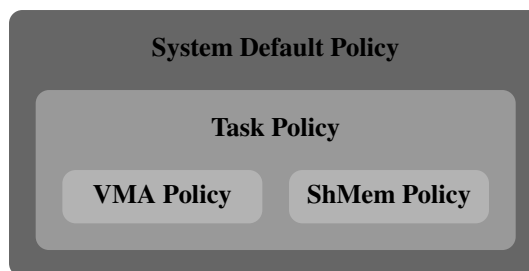


图6.15: 内存策略层次结构

共享内存段。如果特定地址没有相应的策略，则使用任务的策略。如果任务也没有指定策略，则使用系统的默认策略。

系统默认情况下会为请求内存的线程分配本地节点的内存。默认不提供任务和虚拟内存区域（VMA）策略。对于一个多线程进程，本地节点是该进程最初运行的那个“家”节点。上述提到的系统调用可以用来选择不同的策略。

### 6.5.2 指定策略

调用 set\_mempolicy 可以用于设置当前线程（在内核术语中称为任务）的任务策略。只有当前线程受到影响，整个进程不受影响。

```
```c#include <numaif.h>long set_mempolicy(int mode, unsigned long *nodemask, unsigned long maxnode);```
```

模式参数必须是前一节中介绍的MPOL_*常量之一。nodemask参数指定将来分配使用的内存节点，而maxnode表示nodemask中的节点数（即位数）。如果模式为MPOL_DEFAULT，则无需指定任何内存节点，并且会忽略nodemask参数。对于MPOL_PREFERRED模式，若传入的nodemask为空指针，则选择本地节点；否则，MPOL_PREFERRED将使用nodemask中对应位设置的最小节点编号。

设置策略不会对已经分配的内存产生任何影响。页面不会自动迁移；只有未来的分配会受到影响。注意区分内存分配和地址空间预留之间的区别：使用mmap建立的地址空间区域通常不会被自动分配。首次对该内存区域进行读取或写入操作时，才会分配相应的页面。如果在访问同一地址空间区域内不同页面之间更改策略，或者策略允许从不同的节点分配内存，则看似统一的地址空间区域可能会分散到许多内存节点上。

6.5.3 页面交换与策略

当物理内存不足时，系统必须丢弃干净的页面并将脏页面保存到交换空间。Linux 的交换实现会在将页面写入交换空间时丢弃节点信息。这意味着在重新使用该页面并将其换入时，所选择的节点将会从头开始确定。线程的策略很可能会导致选择一个距离执行处理器较近的节点，但这个节点可能与之前使用的节点不同。

这种关联的变化意味着节点关联不能被程序作为页面属性存储下来。该关联会随时间发生变化。对于与其他进程共享的页面，这种情况也可能发生，因为某个进程请求了它（参见下文关于 `mbind` 的讨论）。内核自身可以在一个节点空间不足而其他节点仍有空闲空间的情况下迁移页面。

任何用户级代码学到的节点关联只在短时间内有效。这更像是一种提示而非绝对信息。每当需要准确知识时，应使用 `get_mempolicy` 接口（参见第 6.5.5 节）。

6.5.4 VMA 策略

要为某个地址范围设置 VMA 策略，需要使用不同的接口：

```
#include <numaif.h>
long mbind(void *start, unsigned long len,
           int mode,
           unsigned long *nodemask,
           unsigned long maxnode,
           unsigned flags);
```

此接口为地址范围 `[start, start + len)` 注册一个新的虚拟内存区域 (VMA) 策略。由于内存管理以页为单位进行操作，起始地址必须是页对齐的。`len` 值会被向上取整到下一个页面大小。

模式参数再次指定了策略；必须从第 6.5.1 节列出的选项中选择值。与 `set_mempolicy` 类似，`nodemask` 参数仅用于某些策略，并且其处理方式相同。

`mbind` 接口的语义取决于 `flags` 参数的值。默认情况下，如果 `flags` 为零，则系统调用会设置地址范围内的 VMA 策略。现有映射不受影响。如果不满足需求，目前有三种标志可以修改此行为；这些标志可以单独选择或组合使用：

`MPOL_MF_STRICT` 如果指定的 `nodemask` 中未包含所有页面，则 `mbind` 调用将失败。

当此标志与 `MPOL_MF_MOVE` 和/或 `MPOL_MF_MOVEALL` 一起使用时，如果任何一页无法移动，则调用将失败。

`MPOL_MF_MOVE` 内核将尝试移动分配在由 `nodemask` 指定的集合之外节点上的地址范围内的任何页面。默认情况下，只有当前进程的页表独占使用的页面会被移动。

`MPOL_MF_MOVEALL` 类似于 `MPOL_MF_MOVE`，但内核会尝试移动所有页面，而不仅仅是当前进程的页表使用的那些页面。此操作具有系统范围的影响，因为它会影响其他进程（这些进程可能不属于同一用户）的内存访问。因此，`MPOL_MF_MOVEALL` 是一个特权操作（需要 `CAP_NICE` 权限）。

注意，对 `'MPOL_MF_MOVE'` 和 `'MPOL_MF_MOVEALL'` 的支持仅在 2.6.16 版本的 Linux 内核中添加。不带任何标志调用 `'mbind'` 在为新预留的地址范围指定策略（在实际分配任何页面之前）时最为有用。

```
``cvoid *p = mmap(NULL, len, PROT_READ | PROT_WRITE, MAP_ANON, -1, 0); if (p != MAP_FAILED) mbind(p, len, mode, nodemask, maxnode, 0);``
```

此代码序列预留了一个长度为 `len` 字节的地址空间范围，并指定引用 `nodemask` 中内存节点时应使用的策略模式。除非在 `mmap` 调用中使用了 `MAP_POPULATE` 标志，否则在 `mbind` 调用时不会分配任何内存，因此新的策略适用于该地址空间区域内的所有页面。

仅使用 `MPOL_MF_STRICT` 标志可以确定由 `mbind` 函数的 `start` 和 `len` 参数描述的地址范围内的任何页面是否分配在除 `nodemask` 指定节点以外的其他节点上。不会更改已分配的页面。如果所有页面都分配在指定的节点上，则根据 `mode` 更改该地址空间区域的 VMA 策略。

有时需要重新平衡内存，在这种情况下，可能需要将分配在某一节点上的页面移动到另一个节点。调用带有 `MPOL_MF_MOVE` 标志的 `mbind` 函数会尽力实现这一目标。只有那些仅由进程的页表树引用的页面才会被考虑进行迁移。这部分页表树可能会有多个使用者，如线程或其他共享该部分内存的进程。无法影响其他恰好映射相同数据的进程。这些页面不共用相同的页表项。

如果在调用 `mbind` 时传递的 `flags` 参数中同时设置了 `MPOL_MF_STRICT` 和 `MPOL_MF_MOVE` 标志，内核将尝试移动所有未分配到指定节点上的页面。如果无法完成此操作，则调用将会失败。这种调用可能有助于确定是否存在一个节点（或一组节点），能够容纳所有的页面。可以依次尝试多种组合，直到找到合适的节点为止。

使用 `MPOL_MF_MOVEALL` 操作较难进行合理化解释，除非运行当前进程是计算机的主要目的。原因是即使出现在多个页表中的页面也会被移动，这很容易对其他进程产生负面影响。因此，在使用此操作时应格外小心。

6.5.5 查询节点信息

获取内存策略接口可以用于查询给定地址的 NUMA 状态的各种信息。

```
```c#include <numaif.h>long get_mempolicy(int *policy, c
onst unsigned long *nmask, unsigned long maxnode, void *
addr, int flags);```
```

当调用 `get_mempolicy` 并将标志参数设置为零时，地址 `addr` 的策略信息会被存储在由 `policy` 指向的单词中以及由 `nmask` 指向的节点位掩码中。如果 `addr` 落在一个已指定 VMA 策略的地址空间区域，则返回该策略的信息。否则，将返回任务策略或必要时的系统默认策略信息。

如果标志中设置了 `MPOL_F_NODE` 标志，并且管理 `addr` 的策略是 `MPOL_INTERLEAVE`，那么 `policy` 指向的单词中的值将是下一个分配将在其上发生的节点的索引。此信息可以用于设置即将在新分配的内存上工作的线程的亲合性。如果该线程尚未创建，这可能是实现接近性的更低成本的方法。

使用 `MPOL_F_ADDR` 标志可以获取另一个完全不同的数据项。如果使用此标志，`policy` 指向的单词中存储的值是包含 `addr` 的页面所分配的内存节点的索引。这一信息可用于决定可能的页面迁移、确定哪个线程能最高效地处理该内存位置，并且还可以用于许多其他用途。

CPU 及其所使用的内存节点比其内存分配要不稳定得多。内存页在没有显式请求的情况下只会被移动。

极端情况下，线程可能会因为重新平衡 CPU 负载而被分配到另一个 CPU 上。因此，关于当前 CPU 和节点的信息可能是短暂的。调度器会尽量将线程保留在同一个 CPU 上，甚至在同一核心上，以减少由于缓存失效导致的性能损失。这意味着查看当前的 CPU 和节点信息是有用的；只是不要假设这种关联不会发生变化。

`libNUMA` 提供了两种接口来查询给定虚拟地址空间范围的节点信息：

```
```c#include <libNUMA.h>int NUMA_mem_get_node_idx(void *addr
);int NUMA_mem_get_node_mask(void *addr, size_t size, size_t __de
stsize, memnode_set_t *dest);```
```

`NUMA_mem_get_node_mask` 根据管理策略在 `dest` 中设置所有内存节点的位，这些节点上分配了（或将会分配）范围 `[addr, addr + size)` 内的所有页面。而 `NUMA_mem_get_node` 只检查地址 `addr` 并返回该地址所在（或将要被分配到）的内存节点索引。与 `get_mempolicy` 接口相比，这两个接口更易于使用，可能更适合优先选用。

当前线程使用的 CPU 可以通过调用 `sched_getcpu`（参见第 6.4.3 节）来查询。利用这一信息，程序可以使用 `libNUMA` 中的 `NUMA_cpu_to_memnode` 接口确定与该 CPU 本地相关的内存节点。

```
```c#include <libNUMA.h>int numa_cpu_to_memnode(size_t cpuset
size, const cpu_set_t *cpuset, size_t memnodesize, memnode_set_t *me
mnodeset);```
```

调用此函数将会在由第四个参数指向的内存节点集中设置所有对应于第二个参数所指 CPU 集合中任意一个 CPU 本地内存节点的位。就像 CPU 信息本身一样，这些信息仅在机器配置未发生变化（例如，CPU 被移除或添加）的情况下才正确有效。

`memnode_set_t` 对象中的位可以在调用低级函数（如 `get_mempolicy`）时使用。为了方便起见，可以利用 `lib-numa` 中的其他函数。反向映射可通过以下方式获得：

```
#include <libNUMA.h>
```

```
int NUMA_memnode_to_cpu(size_t memnodesize,
 const memnode_set_t *
 memnodeset,
 size_t cpusetsize,
 cpu_set_t *cpuset);
```

结果中的cpuset位集是那些与memnodeset中相应位置的内存节点本地CPU对应的位。对于这两种接口，程序员需要注意这些信息会随时间变化（特别是在热插拔CPU的情况下）。在许多情况下，输入位集中只设置了一个位，但也有意义的是，将通过调用sched\_getaffinity获取的所有CPU传递给NUMA\_cpu\_to\_memnode，以确定线程可以直接访问的内存节点。

### 6.5.6 中央处理器和节点集

在无法获取源代码的情况下，通过修改代码以使用之前描述的接口来适应SMP和NUMA环境可能会代价高昂（甚至不可能实现）。此外，系统管理员可能希望对用户和/或进程可以使用的资源施加限制。为应对这些情况，Linux内核支持所谓的CPU集。这个名字有些误导，因为内存节点也被包含在内。它们与cpu\_set\_t数据类型没有任何关系。

当前，CPU接口是以一个特殊的文件系统形式存在的。通常情况下，这个文件系统并未挂载（至少到目前为止是这样的）。这一情况可以通过以下方式更改：

挂载 cgroup 的 cpuset 子系统：``mount -t cpuset none /dev/cpuset``/\* Cannot get node, pick one. \*/

/mdev/cpuset 挂载点必须在此时存在。该目录的内容描述了默认（根）CPU集。初始状态下，它包含所有CPU和所有内存节点。该目录中的cpus文件显示CPU集中的CPU列表，mems文件显示内存节点列表，tasks文件则列出进程。

为了创建一个新的CPU集，只需在层次结构中的某个位置创建一个新目录。新的CPU集将继承所有来自父级的设置。然后通过向新目录下的cpus和mems伪文件写入新的值来更改该新CPU集的CPU和内存节点。

如果一个进程属于某个CPU集，则该CPU集和内存节点的设置将作为亲和性掩码和内存策略位图的屏蔽。这意味着程序不能选择任何不在该进程使用的CPU集中（即在tasks文件中列出的位置）的affinity掩码中的CPU。同样，对于内存策略的节点掩码和mems文件也是如此。

程序在屏蔽操作后不会出现任何错误，除非位掩码为空，因此CPU集成为控制程序执行提供了一种几乎看不见的手段。此方法尤其适用于具有大量CPU和/或内存节点的机器。将进程移动到新的CPU集中，只需将进程ID写入相应CPU集的任务文件中即可。

CPU集目录包含多个其他文件，这些文件可用于指定在内存压力下的行为以及对CPU和内存节点的独占访问等细节。有兴趣的读者可以参阅内核源码树中的Documentation/cpusets.txt文件。

### 6.5.7 显式NUMA优化

所有节点上的线程都需要访问相同的内存区域时，本地内存亲和性规则就无法发挥作用了。当然，可以简单地限制线程数量，使其不超过直接连接到内存节点的处理器所能支持的数量。然而，这种方法并未充分利用SMP NUMA机器的优势，因此并不是一个真正的解决方案。

如果要读取的数据是只读的，有一个简单的解决方案：数据复制。每个节点都可以拥有自己的数据副本，从而不需要进行跨节点访问。实现这一功能的代码可以如下所示：

```
void *local_data(void) {
 static void *data[NNODES];
 int node =
 NUMA_memnode_self_current_idx();
 if (node == -1)
 /* Cannot get node, pick one. */
 node = 0;
 if (data[node] == NULL)
 data[node] = allocate_data();
 return data[node];
}

void worker(void) {
 void *data = local_data();
 for (...)
 compute using data
}
```

在此代码中，worker函数通过调用local\_data获取本地数据副本的指针。然后它继续执行循环，使用该指针。local\_data函数维护一个已分配的数据副本列表。每个系统具有有限数量的内存节点，因此指向每个节点内存副本的指针数组大小是有限制的。lib NUMA中的NUMA\_memnode\_system\_count函数返回这个数字。如果当前节点尚未为其分配内存（通过data中为空指针识别），则需要进行内存分配。

由 ``NUMA_memnode_self_current`` 调用返回的索引 (idx) 所指向的位置上, 分配一个新的副本。

需要注意的是, 如果在调用 `getcpu` 系统调用之后, 线程被调度到连接到不同内存节点的另一个CPU上, 也不会发生什么严重的问题。这仅仅意味着使用 `data` 变量进行访问的操作会在另一个内存节点上的内存中执行, 从而减慢程序的速度, 直到重新计算数据为止, 但除此之外没有其他影响。内核总是会避免不必要的各CPU运行队列的重新平衡操作。如果确实发生了这样的迁移, 通常是有充分理由的, 并且在未来一段时间内不会再次发生。

当涉及的内存区域可写时, 情况会更加复杂。简单地进行复制在这种情况下无法奏效。根据具体情况, 可能有多种解决方案。

例如, 如果可写内存区域用于累积结果, 则可以先为每个内存节点创建一个单独的区域来累积这些结果。当这项工作完成后, 将所有节点的内存区域合并以获得总结果。即使实际的工作从未真正停止, 但需要中间结果时, 此技术仍然有效。这种方法的要求是结果的累积是无状态的, 即它不依赖于之前收集的结果。

直接访问可写内存区域始终更为理想。如果对该内存区域的访问次数较多, 强制内核将相关内存页迁移到本地节点可能是个好主意。若访问次数非常高, 并且不同节点上的写操作不会并发发生, 则这可能会有所帮助。但请注意, 内核无法创造奇迹: 页面迁移是一项复制操作, 因此成本较高。此成本需要摊薄。

#### 6.5.8 利用全部带宽

图5.4中的数据表明, 当缓存无效时访问远程内存的速度并不会比访问本地内存慢很多。这意味着程序可以通过将不需要再次读取的数据写入另一处理器连接的内存来节省对本地内存的带宽。由于连接到DRAM模块的带宽和互连带宽大部分是独立的, 因此并行使用可以提高整体性能。

这是否真的可行取决于许多因素。必须确保缓存无效, 否则远程访问相关的性能下降是可以测量的。另一个大问题是远程节点是否有其自身的内存带宽需求。

在详细探讨这种方法之前, 必须仔细审视这种可能性。理论上, 充分利用处理器可用的所有带宽可能会产生积极影响。10h系列Opteron处理器可以直接连接多达四个其他处理器。如果整个系统配合得当, 利用所有这些额外的带宽, 并结合适当的预取操作 (特别是写入预取), 可能带来改进。

---

<sup>43</sup>The user-level `sched_getcpu` interface is implemented using the `getcpu` system call which should not be used directly and has a different interface.

## 7 内存性能工具

有许多工具可以帮助程序员理解程序的性能特征，包括缓存和内存使用情况等。现代处理器配备了可以用于性能监测的硬件。有些事件难以精确测量，因此也有模拟的空间。在更高层次的功能方面，有一些专门的工具可以用来监控进程的执行情况。我们将介绍一套大多数Linux系统上常见的常用工具。

### 7.1 内存操作 profiling

profiling内存操作需要硬件的支持。虽然可以在软件中收集一些信息，但这些信息要么是粗粒度的，要么仅仅是模拟的结果。第七章第二节和第五节将展示具体的模拟示例。在这里，我们将专注于可测量的内存效应。

在Linux系统中，性能监控硬件的访问由oprofile提供。oprofile提供了连续 profiling 功能，首次描述见文献[2]；它通过一个易于使用的界面进行统计性的、系统的性能分析。oprofile绝不是利用处理器性能测量功能的唯一方式；Linux开发人员正在开发pfmon，该工具未来可能广泛部署，并在此予以介绍。

接口oprofile提供的功能简单且精简，即使使用可选的图形用户界面也是如此。用户需要从处理器可以记录的事件中进行选择。处理器架构手册会描述这些事件，但很多时候，解读数据需要深入了解处理器本身的知识。另一个问题是收集的数据如何解释。性能测量计数器是绝对值，并且可以任意增长。对于给定的计数器来说，数值达到多少就算过高？

为了解决这个问题，可以避免关注绝对值，而是将多个计数器相互关联起来。处理器可以监控多个事件；随后可以考察收集到的绝对值之间的比率。这样可以获得一些可比较的良好结果。通常除数是处理时间的度量，比如时钟周期的数量或指令数量。作为初步评估程序性能的方法，仅通过这两个数字进行关联就很有用。

图7.1展示了简单随机“Follow”测试用例在不同工作集大小下的每指令周期数（CPI）。对于大多数Intel处理器而言，收集此类信息的事件名称为CPU\_CLK\_UNHALTED和INST\_RETIRED。顾名思义，前者计数CPU的时钟周期数，后者则统计执行的指令数量。我们看到的画面与之前使用的每列表元素周期数测量结果类似：在小工作集大小下，该比率是1.0或更低。

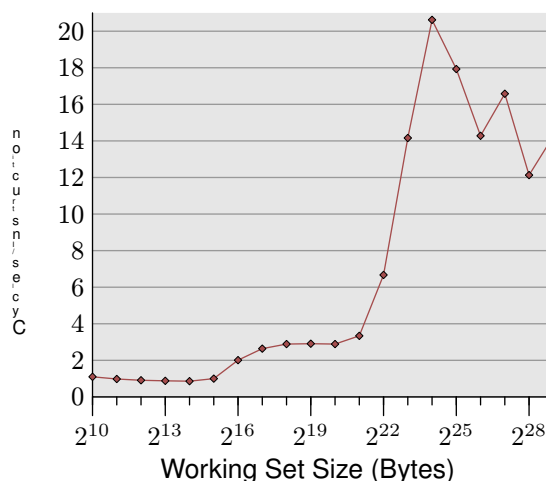


图7.1：每指令周期（随机访问）

在英特尔酷睿2处理器上进行了测量，该处理器是多标量架构，并且可以同时处理多个指令。对于不受内存带宽限制的程序而言，比率可能显著低于1.0，但在这种情况下，1.0已经相当不错了。

一旦L1d缓存不再足够大以容纳工作集，CPI会跃升至接近3.0。请注意，CPI比率平均了所有指令访问L2的惩罚，而不仅仅是内存指令。通过使用列表元素数据所需的周期数，可以计算出每个列表元素需要多少条指令。如果连L2缓存也不够用，CPI比率将跳增至超过20。这些是预期的结果。

但是性能测量计数器应该能提供更多关于处理器内部情况的洞察。为此，我们需要考虑处理器的具体实现。在这份文档中，我们关注缓存处理细节，因此需要查看与缓存相关的事件。这些事件、它们的名称以及所记录的内容都是特定于处理器的。这就是oprofile目前难以使用的地方，无论其简单的用户界面如何：用户必须自行弄清楚性能计数器的详细信息。在附录B中，我们将看到一些处理器的具体细节。

对于Core 2处理器，需要关注的事件是L1D\_REPL、DTLB\_MISSES和L2\_LINES\_IN。后者可以测量所有未命中以及由指令而非硬件预取引起的未命中。随机“Follow”测试的结果如图7.2所示。

所有比率均基于已提交的指令数（INST\_RETIRED）计算。这意味着，为了计算缓存缺失率，需要从INST\_RETIRED值中减去大量的加载和存储指令数量，这表明实际的内存操作缓存缺失率甚至高于图表所示数值。

L1数据缓存缺失数量远远超过其他类型的缓存缺失，因为L2缓存缺失会

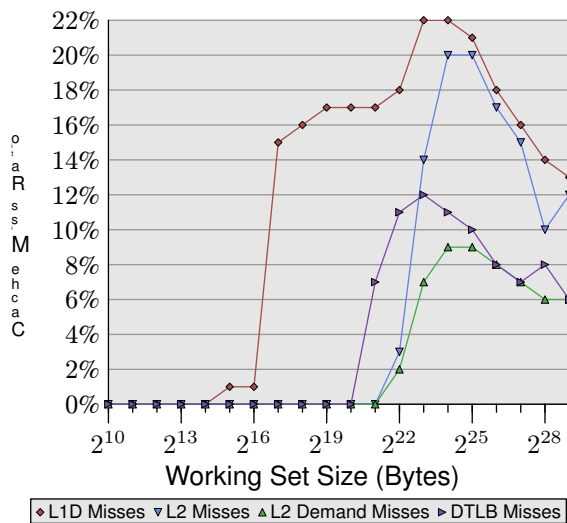


图7.2: 测量的缓存未命中（随机访问）

这表明，在使用包含缓存的情况下，对于Intel处理器来说L1d缺失率会上升。该处理器的L1d大小为32 KB，因此我们可以看到，如预期的一样，当工作集大小达到约32KB时（由于除了列表数据结构之外还有其他缓存用途），L1d命中率从零开始上升（实际上增加发生在16KB和32KB之间）。值得注意的是，在工作集大小不超过64KB的情况下，硬件预取机制能够将缺失率保持在1%。超过这个范围后，L1d的缺失率会急剧上升。

L2缓存缺失率在L2耗尽之前保持为零；由于其他用途导致的少量缺失对数据影响不大。一旦L2大小（ $2^{21}$ 字节）被超过，缺失率开始上升。值得注意的是，L2需求缺失率不为零。这表明硬件预取器并未加载所有后续指令所需的缓存行。这是可以预期的结果，访问的随机性阻止了完美的预取效果。将此与图7.3中的顺序读取数据进行比较。

在这张图中我们可以看到，L2缺失率基本上为零（请注意该图的刻度与图7.2不同）。对于顺序访问的情况，硬件预取器工作得非常完美：几乎所有的L2缓存缺失都是由预取器引起的。L1d和L2缺失率相同这一事实表明，所有L1d缓存缺失都直接由L2缓存处理而没有进一步的延迟。这是对程序的理想情况，但当然很少能够实现。

在两张图中的第四条线是数据快表（DTLB）缺失率（Intel 将指令和数据的快表分开，DTLB 是数据快表）。对于随机访问情况，DTLB 缺失率显著，并且对延迟有较大影响。有趣的是，在 L2 缓存缺失之前，DTLB 的惩罚就已经显现出来了。而在顺序访问情况下，DTLB 的开销基本上可以忽略不计。

回到第几节中的矩阵乘法示例。

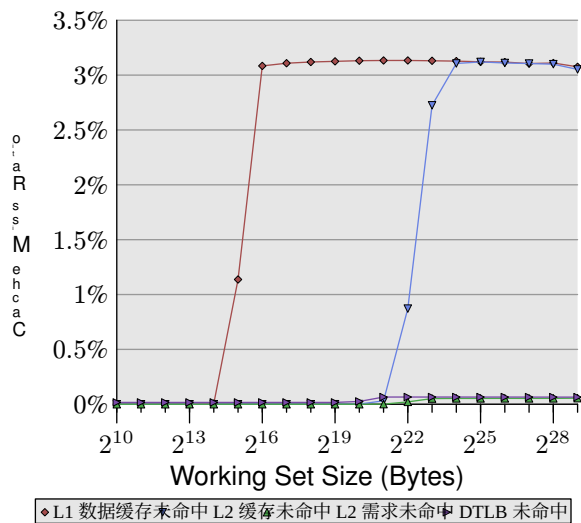


图7.3: 测量的缓存未命中（顺序访问）

根据第6.2.1节和附录A.1中的示例代码，我们可以利用三个额外的计数器。SSE\_HIT\_PRE、SSE\_PRE\_MISS 和 LOAD\_PRE\_EXEC 计数器可以用来衡量软件预取的效果。如果运行附录A.1中的代码，我们将得到以下结果：

Description	Ratio
Useful NTA prefetches	2.84%
Late NTA prefetches	2.65%

低效的非临时对齐（NTA，Non-Temporal Aligned）预取比例表明，许多预取指令被执行在已经加载的缓存行上，因此实际上没有额外的工作需要完成。这意味着处理器浪费了时间来解码预取指令并查找缓存。然而，也不能过于苛责代码本身。这很大程度上取决于所用处理器的缓存大小；硬件预取器也起到了一定作用。

低延迟NTA预取比例具有误导性。该比率意味着所有预取指令中有2.65%的指令发出得太晚，导致需要数据的指令在数据能够被预取到缓存之前就已经执行了。必须注意的是，只有约284%至265%之间的55%的预取指令是有用的。在这部分有用的NTA预取指令中，有48%未能及时完成。因此代码可以进一步优化：

- 大多数预取指令都是不必要的。
- 使用预取指令的调整可以更好地匹配硬件。

留作习题，请读者自行确定适用于现有硬件的最佳解决方案。具体的硬件规格起着重要作用。在Core 2处理器上，SSE算术运算的延迟为1个时钟周期。较旧的

```
$ \time ls /etc
[...]
```

0.00user	0.00system	0:00.02elapsed	17%CPU	(0avgtext+0avgdata	0maxresident)	k
0inputs+0outputs	(1major+335minor)	pagefaults	0swaps			

图7.4: time实用程序的输出

版本的延迟为2个周期，这意味着硬件预取器和预取指令有更多的时间将数据加载进来。

为了确定何处可能需要预取操作或预取操作是不必要的，可以使用 `opannotate` 程序。该程序会列出程序的源代码或汇编代码，并显示事件被识别出的指令位置。需要注意的是，存在两种模糊性来源：

**OProfile** 进行随机采样分析。只有每第  $N$  个事件（其中  $N$  是每个事件的阈值，并且有一个强制执行的最小值）会被记录下来，以避免过多地拖慢系统运行速度。可能存在某些代码行会产生 100 个事件，但它们在报告中却并未显示出来。

2. 并非所有事件都能被准确记录。例如，某个特定事件发生时的指令计数器可能不正确。由于处理器是多发射的，因此很难给出一个100%正确的答案。不过，某些处理器上的少数事件是可以精确记录的。

注解列表不仅有助于确定预取信息。每个事件都与指令指针一起记录，因此也可以精确定位程序中的其他热点区域。产生大量 `INST_RETIRED` 事件的位置执行频繁，值得优化。报告许多缓存缺失的位置可能需要添加预取指令以避免缓存缺失。

一种无需硬件支持即可测量的事件是缺页异常。操作系统负责解决缺页异常，并在这些情况下对其进行计数。它区分两种类型的缺页异常：

对于匿名页面（即没有文件作为后援的页面）、尚未使用的拷贝-on-写页面以及其他内容已经在内存中某处的页面。

**重大缺页异常** 解决这些异常需要访问磁盘以检索由文件支持（或换出）的数据。

显然，重大页故障比普通页故障要昂贵得多。但后者也不便宜。无论是哪种情况，都需要进入内核，找到新的页面，清除或填充适当的页面数据，并且需要进行

页表树必须相应地进行修改。最后一步需要与其他读取或修改页表树的任务同步，这可能会引入进一步的延迟。

获取页面故障计数信息最简单的方式是使用 `time` 工具。注意：请使用实际的 `time` 工具，而不是 `shell` 内置命令。输出结果如图7.4所示。<sup>44</sup>

这里有趣的部分是最后一行。时间工具报告了1次大页故障和335次小页故障。具体数字会有所不同；特别是，如果立即重复运行，可能会发现根本没有大页故障。如果程序执行相同的操作，并且环境没有任何变化，总的页故障数量将是稳定的。

关于缺页异常，程序启动阶段尤为敏感。每个被使用的页面都会产生一次缺页异常；直观的效果是（尤其是对于图形用户界面应用程序），使用的页面越多，程序启动所需的时间就越长。在第7.5节中，我们将介绍一个专门用于测量这一效果的工具。

底层实现中，`time` 工具使用了 `rusage` 功能。`wait4` 系统调用在父进程等待子进程终止时填充一个 `struct rusage` 对象；这正是 `time` 工具所需的信息。此外，进程也可以请求获取自身资源使用的相关信息（这就是 `rusage` 这个名字的由来），或者已终止子进程的资源使用情况。

```
```c#include <sys/resource.h>int getrusage(__rusage_who_t who, struct rusage *usage);```
```

参数 `who` 指定请求信息的目标进程。当前仅定义了 `RUSAGE_SELF` 和 `RUSAGE_CHILDREN`。当每个子进程终止时，其资源使用情况会被累积计算。这是一个总计值，并非单个子进程的使用情况。目前已有提议允许请求线程特定的信息，因此在未来不久我们可能会看到 `RUSAGE_THREAD` 的出现。`rusage` 结构体被定义为包含各种度量指标，包括执行时间、发送的 `IPC` 消息数量等。

⁴⁴The leading backslash prevents the use of the built-in command.

```

==19645== I   refs:      152,653,497
==19645== I1  misses:      25,833
==19645== L2i misses:      2,475
==19645== I1  miss rate:      0.01%
==19645== L2i miss rate:      0.00%
==19645==
==19645== D   refs:      56,857,129 (35,838,721 rd + 21,018,408 wr)
==19645== D1  misses:      14,187 ( 12,451 rd + 1,736 wr)
==19645== L2d misses:      7,701 ( 6,325 rd + 1,376 wr)
==19645== D1  miss rate:      0.0% ( 0.0% + 0.0% )
==19645== L2d miss rate:      0.0% ( 0.0% + 0.0% )
==19645==
==19645== L2  refs:      40,020 ( 38,284 rd + 1,736 wr)
==19645== L2  misses:      10,176 ( 8,800 rd + 1,376 wr)
==19645== L2  miss rate:      0.0% ( 0.0% + 0.0% )

```

图7.5: Cachegrind汇总输出

内存使用情况以及缺页异常的数量。后者的信息可以在结构体的 `ru_minflt` 和 `ru_majflt` 成员中获得。

一个试图确定其程序因缺页异常而性能下降位置的程序员可以定期请求相关信息，并将返回的结果与之前的结果进行比较。

从外部来看，如果请求者具有相应的权限，也可以看到这些信息。伪文件 `/proc/<PID>/stat` 中包含了我们感兴趣的进程的进程标识符 `<PID>`，该文件第十至第十四字段中记录了缺页异常的数量。具体来说，它们分别是该进程及其子进程累计发生的少量页面错误和大量页面错误的对数。

7.2 模拟CPU缓存

虽然缓存的工作原理从技术上讲相对容易理解，但要观察实际程序在缓存方面的行为却并非易事。程序员并不直接关心地址的值，无论是绝对地址还是相对地址。地址部分由链接器确定，在运行时又由动态链接器和内核进一步决定。生成的汇编代码预期能够处理所有可能的地址，并且在源语言中甚至没有留下任何绝对地址值的痕迹。因此，要了解程序如何使用内存是非常困难的。⁴⁵

CPU级性能分析工具，如第7.1节中描述的oprofile，有助于理解缓存使用情况。收集到的数据对应实际硬件，并且如果不需要细粒度采集的话，可以相对较快地获取。一旦需要更细粒度的数据，oprofile就不再适用；此时线程会被过于频繁地中断。此外，为了观察程序在不同处理器上的内存行为，实际上必须拥有相应的机器并执行该程序。

此程序有时（经常）无法在它们上面运行。例如，图3.8中的数据。要使用oprofile收集此类数据，则需要24台不同的机器，其中许多并不存在。

该图中的数据是使用缓存模拟器收集的。此程序名为cachegrind，它基于valgrind框架开发，最初用于检测程序中与内存处理相关的错误。valgrind框架通过模拟程序执行，并在此过程中允许各种扩展（如cachegrind）挂接到执行框架上。cachegrind工具利用这一点拦截所有内存地址的使用；然后根据给定的大小、缓存行大小和关联性，模拟L1i、L1d和L2缓存的操作。

要使用该工具，必须用 valgrind 作为包装器来运行程序：

```
valgrind --tool=cachegrind 命令 参数
```

在此最简单的形式中，程序命令使用参数 `arg` 执行，并模拟与运行处理器对应的三级缓存大小和关联性。当程序运行时，一部分输出会打印到标准错误流；这部分输出包括如图7.5所示的总缓存使用的统计信息，其中包括总的指令数和内存引用次数、L1i/L1d 和 L2 缓存产生的缺失次数、缺失率等。该工具甚至能够将 L2 访问拆分为指令访问和数据访问，并且所有数据缓存使用情况都被拆分为读取访问和写入访问。

当模拟缓存的细节发生变化并进行结果比较时，情况变得更加有趣。通过使用 `--I1`、`--D1` 和 `--L2` 参数，cachegrind 可以被指示忽略处理器的实际缓存布局，并采用命令行上指定的配置。例如：

```
valgrind --tool=cachegrind \
--L2=8388608,8,64 command arg
```

⁴⁵When programming close to the hardware this might be different, but this is of no concern to normal programming and, in any case, is only possible for special addresses such as memory-mapped devices.

Ir	I1mr	I2mr	Dr	D1mr	D2mr	Dw	D1mw	D2mw	file:function
53,684,905	9	8	9,589,531	13	3	5,820,373	14	0	???:_IO_file_xsputn@@GLIBC_2.2.5
36,925,729	6,267	114	11,205,241	74	18	7,123,370	22	0	???:vfprintf
11,845,373	22	2	3,126,914	46	22	1,563,457	0	0	???:__find_specmb
6,004,482	40	10	697,872	1,744	484	0	0	0	???:strlen
5,008,448	3	2	1,450,093	370	118	0	0	0	???:strcmp
3,316,589	24	4	757,523	0	0	540,952	0	0	???:_IO_padr
2,825,541	3	3	290,222	5	1	216,403	0	0	???:_itoa_word
2,628,466	9	6	730,059	0	0	358,215	0	0	???:_IO_file_overflow@@GLIBC_2.2.5
2,504,211	4	4	762,151	2	0	598,833	3	0	???:_IO_do_write@@GLIBC_2.2.5
2,296,142	32	7	616,490	88	0	321,848	0	0	dwarf_child.c:__libdw_find_attr
2,184,153	2,876	20	503,805	67	0	435,562	0	0	???:__dcigettext
2,014,243	3	3	435,512	1	1	272,195	4	0	???:_IO_file_write@@GLIBC_2.2.5
1,988,697	2,804	4	656,112	380	0	47,847	1	1	???:getenv
1,973,463	27	6	597,768	15	0	420,805	0	0	dwarf_getattnrs.c:dwarf_getattnrs

图7.6: cg annotate 输出

会模拟一个8MB的L2缓存，该缓存具有8路组相联性和64字节的缓存行大小。请注意，在命令行中，--L2选项应在被模拟程序名称之前出现。

这并不是cachegrind的全部功能。在进程退出前，它会生成一个名为cachegrind.out.XXXXXX的文件，其中XXXXXX是该进程的PID。此文件包含了缓存使用的汇总信息以及每个函数和源文件中的详细信息。这些数据可以通过cg annotate程序进行查看。

该程序产生的输出包含缓存使用概要，这与进程终止时打印的信息相同，并且还包括程序中每个函数的详细缓存行使用情况。生成这些按功能划分的数据需要cg annotate能够将地址匹配到相应的函数上。这意味着为了获得最佳效果，应提供调试信息。如果没有调试信息，ELF符号表也能提供一些帮助，但由于内部符号未列在动态符号表中，因此结果不完整。图7.6显示了与图7.5相同程序运行的部分输出。

Ir、Dr和Dw列显示了总缓存使用情况，而不是缓存未命中次数，后者在接下来的两列中给出。这些数据可用于识别产生最多缓存未命中的代码。首先，可能会重点关注L2缓存未命中，然后继续优化L1i/L1d缓存未命中。

cg annotate 可以提供更详细的数据。如果给出了源文件的名称，它还会为该源文件的每一行标注对应的缓存命中次数和未命中次数（这也是程序名称的由来）。这些信息允许程序员精确定位到导致缓存未命中的具体代码行。程序接口略显原始：截至撰写本文时，cachegrind 数据文件和源文件必须位于同一目录中。

在此需要再次强调：cachegrind 是一个模拟器，它不使用处理器的测量数据。实际缓存实现中，处理器的

处理器的情况可能大不相同。cachegrind 模拟了最近最少使用（LRU）淘汰策略，这在关联度较大的缓存中可能会过于昂贵。此外，该模拟没有考虑上下文切换和系统调用，这两者都会破坏大部分 L2 缓存，并且必须清空 L1i 和 L1d 缓存。这导致总的缓存缺失数量低于实际体验中的数值。然而，cachegrind 是一个很好的工具，用于了解程序的内存使用情况及其与内存相关的问题。

7.3 测量内存使用情况

了解程序分配了多少内存以及可能的分配位置是优化其内存使用的第一步。幸运的是，有一些易于使用的工具可供利用，这些工具无需重新编译或专门修改程序即可使用。

对于第一个工具，名为massif，只需保留编译器自动生成的调试信息即可。它提供了内存使用情况随时间累积的概览。图7.7展示了生成的输出示例。类似于cachegrind（参见第7.2节），massif是一个基于valgrind基础设施的工具。它的启动方式为

valgrind --tool=massif 命令 参数

其中 command arg 是要观察的程序及其参数。该程序将被模拟，所有对内存分配函数的调用都将被识别。调用位置和时间戳值会被记录；新分配的大小会同时添加到整个程序的总和以及特定调用点的总计中。对于释放内存的函数来说也是如此，在这种情况下，释放块的大小从相应的总数中减去。这些信息可以用来创建一个图表，展示程序生命周期内的内存使用情况，并根据请求分配的位置拆分每个时间值。

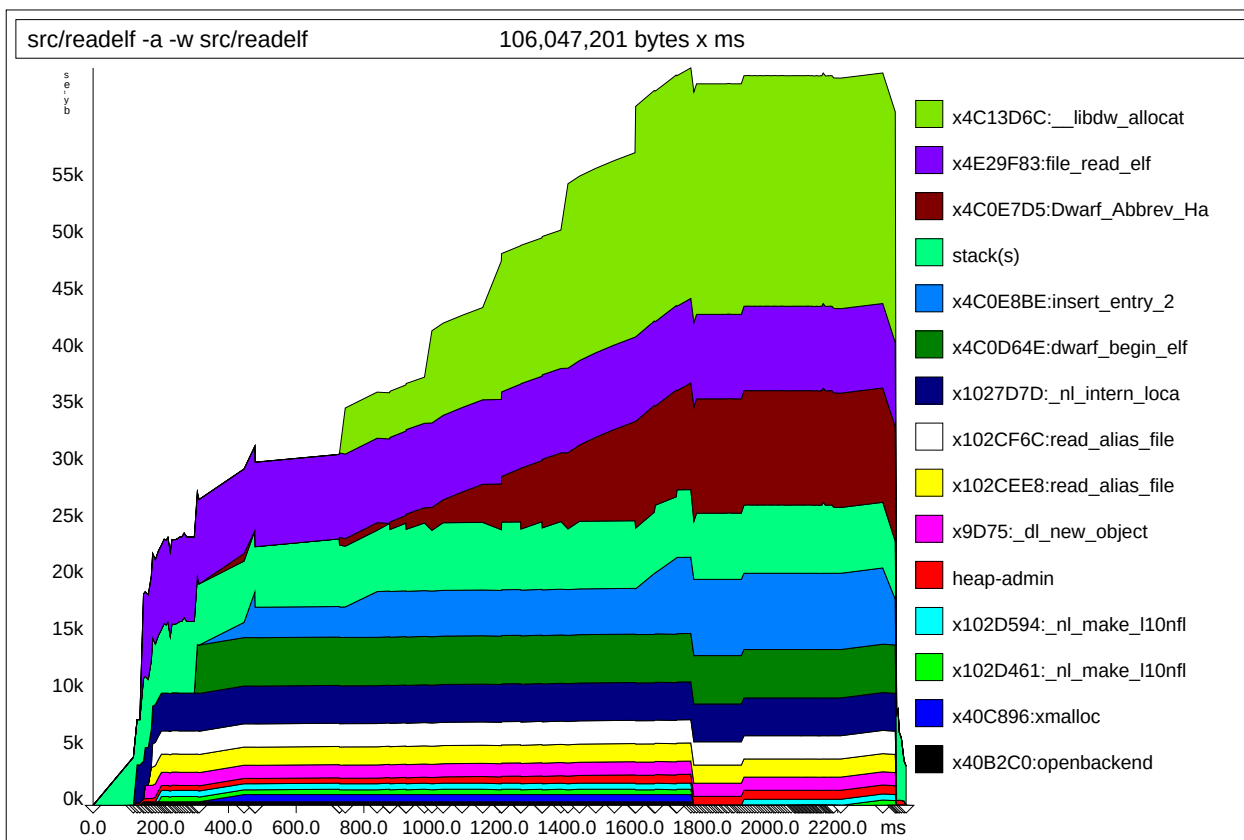


图7.7: Massif 输出

在进程终止前，massif会创建两个文件：massif.XXX XX.txt和massif.XXXXX.ps；XXXXX仍然是该进程的PID。.txt文件汇总了所有调用点的内存使用情况，而.ps文件则如图7.7所示。

Massif 还可以记录程序的栈使用情况，这有助于确定应用程序的总内存占用。但在某些情况下（例如一些线程栈或当使用 `signalstack` 时），`valgrind` 运行时无法获知栈的边界。在这种情况下，将这些栈的大小加入到总计中也没有太大意义。还有其他几种类似的情况也是如此。如果程序受此影响，应通过添加选项 `--stacks=no` 来启动 Massif。请注意，这是 `valgrind` 的一个选项，因此必须在要观察的程序名称之前指定。

一些程序提供了自己的内存分配实现或围绕系统分配函数的包装函数。在第一种情况下，分配操作通常不会被记录；而在第二种情况下，由于只记录了包装函数中调用地址，因此实际的调用位置信息会被隐藏。出于这个原因，可以将额外的函数添加到分配函数列表中。使用 `--alloc-fn{xmalloc}` 参数指定 `xmalloc` 函数也是一个分配函数，这在 GNU 程序中很常见。对 `xmalloc` 的调用会被记录下来，但不会记录

从 `xmalloc` 内部发出的信号量调用。

该工具名为 `memusage`，它是 GNU C 库的一部分。它是一个简化版的 `massif`（但早在 `massif` 出现之前就已经存在）。它仅记录堆内存的总使用量（如果提供了 `-m` 选项，则包括可能调用的 `mmap` 等函数），以及可选地记录栈内存的使用情况。结果可以以时间为横轴显示总内存使用的图表，或者以分配函数调用次数为横轴线性展示。这些图表是由 `memusage` 脚本单独生成的，就像 `valgrind` 一样，该脚本必须用于启动应用程序：

memusage 命令 参数

使用 `-p IMGFILE` 选项指定应在文件 `IMGFILE` 中生成图表。这是一个 PNG 文件。收集数据的代码在实际程序中运行，而不是像 `valgrind` 那样的模拟工具。这意味着 `memusage` 比 `massif` 更快，并且可以在 `massif` 不适用的情况下使用。除了总内存消耗外，该代码还记录分配大小，在程序终止时显示已用分配大小的直方图。这些信息被写入标准错误输出。

有时直接调用应被观察的程序是不可能的（或不可行的）。例如，`gcc` 编译器的编译阶段是由 `gcc` 驱动程序启动的。在这种情况下，需要调用程序的名称为：

应通过 `-n NAME` 参数将需要观察的程序名称提供给 `memusage` 脚本。此参数在被观察的程序启动其他程序时也非常有用。如果没有指定程序名，则所有启动的程序都将被 `profiling`。

BOTH程序（`massif`和`memusage`）都提供了额外的选项。如果程序员发现自己需要更多功能，应首先查阅手册或帮助信息，确保所需的功能尚未实现。

注意：此处 标记是为了区分指令与翻译内容，并非原文部分，请忽略该符号。

现在我们已经了解了如何捕获内存分配的数据，接下来需要讨论这些数据在内存和缓存使用上下文中的解释方法。高效动态内存分配的主要方面是线性分配以及已用部分的紧凑性。这涉及到使预取操作高效并减少缓存未命中。

一个需要读取任意数量数据以供后续处理的程序可以通过创建一个列表来实现，其中每个列表元素包含一个新的数据项。这种分配方法的开销可能很小（单链表只需一个指针），但在使用数据时缓存效应可能会显著降低性能。

例如，并不能保证顺序分配的内存会在物理内存中连续排列。造成这种情况的原因有很多：

- 大块内存中的内存块实际上是由后向前归还给内存分配器的。
- 一个内存块耗尽后，在地址空间的另一部分开始一个新的内存块；
- 分配请求的大小不同，分别从不同的内存池中服务；
- 多线程程序中各线程的交错分配。

如果数据必须预先分配以便后续处理，使用链表的方式显然是一个糟糕的选择。无法保证列表中的连续元素在内存中也是连续排列的。为了确保连续分配，内存不能被分成小块进行分配。需要使用另一层内存管理机制；这可以很容易地由程序员实现。另一种选择是使用GNU C库提供的obstack实现。这种分配器会从系统的分配器请求大块内存，并随后分发任意大小的内存块。这些分配总是连续的，除非大的内存块耗尽，这种情况根据所需的分配大小来说相当罕见。Obstack并不是完全替代内存管理的解决方案。

分配器本身具有有限的释放对象的能力。详情请参阅GNU C库手册。

因此，如何从图表中识别出使用obstack（或其他类似技术）是有益的情况？不查看源代码的情况下，无法确定具体的变化候选者，但图表可以为搜索提供一个切入点。如果大量分配操作来自同一位置，则可能意味着批量分配会有所帮助。在图7.7中，我们可以看到地址0x4c0e7d5处的分配可能是这样的一个候选者。从运行的大约800毫秒到1800毫秒这段时间内，这是唯一增长的区域（除了顶部绿色的那个）。此外，斜率并不陡峭，这意味着我们有大量的相对较小的分配操作。这确实是一个使用obstack或其他类似技术的候选场景。

另一个问题图表可以展示的是当总的分配次数很高的情况。如果图表不是随着时间线性绘制，而是根据调用次数（`memusage` 的默认设置）线性绘制的话，这种情况就更容易观察到。在这种情况下，图中的平缓斜率表示有大量的小规模分配。虽然 `memusage` 不会指出这些分配发生的位置，但与 `massif` 输出的比较可以提供这方面的信息，或者程序员可能立即就能识别出来。大量的小额分配应该被合并以实现线性内存使用。

但是，这一类情况还有另一个同样重要的方面：许多分配操作也意味着更高的管理数据开销。这本身可能并不会造成太大问题。在massif图中以红色区域“堆管理”表示的这部分开销相对较小。然而，根据不同的malloc实现方式，这些管理数据会与数据块一起在同一内存区域内进行分配。对于当前GNU C库中的malloc实现而言，情况确实如此：每个分配的数据块至少包含一个2词（在32位平台上为8字节，在64位平台上为16字节）的头部。此外，由于内存管理方式的不同，数据块的实际大小往往略大于所需大小（通常会将块大小向上取整到特定倍数）。

这也就意味着程序使用的内存与分配器仅用于管理
目的内存交错分布。我们可能会看到如下情况：



每个块代表一个内存字。在这个小的内存区域中有四个已分配的块。由于块头和填充带来的开销占50%。由于头部的位置，这自动意味着处理器的有效预取率最多会降低50%。如果按顺序处理这些块（以充分利用预取），处理器会将所有的块头和填充字读入缓存，尽管它们实际上从不被使用。

该块的数据部分应该是由应用程序本身进行读取或写入的。只有运行时环境会使用头部字节，而且运行时环境仅在释放该块时起作用。

在此阶段可以提出，实现方式应改为将管理数据放置在其他地方。确实有些实现采用了这种方法，并且这可能是个好主意。不过，需要考虑许多方面，安全性问题不容忽视。无论未来是否会发生变化，填充问题都不会消失（例如，在忽略头部的情况下，占用了示例中16%的数据）。只有当程序员直接控制分配时，才能避免这种情况的发生。在对齐要求起作用时仍可能存在空洞，但这同样是在程序员的掌控范围内。

7.4 提高分支预测精度

在6.2.2节中，提到通过分支预测和块重排来改进L1i使用的两种方法：静态预测通过`__builtin_expect`以及基于剖析的优化（PGO）。正确的分支预测会对性能产生影响，但在这里我们更关注内存使用方面的改进。

使用`'__builtin_expect'`（或更好的是`'likely'`和`'unlikely'`宏）非常简单。定义放在一个中央头文件中，编译器会处理其余部分。不过有一个小问题：程序员很容易在本意为`'unlikely'`的地方误用`'likely'`，反之亦然。即使有人使用像`oprofile`这样的工具来测量不正确的分支预测和L1i缺失，这些问题也很难被发现。

有一种简单的方法。附录A.2中的代码展示了另一种定义的`likely`和`unlikely`宏，这些宏在运行时主动测量静态预测是否正确。程序员或测试人员可以检查结果并进行调整。这些测量并不实际考虑程序的性能，它们只是检验程序员所作的静态假设。更多细节以及相关代码可以在上述引用的部分找到。

使用`gcc`进行PGO目前相当简单，但这是一个三步过程，并且必须满足某些要求。首先，所有源文件都必须添加`-fprofile-generate`选项进行编译。此选项需要传递给所有的编译器运行以及链接程序的命令。可以混合编译带有和不带该选项的对象文件，但是PGO对于未启用该选项的部分将不起作用。

编译器生成的二进制文件在正常行为的基础上，体积显著增大且运行速度变慢，因为它记录（并存储）了分支是否被取走的信息。编译器还会为每个输入文件生成一个扩展名为`.gcno`的文件，该文件包含

与代码中的分支相关的信息必须保留以备后用。

一旦程序二进制文件可用，应使用它来运行一组具有代表性的工作负载。无论选用何种工作负载，最终的二进制文件都将被优化以高效完成该任务。连续多次运行程序是可能且通常必要的；所有这些运行结果将贡献于同一个输出文件中。在程序终止前，收集的数据会被写入扩展名为`.gcda`的文件中。这些文件会在包含源文件的目录下创建。程序可以从任何目录执行，并且二进制文件可以被复制，但必须确保源代码所在的目录是可访问和可写的。同样地，每个输入源文件都会生成一个输出文件。如果多次运行该程序，则需要确保在源代码目录中能找到前一次运行产生的`.gcda`文件，否则无法将各次运行的数据合并到同一个文件中。

当一组代表性的测试运行完毕后，是时候重新编译应用程序了。编译器必须能够在与源文件相同的目录中找到`.gcda`文件。这些文件不能被移动，否则编译器将无法找到它们，并且文件中的嵌入式校验和也将不再匹配。在重新编译时，用`-fprofile-use`参数替换原来的`-fprofile-generate`参数。至关重要的是，在任何可能改变生成代码的方式下都不能修改源代码。这意味着：可以更改空白字符并编辑注释，但增加更多的分支或基本块将使收集的数据失效，并导致编译失败。

这是程序员需要做的全部工作；这是一个相当简单的过程。最重要的是要正确选择具有代表性的测试来执行测量。如果测试负载与实际使用程序的方式不匹配，所进行的优化可能会适得其反。因此，通常很难将PGO用于库文件。库可以在许多——有时是截然不同的——场景中被使用。除非用例确实相似，否则通常最好完全依赖静态分支预测，使用`__builtin_expect`。

关于`.gcno`和`.gcda`文件。这些是二进制文件，无法直接用于检查。不过可以使用`'gcov'`工具（该工具也是`gcc`包的一部分）来查看它们。此工具主要用于覆盖率分析（因此得名），但所使用的文件格式与PGO相同。`'gcov'`工具为每个包含已执行代码的源文件生成扩展名为`.gcov`的输出文件（这可能包括系统头文件）。这些文件是根据给定参数进行标注的源代码列表，其中包含了分支计数器、概率等信息。

0 0x3000000000 C	0 0x3000000B50: (within /lib64/ld-2.5.so)
1 0x 7FF000000 D	3320 0x3000000B53: (within /lib64/ld-2.5.so)
2 0x3000001000 C	58270 0x3000001080: _dl_start (in /lib64/ld-2.5.so)
3 0x3000219000 D	128020 0x30000010AE: _dl_start (in /lib64/ld-2.5.so)
4 0x300021A000 D	132170 0x30000010B5: _dl_start (in /lib64/ld-2.5.so)
5 0x3000008000 C	10489930 0x3000008B20: _dl_setup_hash (in /lib64/ld-2.5.so)
6 0x3000012000 C	13880830 0x3000012CC0: _dl_sysdep_start (in /lib64/ld-2.5.so)
7 0x3000013000 C	18091130 0x3000013440: brk (in /lib64/ld-2.5.so)
8 0x3000014000 C	19123850 0x3000014020: strlen (in /lib64/ld-2.5.so)
9 0x3000002000 C	23772480 0x3000002450: dl_main (in /lib64/ld-2.5.so)

图7.8: pagein 工具的输出

7.5 缺页异常优化

在支持按需分页的操作系统（如 Linux）中，mmap 调用仅修改页表。它确保对于由文件支持的页面，底层数据可以找到；而对于匿名内存，在访问时提供初始化为零的页面。在 mmap 调用时刻，并未实际分配内存。⁴⁶

当内存页面首次被访问时（无论是读取、写入数据，还是执行代码），就会发生分配过程。随后的缺页异常触发内核接管，并利用页表树确定该页面上必须存在的数据。虽然这种缺页中断的处理代价不菲，但它会针对进程使用的每一个页面都进行一次。

为了减少缺页异常的成本，必须降低使用的页面总数。优化代码大小有助于实现这一目标。为了减少特定代码路径（例如启动代码）的开销，也可以重新组织代码，使得在该代码路径中访问到的页面数最小化。然而，确定正确的顺序并不容易。

作者编写了一个基于 Valgrind 工具集的工具，用于测量缺页异常发生时的原因，而不仅仅是记录发生的次数。该 pagein 工具会输出关于缺页异常顺序和时间的信息。输出结果会被写入名为 pagein.<PID> 的文件中，如图 7.8 所示。第二列指明了被换入的页面地址。第三列则通过 ‘C’ 或 ‘D’ 分别表示该页是代码页还是数据页。第四列表明自首次发生缺页异常以来经过的周期数。其余部分则是 Valgrind 尝试为导致缺页异常的地址找到一个名称的结果。虽然地址值本身是正确的，但如果缺少调试信息，则该名称并不总是准确的。

在图7.8的例子中，执行从地址3000000B50₁₆开始，这迫使系统将地址为3000000000₁₆的页面调入内存。不久之后，紧接着的下一个页面也被调入；此时被调用的函数是

该页面为 _dl_start。初始代码访问了位于页7FF00000的变量v7。这发生在首次缺页异常后的第3,320个周期，很可能是程序中的第二条指令（距离第一条指令仅三个字节）。如果查看程序，会发现这段内存访问有些特别之处。相关指令是一条调用指令，并未明确加载或存储数据。不过它确实将返回地址压入了栈中，而这正是这里发生的情况。需要注意的是，这不是进程的官方栈，而是valgrind为应用程序内部维护的一个栈。这意味着在解释页面加载的结果时，必须牢记valgrind引入了一些伪迹。

页表加载（pagein）的输出可用于确定程序代码中哪些代码序列应理想地相邻。快速查看 /lib64/ld-2.5.so 代码可以发现，第一条指令立即调用了函数 _dl_start，并且这两个地方位于不同的页面上。通过重新排列代码将这些代码序列移到同一页面上，可以避免——或者至少推迟——缺页异常的发生。到目前为止，确定最佳的代码布局仍是一个繁琐的过程。由于第二次使用某个页面的设计并未记录下来，因此需要采用试错法来观察更改的效果。利用调用图分析，可以推测可能的调用序列；这可能会加快对函数和变量排序过程的速度。

在非常粗略的层面上，可以通过查看可执行文件或 DSO（动态共享对象）所包含的对象文件来观察调用序列。从一个或多个入口点（即函数名）开始，可以计算依赖链。在对象文件级别上，这很容易实现。每一轮中，确定哪些对象文件包含了所需的函数和变量。初始种子集必须明确指定。然后确定这些对象文件中的所有未定义引用，并将其添加到所需符号集中。重复此过程直到集合稳定为止。

第二步是确定一个顺序。各种对象文件需要被分组以尽量减少所占用的页面数量。作为额外的好处，任何函数都不应跨越页边界。然而，在最佳安排这些对象文件时，必须了解链接器后续的操作。重要的是要知道这一点。

⁴⁶If you want to say “Wrong!” wait a second, it will be qualified later that there are exceptions.

链接器会按照输入文件（例如档案）和命令行中对象文件的出现顺序将它们放入可执行文件或DSO中。这为程序员提供了足够的控制权。

对于愿意投入更多时间的开发者，已经有人通过使用 gcc 在调用 `-finstrument-functions` 选项时插入的 `__cyg_profile_func_enter` 和 `__cyg_profile_func_exit` 回调钩子实现了自动函数调用追踪并重新排序的成功尝试[17]。有关这些 `__cyg_*` 接口的更多信息，请参阅 gcc 手册。通过创建程序执行的跟踪记录，程序员可以更准确地确定调用链。文献 [17] 中的结果表明，仅通过重新排列函数顺序即可使启动成本降低 5%。主要的好处是减少了缺页异常的数量，但快表缓存也起到了重要作用——在虚拟化环境中，由于 TLB 缺失变得更加昂贵，其重要性日益增加。

通过结合页面加载工具的分析与调用序列信息，应该可以优化程序的某些阶段（如启动阶段），以最小化缺页异常的数量。

Linux 内核提供了两种额外的机制来避免缺页异常。第一种是在 `mmap` 调用中添加一个标志，指示内核不仅修改页表，而且预处理映射区域中的所有页面。这可以通过在 `mmap` 调用的第四个参数中加入 `MAP_POPULATE` 标志来实现。这样会使 `mmap` 调用变得更为昂贵，但如果调用所映射的所有页面立即被使用，则其带来的好处也是显著的。程序将不再经历多次缺页异常（每次由于同步需求等开销而显得非常昂贵），而是会有一个更昂贵但代价相对较小的 `mmap` 调用。然而，在大量映射页面在调用后不久（或永远）未被使用的场景下，使用该标志会产生不利影响。未使用的映射页面显然是时间和内存的浪费。那些立即预处理但后来才使用的页面也可能堵塞系统。内存会在实际使用前就被分配，这可能会导致在此期间出现内存短缺的情况。另一方面，在最坏的情况下，这些页面可以重新用于新的目的（因为它们尚未被修改），虽然这种操作的成本不高，但它与分配一起仍然会增加一些开销。

`MAP_POPULATE` 的粒度确实过于粗糙。其次，还可能存在另一个问题：这是一个优化操作；并非所有页面都必须被映射是关键性的要求。如果系统太忙而无法执行该操作，则可以放弃预故障处理。一旦页面真正被使用时，程序会触发缺页异常，但这并不比人为制造资源稀缺更糟糕。另一种方法是使用 `posix_madvise` 函数与 `POSIX_MADV_WILLNEED` 建议，这向操作系统发出信号，在不久的将来，程序将需要调用中描述的页面。内核

可以忽略该建议，但也可以预处理页面。这里的优点是粒度更细。任何映射地址空间区域中的单个页面或页面范围都可以被预处理。对于包含大量在运行时未使用的数据的内存映射文件而言，这种方法相比使用 `MAP_POPULATE` 具有巨大优势。

除了主动减少缺页异常的措施之外，还可以采取一种更为被动的方法，这种方法在硬件设计者中较为流行。动态共享对象（DSO）在其地址空间中占用相邻的页面，代码和数据各占一片连续的页面区域。页面大小越小，容纳同一 DSO 所需的页面数量就越多；相应地，缺页异常的数量也会增加。反之亦然，页面尺寸越大，映射所需（或匿名内存）的页面数量就越少；因此，缺页异常的数量也随之减少。

大多数架构支持 4KB 的页面大小。在 IA-64 和 PPC64 架构中，64KB 的页面大小也很流行。这意味着内存分配的最小单位是 64KB。该值需要在编译内核时指定，并且不能动态更改（至少目前如此）。多页大小架构的 ABI 设计允许应用程序使用任意一种页面大小运行。运行时将进行必要的调整，正确编写的应用程序不会察觉到任何不同。较大的页面大小会导致部分使用的页面浪费更多内存，但在某些情况下这是可以接受的。

大多数架构还支持 1MB 或更大的非常大的页面大小。在某些情况下，这样的页面也很有用，但将所有内存都以如此大的单位分配出去是没有意义的，因为物理 RAM 的浪费会太大。但是，非常大的页面也有其优势：如果使用巨大的数据集，在 x86-64 系统上将其存储在 2MB 页面中，相比于使用相同数量的 4KB 页面，可以减少 511 次缺页异常（每大页面）。这会产生显著的影响。解决方案是选择性地请求内存分配，仅针对请求的地址范围使用巨大页面，并且对于同一进程中的所有其他映射，则使用正常的页面大小。

大页尺寸的使用也伴随着一定的代价。由于用于大页的物理内存必须是连续的，随着时间推移，可能会因为内存碎片化而无法分配这样的大页。人们正在研究内存去碎片化和避免碎片化的技术，但这非常复杂。对于 2MB 的大页来说，所需的 512 个连续页面通常很难找到，唯一例外是在系统启动时。这就是为什么当前解决大页问题的方案需要使用一个特殊的文件系统 `hugetlbfs` 的原因。这个伪文件系统是由系统管理员根据需求写入所需预留的大页数量来分配的。

`/proc/sys/vm/巨页数量(nr_hugepages)`

此操作可能会因无法找到足够的连续内存而失败。如果使用虚拟化技术，情况会变得尤为复杂。采用VMM模型的虚拟化系统不直接管理物理内存，因此自身无法分配hugetlbfs。它必须依赖于VMM，并且该特性可能不受支持。对于KVM模型，运行KVM模块的Linux内核可以执行hugetlbfs分配，并有可能将所分配页面的一部分传递给一个客户机械。

后来，当程序需要一个大页时，有多种可能性：

- 该程序可以使用带有 SHM_HUGETLB 标志的 System V 共享内存接口。
- 一种类型为hugetlbfs的文件系统实际上可以被挂载，程序可以在挂载点下创建一个文件，并使用mmap将一个或多个页面映射为匿名内存。

在第一种情况下，无需挂载 hugetlbfs。请求一个或多个大页的代码可能如下所示：

```
key_t k = ftok("/some/key/file", 42); int id = shmget(k,
LENGTH, IPC_CREAT | SHM_HUGETLB | SHM_R
| SHM_W); void *a = shmat(id, NULL, 0);
```

该代码序列的关键部分是使用SHM_HUGETLB标志以及为LENGTH选择合适的值，该值必须是系统大页大小的倍数。不同架构有不同的值。System V共享内存接口的使用存在一个严重的问题，即依赖于key参数来区分（或共享）映射。ftok接口容易产生冲突，因此如果可能的话，最好使用其他机制。

如果挂载 hugetlbfs 文件系统的条件不成问题，最好使用它而不是 System V 共享内存。使用该特殊文件系统的主要问题是内核必须支持它，并且目前还没有标准化的挂载点。一旦文件系统被挂载（例如在 /dev/hugetlb），程序就可以轻松地使用它：

```
``cint fd = open("/dev/hugetlb/file1", O_RDWR | O_
CREAT, 0700); void *a = mmap(NULL, LENGTH, P
ROT_READ | PROT_WRITE, fd, 0);``
```

通过在打开调用中使用相同的文件名，多个进程可以共享同一组大页并进行协作。也可以将页面设置为可执行，在这种情况下，mmap调用中的PROT_EXEC标志也必须被设置。与System V共享内存示例一样，LENGTH的值必须是系统大页大小的整数倍。

一个防御性编写的应用程序（所有程序都应如此）可以在运行时使用类似如下的函数来确定挂载点：

```
``cchar *hugetlbfs_mntpoint(void) { char *result = NULL;
FILE *fp = setmntent(_PATH_MOUNTED, "r"); if (fp != NU
LL) { struct mntent *m; while ((m = getmntent(fp)) !=
NULL) if (strcmp(m->mnt_fstype, "hugetlbfs") == 0) {
result = strdup(m->mnt_dir); break; } } endm
ntent(fp); } return result;}```
```

更多相关信息可以在内核源代码树中的hugetlbpage.txt文件中找到。该文件还描述了IA-64架构所需的特殊处理。

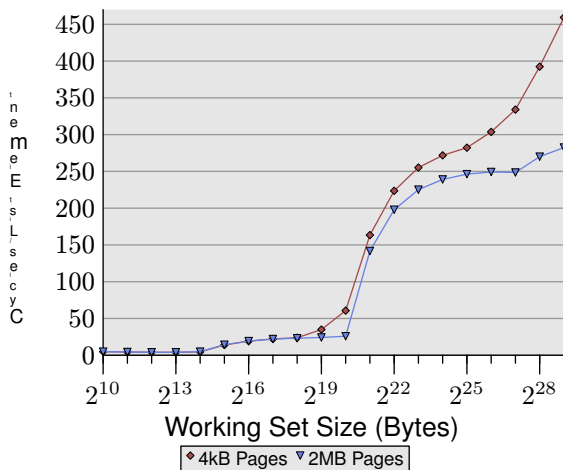


图7.9：使用巨页，NPAD=0

为了说明大页的优势，图7.9展示了NPAD=0随机Follow测试的结果。这与图3.15显示的数据相同，但这次我们还使用分配在大页中的内存来测量数据。如图所示，性能优势可能非常显著。对于2²⁰字节的测试，使用大页时速度提高了57%。这是由于该大小

仍然完全适合于一个单一的2MB页面中，因此不会发生快表未命中。

在此之后，收益最初较小，但随着工作集大小的增加而再次增长。对于512 MB的工作集大小，大页测试快38%。大页测试曲线在大约250个周期时达到平台期。当工作集超过2 GB后，数值又显著上升。平台期出现的原因是64个TLB条目可以覆盖2 GB的2 MB页面。

如这些数据所示，使用大工作集大小的成本中有一大部分来自快表未命中。采用本节描述的接口可以带来显著的好处。图中的数值很可能是上限值，但即使是实际世界中的程序也显示出明显的加速效果。由于数据库使用大量数据，它们是当今使用巨页的程序之一。

目前无法使用大页来映射文件支持的数据。有人对此功能的实现表现出兴趣，但迄今为止提出的方案都涉及显式地使用大页，并且依赖于 `hugetlbfs` 文件系统。这不可接受：在这种情况下，大页的使用必须是透明的。内核可以轻松确定哪些映射为大页并自动使用大页。一个主要问题是内核并不总是了解使用模式。如果原本可作为大页映射的内存后来需要 4K 页面粒度（例如，因为部分内存范围的保护通过 `mprotect` 更改），将浪费大量宝贵的资源，特别是线性物理内存。因此，在这种方案成功实现之前还需要一些时间。

8项即将来临的技术

在前面关于多处理器处理的章节中，我们已经看到如果增加CPU或核心的数量，则必须预期会出现显著的性能问题。但这种扩展正是未来所期待的趋势。处理器将越来越多地配备核心，而程序也必须越来越并行化以充分利用CPU的增强潜力，因为单核性能的增长速度将不再像过去那样快。

8.1 原子操作的问题

访问共享数据结构的同步通常通过以下两种方式实现：

- 通过互斥机制，通常利用系统运行时的功能来实现这一点；
- 通过使用无锁数据结构。

无锁数据结构的问题在于，处理器必须提供能够原子性地执行整个操作的基本原语。这种支持是有限的。在大多数架构中，仅限于原子性地读取和写入一个字。实现这一点主要有两种基本方法（参见第6.4.2节）：

- 使用原子比较和交换（CAS）操作；
- 使用加载锁定/存储条件(LL/SC)成对操作。

可以很容易地看到如何使用LL/SC指令来实现CAS操作。这使得CAS操作成为大多数原子操作和无锁数据结构的基础构建块。

一些处理器，特别是x86和x86-64架构，提供了更为复杂的一组原子操作。其中许多是针对特定目的优化的CAS（比较并交换）操作。例如，可以使用CAS和LL/SC（加载链接/存储条件）操作原子地向内存位置添加一个值，但在x86/x86-64处理器上原生支持的原子增量操作更快。程序员在编程时需要了解这些操作及其提供的内建函数，但这并不是什么新鲜事。

这两种架构的非凡扩展在于它们支持双字CAS（DCAS）操作。这对某些应用来说非常重要，但并非所有应用都是如此（参见[5]）。以无锁数组栈/后进先出数据结构为例，展示如何使用DCAS，可以参考图8.1中使用gcc内置函数的尝试。

此代码显然不是线程安全的。不同线程中的并发访问会修改全局变量 `top`。

```

struct elem {
    data_t d;
    struct elem *c;
};
struct elem *top;
void push(struct elem *n) {
    n->c = top;
    top = n;
}
struct elem *pop(void) {
    struct elem *res = top;
    if (res != NULL)
        top = res->c;
    return res;
}

```

图8.1: 非线程安全的LIFO

不考虑其他线程的修改。元素可能会丢失或被移除，而这些元素又可能神秘地重新出现。可以使用互斥机制，但在这里我们将尝试仅使用原子操作。

首次尝试使用CAS操作来安装或移除列表元素时，生成的代码如图8.2所示。

```

#define CAS __sync_bool_compare_and_swap
struct elem {
    data_t d;
    struct elem *c;
};
struct elem *top;
void push(struct elem *n) {
    do
        n->c = top;
    while (!CAS(&top, n->c, n));
}
struct elem *pop(void) {
    struct elem *res;
    while ((res = top) != NULL)
        if (CAS(&top, res, res->c))
            break;
    return res;
}

```

图8.2: 使用CAS的LIFO

乍一看这似乎是一个可行的解决方案。只有当元素与操作开始时LIFO栈顶的元素匹配时，才会修改top。但是我们必须考虑所有级别的并发性。可能在最糟糕的时刻调度另一个正在处理数据结构的线程。这里的一个例子就是所谓的ABA问题。假设在pop操作中的CAS操作之前恰好调度了第二个线程，并且它执行了以下操作：

```

1. l = pop()
2. push(新元素)
3. push(l)

```

此操作的最终结果是，原来的LIFO栈顶元素重新回到了栈顶位置，但第二个元素却有所不同。回到最初的线程中，由于栈顶元素未发生变化，CAS操作将会成功。但是，res->c的值并不正确，它指向的是原LIFO中的第二个元素而非newelem。结果就是这个新元素丢失了。

在文献[10]中，你可以找到一些处理器使用特定功能来解决此问题的建议。具体来说，这是指x86和x86-64处理器执行DCAS操作的能力。这一方法被用于图8.3所示代码的第三次实现中。

```

```c
#define CAS __sync_bool_compare_and_swap
struct elem { data_t d; struct elem *c; };
struct lifo { struct elem *top; size_t gen; };
void push(struct elem *n) { struct lifo old, new; do { old = l; new.top = n; new.c = old.top; new.gen = old.gen + 1; } while (!CAS(&l, old, new)); }
struct elem *pop(void) { struct lifo old, new; do { old = l; if (old.top == NULL) return NULL; new.top = old.top->c; new.gen = old.gen + 1; } while (!CAS(&l, old, new)); return old.top; }
```

```

图8.3: 使用双字CAS的LIFO

与前两个示例不同，这里的代码目前是伪代码，因为 gcc 不支持在 CAS 内置函数中使用结构体。不过，这个例子足以帮助理解该方法的思路。在指向 LIFO 栈顶的指针中添加了一个生成计数器。由于每次操作（无论是压栈还是弹栈）都会改变生成计数器的值，因此上述描述中的 ABA 问题不再存在。当第一个线程重新开始工作并实际交换栈顶指针时，生成计数器已经增加了三次。CAS 操作会失败，并且在下一轮循环中，正确的第一次和第二次操作将会进行。

LIFO中的元素被确定，并且LIFO未被破坏。至此完成。

这真的是解决方案吗？文献[10]的作者确实将其说得很有说服力，值得肯定的是，确实可以构建数据结构以支持上述代码用于LIFO。但是，总体而言，这种做法与之前的方案一样注定会失败。我们仍然存在并发问题，只是现在这些问题出现在了不同的地方。假设一个线程执行pop操作，并在对`old.top == NULL`的测试后被中断。此时另一个线程使用pop操作并获得了LIFO中原来的第一个元素的所有权。它可以对该元素进行任何操作，包括更改所有值或，在动态分配元素的情况下，释放内存。

现在第一个线程恢复执行。旧变量仍然填充着之前的后进先出（LIFO）栈顶元素。具体来说，栈顶成员指向了第二个线程弹出的元素。在``new.top =``和``old.top->c``中，第一个线程解引用了该元素中的指针。但是这个指针所引用的元素可能已经被释放掉了。这部分地址空间可能是不可访问的，并且进程可能会因此崩溃。对于通用数据类型实现来说，这种情况是不允许发生的。任何修复这个问题的方法都非常昂贵：内存必须永远不被释放，或者至少在释放之前需要验证没有任何线程再引用该内存了。鉴于无锁数据结构应该更快、更并发，这些额外的要求完全破坏了它们的优势。在支持垃圾收集的语言中，通过垃圾收集来处理内存可以解决这个问题，但这也有其代价。

对于更复杂的数据结构，情况往往更加糟糕。上述引用的论文还描述了一种FIFO实现（在后续论文中进行了改进）。但这段代码存在同样的问题。由于现有硬件（x86、x86-64）上的CAS操作仅限于修改内存中连续的两个字，因此在其他常见情况下毫无帮助。例如，在双向链表中的任意位置原子性地添加或删除元素是不可能实现的。

该问题在于通常涉及多个内存地址，只有当这些地址中的值没有被同时更改时，整个操作才能成功。这一概念在数据库处理中是众所周知的，而解决这种困境的一个最有前景的提议正是由此而来。

8.2 事务内存

在他们开创性的1993年论文[13]中，Herlihy和Moss提出通过硬件实现内存操作的事务处理，因为仅靠软件无法高效地解决这一问题。当时，Digital Equipment Corporation已经在与可扩展性问题作斗争。

⁴⁷As a side note, the developers of the IA-64 did *not* include this feature. They allow *comparing* two words, but replacing only one.

在他们的高端硬件上进行测试，这些硬件配备了几十个处理器。其原理与数据库事务相同：一个事务的结果要么一次性全部可见，要么事务被中止，所有值保持不变。

这里内存发挥了作用，这也是为什么前面部分要开发使用原子操作的算法。事务性内存旨在替代并扩展原子操作，在许多情况下特别是对于无锁数据结构时更是如此。将一个事务系统集成到处理器中听起来像是极其复杂的事情，但实际上，大多数处理器在某种程度上已经具备了类似的功能。

某些处理器实现的LL/SC操作构成一个事务。SC指令根据内存位置是否被访问来终止或提交该事务。事务内存是这一概念的扩展。现在，参与事务的不再是简单的指令对，而是多个指令。为了理解这如何运作，首先看看LL/SC指令是如何实现的是很有价值的。

8.2.1 加载锁定/存储条件实施

当发出LL指令时，内存位置的值会被加载到寄存器中。作为该操作的一部分，此值也会被加载到L1d缓存中。随后执行的SC指令只能在未对该值进行篡改的情况下成功。处理器如何检测这一点？回顾图3.18中的MESI协议描述，答案应该显而易见。如果另一个处理器更改了内存位置的值，则第一个处理器的L1d缓存中的该值副本必须被撤销。当在第一个处理器上执行SC指令时，它会发现需要再次将值加载到L1d中。这是处理器必须已经检测到的情况。

关于上下文切换（在同一处理器上可能的修改）和在另一个处理器写入后意外重新加载缓存行的问题，还有一些细节需要解决。这些问题可以通过一些策略（如在上下文切换时清空缓存）和额外标志位，或者为LL/SC指令使用单独的缓存行来解决。总体而言，LL/SC的实现几乎可以免费地与MESI之类的缓存一致性协议一起实现。

8.2.2 事务内存操作

为了使事务内存具有普遍实用性，一个事务不能仅通过第一条存储指令完成。相反，实现应允许执行一定数量的加载和存储操作；这意味着我们需要单独的提交和中止指令。稍后我们将看到，我们还需要一条允许检查的指令。

⁴⁸This does not mean it is actually implemented like this.

根据事务的当前状态以及它是否已经被中止来决定。

有三种不同的内存操作需要实现：

- 读取内存
- 读取以后会被写入的内存
- 写入内存

在分析MESI协议时，应当清楚这种特殊类型的读操作为何有用。正常情况下，读操作可以通过处于‘E’状态或‘S’状态的缓存行来满足。而第二种类型的读操作则需要一个处于‘E’状态的缓存行。第二种内存读取操作之所以必要，可以从以下讨论中略见一斑，但欲了解更详细的描述，请参阅关于事务性内存的相关文献，特别是参考文献[13]。

此外，我们还需要事务处理，主要包括数据库事务处理中已经熟悉的提交和回滚操作。不过还有一种操作虽然是理论上可选的，但在使用事务内存编写健壮程序时是必需的。该指令允许一个线程检测事务是否仍在正常进行（或许可以稍后提交），或者事务已失败并将被终止。

我们将讨论这些操作如何实际与CPU缓存交互，以及它们如何匹配总线操作。但在继续之前，我们先看一下使用事务内存的实际代码。这将有助于大家更好地理解本节的其余部分。

8.2.3 使用事务内存的示例代码

为了举例说明，我们重新审视之前的示例，并提供一种使用事务内存原语实现的后进先出（LIFO）方案。

这段代码与不安全的线程代码非常相似，这使得使用事务内存编写代码变得更加容易。代码中的新增部分包括 LTX、ST、COMMIT 和 VALIDATE 操作。这四个操作是请求访问事务内存的方式。实际上还有一个 LT 操作未在此处使用。LT 请求非独占读取权限，LTX 请求独占读取权限，而 ST 是将数据存储在事务内存中。VALIDATE 操作检查该事务是否仍然可以提交。如果事务仍处于正常状态，则返回 true；若事务已被标记为需要终止，则会实际终止并返回相应的值。接下来的事务内存指令将开始一个新的事务。因此，在这种情况下，代码使用了一个新的 if 语句块来判断事务是否仍在进行中。

```
``struct elem { data_t d; struct elem *c; };struct
elem *top;void push(struct elem *n){ while (1
){ n->c LTX(top); ST(&top, n); if
(COMMIT()) return; ... delay ... }}str
uct elem *pop(void){ while (1){ struct el
em *res LTX(top); if (VALIDATE()) {
if (res != NULL) ST(&top, res->c); if (C
OMMIT()) return res; } ... delay ...
}}``
```

图8.4：使用事务内存的LIFO

COMMIT 操作完成事务；如果操作成功执行，则返回 true。这意味着这部分程序已完成，线程可以继续执行后续任务。若操作返回 false 值，通常意味着整个代码序列必须重新执行。这正是外层 while 循环在此处所做的事情。不过，在某些情况下放弃当前工作可能是正确的做法。

LT、LTX 和 ST 操作的一个有趣之处在于，它们在不直接发出失败信号的情况下可能会失败。程序可以通过 VALIDATE 或 COMMIT 操作来请求这些信息。对于加载操作而言，这意味着实际被加载到寄存器中的值可能是无效的；这就是为什么在上述示例中需要在解引用指针之前使用 VALIDATE 的原因。在下一节中，我们将看到为何这是一个明智的选择。一旦事务性内存真正广泛可用，处理器可能会实现不同的机制。来自 [13] 的结果表明了我们在哪里所描述的情况。

推入函数可以总结如下：通过读取列表头部的指针来启动事务。由于稍后在函数中会写入此变量，因此该读请求获取独占所有权。如果另一个线程已经启动了一个事务，则加载将失败，并标记未出生的事务为已中止；在这种情况下，实际加载的值可能是无效数据。无论其状态如何，这个值都会被存储在下一个字段中。

新列表成员加入。这没有问题，因为此成员尚未使用，并且仅由一个线程访问。然后将指向列表头部的指针赋值给新元素的指针。如果事务仍然有效，则该写操作可以成功执行。这是正常情况，除非有线程使用除提供的 `push` 和 `pop` 函数之外的代码来访问此指针，否则不会失败。如果在执行 `ST` 指令时事务已经被中止，则不做任何事情。最后，线程尝试提交事务。如果提交成功，则工作完成；其他线程现在可以开始它们的事务。如果事务失败，则必须从头开始重试。但在这样做之前，最好插入一个延迟。如果不这样做，线程可能会陷入忙循环（浪费能量，使 CPU 过热）。

`pop` 函数稍微复杂一些。它首先读取包含链表头部的变量，并请求独占所有权。然后立即检查 `LTX` 操作是否成功。如果不成功，则本轮仅延迟到下一轮执行。如果顶部指针读取成功，这意味着其状态良好；我们现在可以解引用该指针。请记住，这正是使用原子操作时代码的问题所在；而在事务性内存中，这种情况可以轻松处理。随后的 `ST` 操作仅在 `LIFO` 不为空的情况下进行，就像原始的线程不安全代码一样。最后提交事务。如果成功，则函数返回旧头部指针；否则延迟并重试。这段代码中的难点在于要记住，`VALIDATE` 操作会因已失败而终止事务。接下来的事务性内存操作将启动一个新的事务，因此我们必须跳过该函数中剩余的代码。

当事务内存能够在硬件中实现时，延迟代码的工作原理将会一目了然。如果这方面处理不当，系统性能可能会显著下降。

8.2.4 事务内存的总线协议

现在我们已经了解了事务内存的基本原理，可以深入探讨其实现细节。请注意，这基于实际硬件的实现，并且是根据原始的事务内存设计和缓存一致性协议的知识来构建的。省略了一些细节，但仍能够洞察其性能特征。

尽管名称如此，事务性内存实际上并不是作为独立的内存来实现的；在多线程地址空间中的任何位置都需要进行事务处理，因此这样的设计是没有意义的。相反，它被实现在一级缓存处理的一部分中。理论上，这可以在正常的 `L1d` 缓存中实现，但正如文献[13]指出的那样，这不是一个好主意。我们更有可能看到事务缓存在以下方式中实现：

与 `L1d` 缓存平行存在事务缓存。所有访问都将像使用 `L1d` 缓存一样使用更高层次的缓存。事务缓存很可能比 `L1d` 小得多。如果它是全相联的，其大小由一个事务可以包含的操作数量决定。实现中可能会对架构和/或特定处理器版本设定限制。很容易想象一个只有16个元素甚至更少的事务缓存。在上述示例中，我们只需要一个单一的内存位置；具有更大事务工作集的算法会变得非常复杂。有可能我们会看到支持在同一时间有多个活动事务的处理器。此时缓存中的元素数量将增加，但仍足够小以实现全相联。

事务缓存与 `L1d` 是互斥的。这意味着一个缓存行最多只会存在于这两个缓存中的一个，而不会同时存在于两者之中。事务缓存中的每个槽位在任意时刻都处于 `MESI` 协议规定的四种状态之一。此外，每个槽位还具有一个事务状态。这些状态如下（名称参考文献 [13]）：

缓存槽为空，不包含任何数据。`MESI` 状态始终为 ‘`I`’。

`NORMAL` 缓存槽包含已提交的数据。这些数据也可能存在于 `L1d` 中。`MESI` 状态可以是 ‘`M`’、‘`E`’ 和 ‘`S`’。允许存在 ‘`M`’ 状态意味着事务提交不会强制将数据写入主内存（除非该内存区域被声明为未缓存或写通）。这有助于显著提高性能。

`XABORT` 指缓存槽中的数据在中止时将被丢弃。这显然与 `XCOMMIT` 相反。事务过程中创建的所有数据都保留在事务缓存中，只有在提交时才会写入主内存。这限制了最大事务大小，但同时也意味着，在除了事务缓存之外的其他任何内存位置上，无需了解单个内存位置上的 `XCOMMIT/XABORT` 二元性。可能的 `MESI` 状态为 ‘`M`’、‘`E`’ 和 ‘`S`’。

`XCOMMIT` 操作会丢弃缓存槽中的数据。这是一种处理器可以实现的优化方法。如果使用事务操作更改了内存位置，旧的内容不能直接丢弃：如果事务失败，则需要恢复旧内容。`MESI` 状态与 `XABORT` 相同。与 `XABORT` 不同的是，当事务缓存已满时，处于 ‘`M`’（修改）状态的任何 `XCOMMIT` 条目可以写回到内存中，然后对于所有状态，这些条目将被丢弃。

当启动 `LT` 操作时，处理器会在缓存中分配两个槽位。首先根据操作的地址查找 `NORMAL` 状态的槽位选择替换候选，即

缓存命中。如果找到这样的条目，则定位第二个槽位，复制值，并将一个条目标记为XABORT，另一个条目标记为XCOMMIT。

如果地址尚未缓存，则查找空闲的 EMPTY 缓存槽。若未找到，再寻找 NORMAL 槽。如果 MESI 状态为 ‘M’，则必须将旧内容刷新到内存中。如果没有可用的 NORMAL 槽，可以牺牲 XCOMMIT 条目。这可能是实现细节。事务的最大大小由事务缓存的大小决定，并且由于每个事务操作所需的槽位数量是固定的，因此可以在需要驱逐 XCOMMIT 条目之前限制事务的数量。

如果地址未在事务缓存中找到，则会在总线上发出一个 T READ 请求。这与正常的 READ 总线请求类似，但表示这是为事务缓存服务的。就像正常 READ 请求一样，其他处理器中的所有缓存首先有机会响应。如果没有缓存响应，则从主内存读取值。MESI 协议确定新缓存行的状态是 “E” 还是 “S”。当缓存行当前被另一个处理器或核心上的活动事务使用时，T READ 和 READ 的区别就显现出来了。在这种情况下，T-READ 操作显然失败了，并且没有数据传输。生成 T READ 总线请求的事务会被标记为失败，操作中使用的值（通常是简单的寄存器加载）是未定义的。回顾之前的示例，我们可以看到，只要正确使用事务内存操作，这种行为不会造成问题。在事务中加载的值被使用之前，必须通过 VALIDATE 进行验证。这几乎从来不是额外负担。正如我们在尝试使用原子操作实现 FIFO 时所见，我们添加的检查是使无锁代码工作的缺失特性。

LTX操作几乎与LT相同。唯一的区别在于总线操作使用的是T RFO而不是T READ。T RFO类似于正常的RFO总线消息，请求缓存行的独占所有权。结果缓存行的状态为 ‘E’。和T READ总线请求一样，T RFO也可能失败，在这种情况下使用的值也是未定义的。如果缓存行已经在本地事务缓存中，并且状态是 ‘M’ 或 ‘E’，则无需执行任何操作。如果本地事务缓存中的状态是 ‘S’，则必须发出总线请求以使所有其他副本失效。

ST操作与LTX类似。首先，值仅在本地事务缓存中独占可用。然后，ST操作将该值复制到缓存中的第二个槽，并标记该条目为XCOMMIT。最后，另一个槽被标记为XABORT，并将新值写入其中。如果事务已经中止，或者由于隐式LTX失败而新中止，则不进行任何写入操作。

Neither the VALIDATE nor the COMMIT operations automatically receive a signal that the transaction has been aborted.

事务性内存通过隐式和显式地创建总线操作，从而在并发处理上具有原子操作无法比拟的巨大优势。使用原子操作时，并发是通过将更改后的值写回主内存来实现的。如果您读过本文档至此，应该了解这样做代价高昂。而在事务性内存中，并不需要强制访问主内存。如果缓存没有空闲槽位，则必须驱逐当前内容；对于处于 “M” 状态的槽位，其内容必须写回到主内存。这与常规缓存并无不同，回写操作无需特殊的原子性保证即可完成。若缓存大小足够大，内容可以长时间驻留其中。如果对同一内存位置反复进行事务处理，则性能提升可能极其显著，因为在一种情况下每轮需要一到两次主内存访问，而在使用事务性内存时所有访问都命中了事务缓存，其速度与L1d缓存相当。

所有事务在中止时执行的VALIDATE和COMMIT操作仅是将标记为XABORT的缓存槽标记为空，并将XCOMMIT槽标记为NORMAL。同样，当COMMIT成功完成一个事务后，会将XCOMMIT槽标记为空，而将XABORT槽标记为NORMAL。这些操作在事务缓存中非常快速。不会向其他希望执行事务的处理器发出显式的通知；那些处理器只需继续尝试即可。高效地实现这一点是另一回事。在上面示例代码中的适当位置我们简单插入了...延迟...指令。我们可能会看到实际处理器对以有效方式延迟的支持。

总之，事务内存操作仅在启动新事务以及将不在事务缓存中的新缓存行添加到仍成功的事务时才引起总线操作。中止的事务的操作不会导致总线操作。多线程尝试使用同一段内存不会造成缓存行乒乓效应。

8.2.5 其他考虑因素

在第6.4.2节中，我们已经讨论了如何利用x86和x86-64架构中的锁前缀指令，在某些情况下避免编写原子操作代码。然而，当存在多个线程且这些线程不竞争同一内存时，所提出的技巧就显得不足了。在这种情况下，会不必要的使用原子操作。采用事务性内存则可以解决这个问题。昂贵的读取-修改总线消息（RFO）仅在不同CPU并发或顺序地访问同一内存位置时才会发出；只有在这种情况下才需要这样做。几乎不可能做得更好。

细心的读者可能会注意到延迟的问题。最坏的情况会怎样？如果拥有活跃事务的线程被取消调度，或者收到信号并可能终止，又或者决定要

使用`siglongjmp`跳转到外部作用域？答案是：事务将被中止。每当线程进行系统调用或接收信号（即发生环级变化）时，都可以中止事务。也可能在操作系统执行系统调用或处理信号时，中止事务是其职责的一部分。我们必须等到实现可用后才能看到实际的操作方式。

事务内存的最后一个方面是人们今天可能需要考虑的问题。事务缓存像其他缓存一样，以缓存行为基础进行操作。由于事务缓存是一个独占缓存，因此在同一缓存行上同时执行事务和非事务操作会引发问题。因此，确保这一点非常重要。

- 将非事务性数据移出缓存行
- 为单独的事务使用独立的缓存行

第一个观点并不新鲜，同样的努力在今天的原子操作中也会有所回报。第二个观点则更为棘手，因为如今对象几乎从未对齐到缓存行，这是由于相关联的高成本所致。如果用于原子操作修改的数据与被使用的数据位于同一缓存行上，则可以少用一条缓存行。这不适用于互斥（其中互斥体对象应始终拥有自己的缓存行），但确实存在将原子操作与其他数据一起使用的情况。对于事务内存，将缓存行用于两个目的很可能会导致致命问题。每次对数据⁴⁹的正常访问都会从事务缓存中移除该缓存行，从而导致事务失败。未来，数据对象的缓存对齐不仅关乎性能，还关乎正确性。

事务内存实现可能会采用更为精确的记账机制，从而不会因为正常访问属于事务范围内缓存行中的数据而出现问题。不过，这需要付出更多的努力，因为在那种情况下，MESI协议信息将不再足够。

8.3 提高延迟

关于未来内存技术的发展，有一点几乎是确定的：延迟将继续增加。我们在第2.2.4节中已经讨论过，即将推出的DDR3内存技术将比当前的DDR2技术具有更高的延迟。如果FB-DRAM得以部署，它也可能具有潜在的更高延迟，尤其是在FB-DRAM模块级联的情况下。请求和结果的传递并非没有代价。

⁴⁹From the cache line in question. Access to arbitrary other cache lines does not influence the transaction.

延迟的第二个来源是NUMA（非统一内存访问）架构的广泛应用。AMD的Opteron处理器如果包含多个处理器，则构成NUMA系统。每个CPU都配有独立的内存控制器和本地内存，但在SMP主板上，其余内存需要通过Hypertransport总线进行访问。英特尔的CSI技术也将采用类似的技术。由于每处理器带宽限制以及服务多张10Gb/s以太网卡等需求的存在，即使每插槽核心数量增加，多插座主板也不会消失。

延迟的第三个来源是协处理器。我们认为在20世纪90年代初数学协处理器不再必要后就解决了这个问题，但如今它们又卷土重来。英特尔的Geneseo和AMD的Torrenza是平台扩展，允许第三方硬件开发商将其产品集成到主板上。也就是说，协处理器不必再位于PCIe卡上，而是更靠近CPU的位置，这为它们提供了更大的带宽。

IBM采取了不同的路线（尽管类似Intel和AMD的扩展仍然是可能的），开发了Cell处理器。Cell处理器除了包含一个PowerPC内核之外，还配备了8个协同处理单元（SPU），这些专门处理器主要用于浮点运算。

协同处理器和SPU（可编程单元）的共同点在于，它们很可能拥有比实际处理器更慢的内存逻辑。这在一定程度上是由必要的简化所导致：所有缓存处理、预取等操作都相当复杂，尤其是在需要缓存一致性的情况下更是如此。高性能程序将越来越多地依赖于协同处理器，因为性能差异可能会非常显著。理论上，Cell CPU的峰值性能可达210GFLOPS，而高端CPU则为50-60GFLOPS。目前使用的图形处理单元（GPU，即显卡上的处理器）能够实现更高的数值（超过500GFLOPS），这些GPU可能只需稍作努力就能集成到Geneseo/Torrenza系统中。

由于所有这些发展，程序员必须得出结论，预取将变得越来越重要。对于协处理器而言，这将是绝对关键的。对于CPU来说，尤其是随着核心数量越来越多，需要始终保持前端总线忙碌，而不是批量堆积请求。这要求通过高效使用预取指令尽可能地让CPU对未来流量有更多了解。

8.4 向量操作

当今主流处理器中的多媒体扩展仅以有限的方式实现向量操作。向量指令的特点是一条指令执行大量运算（单指令多数据，SIMD）。与标量运算相比，这可以

关于多媒体指令的说法还有很多，但与像Cray-1这样的向量计算机或IBM 3090之类的机器的向量单元所实现的功能相比，这些还相差甚远。

为了补偿单条指令执行的操作数量有限（大多数机器上为四次浮点运算或两次双精度运算），周围的循环必须被执行更多次数。第A.1节中的示例清楚地展示了这一点，每个缓存行都需要执行SM次迭代。

随着更宽的向量寄存器和操作，循环迭代次数可以减少。这不仅改善了指令解码等方面的性能；更重要的是内存效应方面的改进。单条指令加载或存储更多数据后，处理器能够更好地了解应用程序的内存使用情况，并不需要从个别指令的行为中拼凑信息。此外，提供不影响缓存的加载或存储指令变得更加有用。在x86 CPU中，如果SSE寄存器进行16字节宽的加载操作，则使用未缓存加载是一个糟糕的想法，因为后续对同一缓存行的访问需要再次从内存中加载数据（在缓存缺失的情况下）。相反，如果向量寄存器足够宽以容纳一个或多个缓存行，那么未缓存加载或存储不会产生负面影响。处理无法完全放入缓存的数据集的操作变得更加实际。

拥有大型向量寄存器并不一定意味着指令的延迟会增加；向量指令不必等待所有数据都被读取或存储。如果向量单元能够识别代码流，它可以先从已经读取的数据开始处理。这意味着，例如，在一个向量寄存器被加载后紧接着对所有的向量元素与标量相乘的情况下，CPU可以在第一部分向量加载完成后立即开始乘法操作。这仅仅取决于向量单元的复杂程度。这表明，在理论上，向量寄存器可以变得非常宽大，并且程序设计时也可以考虑到这一点。实际上，由于处理器被用于多进程和多线程的操作系统中，向量寄存器的大小会受到限制。因此，包括存储和加载寄存器值在内的上下文切换时间变得非常重要。

随着向量寄存器的宽度增加，操作的输入和输出数据在内存中无法顺序排列的问题也随之出现。这可能是由于矩阵是稀疏的、按列而不是按行访问矩阵以及其他许多因素造成的。为了应对这种情况，向量单元提供了以非顺序模式访问内存的方法。单个向量加载或存储指令可以参数化，并被指示从地址空间中的多个不同位置加载数据。而使用当今的多媒体指令集，这是完全不可能实现的。值必须逐一显式地加载，然后费力地组合到一个向量寄存器中。

过去的向量单元具有不同的模式以适应各种计算需求。

允许最有用的访问模式：

- 使用 *striding*，程序可以指定两个相邻向量元素之间的间隔大小。所有元素间的间隔必须相同，但这样就可以轻松地用一条指令将矩阵的一列读入向量寄存器，而无需为每一行分别使用一条指令。
- 使用间接寻址，可以创建任意访问模式。加载或存储指令接收指向一个数组的指针，该数组包含需要被加载的真实内存位置的地址或偏移量。

目前尚不清楚主流处理器的未来版本是否会重新引入真正的向量操作。也许这项工作会被分配给协处理器来完成。无论如何，如果我们能够访问到向量操作，正确组织执行这些操作的代码就显得尤为重要了。代码应该具有自包含性和可替换性，并且接口应当足够通用，以便高效地应用向量操作。例如，接口应允许整个矩阵相加，而不仅仅是行、列或元素组的操作。构建块越大，使用向量操作的机会也就越好。

在文献[11]中，作者们强烈呼吁复兴向量操作。他们指出了许多优势，并试图驳斥各种神话。然而，他们的描述过于简单化了。如上所述，大型寄存器集意味着较高的上下文切换时间，在通用操作系统中必须避免这种情况。参见IA-64处理器在执行密集型上下文切换操作时遇到的问题。如果中断涉及向量操作的长时间执行也是一个问题。一旦发生中断，处理器必须停止当前工作并开始处理中断。之后，它必须继续执行被中断的代码。通常，在指令执行过程中中断是一个大问题；虽然并非不可能实现，但确实很复杂。对于长时间运行的指令来说，要么必须进行这样的中断处理，要么这些指令需要以可重新启动的方式实现，否则中断响应时间会过高。后者是不可接受的。

向量单元在内存访问对齐方面也较为宽容，这影响了相应算法的设计。如今的一些处理器（尤其是精简指令集处理器）要求严格的对齐规则，因此扩展到完整的向量操作并非易事。然而，拥有向量操作特别是支持步进和间接寻址的情况下，具有很大的潜在优势，所以我们有在未来看到这一功能的实现。

这是第6.2.1节中矩阵乘法的完整基准程序。关于所用内联汇编指令的详细信息，请参阅Intel的参考手册。

```
#include <stdlib.h> #include <stdio.h> #include <emmintrin.h> #define N 1000 double res[N][N] __attribute__
__ ((aligned (64))); double mul1[N][N] __attribute__ ((aligned (64))); double mul2[N][N] __attribute__
((aligned (64))); #define SM (CLS / sizeof (double)) int main (void) { // ... 初始化 mul1 和 mul2 int i
, i2, j, j2, k, k2; double *restrict rres; double *restrict rmul1; double *restrict rmul2; for (i = 0; i
< N; i += SM) for (j = 0; j < N; j += SM) for (k = 0; k < N; k += SM) { for (i2 = 0, rr
es = &res[i][j], rmul1 = &mul1[i][k]; i2 < SM; ++i2, rres += N, rmul1 += N) { __mm_prefetch (
&rmul1[8], _MM_HINT_NTA); for (k2 = 0, rmul2 = &mul2[k][j]; k2 < SM; ++k2, rmul2 += N
) { __m128d m1d = _mm_load_sd (&rmul1[k2]); m1d = _mm_unpacklo_pd (
m1d, m1d); for (j2 = 0; j2 < SM; j2 += 2) { __m128d m2 = _mm_load_pd (
&rmul2[j2]); __m128d r2 = _mm_load_pd (&rres[j2]); __mm_store_pd (
&rres[j2], _mm_add_pd (_mm_mul_pd (m2, m1d), r2)); } } } } // ... 使用 res 矩阵 return 0; }
```

循环的结构与第6.2.1节中的最终版本几乎相同。唯一的大变化是将加载rmul1[k2]值的操作从内层循环中提取出来，因为我们必须创建一个向量，其中两个元素具有相同的值。这就是_mm_unpacklo_pd()内在函数的作用。

值得注意的是，我们将三个数组显式地对齐，使得预期在同一缓存行中的值确实位于该缓存行中。

如果按照第6.2.2节的建议使用likely和unlikely的定义，那么很容易在调试模式下检查这些假设是否真正成立。宏的定义可以替换为如下形式：

```
#ifndef DEBUGPRED
``c#define 不大可能(expr) __builtin_expect (!!(expr), 0)#define 很可能(expr)
__builtin_expect (!!(expr), 1)``
#else
``asm (".section predict_data, \"aw\"; .previous\n\" ".section predict_line,
\"a\"; .previous\n\" ".section predict_file, \"a\"; .previous");``

#ifdef __x86_64__
``c#define debugpred__(e, E) \({ long int _e = v1 !!(e); \ asm volatile (".pushsection predict_data\n\" \
"..predictcnt%=: .quad 0; .quad 0\n\" \          ".section predict_line; .quad %c1\n\" \          ".section p
redict_file; .quad %c2; .popsection\n\" \          "addq $1,..predictcnt%= (,%0,8) " \          :: "r" (_e E
), "i" (__LINE__), "i" (__FILE__)); \ __builtin_expect (_e, E); \})``

# elif defined __i386__
``c#define debugpred__(e, E) \({ long int _e = !(e); \ asm volatile (".pushsection predict_data\n\" \
"..predictcnt%=: .long 0; .long 0\n\" \          ".section predict_line; .long %c1\n\" \          ".section pr
edict_file; .long %c2; .popsection\n\" \          "incl ..predictcnt%= (,%0,4) " \          :: "r" (_e E), "i"
(__LINE__), "i" (__FILE__)); \ __builtin_expect (_e, E); \})``

``#else#error "debugpred__ 定义缺失"#endif#define unlikely(expt) d
ebugpred__((expr), 0)#define likely(expr) debugpred__((expr), 1)#en
dif``
```

这些宏在创建 ELF 文件时大量使用了 GNU 汇编器和链接器提供的功能。在 DEBUGPRED 情况下的第一条 a sm 语句定义了三个额外的段；主要目的是向汇编器提供有关如何创建这些段的信息。所有段都在运行时可用，而 predict_data 段是可写的。重要的是，所有的段名都必须有效的 C 标识符。稍后会解释原因。

新的 likely 和 unlikely 宏的定义引用了机器特定的 debugpred__ 宏。该宏具有以下任务：

1. 在预测数据段分配两个字节来存储正确预测和错误预测的计数。通过使用%=为这两个字段赋予唯一名称，前面的点号确保这些符号不会污染符号表。
2. 在预测行段分配一个字节来存储可能或不可能宏使用的行号。

在 `predict_file` 部分分配一个字来存放可能或不可能使用的宏的文件名指针。

4. 根据表达式 `e` 的实际值，增加为此宏创建的“正确”或“错误”计数器。这里不使用原子操作，因为它们发生在碰撞时会慢得多，并且在这种不太可能的情况下绝对精确度并不重要。如果需要更改也很简单。

《`.pushsection`》和《`.popsection`》伪操作的详细描述请参见汇编手册。感兴趣的读者可借助相关手册并通过尝试与错误来进一步探索这些定义的细节。

这些宏自动且透明地负责收集关于正确和错误分支预测的信息。所缺少的是获取结果的方法。最简单的方式是为该对象定义一个析构函数，并在其中打印出结果。这可以通过如下定义的函数实现：

```
extern long int __start_predict_data;extern long int __stop_predict_data;extern long int __start_predict_line;extern const char *__start_predict_file;static void __attribute__((destructor)) predprint(void) { long int *s = &__start_predict_data; long int *e = &__stop_predict_data; long int *sl = &__start_predict_line; const char **sf = &__start_predict_file; while (s < e) { printf("%s:%ld: incorrect %ld, correct %ld%s\n", *sf, *sl, s[0], s[1], s[0] > s[1] ? " WARNING" : ""); (*sl)++; (*sf)++; s += 2; }}
```

这里节名是有效的C语言标识符这一事实起到了关键作用；GNU链接器利用这一点，在必要时自动定义该节的两个符号。`__start_XYZ` 符号对应于节 `XYZ` 的开始位置，而 `__stop_XYZ` 则表示紧接在节 `XYZ` 之后的第一个字节的位置。这些符号使得可以在运行时遍历节的内容成为可能。需要注意的是，由于节内容可以来自链接器在连接过程中使用的所有文件，编译器和汇编器没有足够的信息来确定该节的大小。只有借助这些由链接器生成的特殊符号，才能实现对节内容的遍历。

代码不仅针对一个部分进行迭代；涉及三个部分。由于我们知道每向 `predict_data` 部分添加两个单词，就会向 `predict_line` 和 `predict_file` 部分各添加一个单词，因此无需检查这两个部分的边界。我们只需同步移动指针并递增它们即可。

该代码会为每一条出现在其中的预测输出一行信息，并突出显示那些预测错误的情况。当然，这可以进行调整，调试模式也可以仅标记那些错误预测多于正确预测的条目。这些是需要更改的候选者。还有一些细节使问题变得复杂；例如，如果分支预测发生在多个地方使用的宏内部，则在做出最终判断之前必须一起考虑所有宏的使用情况。

两个最后的说明：用于此次调试操作的数据量不小，特别是在涉及动态共享对象（DSO）时会更加昂贵（预测文件部分必须重新定位）。因此，在生产二进制文件中不应启用调试模式。最后，每个可执行文件和动态共享对象都会生成自己的输出，在分析数据时需注意这一点。

本节包含了用于测量在同一缓存行上使用变量与在不同缓存行上使用变量的开销的测试程序。

```
``c#include <error.h>#include
e <pthread.h>#include <stdlib
.h>``

#define N (atomic ? 10000000 : 500000000) #define N ((atomic) ? (10000000) : (500000000))
```

```
``cstatic int atomic;static unsigned nthreads;static unsigned
disp;static long **reads;static pthread_barrier_t b;static voi
d *tf(void *arg) { long *p = (long *) arg; if (atomic)
for (int n = 0; n < N; ++n) __sync_add_and_fetch(p,
1); else for (int n = 0; n < N; ++n) { (*p)++;
asm volatile("" : : "m"(*p)); } return NULL;}int main
(int argc, char *argv[]) { if (argc < 2) disp = 0; else
disp = atol(argv[1]); if (argc < 3) nthreads = 2; el
se nthreads = atol(argv[2]) ? 1; if (argc < 4) ato
mic = 1; else atomic = atol(argv[3]); pthread_barrie
r_init(&b, NULL, nthreads);`` 注意：上述代码片段在最
后一行的`pthread_barrier_init`函数调用中，参数`nthread
`应为`nthreads`。
```

s);

```

void *p;posix_memalign(&p, 64, (nthreads * disp ?: 1) * sizeof(long));long *mem = p;pthread_t th[nthreads]
;pthread_attr_t a;pthread_attr_init(&a);cpu_set_t c;for (unsigned i = 1; i < nthreads; ++i){ CPU_ZERO(&c);
CPU_SET(i, &c); pthread_attr_setaffinity_np(&a, sizeof(c), &c); mem[i * disp] = 0; pthread_create(&th[i]
, &a, tf, &mem[i * disp]);}CPU_ZERO(&c);CPU_SET(0, &c);pthread_setaffinity_np(pthread_self(), sizeof(c),
&c);mem[0] = 0;tf(&mem[0]);if ((disp > 0 && mem[0] != nthreads * N) || (disp <= 0 && mem[0] != N)) erro
r(1, 0, "mem[0] wrong: %ld instead of %d", mem[0], disp > 0 ? nthreads * N : N);for (unsigned i = 1; i < nthrea
ds; ++i){ pthread_join(th[i], NULL); if (disp <= 0 && mem[i * disp] != N) error(1, 0, "mem[%u] wron
g: %ld instead of %d", i, mem[i * disp], N);}return 0;

```

此处提供的代码主要用于演示如何编写一个测量缓存行开销等效果的程序。有趣的部分是 `tf` 函数中循环体的内容。已知编译器的 `__sync_add_and_fetch` 内在函数会生成原子加指令。在第二个循环中，我们必须“消费”增量操作的结果（通过内联汇编语句）。该 `asm` 语句并不会实际引入任何代码；相反，它防止了编译器将增量操作移出循环。

程序将线程固定在特定的处理器上，这是第二个有趣的部分。代码假设处理器编号为0到3，这通常是机器具有四个或更多逻辑处理器时的情况。代码本可以使用 `libNUMA` 接口来确定可用处理器的编号，但这个测试程序应该广泛适用而不引入这种依赖关系。无论如何，稍作修改即可解决这个问题。

附录 B 几个 OProfile 使用技巧

以下内容并非旨在作为使用oprofile的教程。关于这一主题已有完整的文档进行详细介绍。相反，这里提供一些高层次的建议，帮助您审视自己的程序以发现潜在的问题所在。但在那之前，我们至少需要一个简要的入门介绍。

OProfile 基础知识

OProfile 分为两个阶段：采集和分析。采集由内核执行；由于测量使用了 CPU 的性能计数器，因此无法在用户级进行，因为这些计数器的访问需要读取 MSR，而读取 MSR 需要相应的权限。

每个现代处理器都提供了一套性能计数器。在某些架构中，部分计数器由所有处理器实现提供，而其他计数器则会因版本不同而有所差异。这使得给出关于oprofile使用的一般性建议变得困难。目前（尚且）没有一个更高层次的抽象来隐藏这些细节。

处理器版本还决定了任何时刻可以同时追踪多少事件及其组合方式。这又增加了复杂性。

如果用户了解性能计数器的相关细节，可以使用opcontrol程序来选择需要统计的事件。对于每个事件，必须指定“溢出数量”（即在CPU中断以记录事件之前必须发生的事件次数），以及该事件是否应在用户态和/或内核态下进行计数，并最终指定一个“单位掩码”（它用于选择性能计数器的子功能）。

为了在x86和x86-64处理器上计数CPU周期，需要发出以下命令：

```
opcontrol --事件 CPU_CLK_UNHALTED:30000:0:1:1
```

数字30000是溢出数量。选择一个合理的值对于系统的运行行为和收集的数据至关重要。要求接收每次事件发生时的所有数据是一个糟糕的主意，因为对于许多事件来说，这会使机器陷入停滞状态，因为它只会忙于处理事件溢出的数据采集工作；这就是为什么oprofile强制执行最小值的原因。由于不同的事件在正常代码中被触发的概率不同，因此每个事件的最小值也各不相同。

选择一个非常高的数值会降低分析的分辨率。在每次溢出时，oprofile记录下当前执行指令的地址；对于x86和PowerPC架构，在某些情况下还可以记录回溯信息。采用较低的分辨率可能导致热点区域无法获得具有代表性的命中次数；这完全取决于概率问题，这也是为什么oprofile被称为概率分析工具的原因。溢出数值越低，对系统性能的影响越大（表现为延迟增加），但同时分辨率也会更高。

如果需要对特定程序进行性能分析，且系统不用于生产环境，则通常最有效的方法是使用尽可能低的溢出值。每个事件的确切值可以通过查询获得。

```
opcontrol --list-events
```

如果被分析的程序与其他进程交互，而性能分析导致了显著的延迟，则可能会出现这个问题。同样地，如果某个进程具有实时性要求，在频繁中断的情况下无法满足这些需求时也会产生问题。在这种情况下，需要找到一个折中的解决方案。同样的情况也适用于长时间对整个系统进行性能分析的情形。过低的中断次数会导致严重的性能下降。无论如何，oprofile和其他任何性能分析工具一样，都会引入不确定性和不准确性。

性能分析需要通过`opcontrol --start`命令启动，并可通过`opcontrol --stop`命令停止。在oprofile处于活动状态时，它会收集数据；这些数据首先由内核收集，然后以批处理的形式发送到用户级守护进程，在那里进行解码并写入文件系统。使用`opcontrol --dump`可以请求将内核中缓存的所有信息释放给用户级。

收集的数据可能包含来自不同性能计数器的事件。除非用户选择在单独的oprofile运行之间清除存储的数据，否则所有数字都将并行保存。可以在不同的时间点累积同一事件的数据。如果在不同的分析运行中遇到相同的事件，并且用户选择了相应的选项，则这些数字将被累加。

数据收集过程的用户级部分将数据解复用。每个文件的数据分别存储。它是

希望所有架构最终都能提供回溯支持。

甚至可以区分各个可执行文件使用的DSO（动态共享对象），甚至是各个线程的数据。由此产生的数据可以通过oparchive进行归档。此命令生成的文件可以传输到另一台机器上并在那里进行分析。

通过`opreport`程序可以生成性能分析结果的报告。使用`opannotate`可以查看各种事件发生的位置：是哪条指令，在数据可用的情况下，还可以看到是在源代码中的哪一行。这使得查找热点区域变得容易。统计CPU周期可以指出最耗时的地方（这包括缓存未命中），而统计已提交的指令则可以帮助找到执行最多的指令所在位置——这两者之间存在很大差异。

对地址的单次访问通常没有意义。统计剖析的一个副作用是，仅执行几次甚至只执行一次的指令也可能被记录为命中。在这种情况下，需要通过重复测试来验证结果。

附录 B.2 外观示意图

一个oprofile会话可以简单到如下形式：

```
``opcontrol -i cachebenchopcontrol -e INST_RETIRED:6000:0:0:1
--start./cachebench ...opcontrol -h``
```

请注意，这些命令（包括实际程序）都是以root用户身份运行的。在这里以root用户身份运行程序只是为了简化操作；实际上该程序可以由任何用户执行，并且oprofile能够捕获其相关信息。接下来的步骤是分析数据。使用opreport我们可以看到：

CPU：核心数 2，频率 1596 MHz（估计值） 记录 INST_RETIRED.ANY_P 事件（已提交的指令数量），单位掩码为 0x00（无单元掩码），计数值为 6000INST_RETIRED:6000 | 样本数 | 百分比 | -----116452 100.000 cachebench

这意味着我们收集了大量的事件；现在可以使用opannotate来更详细地查看数据。我们可以看到程序中记录最多事件的位置。部分opannotate --source输出如下所示：

```
静态 void inc(struct l *l, unsigned n) { while (n-- > 0) /* inc total: 13980
11.7926 */ { long v4 = l->pad[0].l; // 13974 11.7875 l->n++; // 1 8.4e-04
asm volatile ("": "r" (l)); }}
```

这是测试的内部功能，其中大部分时间都花费在这里。我们可以看到样本分布在循环中的三条指令行上。主要原因在于采样时记录的指令指针并不总是100%准确。CPU会以乱序执行指令；重构确切的执行顺序以生成正确的指令指针非常困难。最近的CPU版本尝试为少数几个事件完成这一任务，但总体而言这样做不值得或根本不可能实现。在大多数情况下这其实无关紧要。程序员应该能够根据一组正态分布的样本确定程序的运行情况。

B.3 开始 profiling

在开始分析一段代码时，确实可以从程序中耗时最多的部分入手。这部分代码当然需要尽可能地进行优化。但接下来呢？程序的其余部分又在哪里花费了大量时间？这个问题并不容易回答。

在这种情况下，问题之一是绝对值并不能说明真实情况。程序中的某个循环可能占用了大部分时间，这倒是可以接受的。然而，导致高CPU利用率的原因有很多。更常见的情况是，CPU使用率在整个程序中分布得比较均匀。在这种情况下，绝对值会指向很多地方，而这并没有太大用处。

在许多情况下，观察两个事件的比例是有帮助的。例如，如果没有任何衡量标准来确定一个函数被执行的频率，那么该函数中预测错误的分支数量就毫无意义。是的，绝对值对于程序性能来说很重要。但是，每调用一次产生的预测错误次数更能反映函数代码的质量。英特尔针对x86和x86-64架构的优化手册[15]描述了一些值得研究的比例（在引用文档的附录B.7中列出了Core 2事件的相关比例）。与内存处理相关的几个重要比例如下所示。

| | | |
|--------------------------|---|--|
| Instruction Fetch Stall | $\text{CYCLES_L1I_MEM_STALLED} / \text{CPU_CLK_UNHALTED.CORE}$ | Ratio of cycles during which in instruction decoder is waiting for new data due to cache or ITLB misses. |
| ITLB Miss Rate | $\text{ITLB_MISS_RETIRED} / \text{INST_RETIRED.ANY}$ | ITLB misses per instruction. If this ratio is high the code is spread over too many pages. |
| L1I Miss Rate | $\text{L1I_MISSES} / \text{INST_RETIRED.ANY}$ | L1i misses per instruction. The execution flow is unpredictable or the code size is too large. In the former case avoiding indirect jumps might help. In the latter case block reordering or avoiding inlining might help. |
| L2 Instruction Miss Rate | $\text{L2_IFETCH.SELF.I_STATE} / \text{INST_RETIRED.ANY}$ | L2 misses for program code per instruction. Any value larger than zero indicates code locality problems which are even worse than L1i misses. |
| Load Rate | $\text{L1D_CACHE_LD.MESI} / \text{CPU_CLK_UNHALTED.CORE}$ | Read operations per cycle. A Core 2 core can service one load operation. A high ratio means the execution is bound by memory reads. |
| Store Order Block | $\text{STORE_BLOCK.ORDER} / \text{CPU_CLK_UNHALTED.CORE}$ | Ratio if stores blocked by previous stores which miss the cache. |
| L1d Rate Blocking Loads | $\text{LOAD_BLOCK.L1D} / \text{CPU_CLK_UNHALTED.CORE}$ | Loads from L1d blocked by lack of resources. Usually this means too many concurrent L1d accesses. |
| L1D Miss Rate | $\text{L1D_REPL} / \text{INST_RETIRED.ANY}$ | L1d misses per instruction. A high rate means that prefetching is not effective and L2 is used too often. |
| L2 Data Miss Rate | $\text{L2_LINES_IN.SELF.ANY} / \text{INST_RETIRED.ANY}$ | L2 misses for data per instruction. If the value is significantly greater than zero, hardware and software prefetching is ineffective. The processor needs more (or earlier) software prefetching help. |
| L2 Demand Miss Rate | $\text{L2_LINES_IN.SELF.DEMAND} / \text{INST_RETIRED.ANY}$ | L2 misses for data per instruction for which the hardware prefetcher was not used at all. That means, prefetching has not even started. |
| Useful NTA Prefetch Rate | $\text{SSE_PRE_MISS.NTA} / \text{SSS_PRE_EXEC.NTA}$ | Ratio of useful non-temporal prefetch relative to the total number of all non-temporal prefetches. A low value means many values are already in the cache. This ratio can be computed for the other prefetch types as well. |
| Late NTA Prefetch Rate | $\text{LOAD_HIT_PRE} / \text{SSS_PRE_EXEC.NTA}$ | Ratio of load requests for data with ongoing prefetch relative to the total number of all non-temporal prefetches. A high value means the software prefetch instruction is issued too late. This ratio can be computed for the other prefetch types as well. |

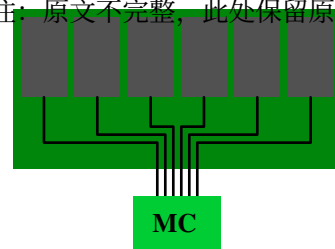
对于所有这些比例，程序应与oprofile一起运行，并指示其测量两个事件。这保证了两个计数结果可以比较。在进行除法运算之前，必须确保考虑到可能不同的溢出值。最简单的方法是通过将每个事件计数器乘以相应的溢出值来实现这一点。

这些比率在分析整个程序、可执行文件或动态共享对象级别，甚至函数级别时都有意义。越深入地剖析程序，所包含的误差就越多。

要理解这些比率，需要有基准值。这并非像看起来那么简单。不同类型的代码具有不同的特性，某个程序中表现不佳的比率在另一个程序中可能是正常的。

尽管高效编程不需要掌握这些更详细的内存类型技术细节，但描述“带缓冲”的与“不带缓冲”的以及具有纠错码（ECC）和无纠错码的DRAM类型的差异可能会有所帮助。

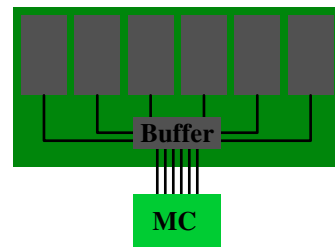
“registered”和“buffered”这两个术语在描述时可互换使用，当——（注：原文不完整，此处保留原句结构以待补充。）描述一种具有额外组件的DRAM类型：在DRAM模块上增加了一个缓冲器。所有DDR内存类型都可以采用带缓冲和不带缓冲的形式。对于不带缓冲的模块，内存控制器直接连接到模块上的所有芯片。图C.1展示了这种配置。



图C.1：无注册DRAM模块

从电气特性来看，这是一项相当艰巨的任务。内存控制器必须能够处理所有内存芯片的容量（图中所示的六片之外还有更多）。如果内存控制器（MC）存在限制，或者需要使用多个内存模块，则这种配置并不理想。

缓冲内存（或带寄存器的内存）改变了这种情况：不再是直接将DRAM模块上的RAM芯片连接到内存时，这些芯片会先连接到一个缓冲器，该缓冲器再连接到内存控制器。这显著降低了电气连接的复杂性。内存控制器驱动DRAM模块的能力因此提高了与节省的连接数量相对应的倍数。



图C.2：注册式DRAM模块

具有这些优势，问题来了：为什么不是所有的DRAM模块都采用缓冲设计？原因有几点。显然，带缓冲的模块稍微复杂一些，因此成本也更高。不过，成本并非唯一因素。缓冲会略微延迟RAM芯片发出的信号；

延迟必须足够高，以确保来自RAM芯片的所有信号都能被缓冲。结果是DRAM模块的时延增加。这里值得一提的是，额外的电气组件增加了能耗成本。由于该缓存器需要在总线频率下运行，其功耗可能相当显著。

使用DDR2和DDR3模块的其他因素通常使得每个银行中不能有超过两个DRAM模块。内存控制器的引脚数量限制了银行的数量（在商用硬件中为两个）。大多数内存控制器能够驱动四个DRAM模块，因此非注册模块已经足够。但在对内存要求很高的服务器环境中情况可能会有所不同。

一些服务器环境的另一个特点是它们无法容忍错误。由于RAM单元中的电容所带电量微小，因此可能发生错误。人们常常开玩笑说宇宙射线会导致这种问题，但这确实是一种可能性。与 α 衰变和其他自然现象一起，这些因素可能导致RAM单元的内容从0变为1或相反。使用的内存越多，此类事件发生的概率就越高。

如果此类错误不可接受，可以使用带有纠错码（Error Correction Code, ECC）的DRAM。纠错码使硬件能够识别不正确的单元内容，并在某些情况下纠正这些错误。在过去，奇偶校验检查只能检测到错误，在发现错误时必须停止机器运行。而使用ECC，则可以在一定范围内自动修正少量的错误位。然而，如果错误数量过高，内存访问将无法正确执行，机器仍然会停止运行。这种情况对于正常工作的DRAM模块来说相当罕见，因为需要在同一模块中同时发生多个错误。

当我们谈到ECC内存时，实际上并不完全准确。执行错误检查的并不是内存本身，而是内存控制器。DRAM模块只是提供了额外的存储空间，并在传输实际数据的同时传递这些非数据位。通常，ECC内存为每8个数据位提供一个额外的校验位。为什么使用8个数据位将在稍后解释。

写入数据到内存地址时，内存控制器会在数据传输前实时计算新内容的纠错码（ECC），然后将数据和ECC一起发送到内存总线上。读取数据时，内存控制器接收包含ECC的数据，并重新计算该数据的ECC，将其与DRAM模块传输过来的ECC进行比较。如果两个ECC匹配，则一切正常。如果不匹配，内存控制器会尝试纠正错误。如果无法纠正错误，则记录该错误并可能停止机器运行。

| Data Bits W | SEC | | SEC/DED | |
|---------------|--------------|----------|--------------|----------|
| | ECC Bits E | Overhead | ECC Bits E | Overhead |
| 4 | 3 | 75.0% | 4 | 100.0% |
| 8 | 4 | 50.0% | 5 | 62.5% |
| 16 | 5 | 31.3% | 6 | 37.5% |
| 32 | 6 | 18.8% | 7 | 21.9% |
| 64 | 7 | 10.9% | 8 | 12.5% |

表C.1: ECC与数据位的关系

几种错误校正技术正在使用，但对于DRAM ECC（纠错码），通常采用汉明码。最初的汉明码用于对四个数据位进行编码，并具备识别和纠正一个翻转位的能力（SEC，单错误校正）。该机制可以轻松扩展到更多数据位。数据位的数量 W 与错误代码所需位数 E 之间的关系由以下方程描述：

$$E = \lceil \log_2(W + E + 1) \rceil$$

求解该方程的迭代结果如表C.1中的第二列所示。通过增加一个额外位，我们可以使用简单的奇偶校验位识别两个翻转的位。这被称为SEC/DED（单错误纠正/双错误检测）。有了这个额外位，我们得到了表C.1中第四列的值。对于 $W = 64$ 来说，开销足够低，并且数字(64, 8)是8的倍数，因此这是ECC的一个自然选择。在大多数模块上，每个RAM芯片产生8位，因此任何其他组合都会导致更不高效的解决方案。

| | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|-----------------------|---|---|---|---|---|---|---|
| ECC Word | D | D | D | P | D | P | P |
| P ₁ Parity | D | – | D | – | D | – | P |
| P ₂ Parity | D | D | – | – | D | P | – |
| P ₄ Parity | D | D | D | P | – | – | – |

汉明码的计算很容易通过一段代码来演示。

使用 $W = 4$ 和 $E = 3$ 。我们在编码字的特定位置计算奇偶校验位。表C.2显示了这一原则。在对应于2的幂的位置上添加奇偶校验位。第一个奇偶校验位 P_1 的奇偶校验和包含每第二个比特。第二个奇偶校验位 P_2 的奇偶校验和包含数据比特1、3和4（这里编码为3、6和7）。类似地，计算 P_4 。

表C.2: 汉明生成矩阵构造

奇偶校验位的计算可以更优雅地用矩阵乘法来描述。我们构造一个矩阵 $G = [I|A]$

其中 I 是单位矩阵， A 是奇偶校验生成矩阵，这些都可以从表C.2中确定。

$$G = \left[\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{array} \right]$$

矩阵 A 的列是由计算 P_1 、 P_2 和 P_4 时使用的比特构建而成。如果我们现在将每个输入数据项表示为一个4维向量 d ，我们可以计算出 $r = d \cdot G$ 并得到一个7维向量 r 。这就是在ECC DDR情况下存储的数据。

为了解码数据，我们构建一个新的矩阵 $H = [A^T|I]$ ，其中 A^T 是计算 G 时生成的校验矩阵的转置。这意味着：

$$H = \left[\begin{array}{cccc|ccc} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{array} \right]$$

$H \cdot r$ 的结果表明存储的数据是否损坏。如果不是这种情况，则产品是三维向量 $(0\ 0\ 0)^T$ 。否则，将该产品的值解释为二进制数时，表示翻转位所在的列号。

作为示例，假设 $d = (1\ 0\ 0\ 1)$ 。这将导致

$$r = (1 \ 0 \ 0 \ 1 \ 0 \ 0 \ 1)$$

使用与 H 相乘进行测试的结果为

$$s = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

现在假设存储数据发生了损坏，从内存读取的数据为 $r' = (1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1)$ 。在这种情况下我们得到

$$s' = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$

该向量不是零向量，当其被视为一个数值时，寄存器 s' 的值为 5。这表示我们在寄存器 r' (中翻转的位的位置是从 1 开始计数的第 5 个位。内存控制器可以纠正这个位，并且程序不会察觉到有任何问题。

处理 DED 部分的额外位只是稍微复杂一些。通过更多的努力，可以创建能够纠正两个或更多翻转位的代码。概率和风险决定了是否需要这样做。一些内存制造商表示，在 256MB 的 RAM 中每 750 小时可能发生一次错误。将内存容量加倍可以使时间减少 75%。在有足够的内存的情况下，短时间内经历错误的概率可能会变得显著，此时 ECC 内存就成为必需品。甚至有可能时间范围如此之短，以至于 SEC/DED 实现都不足以应对。

服务器主板具备在指定时间段内自动读取所有内存的能力，而不是实现更多的错误校正功能。这意味着，无论处理器是否实际请求了该内存数据，内存控制器都会读取这些数据，并且如果 ECC 检查失败，则将纠正后的数据写回内存中。只要在一个时间框架内发生少于两次内存错误的概率是可以接受的，那么 SEC/DED 错误校正就是一个完全合理的解决方案。

与注册型 DRAM 类似，一个问题必须被提出：为什么纠错码 (ECC) DRAM 不是标准配置？这个问题的答案与关于注册型 RAM 的问题答案相同：额外的 RAM 芯片增加了成本，而奇偶校验计算增加了延迟。非注册、无 ECC 内存可以显著更快。由于注册型和 ECC DRAM 问题的相似性，通常只找到同时具备注册功能和 ECC 功能的 DRAM，而不是仅具有注册功能而不具备 ECC 功能的 DRAM。

有一种方法可以克服内存错误。一些制造商提供所谓的“内存 RAID”功能，尽管这个名称常常不准确，其实是指数据在多个 DRAM 模块或至少是 RAM 芯片上冗余分布。具有此特性的主板可以使用无注册的 DRAM 模块，但内存总线上增加的数据流量可能会抵消 ECC 和非 ECC DRAM 模块访问时间上的差异。

D NUMA库介绍

尽管程序员需要的信息以优化线程调度、适当分配内存等目的而言是可用的，但获取这些信息却相当繁琐。现有的非统一内存访问（NUMA）支持库（libnuma，在RHEL/Fedora系统中的numactl包内）远不足以提供足够的功能。

作者提出了一种新的库，该库提供了所有必要的功能以支持非统一内存访问（NUMA）系统。由于内存和缓存层次结构处理存在重叠，此库对于多线程和多核处理器的非NUMA系统也非常有用——几乎涵盖了当前所有的机器。

该新库的功能迫切需要遵循本文档中的建议。这是此处提及它的唯一原因。目前（撰写之时），此库尚未完成、未经过审阅、未经打磨，且未广泛分发。未来可能会有重大改动。当前可在此处获取：

<http://people.redhat.com/drepper/libNUMA.tar.bz2>

此库的接口依赖于/sys 文件系统导出的信息。如果该文件系统未被挂载，许多函数将无法正常工作或提供不准确的信息。这一点在进程处于 chroot 监狱环境中时尤为重要。

当前，库的接口头文件包含以下定义：

```
typedef memnode_set_t;

#define MEMNODE_ZERO_S(集合大小, 内存节点集指针) #define MEMNODE_SET_S(节点, 集合大小, 内存节点集指针) #define MEMNODE_CLR_S(节点, 集合大小, 内存节点集指针) #define MEMNODE_ISSET_S(节点, 集合大小, 内存节点集指针) #define MEMNODE_COUNT_S(集合大小, 内存节点集指针)

```c#define MEMNODE_EQUAL_S(集合大小, 内存节点集1, 内存节点集2)```

#define MEMNODE_AND_S(集合大小, 目标集, 源集1, 源集2) #define MEMNODE_OR_S(集合大小, 目标集, 源集1, 源集2) #define MEMNODE_XOR_S(集合大小, 目标集, 源集1, 源集2)

#define MEMNODE_ALLOC_SIZE(计数) #define MEMNODE_ALLOC(计数) #define MEMNODE_FREE(内存节点集)

```cint NUMA_系统CPU数量(void);int NUMA_系统CPU掩码(size_t 目标大小, cpu_set_t *目标);```

```cint NUMA_本节点CPU数量(void);int NUMA_本节点CPU掩码(size_t 目标大小, cpu_set_t *目标);```

```cint NUMA_获取当前CPU索引(void);int NUMA_获取当前CPU掩码(size_t 目标大小, cpu_set_t *目标);```

ssize_t NUMA_节点级别CPU掩码(size_t 目标大小, cpu_set_t *目标, size_t 源大小, const cpu_set_t *源, unsigned int 级别);

int NUMA_系统内存节点计数(void); int NUMA_系统内存节点掩码(size_t destsize, memnode_set_t *dest); int NUMA_自身内存节点掩码(size_t destsize, memnode_set_t *dest); int NUMA_自身当前内存节点索引(void); int NUMA_自身当前内存节点掩码(size_t destsize, memnode_set_t *dest); int NUMA_将CPU映射到内存节点(size_t cpusetsize, const cpu_set_t *cpuset, size_t __memnodesize, memnode_set_t *memnodeset); int NUMA_将内存节点映射到CPU(size_t memnodesize, const memnode_set_t *memnodeset,
```

```
size_t cpusetsize, cpu_set_t *cpuset);

``cint NUMA_mem_get_node_idx(void *addr);int NUMA_mem_get_node_mask(void *addr, size_t size, size_t destsize, memnode_set_t *dest);``
```

MEMNODE_* 宏在形式和功能上与第 6.4.3 节介绍的 CPU_* 宏类似。这些宏没有非 _S 版本，都需要一个大小参数。memnode_set_t 类型类似于 cpu_set_t，但这次是针对内存节点的。需要注意的是，内存节点的数量可能与 CPU 数量无关，反之亦然。可能存在每个内存节点包含多个 CPU 的情况，甚至也可能存在没有任何 CPU 的内存节点。因此，动态分配的内存节点位集的大小不应由 CPU 数量决定。

NUMA_memnode_system_count 接口应被使用。它返回当前已注册的节点数量。这个数字可能会随着时间的增长或减少，但大多数情况下会保持不变，因此它是用于确定内存节点位集大小的一个良好值。分配过程与 CPU_* 宏类似，分别通过 MEMNODE_ALLOC_SIZE、MEMNODE_ALLOC 和 MEMNODE_FREE 来实现。

作为与 CPU_* 宏的最后一个类比，该库还提供了用于比较内存节点位集是否相等以及执行逻辑运算的宏。

NUMA_cpu_* 函数提供处理 CPU 集合的功能。部分接口只是在新名称下提供了现有功能。NUMA_cpu_system_count 返回系统中的 CPU 数量，而 NUMA_CPU_system_mask 变体则返回一个位掩码，并将相应的位设置为 1——这种功能在其他地方不可用。

NUMA_cpu_self_count 和 NUMA_cpu_self_mask 返回当前线程被允许运行的 CPU 的信息。NUMA_cpu_self_current_idx 返回当前使用的 CPU 索引。由于内核可以做出调度决策，返回的信息可能已经过时；因此必须假定这些信息是不准确的。NUMA_cpu_self_current_mask 返回相同的信息，并在位集中设置相应的位。

NUMA_memnode_system_count 已经介绍过。NUMA_memnode_system_mask 是与此功能等效的函数，用于填充位集。NUMA_memnode_self_mask 根据当前线程可以运行的任何 CPU 直接连接的内存节点来填充位集。

NUMA_memnode_self_current_idx 和 NUMA_memnode_self_current_mask 这两个函数会返回更为专门的信息。这些信息指的是当前线程正在其上运行的处理器所连接的内存节点。与 NUMA_cpu_self_current_* 函数一样，当函数返回时，此信息可能已经过时；它只能用作提示。

NUMA_cpu_to_memnode 函数可以将一组 CPU 映射到直接连接的内存节点集合。如果在 CPU 集中的单个位被设置，则可以确定每个 CPU 所属的内存节点。目前，Linux 不支持一个 CPU 属于多个内存节点；理论上，这种情况在未来可能会发生变化。要进行反向映射，可以使用 NUMA_memnode_to_cpu 函数。

如果内存已经分配，有时需要知道其具体分配位置。这正是 NUMA_mem_get_node_idx 和 NUMA_mem_get_node_mask 这两个函数可以提供给程序员的信息。前者返回参数指定地址对应的页面所在的内存节点索引——或者根据当前安装的策略将要被分配到的内存节点索引（如果该页面尚未分配）。后者则可以为整个地址范围执行相同的工作，它以位集的形式返回信息。该函数的返回值表示使用的不同内存节点的数量。

在本节的其余部分，我们将看到这些接口的一些用例示例。在所有情况下，我们都省略了错误处理以及 CPU 数量和/或内存节点数量过大导致 cpu_set_t 和 memnode_set_t 类型无法容纳的情况。如何使代码更健壮留作练习。

确定给定 CPU 的线程兄弟关系

为了调度辅助线程或其他受益于在特定 CPU 线程上调度的线程，可以使用如下代码序列：

```
``ccpu_set_t cur; CPU_ZERO(&cur); CPU_SET(cpunr, &cur);cpu_set_t hyperths; NUMA_cpu_level_mask(sizeof(hyperths), &hyperths, sizeof(cur), &cur, 1);CPU_CLR(cpunr, &hyperths);``
```

代码首先为由 cpunr 指定的CPU生成一个位集。然后将该位集传递给 NUMA_cpu_level_mask，并通过第五个参数指定我们正在查找超线程。结果返回在 hyperths 位集中。最后一步是清除对应于原始 CPU 的位。

确定给定CPU的核心兄弟关系

如果两个线程不应被调度在同一超线程上，但可以受益于缓存共享，则需要确定处理器的其他核心。以下代码序列可以实现这一目的。

```
cpu_set_t cur; CPU_ZERO(&cur); CPU_SET(cpunr, &cur);cpu_set_t hyperths; int nhts = v0;NUMA_cpu_level_mask(sizeof(hyperths), &hyperths, sizeof(cur), &cur, 1);cpu_set_t coreths; int ncs = v1;NUMA_cpu_level_mask(sizeof(coreths), &coreths, sizeof(cur), &cur, 2);CPU_XOR(&coreths, &coreths, &hyperths);ncs -= nhts;
```

代码的第一部分与确定超线程的代码相同，这并非偶然，因为我们必须将给定CPU的超线程与其他核心区分开。这一功能在第二部分中实现，该部分再次调用NUMA_cpu_level_mask函数，但这次级别为2。接下来要做的就是从结果中移除给定CPU的所有超线程。变量nhts和ncs用于记录各自位集中的设置位的数量。

得到的掩码可以用于调度另一个线程。如果没有其他需要显式调度的线程，使用哪个核心的选择可以留给操作系统。否则，可以迭代地运行以下代码：

```
while (ncs > 0) { size_t idx = 0; while (!CPU_ISSET(idx, &ncs)) { ++idx; } CPU_ZERO(&cur); CPU_SET(idx, &cur); nhts = NUMA_cpu_level_mask(sizeof(hyperths), &hyperths, sizeof(cur), &cur, 1); CPU_XOR(&coreths, &coreths, hyperths); ncs -= nhts; ... schedule thread on CPU idx ...}
```

在每次迭代中，循环从剩余的已用核心中选择一个CPU编号。然后计算该CPU的所有超线程。随后使用CPU_XOR将得到的位集从可用核心的位集中减去。如果XOR操作没有移除任何内容，则说明出现了严重问题。更新ncs变量后，我们准备进入下一轮迭代，但在那之前需要做出调度决策。最终，可以根据程序的需求使用idx、cur或hyperths中的任何一个来调度线程。通常最好尽可能地给操作系统留出自由度，因此建议使用hyperths位集，以便让操作系统选择最佳的超线程。

E 索引

ABI, 54 访问模式, 4 激活, 9 地址解复用, 6 地址监听, 16 地址空间布局随机化, 59 别名, 15 别名效应, 50 α 衰变, 105 算术逻辑单元, 29 应用程序二进制接口, 54 地址空间标识符, 40 地址空间布局随机化, 59 异DRAM, 8

瓶颈,

缓存,

dcbz 48 DDR, DDR1 10 DDR2 8, 10 DDR3 1 Demultiplexer 6 Direct Cache Access 64 直接映射缓存 18 脏标志

DMA, 3, 12 双错误检测, 106 双泵技术, 10D RAM, 3, 5 动态随机存取存储器 (Dynamic RAM), 5 ECC DRAM, 105 纠错码 (Error Correction Code) DRAM, 105 独占缓存, 16 伪共享, 6 5FB-DRAM, 11 Fortran, 50 前端总线 (Front Side Bus), 8 FSB, 3, 8f., 11, 14, 70 全相联缓存, 17 全双工总线, 12 gcov, 85 Geneseo, 95 getconf, 50 汉明码 (Hamming Code), 106 硬件预取技术, 60 辅助线程, 63 HUGE_TLB, 8 hugetlbfs, 88 Hyper Transport, 43 超线程 (Hyper-Thread), 29, 63 虚拟机监控器 (Hypervisor), 31, 39 包含缓存, 16, 78 延迟, 8, 23, 30, 60, 95 漏电效应, 5 加载条件 (LL/SC), 68, 89 循环流检测器 (Loop Stream Detector), 57 LSD, 57 MAP_FIXED, 60 大规模存储, 1 内存通道 (Memory Channel), 11 内存控制器 (Memory Controller), 1, 4 内存管理单元 (Memory Management Unit), 30 内存模型 (Memory Model), 16 内存排序 (Memory Ordering), 48 RAID, 4 MESI 协议, 26 MMU, 30 MPI, 43 多核处理器 (Multi-core), 15 多路复用器 (Multiplexer), 6 北桥 (Northbridge), 3, 70 非一致性内存访问架构 (NUMA), 4 NUMA 因子, 4, 43 乱序执行 (OOO), 60

opcontrol.....102opr
eport.....103OProfi
le.....102Out-Of-O
rder.....60

帕西法, 40 页
表查找, 37 并行
端口, 3PATA, ...
..... 3PCI Express,
..... 3PCI-E,
..... 3 程序引导优化,.....
..... 85 物理地址,.....
..... 6 预取页错误,.....
..... 87 预充电,.....
.... 9 预取触发器,..... 60
预取, 14, 20, 60

四倍泵送, 8

RAM,

——

—— ——

SATA,, 3 _SC_LEVEL1_DCACHE_LINESI
ZE,, 50 SDRAM,, 3、8、10 自修改
代码,, 31 感测放大器,6 串行端口
,, 3 组相联,18 SIMD,, 51
、95 单纠错,105 SMC,, 31 SMP
,, 16 SMT,, 29 软件预取,6
1 南桥,3 空间局部性,13 推测,
....., 62 投机加载,63 SPU,, 95 S
RAM,, 5 静态RAM,, 5 流式预取,
....., 60 流读缓冲区,48 步长,6
0、96 对称多线程,29

标签,15 时间局部
性,13 快表,.....
.....22, 39

Torrenza,95 追
踪缓存,31 事务
性内存,91 晶体管
,5 快表,
.....39 tRAS, .
.....9 tRCD, ...
.....8 tRP,
.....9

未注册DRAM,
105 USB,
3, 12

向量操作,95
虚拟地址,6
虚拟机监控器,31
VMM,31,
39 诺依曼架构,
..14

我们, 9 写使
能, 9 写合并
, 25, 48

参考文献列表

- [1] AMD. 先进微设备公司, 2006年6月. 网址 <http://www.amd.com/us-en/assets/content type/white papers and tech docs/40555.pdf>. 5.4
- [2] Jennifer M. Anderson, Lance M. Berc, Jeffrey Dean, Sanjay Ghemawat, Monika R. Henzinger, Shun-Tak A. Leung, Richard L. Sites, Mark T. Vandevoorde, Carl A. Waldspurger 和 William E. Weihl. 连续 profiling: 所有的时钟周期都去哪儿了. 在 *Proceedings of the 16th ACM Symposium of Operating Systems Principles*, 页码 1–14, 1997年10月. URL <http://citeseer.ist.psu.edu/anderson97continuous.html>. 7.1
- Vinodh Cuppu, Bruce Jacob, Brian Davis 和 Trevor Mudge. 工作站环境中高性能 DRAM 的应用. *IEEE Transactions on Computers*, 50(11):1133–1153, 2001年11月. URL <http://citeseer.ist.psu.edu/476689.html>. 2.1.2, 2.2, 2.2.1, 2.2.3, 10
- [4] Arnaldo Carvalho de Melo. 七个小矮人: 超越 GDB 的调试信息. 在 *Proceedings of the Linux Symposium*, 2007. URL <https://ols2006.108.redhat.com/2007/Reprints/melo-Reprint.pdf>. 6.2.1
- Simon Doherty, David L. Detlefs, Lindsay Grove, Christine H. Flood, Victor Luchangco, Paul A. Martin, Mark M.oir, Nir Shavit 和 Jr. Guy L. Steele. DCAS 不是无锁算法设计的万能解决方案. 载于 *SPAA '04: Proceedings of the Sixteenth Annual ACM Symposium on Parallelism in Algorithms and Architectures*, 2004 年, 第 216–224 页, 纽约, 美国. ACM Press 出版. ISBN 1-58113-840-7. 网址 <http://research.sun.com/scalable/pubs/SPAA04.pdf>. 8.1
- 【6】M. Dowler. DDR2 内存简介: DDR 的继任者. <http://www.pcstats.com/articleview.cfm?articleID=1573>, 2004年5月. 2.2.1
- 乌尔里希·德雷珀. 关于 Futex 的复杂性, 2005年12月. URL <http://people.redhat.com/drepper/futex.pdf>. 6.3.4
- 乌尔里希·德雷珀. 线程本地存储的ELF处理. 技术报告, 红帽公司, 2003年. URL <http://people.redhat.com/drepper/tls.pdf>. 6.4.1
- 乌尔里希·德雷珀. 红帽企业版Linux的安全增强, 2004年. URL <http://people.redhat.com/drepper/nonselsec.pdf>. 4.2
- 冯博瑞, 奥拉雷, 莱茨. 无锁技术在共享对象并发访问中的应用. 载于《GMEM》第9卷, 编辑: GMEM, 页码143–150, 2002年. 网址 <http://www.grame.fr/pub/fober-JIM2002.pdf>. 8.1, 8.1
- 乔·吉比斯和大卫·帕特森. 拥抱并扩展二十世纪指令集架构. *Computer*, 40(4):68–75, 2007年4月. 8.4
- 大卫·戈德伯格. 每位计算机科学家都应了解的浮点运算知识. *ACM Computing Surveys*, 23(1):5–48, 1991年3月. URL <http://citeseer.ist.psu.edu/goldberg91what.html>. 1
- 赫尔利 (Maurice Herlihy) 和摩斯 (J. Eliot B. Moss). 事务性内存: 无锁数据结构的架构支持. 载于《v13》, 1993年. 网址 <http://citeseer.ist.psu.edu/herlihy93transactional.html>. 8.2, 8.2.2, 8.2.3, 8.2.4
- [14] 拉姆·胡加哈拉利、拉维·艾耶尔和斯科特·泰特里克. 高带宽网络I/O的直接缓存访问, 2005. URL <http://www.stanford.edu/group/comparch/papers/huggahalli05.pdf>. 6.3.5
- 英特尔公司, 2007年5月. URL: <http://www.intel.com/design/processor/manuals/248966.pdf>. B.3
- 威廉·马戈、保罗·彼得森和桑吉夫·沙赫. 超线程技术: 对计算密集型工作负载的影响. 《英特尔技术期刊》第6卷第1期, 2002年. URL ftp://download.intel.com/technology/itj/2002/volume06issue01/art06_computeintensive/vol6iss1_art06. 3.3.4
- 控制符号顺序. <http://blogs.linux.ie/caolan/2007/04/24/controlling-symbol-ordering/>, 2007年4月. 7.5
- 【美光科技, 2003年. 修订版L 6/06英文版. 2.2.2节, 第10页】
- [19] Jon “Hannibal” Stokes. Ars Technica 内存指南, 第二部分: 异步和同步 DRAM. http://arstechnica.com/paedia/r/ram_guide/ram_guide.part2-1.html, 2004. 2.2[20] 维基百科. 静态随机访问内存. http://en.wikipedia.org/wiki/Static_Random_Access_Memory, 2006. 2.1.1

修订历史记录

2007-6-24 First internal draft.

2007-8-11 Add first set of Johnray's edits.

2007-9 to 2007-11 Lots of language cleanup from Jonathan Corbet and team.

2007-11-21 Version 1.0.

| | |
|----------------------------|----------------------------------|
| Source File Identification | |
| Last Modification | D:20071121175205-08'00' |
| Source File Size | 553180 Bytes |
| Source File MD5 | 0299BAC7C5B6A501077DEB5714C944DF |
| MPost Last Modification | D:20071104104838-08'00' |
| MPost Source File Size | 138021 Bytes |
| MPost Source File MD5 | 7732297065D660A939A9D651A7992FA2 |